Introduction to Causal Econometrics with Observational Data

Xiang Ao, HBS

2026-08-15

Table of contents

Preface

Most applied questions are causal. Did a program raise earnings? Did a drug reduce mortality? Did a policy change behavior? With experimental data, a difference in means can sometimes answer the question. With observational data, it almost never can. Causal econometrics is the business of saying what would have to be true for a number we can compute to mean the causal object we want.

Prerequisites and target audience

This guide assumes graduate-level (or advanced undergraduate) familiarity with probability and statistics – expectations, conditional expectations, basic asymptotic theory (consistency, asymptotic normality) – and with regression analysis at the level of an introductory econometrics course (OLS, instrumental variables, panel data fixed effects). It does not assume measure-theoretic probability. Familiarity with directed acyclic graphs (DAGs) is helpful but not required; the graph-theoretic chapters build up the needed concepts from scratch.

On the computational side, the code examples are in R and assume basic fluency with the language, including the `tidyverse` (`dplyr`, `tidyr`, `ggplot2`) for data manipulation and plotting. See the R Packages Used in This Book appendix for the full list of packages and an installation script.

This book is a working guide in R. It is for applied researchers who already know regression and probability, and who want to see the methods carried out end to end. Each chapter gives the identifying assumptions and then runs code on a real or simulated data set. The book does not try to teach every package. It shows when a package is useful and how the econometric object maps to the code.

The book is organized in the order in which an applied project usually runs. Part I is identification. Part II is estimation. Part III is designs: difference-in-differences, instrumental variables, regression discontinuity, and shift-share IV. The remaining parts cover longitudinal settings, survival outcomes, mediation, and causal discovery. The appendix lists the R packages used in the examples.

A companion volume, Causal Econometrics with Julia, covers the same ground in Julia. Chapters are cross-linked where the two books diverge — usually because a particular estimator is easier or faster in one language than the other. A messier working notebook, Topics on Econometrics and Causal Inference, holds the rougher posts that fed into both books. Source files and datasets are available in the repository linked above, and the README lists the required R packages (package versions are not pinned with a lockfile); the "Edit this page" link at the foot of each chapter goes directly to the corresponding `.qmd` file. Corrections and suggestions are welcome through the issues tracker.

How to cite

Ao, Xiang. *Introduction to Causal Econometrics with Observational Data*. https://xiangao.github.io/causal_econometrics_guide/.

```
@book{ao_causal_econometrics_guide,
  author = {Ao, Xiang},
  title  = {Introduction to Causal Econometrics with Observational Data},
  url    = {https://xiangao.github.io/causal_econometrics_guide/}
}
```

Part I.

Identification

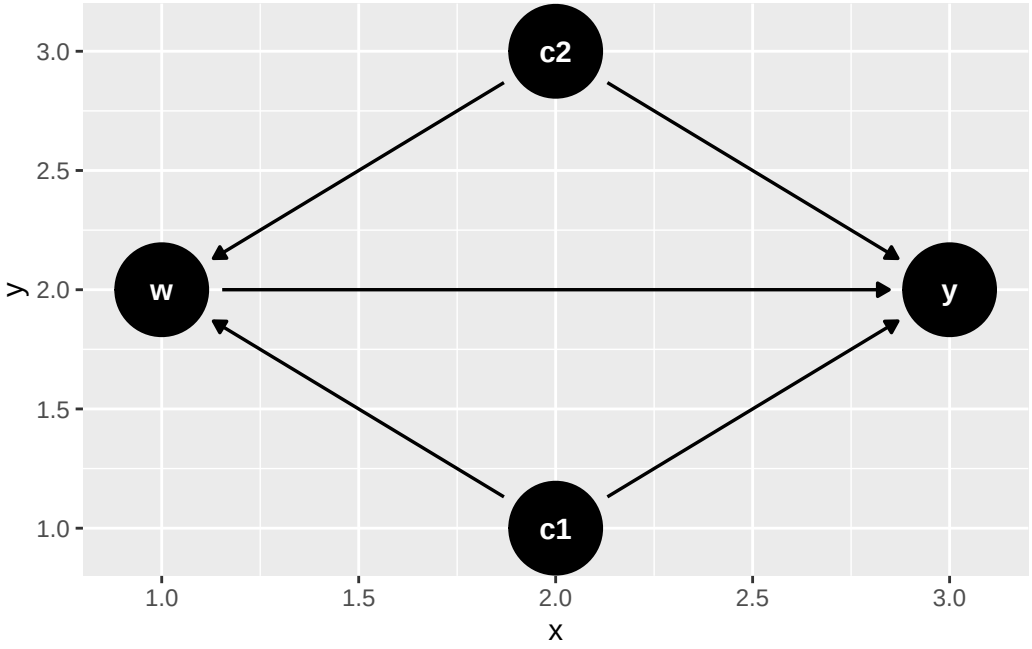
1. Identification & Potential Outcomes

1.1. Why Do We Need Tools for Causal Inference?

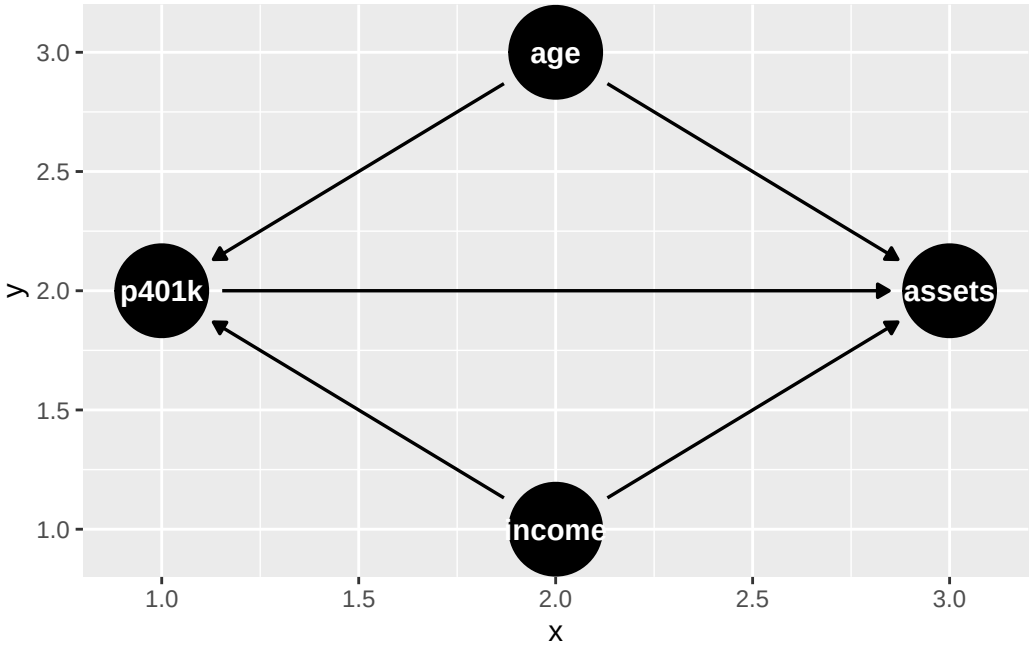
Association is not causation. We have known how to measure associations for a long time, but most econometric models are associational. The gap between the two arises from confounding.

1.2. Confounders

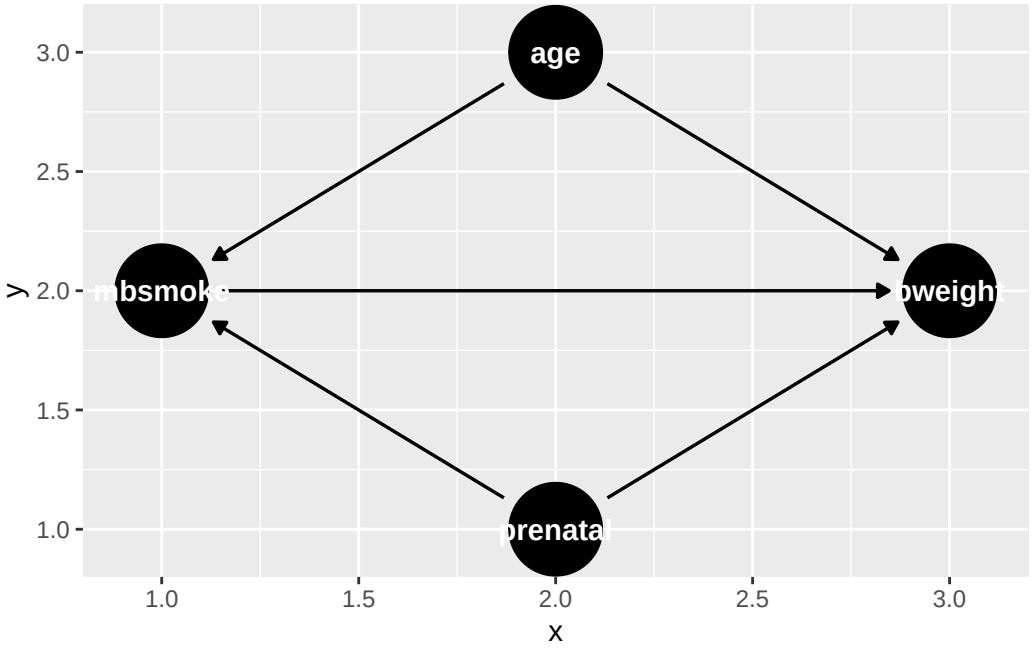
In this graph, w is the treatment, y the outcome, and c_1, c_2 are confounders. Both confounders affect both w and y, so the observed association between w and y may reflect confounding rather than a causal effect.



A concrete example: income and age may both drive 401(k) participation and assets. To identify the causal effect of participation on assets, we need to control for income and age.



And another example with birth weight and maternal smoking. It could be the case that prenatal care and maternal age are causing both maternal smoking and birth weight. It could be that maternal smoking has nothing to do with birth weight. But age and prenatal care are causing both maternal smoking and birth weight, therefore maternal smoking appears to cause birth weight.



1.3. Controlled and Uncontrolled Confounders

If we can control for all confounders, correlation does imply causation. The two questions are: do we observe all confounders, and how do we control for them? In a randomized experiment neither question arises. In an observational study both do. If c_2 is unobserved, any estimate of the effect of w on y is biased.

1.4. Potential Outcome Setup

We study causal effects through the potential outcomes (counterfactual) framework. Each unit has two potential outcomes, $Y(1)$ and $Y(0)$, one for each treatment state. These are fixed before any assignment; the experiment tries to recover them. The fundamental problem is that we never observe both: only the realized outcome $Y = WY(1) + (1 - W)Y(0)$ is seen. Individual treatment effects $\tau_i = Y_i(1) - Y_i(0)$ are not identifiable; only averages can be estimated.

We write W for treatment assignment and X for covariates.

1.5. ATE, ATT and ATU

The standard estimands are:

$$\tau_{ATE} = E[Y(1) - Y(0)] \tag{1.1}$$

$$\tau_{ATT} = E[Y(1) - Y(0)|W = 1] \tag{1.2}$$

$$\tau_{ATU} = E[Y(1) - Y(0)|W = 0] \tag{1.3}$$

$$\tau_{ATE} = \rho\tau_{ATT} + (1 - \rho)\tau_{ATU} \tag{1.4}$$

where $\rho = P(W = 1)$.

1.6. Identification: From Potential Outcome to Observed

We focus on τ_{ATT}. To identify it, we need:

$$E[Y(1)|W = 1] \tag{1.5}$$

which is easy, because we observe it.

And we need

$$E[Y(0)|W = 1] \tag{1.6}$$

To identify this, we'll need unconfoundedness:

$$E[Y(0)|W, X] = E[Y(0)|X] \tag{1.7}$$

and overlap:

$$\pi(x) = P(W = 1|X = x) < 1 \tag{1.8}$$

(for the ATT only $\pi(x) < 1$ is needed; the ATE requires $0 < \pi(x) < 1$).

Unconfoundedness says that within each subgroup $X = x$, treated and control units have the same mean of $Y(0)$: $E[Y(0)|W = 1, X = x] = E[Y(0)|W = 0, X = x] = E[Y|W = 0, X = x]$. Averaging over the covariate distribution of the treated,

$$E[Y(0)|W = 1] = E\big[E[Y|W = 0, X] \mid W = 1\big], \tag{1.9}$$

which involves only observed data. Therefore ATT is identified:

$$\begin{aligned} \tau_{ATT} &= E[Y(1) - Y(0)|W = 1] \\ &= E[Y|W = 1] - E\big[E[Y|W = 0, X] \mid W = 1\big] \end{aligned} \tag{1.10}$$

Note the conditional-on-X assumption does NOT imply the marginal equality $E[Y(0)|W = 1] = E[Y(0)|W = 0]$: treated and control units have different covariate distributions, so the adjustment over X is essential. In a randomised experiment, unconfoundedness holds without conditioning on X (i.e. $E[Y(0)|W] = E[Y(0)]$); then $E[Y(0)|W = 1] = E[Y(0)|W = 0] = E[Y|W = 0]$, the formula collapses to $E[Y|W = 1] - E[Y|W = 0]$, and a difference-in-means estimator is consistent.

$$\hat{\tau}_{ATT} = \bar{Y}_1 - \bar{Y}_0 \tag{1.11}$$

In observational studies, unconfoundedness is rarely credible without further justification. The rest of this book develops the tools for doing so.

1.7. Unconfoundedness and Overlap

Weaker Assumptions: Unconfoundedness (conditional independence, or ignorability), stated jointly for both potential outcomes since we are after the ATE here (as opposed to the ATT case above, which only required it for $Y(0)$):

$$E[Y(d)|W, X] = E[Y(d)|X] \quad \text{for } d \in \{0, 1\} \tag{1.12}$$

Overlap:

$$0 < \pi(x) = P(W = 1|X = x) < 1 \tag{1.13}$$

SUTVA (stable unit treatment value assumption): 1. No interference between units. No spillovers. 2. Consistency. No hidden versions of treatment or control. Treatment is clearly defined.

Within each subgroup of $X = x$, W is independent of $Y(0)$. With the overlap assumption, we ensure there is a comparable control unit for each treated unit.

However, these two criteria are often contradictory: we may need more covariates to satisfy conditional independence; meanwhile, if the dimension of covariates goes high, then it's likely in some partition we don't have control units.

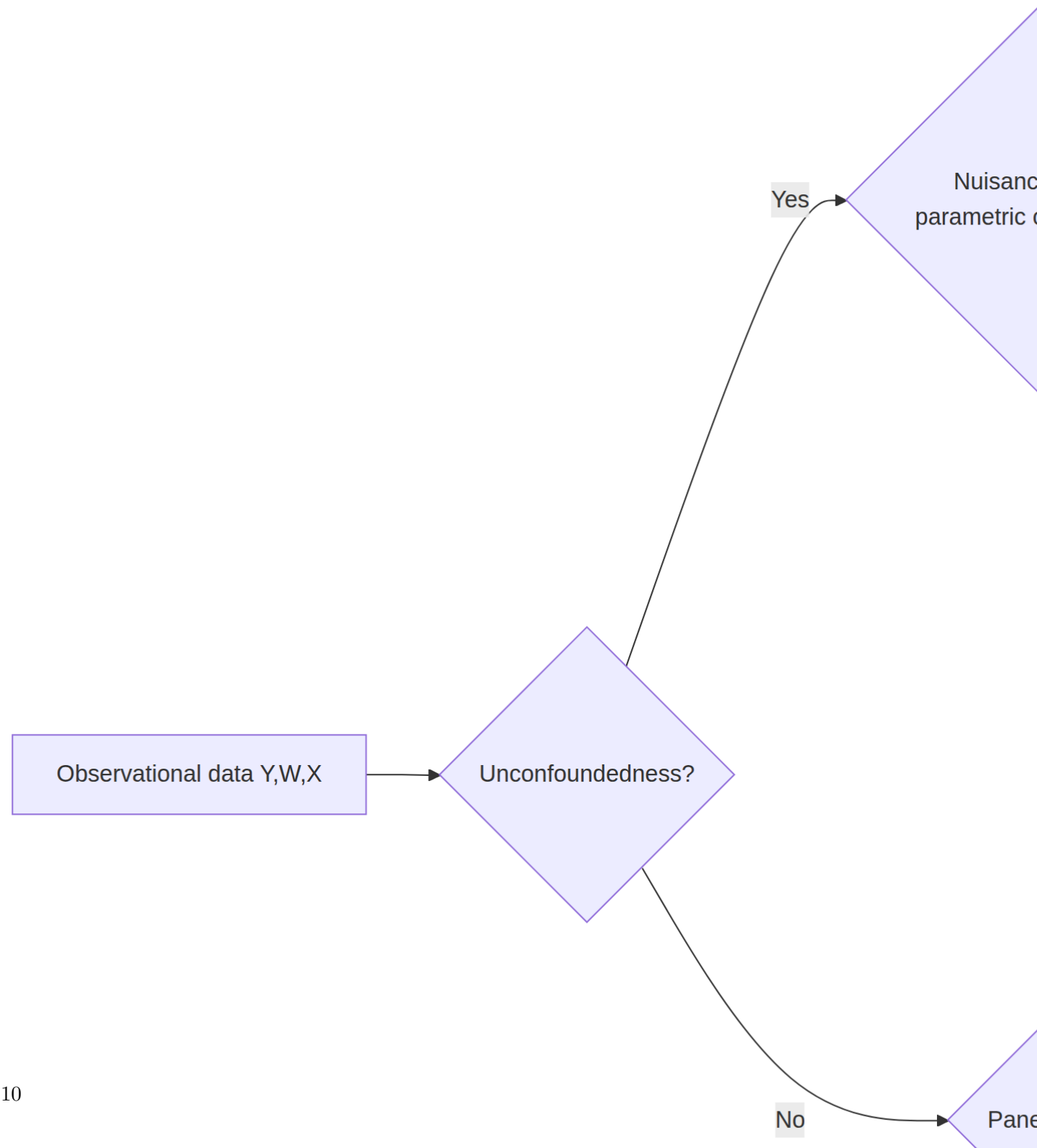
We need four assumptions for identification:

- Unconfoundedness (conditional independence, or ignorability)
- Overlap (positivity, common support)
- No interference
- Consistency

1.8. Observational Data Flowchart

In observational studies, I summarise the decision what method to use for identification of causal effect:

Note: AIPW and IPWRA are themselves influence-function-based, doubly-robust estimators – they can be implemented with either parametric nuisance models (logistic regression, OLS) or flexible machine-learning nuisance models (with cross-fitting, as in DoubleML/TMLE), so they appear on both sides of the split below. The split is really about *how the nuisance (propensity/outcome) models are estimated*, not about the estimator family itself.


Observational data Y,W,X
Unconfoundedness?
Yes
Nuisanc
parametric
No
Pane

2. Five Estimands on One DGP

```
library(ggplot2)
library(dplyr)
library(gridExtra)
```

Before choosing an estimator, we need to know the estimand. ATE, ATT, LATE, CATE and QTE are not five ways to estimate the same object. They are five different objects. Sometimes they are numerically close, but that is a feature of the data-generating process, not a general result.

Here I use one simulated data set so we can calculate all of them. The simulation is useful because we observe both potential outcomes. In real data we do not, which is exactly why identification assumptions matter.

2.1. The data-generating process

We draw $n = 20{,}000$ people. Each has an observed covariate $X_i \sim U(0,1)$, an unobserved type $U_i \sim N(0,1)$, and a randomly assigned offer $Z_i \sim \text{Bernoulli}(0.5)$. The offer is the instrument. It raises the probability of treatment but does not enter the outcome. The treatment probabilities are $P(D = 1 \mid Z = 0, U) = 0.10 + 0.20U$ and $P(D = 1 \mid Z = 1, U) = 0.10 + 0.20U + 0.50$, both clipped to $[0,1]$. One uniform draw V_i is compared against both thresholds, so $D_i(1) \geq D_i(0)$ for every unit and there are no defiers. The potential outcomes are $Y_i(0) = 0.5X_i + 0.3U_i + \varepsilon_i$ with $\varepsilon_i \sim N(0,1)$, and $Y_i(1) = Y_i(0) + \tau(X_i)$ with $\tau(x) = 1 + 2x$. We observe $D_i = Z_i D_i(1) + (1 - Z_i)D_i(0)$ and $Y_i = D_i Y_i(1) + (1 - D_i)Y_i(0)$.

Two features of this design drive everything below. Selection into treatment runs through U and V, never through X. The treatment effect runs through X alone. And X is drawn independently of U, V and Z.

```
set.seed(42)
n <- 20000

X <- runif(n, 0, 1)         # observed covariate, in [0,1]
U <- rnorm(n)               # latent type, unobserved
Z <- rbinom(n, 1, 0.5)     # random instrument

# Treatment assignment: depends on Z (the instrument) and U (selection on type)
# Higher U → more likely to take treatment regardless of Z (always-takers)
# Lower  U → less likely to take treatment regardless of Z (never-takers)
# Middle U → responsive to Z (compliers)
```

```r
pD0 <- pmin(pmax(0.10 + 0.20 * U,        0), 1)  # P(D=1 | Z=0, U)
pD1 <- pmin(pmax(0.10 + 0.20 * U + 0.50, 0), 1)  # P(D=1 | Z=1, U) - adds 0.50

# One latent draw V per person, thresholded against both probabilities.
# Since pD1 >= pD0, V < pD0 implies V < pD1, so D1 >= D0 for every unit:
# monotonicity (no defiers) holds by construction, as the LATE theorem requires.
V  <- runif(n)
D0 <- as.integer(V < pD0)   # potential treatment if not offered slot
D1 <- as.integer(V < pD1)   # potential treatment if offered slot
D  <- ifelse(Z == 1, D1, D0)

# Heterogeneous treatment effect: depends on X
# Y(d) = baseline + d * tau(X) + 0.3 * U + noise
tau_fn <- function(x) 1.0 + 2.0 * x   # the true individual treatment effect

Y0 <- 0.5 * X + 0.3 * U + rnorm(n)
Y1 <- Y0 + tau_fn(X)
Y  <- ifelse(D == 1, Y1, Y0)

df <- data.frame(X = X, Z = Z, D = D, Y = Y)
head(df)
```

```
          X Z D          Y
1 0.9148060 1 1  4.1619422
2 0.9370754 0 0 -0.8962833
3 0.2861395 1 1  1.7657727
4 0.8304476 1 1  3.0306957
5 0.6417455 0 0  0.1362524
6 0.5190959 0 0 -0.2396664
```

The printed rows are the data an analyst would receive: X, Z, D and Y. We keep `Y0` and `Y1` aside, and every "true" number in this chapter is computed from them. With real data each observation reveals only one of the two.

2.2. The five estimands

2.2.1. ATE — average treatment effect

$$\text{ATE} = \mathbb{E}[Y(1) - Y(0)] \tag{2.1}$$

ATE is the mean effect in the whole population. It compares the mean outcome if everyone were treated with the mean outcome if nobody were treated. Here it is the average of $\tau(x) = 1 + 2x$ over $X \sim U(0, 1)$, which is exactly 2. We average the individual effects in the sample and compare.

```r
ate_true <- mean(Y1 - Y0)
cat(sprintf("ATE (population mean of Y1 - Y0) = %.3f\n", ate_true))
```

```
ATE (population mean of Y1 - Y0) = 1.996
```

```r
# Theoretical ATE: integral of (1 + 2x) over Uniform[0,1] = 1 + 1 = 2
cat("Theoretical ATE = integral(1 + 2x)dx on [0,1] = 2.000\n")
```

```
Theoretical ATE = integral(1 + 2x)dx on [0,1] = 2.000
```

The sample average is 1.996 against a population value of 2. The 0.004 gap is simulation noise at $n = 20{,}000$.

2.2.2. ATT — average treatment effect on the treated

$$\text{ATT} = \mathbb{E}[Y(1) - Y(0) \mid D = 1] \tag{2.2}$$

ATT is the mean effect for the units that actually took treatment. It differs from ATE when treatment selection is related to treatment-effect heterogeneity. We average the individual effects over the treated units only.

```r
att_true <- mean(Y1[D == 1] - Y0[D == 1])
cat(sprintf("ATT (mean of Y1-Y0 conditional on D=1) = %.3f\n", att_true))
```

```
ATT (mean of Y1-Y0 conditional on D=1) = 1.995
```

ATT is 1.995 against an ATE of 1.996. The two are analytically identical in this DGP, exactly 2.0 in the population, so the small difference is simulation noise and not a true gap between the estimands. The reason is mechanical: the individual treatment effect $\tau_i = 1 + 2X_i$ depends only on X_i, while selection into treatment depends on U_i (and the instrument Z_i below), and X_i is generated independently of U_i and Z_i. Since treatment-effect heterogeneity is unrelated to who selects into treatment or who complies with the instrument, averaging τ_i over the treated subpopulation or over compliers gives the same population average as over everyone – hence ATE = ATT = LATE = 2.0 exactly here. If selection (or compliance) also depended on X, these estimands would diverge, since ATE, ATT, and LATE are different averages of τ_i across different, non-identical subpopulations.

2.2.3. LATE — local average treatment effect (Imbens-Angrist)

$$\text{LATE} = \mathbb{E}[Y(1) - Y(0) \mid D(1) > D(0)] \tag{2.3}$$

LATE is the mean effect for compliers, the units whose treatment status is changed by the instrument. Under the usual IV assumptions, the Wald estimator identifies this effect, not the ATE. Identifying LATE by the Wald ratio additionally requires monotonicity (no defiers): $D(1) \geq D(0)$ for all i. This DGP imposes it by generating both potential treatments from one shared latent draw V_i with ordered thresholds ($D(z) = \mathbb{1}\{V_i < p_{Dz}\}$ with $p_{D1} \geq p_{D0}$), so no unit is pushed *out* of treatment by the offer. Raising the *probability* alone would not be enough: with independent draws for $D(0)$ and $D(1)$, roughly 3% of units would be defiers and the Wald ratio would no longer equal the complier mean.

We know $D_i(0)$ and $D_i(1)$ for every unit, so we can label the compliers and average their individual effects directly. We then compute the Wald ratio from Y, D and Z alone, which is all an analyst would have, and compare.

```
compliers <- (D1 == 1) & (D0 == 0)
late_true <- mean(Y1[compliers] - Y0[compliers])
cat(sprintf("LATE (mean of Y1-Y0 conditional on complier status) = %.3f\n", late_true))
```

```
LATE (mean of Y1-Y0 conditional on complier status) = 1.999
```

```
cat(sprintf("Share of compliers: %.3f\n", mean(compliers)))
```

```
Share of compliers: 0.456
```

```
# Wald estimator from the data alone
wald <- (mean(Y[Z == 1]) - mean(Y[Z == 0])) /
        (mean(D[Z == 1]) - mean(D[Z == 0]))
cat(sprintf("Wald IV estimate (should match LATE): %.3f\n", wald))
```

```
Wald IV estimate (should match LATE): 1.949
```

Compliers are 45.6% of the sample and their mean effect is 1.999, again the population value of 2. The Wald ratio gives 1.949. The two agree up to sampling noise: the ratio divides a difference in mean outcomes by a treatment rate difference of 0.456, which magnifies the noise in the numerator, and the 0.05 gap is about one standard error of the ratio. With heterogeneous effects IV averages over the people moved by the instrument, and here that group has the same distribution of X as everyone else.

2.2.4. CATE — conditional average treatment effect

$$\mathrm{CATE}(x) = \mathbb{E}[Y(1) - Y(0) \mid X = x] \tag{2.4}$$

CATE keeps X in the estimand. In this DGP, $\mathrm{CATE}(x)$ is simply $\tau(x) = 1 + 2x$. We cannot condition on a single point of a continuous X in a finite sample, so we cut $[0, 1]$ into 20 equal-width bins, average the individual effects within each bin, and print two of them next to the true τ at the bin centre.

```
nbins <- 20
breaks <- seq(0, 1, length.out = nbins + 1)
centers <- (breaks[-1] + breaks[-(nbins + 1)]) / 2
bin_idx <- cut(X, breaks = breaks, include.lowest = TRUE, labels = FALSE)

cate_est <- tapply(Y1 - Y0, bin_idx, mean)

cat(sprintf("CATE at x=%.3f: estimated %.2f, true %.2f\n",
            centers[4], cate_est[4], tau_fn(centers[4])))
```

```
CATE at x=0.175: estimated 1.35, true 1.35
```

```
cat(sprintf("CATE at x=%.3f: estimated %.2f, true %.2f\n",
            centers[16], cate_est[16], tau_fn(centers[16])))
```

```
CATE at x=0.775: estimated 2.55, true 2.55
```

The binned averages are 1.35 at $x = 0.175$ and 2.55 at $x = 0.775$, matching τ at both bin centres to two decimals. The effect at $x = 0.775$ is nearly twice the effect at $x = 0.175$, and no single scalar reports that.

2.2.5. QTE — quantile treatment effect

$$\mathrm{QTE}(q) = F^{-1}_{Y(1)}(q) - F^{-1}_{Y(0)}(q) \tag{2.5}$$

QTE compares the two marginal outcome distributions. It is the difference between a quantile under treatment and the same quantile under control. It does not follow the same person across the two potential outcomes. We take the quantiles of $Y(1)$ and of $Y(0)$ on a grid from 0.05 to 0.95 and difference them, printing the 10th, 50th and 90th percentiles.

```
qte_grid <- seq(0.05, 0.95, by = 0.05)
qte_est  <- quantile(Y1, qte_grid) - quantile(Y0, qte_grid)

cat(sprintf("QTE(0.10) = %.3f\n", quantile(Y1, 0.10) - quantile(Y0, 0.10)))
```

```
QTE(0.10) = 1.709
```

```r
cat(sprintf("QTE(0.50) = %.3f\n", quantile(Y1, 0.50) - quantile(Y0, 0.50)))
```

```
QTE(0.50) = 2.009
```

```r
cat(sprintf("QTE(0.90) = %.3f\n", quantile(Y1, 0.90) - quantile(Y0, 0.90)))
```

```
QTE(0.90) = 2.283
```

QTE rises from 1.709 at the 10th percentile to 2.009 at the median and 2.283 at the 90th. The reason is that $\tau(X) = 1 + 2X$ is largest where X is large, and $Y(0) = 0.5X + 0.3U + \varepsilon$ is also increasing in X, so treatment adds most to the units that were already high in the control distribution. Treatment stretches the distribution rather than shifting it. The median QTE happens to sit near the ATE here, and there is no general reason for that.

2.3. All five estimands on two plots

The left panel plots the binned CATE against the true $\tau(x)$, with ATE, ATT and LATE drawn as horizontal lines so the collapse to a scalar is visible. The right panel plots QTE(q) across the grid with the ATE marked for reference.

```r
df_cate <- data.frame(x = centers, cate = as.numeric(cate_est),
                      true_tau = tau_fn(centers))
df_qte  <- data.frame(q = qte_grid, qte = as.numeric(qte_est))

p1 <- ggplot(df_cate, aes(x = x)) +
  geom_line(aes(y = cate, colour = "CATE(x)"), linewidth = 1.2) +
  geom_line(aes(y = true_tau, colour = "True  (x)"),
            linetype = "dotted", linewidth = 1) +
  geom_hline(aes(yintercept = ate_true,  colour = "ATE"),  linetype = "dashed") +
  geom_hline(aes(yintercept = att_true,  colour = "ATT"),  linetype = "dashed") +
  geom_hline(aes(yintercept = late_true, colour = "LATE"), linetype = "dashed") +
  scale_colour_manual(values = c(
    "CATE(x)" = "#c0392b", "True  (x)" = "#e74c3c",
    "ATE" = "#2c3e50", "ATT" = "#2980b9", "LATE" = "#27ae60"
  )) +
  labs(x = "X (covariate)", y = "Effect", colour = NULL,
       title = "CATE(x) vs scalar estimands") +
  theme_minimal() +
  theme(legend.position = "bottom")

p2 <- ggplot(df_qte, aes(x = q, y = qte)) +
  geom_line(colour = "#8e44ad", linewidth = 1.2) +
  geom_hline(yintercept = ate_true, linetype = "dashed", colour = "#2c3e50") +
  annotate("text", x = 0.07, y = ate_true + 0.06, label = "ATE",
```

```
          colour = "#2c3e50", size = 3) +
  labs(x = "Quantile q", y = "QTE(q)",
       title = "QTE - distribution-level effect") +
  theme_minimal()

grid.arrange(p1, p2, ncol = 2)
```

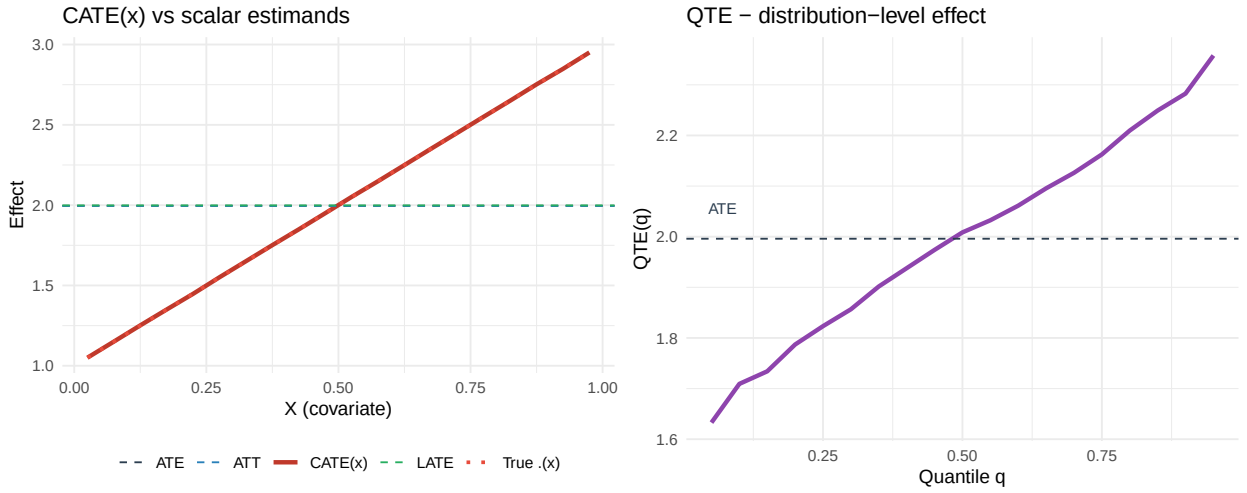


Figure 2.1.: Five views of the same DGP. The horizontal lines for ATE/ATT/LATE collapse the effect to a single scalar. CATE(x) shows how the effect varies with X. QTE(q) shows how the effect varies across the outcome distribution. Each curve answers a different question.

The left panel shows why the scalar estimands can differ. ATE averages $\tau(x)$ over the whole population. ATT averages it over the treated. LATE averages it over compliers. CATE does not collapse the curve.

QTE, in the right panel, is different again. It describes how the treatment changes the outcome distribution. It is not a conditional treatment effect as a function of X.

2.4. When the estimands differ

The three scalar estimands coincided because selection had nothing to do with the effect modifier. We now break that, and see how far apart they move.

The second DGP is the first one with a single change: X enters the treatment probability, $P(D = 1 \mid Z = 0, U, X) = 0.10 + 0.20U + 0.6X$, clipped to $[0, 1]$, with the offer adding 0.50 as before. Everything else is unchanged — a fresh sample of $n = 20{,}000$, $X \sim U(0, 1)$, $U \sim N(0, 1)$, $Z \sim$ Bernoulli(0.5), one shared uniform draw for the two potential treatments, $Y(0) = 0.5X + 0.3U + \varepsilon$ and $\tau(x) = 1 + 2x$.

```r
set.seed(7)
n2 <- 20000
X2 <- runif(n2, 0, 1)
U2 <- rnorm(n2)
Z2 <- rbinom(n2, 1, 0.5)

# High X → much more likely to take treatment (selection on X, the modifier)
pD0_2 <- pmin(pmax(0.10 + 0.20 * U2 + 0.6 * X2, 0), 1)
pD1_2 <- pmin(pmax(pD0_2 + 0.50,                 0), 1)
V2    <- runif(n2)                  # shared draw → D1_2 >= D0_2, no defiers
D0_2  <- as.integer(V2 < pD0_2)
D1_2  <- as.integer(V2 < pD1_2)
D2    <- ifelse(Z2 == 1, D1_2, D0_2)
Y0_2  <- 0.5 * X2 + 0.3 * U2 + rnorm(n2)
Y1_2  <- Y0_2 + tau_fn(X2)
Y2    <- ifelse(D2 == 1, Y1_2, Y0_2)

ate2  <- mean(Y1_2 - Y0_2)
att2  <- mean(Y1_2[D2 == 1] - Y0_2[D2 == 1])
compliers2 <- (D1_2 == 1) & (D0_2 == 0)
late2 <- mean(Y1_2[compliers2] - Y0_2[compliers2])

cat("Selection-on-X DGP:\n")
```

```
Selection-on-X DGP:
```

```r
cat(sprintf("  ATE  = %.3f\n", ate2))
```

```
  ATE  = 2.001
```

```r
cat(sprintf("  ATT  = %.3f  (now higher: treated have higher X → larger  )\n", att2))
```

```
  ATT  = 2.124  (now higher: treated have higher X → larger  )
```

```r
cat(sprintf("  LATE = %.3f  (compliers' mean X drives this)\n", late2))
```

```
  LATE = 1.926  (compliers' mean X drives this)
```

ATE is still 2.001, because the distribution of X has not moved. ATT rises to 2.124 and LATE falls to 1.926. Since $\tau = 1 + 2X$, every subgroup average is $1 + 2\,\mathbb{E}[X \mid \text{subgroup}]$, so the numbers report the subgroups directly: the treated have mean X of 0.562 and the compliers have mean X of 0.463, against 0.5 in the population.

The treated are high-X because high X raises the treatment probability. The compliers are low-X for a less obvious reason. The offer adds 0.50 to a probability capped at 1, so a unit with p_{D0} above

0.5 has a complier window of width $1 - p_{D0}$, narrower than 0.5. High-X units are the ones the cap bites on: mean window width falls from 0.49 in the bottom quartile of X to 0.34 in the top, and the complier rate falls with it, from 0.48 to 0.34. So the instrument moves mostly low-X units, and LATE lands below ATE.

Here the choice between ATE, ATT and LATE changes the substantive answer.

2.5. Which estimand should you choose?

The estimand should come from the research question:

Policy question	Right estimand
"What if we treated everyone?"	ATE
"What did treating the currently-treated achieve?"	ATT
"What can the instrument tell us?"	LATE
"Who benefits most?"	CATE(x)
"Does the effect vary across the outcome distribution?"	QTE(q)
"What is the distribution of individual effects?"	Often unidentifiable; bounds required

It is fine for one paper to report more than one estimand, as long as each is named correctly. What is not fine is to report the coefficient that is easiest to estimate and call it "the causal effect."

2.6. Summary

- ATE, ATT and LATE are different averages of the individual treatment effect.
- CATE(x) keeps heterogeneity by covariates. QTE(q) describes changes in the outcome distribution.
- With heterogeneous effects, IV estimates LATE. It should not be described as ATE unless extra assumptions justify that interpretation. This is the clean case with no covariates. Once we add covariates linearly to 2SLS, even LATE needs a *rich covariates* condition that is rarely defended. See Blandhol et al. (2025) and the IV chapter section "When 2SLS with Covariates Is *Actually* LATE".
- Estimator choice comes after the estimand is fixed.

3. DAGs, ADMGs, Identification, and Estimation

The potential-outcomes chapter asked when an observed data set can tell us about $E[Y(1) - Y(0)]$. Graphs give one way to state the assumptions behind the answer. A graph records which variables are allowed to cause other variables, and whether some observed variables may share unobserved common causes. Once the graph is stated, identification is a graph problem. Estimation comes after that: estimate the functional that the graph identified.

I use `dagitty` for graphs and backdoor adjustment, `causaleffect` for the general Pearl-Shpitser ID algorithm, and small direct estimators for the examples. The goal is to keep the order clear: graph first, identified functional second, estimator third.

3.1. DAGs and ADMGs

A directed acyclic graph, or DAG, has directed arrows such as `X -> A`. In causal work, an arrow means a direct causal relation is allowed by the model. Absence of an arrow is also a substantive assumption.

An acyclic directed mixed graph, or ADMG, adds bidirected arrows such as `A <-> Y`. A bidirected edge represents an unobserved common cause. This is useful because many econometric applications have variables we cannot measure: ability, preferences, latent health, firm quality, and so on.

The graph is not an estimator. It is a compact statement of assumptions. The workflow is:

1. write down a DAG or ADMG;
2. ask whether the target effect is identified;
3. estimate the identified functional from data.

3.2. A DAG: Backdoor Adjustment

Start with a simple observational study. `A` is treatment, `Y` is the outcome, and `X` is an observed confounder:

```
g_dag <- dagitty("dag {
  X -> A
  X -> Y
  A -> Y
}")
coordinates(g_dag) <- list(
  x = c(X = 1, A = 0, Y = 2),
```

```
  y = c(X = 1, A = 0, Y = 0)
)
```

```
ggdag(g_dag) + theme_dag_blank()
```

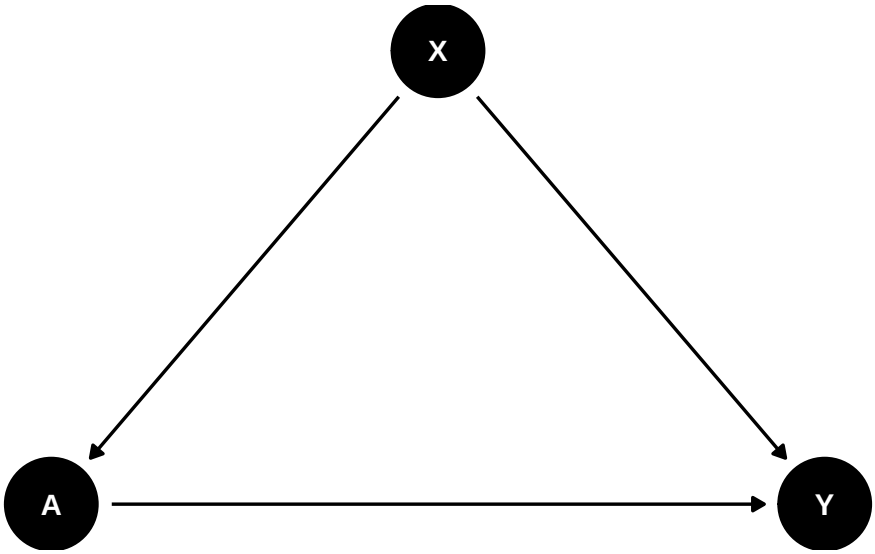


Figure 3.1.: Backdoor DAG: X is an observed confounder of A → Y.

The graph says `X` affects both treatment and outcome. It also says there is no unobserved common cause between `A` and `Y` after we condition on `X`. In this case the treatment effect is identifiable by **backdoor adjustment**.

```
adj_sets <- adjustmentSets(g_dag, exposure = "A", outcome = "Y")
adj_sets
```

```
{ X }
```

The adjustment set is $\{X\}$. Under this graph,

$$E[Y(a)] = E_X\{E(Y \mid A = a, X)\}. \tag{3.1}$$

So the graph has translated a causal query into an observed-data functional.

3.3. Estimating the DAG Effect with AIPW

We simulate data that match this graph, so we can check whether an estimator built from the identified functional recovers an effect we already know.

We draw $n = 400$ observations. The confounder is $X \sim N(0,1)$. Treatment is $A \sim \text{Bernoulli}(\Lambda(-0.2 + 0.8X))$, with Λ the logistic function, so a higher X makes treatment more likely. The outcome is $Y = 1 + 1.5A + 0.7X + 0.3\varepsilon$ with $\varepsilon \sim N(0,1)$. The effect is constant across units, so the ACE is 1.5.

```
set.seed(2026)

logistic <- function(x) 1 / (1 + exp(-x))

n  <- 400
X  <- rnorm(n)
pA <- logistic(-0.2 + 0.8 * X)
A  <- as.numeric(runif(n) < pA)
Y  <- 1 + 1.5 * A + 0.7 * X + 0.3 * rnorm(n)

df_dag <- data.frame(X = X, A = A, Y = Y)
head(df_dag)
```

```
            X A          Y
1  0.52058907 0  1.6530538
2 -1.07969076 0  0.1907760
3  0.13923812 0  1.0984536
4 -0.08474878 0  1.0941604
5 -0.66663962 1  1.8965341
6 -2.51608903 0 -0.8245336
```

The analyst sees only X, A and Y. Because X raises both the chance of treatment and the outcome, a raw contrast of Y by A overstates the effect.

Because the graph is backdoor-identified, use AIPW with the adjustment set the graph chose. The function below implements the AIPW point estimate and an EIF-based standard error.

```
aipw_backdoor <- function(data, treatment, outcome, adjust) {
  fmla_y <- as.formula(paste0(outcome, " ~ ", treatment, " * (",
                              paste(adjust, collapse = " + "), ")"))
  fmla_a <- as.formula(paste0(treatment, " ~ ",
                              paste(adjust, collapse = " + ")))

  mu_fit <- lm(fmla_y, data = data)
  pi_fit <- glm(fmla_a, data = data, family = binomial())

  d1 <- data; d1[[treatment]] <- 1
  d0 <- data; d0[[treatment]] <- 0
  mu1 <- predict(mu_fit, newdata = d1)
  mu0 <- predict(mu_fit, newdata = d0)
  pi1 <- predict(pi_fit, newdata = data, type = "response")
```

```
  w  <- data[[treatment]]; y <- data[[outcome]]
  if1 <- w     * (y - mu1) / pi1       + mu1
  if0 <- (1-w) * (y - mu0) / (1 - pi1) + mu0
  psi  <- if1 - if0

  est  <- mean(psi)
  se   <- sd(psi) / sqrt(length(psi))
  ci   <- est + c(-1, 1) * qnorm(0.975) * se
  data.frame(estimand = "Backdoor ACE",
             estimate = round(est, 4),
             lower_95 = round(ci[1], 4),
             upper_95 = round(ci[2], 4),
             se       = round(se, 4))
}

aipw_backdoor(df_dag, "A", "Y", adjust = "X") |> kable()
```

estimand	estimate	lower_95	upper_95	se
Backdoor ACE	1.4886	1.427	1.5502	0.0314

AIPW returns 1.4886 with an influence-function standard error of 0.0314, and the 95% interval $[1.427, 1.550]$ covers the true ACE of 1.5. The graph chose the adjustment set and the estimator used that set.

> **Note**
>
> The standard error here, `sd(psi) / sqrt(n)`, is the influence-function standard error of the AIPW estimator. It is asymptotically valid when the nuisance functions (μ and π) are regular parametric fits, as they are here. With flexible machine-learning nuisances it is **not** valid as written: you then need cross-fitting (sample splitting) so the nuisance estimates are independent of the fold being evaluated, and the variance should still be computed from the efficient influence function (or by bootstrap). Treating a full-sample ML AIPW estimate with this plug-in SE will generally understate uncertainty.

3.4. An ADMG: Front-Door Identification

Now suppose treatment and outcome share an unobserved common cause. A simple adjustment argument no longer works. The front-door ADMG has a mediator `M` that transmits the effect of `A` to `Y`, while `A` and `Y` are hidden-confounded:

```
g_fd <- dagitty("dag {
  A -> M
  M -> Y
  A <-> Y
```

```
}")
coordinates(g_fd) <- list(
  x = c(A = 0, M = 1, Y = 2),
  y = c(A = 0, M = 0, Y = 0)
)
```

```
ggdag(g_fd) + theme_dag_blank()
```

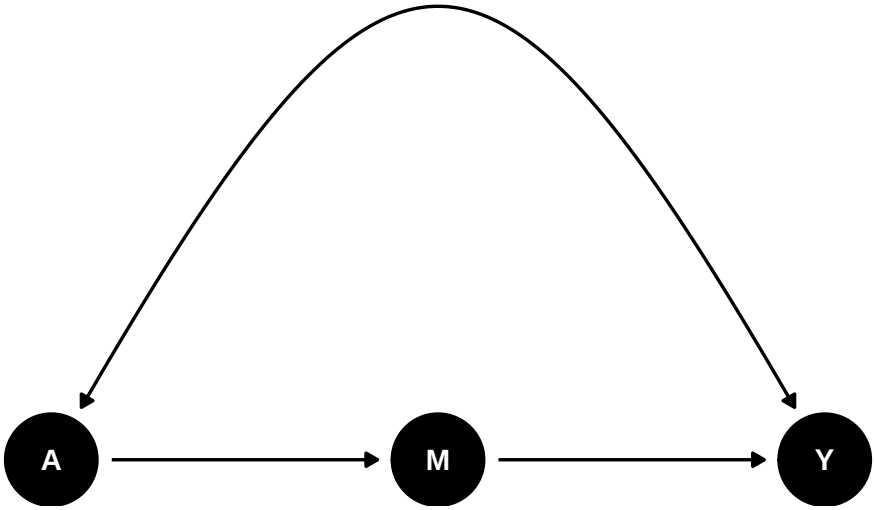


Figure 3.2.: Front-door ADMG: A and Y share an unobserved common cause (curved bidirected edge), but the effect propagates through the mediator M.

The bidirected edge `A <-> Y` means that ordinary backdoor adjustment is not available. Indeed `dagitty` reports no adjustment set:

```
sets_fd <- adjustmentSets(g_fd, exposure = "A", outcome = "Y")
# An empty dagitty adjustment-set list prints nothing at all, which is easy to
# mistake for a chunk that failed to run. Report the count explicitly.
cat("Valid backdoor adjustment sets:", length(sets_fd), "\n")
```

```
Valid backdoor adjustment sets: 0
```

There are none. No set of observed variables blocks the confounding, because the confounder is not observed.

But the effect is still identified by the front-door criterion (Pearl, 1995). I call the Pearl-Shpitser ID algorithm in `causaleffect` to obtain the symbolic expression. The function wants the graph as an `igraph` object with bidirected edges encoded as two directed edges marked `description = "U"`:

```
g_fd_ig <- graph_from_literal(A -+ M, M -+ Y, A -+ Y, Y -+ A)
# Mark the A<->Y bidirected edge (encoded as the two directed edges A->Y and
# Y->A) by endpoint name rather than by hardcoded edge index -- igraph's
# internal edge ordering after graph_from_literal() is not part of its
# documented contract, so indices like c(2, 4) can silently point at the
# wrong edges if igraph's internals, the vertex/edge creation order, or the
# package version ever changes.
edge_ends <- ends(g_fd_ig, E(g_fd_ig))
confounded <- (edge_ends[, 1] == "A" & edge_ends[, 2] == "Y") |
              (edge_ends[, 1] == "Y" & edge_ends[, 2] == "A")
E(g_fd_ig)$description[confounded] <- "U"
expr_fd <- causal.effect(y = "Y", x = "A", G = g_fd_ig, simp = TRUE)
cat(expr_fd, "\n")
```

```
\sum_{M}P(M|A)\left(\sum_{A}P(Y|A,M)P(A)\right)
```

The expression $\sum_M P(M \mid A) \sum_{A'} P(Y \mid A', M) P(A')$ is precisely the front-door formula.

3.5. Estimating the ADMG Effect with the Front-Door Formula

We simulate a front-door setting so we can compare the front-door estimate against a truth we derive below.

We draw $n = 400$ observations. The latent confounder is $U \sim N(0, 1)$. Treatment is $A \sim \text{Bernoulli}(\Lambda(-0.1 + 0.8U))$, the mediator is $M \sim \text{Bernoulli}(\Lambda(-0.4 + 1.2A))$, and the outcome is $Y \sim \text{Bernoulli}(\Lambda(-1.0 + 0.9M + 0.7U))$. All three observed variables are binary. U is not in the observed data; it is what the bidirected edge `A <-> Y` stands for. Two features make the front-door criterion apply: U enters A and Y but not M, and A reaches Y only through M.

```
set.seed(2027)

n     <- 400
U     <- rnorm(n)
A_fd  <- as.numeric(runif(n) < logistic(-0.1 + 0.8 * U))
M     <- as.numeric(runif(n) < logistic(-0.4 + 1.2 * A_fd))
Y_fd  <- as.numeric(runif(n) < logistic(-1.0 + 0.9 * M + 0.7 * U))

df_fd <- data.frame(A = A_fd, M = M, Y = Y_fd)
head(df_fd)
```

```
  A M Y
1 0 0 0
2 0 1 1
3 1 1 1
4 1 1 1
```

```
5 0 1 1
6 1 0 0
```

The front-door point estimator is a direct evaluation of the identification expression:

$$E[Y(a)] = \sum_m P(M = m \mid A = a) \sum_{a'} P(Y = 1 \mid A = a', M = m) P(A = a'). \tag{3.2}$$

The DGP has a known truth. Under $do(A = a)$ the mediator is $M \sim \text{Bern}(\Lambda(-0.4 + 1.2a))$ while U keeps its marginal $N(0, 1)$ distribution, so

$$\tau = \big[\Lambda(0.8) - \Lambda(-0.4)\big]\,\big[g(-0.1) - g(-1.0)\big], \qquad g(c) = E_U\big[\Lambda(c + 0.7U)\big], \tag{3.3}$$

which evaluates to $0.2887 \times 0.1894 = 0.0547$. The naive contrast $E[Y \mid A = 1] - E[Y \mid A = 0]$ equals 0.158 in the population this DGP defines — almost three times the causal effect, because U raises both the chance of treatment and the outcome. That gap is what the front-door step removes. The table below reports the sample version of the naive contrast alongside the front-door estimate.

For binary A, M, Y we plug sample proportions into that formula, and take the standard error from 500 bootstrap resamples:

```r
front_door_ace <- function(data) {
  pA <- mean(data$A)
  p_marginal_A <- c("0" = 1 - pA, "1" = pA)

  # P(M | A): rows indexed by A, columns by M
  pM_given_A <- prop.table(table(data$A, data$M), margin = 1)
  # P(Y=1 | A, M)
  pY_given_AM <- with(data, tapply(Y, list(A, M), mean))

  # E[Y(a)] for a in {0, 1}
  ey_do <- function(a) {
    m_levels <- as.numeric(colnames(pM_given_A))
    sum(sapply(m_levels, function(m) {
      mar <- pM_given_A[as.character(a), as.character(m)]
      inner <- sum(sapply(c(0, 1), function(ap)
        pY_given_AM[as.character(ap), as.character(m)] *
          p_marginal_A[as.character(ap)]))
      mar * inner
    }))
  }
  ey_do(1) - ey_do(0)
}

# Bootstrap for SE
set.seed(99)
B   <- 500
```

```r
boot_ace <- replicate(B, {
  idx <- sample.int(nrow(df_fd), replace = TRUE)
  front_door_ace(df_fd[idx, ])
})

est_fd <- front_door_ace(df_fd)
se_fd  <- sd(boot_ace)
ci_fd  <- est_fd + c(-1, 1) * qnorm(0.975) * se_fd

naive_fd <- mean(df_fd$Y[df_fd$A == 1]) - mean(df_fd$Y[df_fd$A == 0])

data.frame(estimand = c("Naive E[Y|A=1] - E[Y|A=0]", "Front-door ACE", "Truth"),
           estimate = round(c(naive_fd, est_fd, 0.0547), 4),
           lower_95 = c(NA, round(ci_fd[1], 4), NA),
           upper_95 = c(NA, round(ci_fd[2], 4), NA),
           se       = c(NA, round(se_fd, 4), NA)) |> kable()
```

estimand	estimate	lower_95	upper_95	se
Naive E[Y|A=1] - E[Y|A=0]	0.1395	NA	NA	NA
Front-door ACE	0.0775	0.0403	0.1146	0.0189
Truth	0.0547	NA	NA	NA

The naive contrast is 0.1395, more than twice the true 0.0547. The front-door estimate is 0.0775 with a bootstrap standard error of 0.0189, and its 95% interval $[0.040, 0.115]$ covers the truth. The confounding bias is removed; what is left is sampling noise, and the interval is wide relative to an effect of 0.05 because the formula estimates seven proportions from 400 binary observations.

This is the practical advantage of separating identification from estimation. State the graph first. Then use the estimator that matches the identified functional.

3.6. General ID Algorithm

The named routes above are useful because they are easy to estimate: backdoor adjustment and the front-door formula. ADMGs can be more general. The Pearl-Shpitser ID algorithm asks the broader question:

```
Can P(Y | do(A)) be written using only the observed joint law P(V)?
```

The algorithm works recursively:

1. remove variables that are not ancestors of the outcome;
2. add interventions that are irrelevant after graph surgery;
3. split the remaining graph into bidirected districts;
4. identify each district as a factor of the observed law, if possible;

5. return a hedge witness when no such reduction is possible.

For the front-door graph, the general ID algorithm succeeds and returns the same front-door formula we obtained above. For more complex ADMGs it may yield novel functionals that no named estimator covers — this is where the plug-in approach is essential.

3.7. When the Graph Says No

The **bow graph** has both a direct causal edge and unobserved confounding between the same two variables:

```
g_bow <- dagitty("dag {
  A -> Y
  A <-> Y
}")
coordinates(g_bow) <- list(
  x = c(A = 0, Y = 1),
  y = c(A = 0, Y = 0)
)
```

```
ggdag(g_bow) + theme_dag_blank()
```

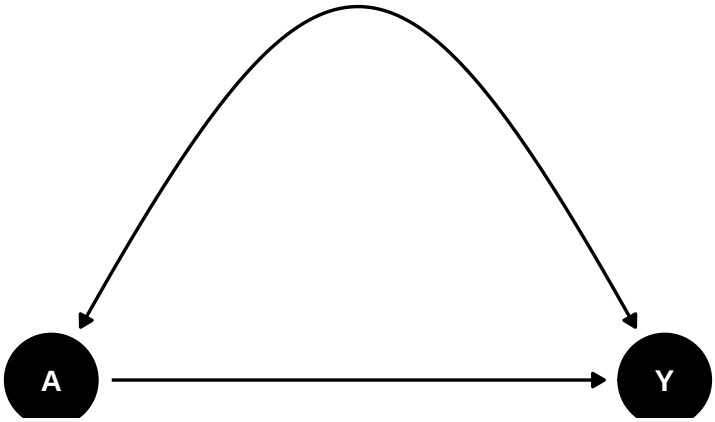


Figure 3.3.: Bow ADMG: a direct edge A → Y plus a bidirected A Y. The effect is not identified.

The effect of `A` on `Y` is not identified in this ADMG. The observed association mixes the direct causal effect with the latent common cause.

```
g_bow_ig <- graph_from_literal(A -+ Y, Y -+ A)
g_bow_ig <- set_edge_attr(g_bow_ig, "description",
                          index = c(1, 2), value = "U")
g_bow_ig <- add_edges(g_bow_ig, c("A", "Y"))   # add the direct A -> Y
```

```
result <- tryCatch(
  causal.effect(y = "Y", x = "A", G = g_bow_ig, simp = TRUE),
  error = function(e) paste("Not identifiable:", conditionMessage(e))
)
result
```

```
[1] "Not identifiable: Not identifiable."
```

`causaleffect` raises an error message indicating the effect is not identifiable. This is a useful failure mode. If the graph does not justify the causal effect, the estimator should not manufacture one.

3.8. Practical Workflow

For applied work, a useful workflow is:

1. write the graph before looking at estimates;
2. use bidirected edges for latent common causes;
3. ask `dagitty::adjustmentSets(graph, exposure, outcome)` — if non-empty, you have a backdoor route;
4. if no adjustment set exists, call `causaleffect::causal.effect(y, x, G)` — if it returns an expression, the effect is identified by a more general route; if it errors, the effect is not identified in this ADMG;
5. estimate using AIPW for backdoor effects, the front-door formula when the algorithm returns that pattern, or a plug-in evaluation of the symbolic expression for general ID functionals;
6. report the graph and the identification route along with the estimate.

4. Applied Graph Workflow: Smoking Cessation

The previous chapter used small graph examples. Here I apply the same graph-identification-estimation workflow to a real data set: the complete-case National Health and Nutrition Examination Survey I Epidemiologic Follow-up Study (NHEFS) data distributed through the `causaldata` package.

The question is:

```
What is the effect of quitting smoking on weight change from 1971 to 1982?
```

The treatment is `qsmk`, an indicator for quitting smoking between baseline and follow-up. The outcome is `wt82_71`, weight change over the follow-up period. This is observational data. The effect is not identified by the data alone. It is identified only after we state the adjustment assumptions in the graph.

4.1. Data

We keep the treatment `qsmk`, the outcome `wt82_71` (weight change in kg), and the nine baseline covariates commonly used in NHEFS smoking-cessation examples: `sex`, `race`, `age`, `school` (years of schooling), `smokeintensity` (cigarettes per day in 1971), `smokeyrs` (years of smoking), `exercise` and `active` (ordinal activity codes), and `wt71` (baseline weight in kg). All are measured at baseline, before quitting.

```
nhefs_cols <- c(
  "qsmk", "wt82_71",
  "sex", "race", "age", "school",
  "smokeintensity", "smokeyrs",
  "exercise", "active", "wt71"
)
nhefs <- causaldata::nhefs_complete |>
  dplyr::select(all_of(nhefs_cols)) |>
  dplyr::mutate(across(everything(), as.numeric))

c(n = nrow(nhefs), variables = ncol(nhefs))
```

```
        n variables
     1566        11
```

The complete-case sample has 1,566 people and these 11 columns. The first five rows:

```
head(nhefs, 5) |>
  dplyr::mutate(across(everything(), \(x) signif(x, 4))) |>
  kable()
```

qsmk	wt82_71	sex	race	age	school	smokeintensity	smokeyrs	exercise	active	wt71
0	-10.090	1	2	42	7	30	29	3	1	79.04
0	2.605	1	1	36	9	20	24	1	1	58.63
0	9.414	2	2	56	11	20	26	3	1	56.81
0	4.990	1	2	68	5	3	53	3	2	59.42
0	4.989	1	1	40	11	20	19	2	2	87.09

Weight change is already in kilograms and can be negative, so the estimand is in kilograms too.

4.2. Adjustment DAG

Start with the grouped graph

```
X -> A -> Y
X -----> Y
```

where `X` is the baseline covariate set, `A` is quitting smoking, and `Y` is later weight change. This graph says the baseline variables may affect both quitting and later weight change, and that there is no unmeasured common cause of quitting and later weight change after conditioning on those baseline variables.

```
conceptual_graph <- dagitty("dag {
  X -> A
  X -> Y
  A -> Y
}")
coordinates(conceptual_graph) <- list(
  x = c(X = 1, A = 0, Y = 2),
  y = c(X = 1, A = 0, Y = 0)
)
```

```
ggdag(conceptual_graph) + theme_dag_blank()
```

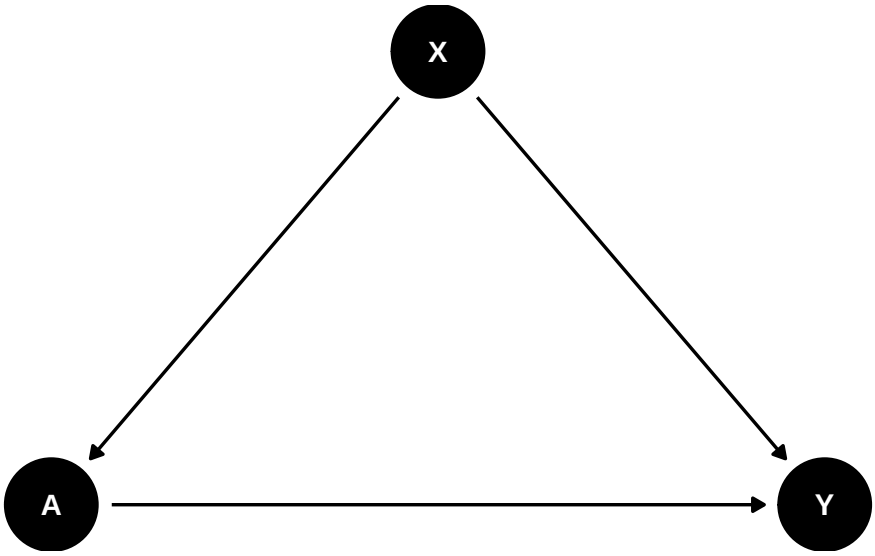


Figure 4.1.: Conceptual adjustment DAG: baseline X confounds A → Y.

For estimation, expand `X` into the observed NHEFS columns. The expanded graph is still an adjustment DAG: each baseline variable is allowed to point into quitting and into later weight change. We build it and count what it contains.

```
baseline_vars <- setdiff(nhefs_cols, c("qsmk", "wt82_71"))

build_grouped_graph <- function(baseline) {
  edges_X_to_A <- paste0(baseline, " -> qsmk")
  edges_X_to_Y <- paste0(baseline, " -> wt82_71")
  body <- paste(c(edges_X_to_A, edges_X_to_Y, "qsmk -> wt82_71"),
                collapse = "\n  ")
  dagitty(paste0("dag {\n  ", body, "\n}"))
}

nhefs_graph <- build_grouped_graph(baseline_vars)
c(vertices = length(names(nhefs_graph)),
  directed_edges = length(edges(nhefs_graph)$v))
```

```
      vertices directed_edges
            11             19
```

Eleven vertices and 19 directed edges: two out of each of the nine baseline variables, plus `qsmk -> wt82_71`.

4.3. Identification

`dagitty::adjustmentSets` checks the graph before estimation. In this adjustment DAG, the minimal sufficient adjustment set is the full baseline:

```
adj_sets <- adjustmentSets(nhefs_graph, exposure = "qsmk",
                           outcome = "wt82_71", type = "minimal")
adj_sets
```

```
{ active, age, exercise, race, school, sex, smokeintensity, smokeyrs,
  wt71 }
```

The graph therefore identifies the causal mean by the adjustment formula

$$E[Y(a)] = E_X\{E(Y \mid A = a, X)\}. \tag{4.1}$$

This formula is not a modeling claim by itself. It is the observed-data functional implied by the graph. Estimation still requires nuisance models for the outcome regression and treatment mechanism.

4.4. Estimation with AIPW

The book keeps identification and estimation as separate steps. The graph supplies the adjustment set; the estimator uses that set.

```
aipw_backdoor <- function(data, treatment, outcome, adjust) {
  fmla_y <- as.formula(paste0(outcome, " ~ ", treatment, " * (",
                              paste(adjust, collapse = " + "), ")"))
  fmla_a <- as.formula(paste0(treatment, " ~ ",
                              paste(adjust, collapse = " + ")))

  mu_fit <- lm(fmla_y, data = data)
  pi_fit <- glm(fmla_a, data = data, family = binomial())

  d1 <- data; d1[[treatment]] <- 1
  d0 <- data; d0[[treatment]] <- 0
  mu1 <- predict(mu_fit, newdata = d1)
  mu0 <- predict(mu_fit, newdata = d0)
  pi1 <- predict(pi_fit, newdata = data, type = "response")
  # Truncate (winsorize) the propensity score away from 0/1. AIPW divides by
  # pi1 and (1-pi1), so an estimated propensity near the boundary produces
  # huge inverse-probability weights and blows up the estimator's variance.
  # Bounding at [0.01, 0.99] trades a small amount of bias (we no longer use
  # the true estimated propensity for the most extreme units) for much
  # lower variance -- a standard and usually favorable trade-off, but worth
```

```
  # flagging explicitly since it is a bias-variance choice, not a free lunch.
  pi1 <- pmin(pmax(pi1, 0.01), 0.99)

  w  <- data[[treatment]]; y <- data[[outcome]]
  if1 <- w     * (y - mu1) / pi1       + mu1
  if0 <- (1-w) * (y - mu0) / (1 - pi1) + mu0
  psi  <- if1 - if0

  est <- mean(psi); se <- sd(psi) / sqrt(length(psi))
  ci  <- est + c(-1, 1) * qnorm(0.975) * se
  data.frame(estimand = "Quit smoking vs continued smoking",
             estimate = signif(est, 4),
             lower_95 = signif(ci[1], 4),
             upper_95 = signif(ci[2], 4),
             se       = signif(se, 4))
}

aipw_backdoor(nhefs, "qsmk", "wt82_71", adjust = baseline_vars) |> kable()
```

estimand	estimate	lower_95	upper_95	se
Quit smoking vs continued smoking	3.293	2.317	4.269	0.4981

Quitters gain 3.29 kg more than continuing smokers, with a standard error of 0.50 and a 95% interval of $[2.32, 4.27]$. Under the DAG this is the average effect of quitting versus continuing on weight change in the complete-case NHEFS population. The causal interpretation depends on the graph, especially the assumption that the measured baseline variables block the relevant backdoor paths.

The `sd(psi) / sqrt(n)` standard error is the AIPW influence-function SE and is valid here because the nuisances are parametric logistic/linear fits. If you replace them with flexible machine learners, use cross-fitting and an influence-function (or bootstrap) variance instead; the plug-in SE on full-sample ML fits is not generally valid.

4.5. Sensitivity Graph

Now encode a different assumption: quitting and later weight change share an unobserved common cause even after conditioning on the measured baseline variables. In an ADMG this is a bidirected edge, `qsmk <-> wt82_71`. We rebuild the same graph with that one edge added and count the sufficient adjustment sets it admits.

```
build_sensitivity_graph <- function(baseline) {
  edges_X_to_A <- paste0(baseline, " -> qsmk")
  edges_X_to_Y <- paste0(baseline, " -> wt82_71")
  body <- paste(c(edges_X_to_A, edges_X_to_Y,
```

```
                  "qsmk -> wt82_71", "qsmk <-> wt82_71"),
                collapse = "\n  ")
  dagitty(paste0("dag {\n  ", body, "\n}"))
}

sensitivity_graph <- build_sensitivity_graph(baseline_vars)
sens_adj <- adjustmentSets(sensitivity_graph, exposure = "qsmk",
                           outcome = "wt82_71", type = "minimal")
cat("Number of sufficient adjustment sets:", length(sens_adj), "\n")
```

```
Number of sufficient adjustment sets: 0
```

The count is zero. One edge changes the identifying conclusion: under this graph the observed NHEFS variables are not enough to identify the effect, and no adjustment set exists. The data are the same as in the previous section, and the 3.29 kg is still computable, but nothing licenses reading it as a causal effect. This is why the graph has to come before the estimator.

4.6. Structured Baseline Graph

The simple adjustment graph is often the clearest starting point. A more detailed DAG can record baseline ordering: demographics may precede education, smoking history, activity, and baseline weight; those variables may then affect quitting and later weight change. We write that graph out, ask for its minimal adjustment set, and check whether the set equals the full baseline we used before.

```
structured_graph <- dagitty("dag {
  age -> school
  age -> smokeyrs
  age -> wt71
  age -> active
  age -> exercise
  age -> qsmk
  age -> wt82_71

  sex -> smokeintensity
  sex -> smokeyrs
  sex -> wt71
  sex -> qsmk
  sex -> wt82_71

  race -> school
  race -> smokeintensity
  race -> qsmk
  race -> wt82_71
```

```
  school -> smokeintensity
  school -> qsmk
  school -> wt82_71

  smokeyrs -> smokeintensity
  smokeyrs -> qsmk
  smokeyrs -> wt82_71

  smokeintensity -> qsmk
  smokeintensity -> wt82_71

  exercise -> wt71
  exercise -> qsmk
  exercise -> wt82_71

  active -> wt71
  active -> qsmk
  active -> wt82_71

  wt71 -> qsmk
  wt71 -> wt82_71

  qsmk -> wt82_71
}")

struct_adj <- adjustmentSets(structured_graph, exposure = "qsmk",
                             outcome = "wt82_71", type = "minimal")
struct_adj
```

```
{ active, age, exercise, race, school, sex, smokeintensity, smokeyrs,
  wt71 }
```

```
data.frame(
  matches_full_baseline = setequal(unlist(struct_adj[[1]]), baseline_vars)
) |> kable()
```

matches_full_baseline
TRUE

The check returns `TRUE`. For this treatment effect the detailed baseline DAG is adjustment-equivalent to the simpler expanded DAG: same sufficient adjustment set, same identifying functional, and therefore the same 3.29 kg estimate. The extra arrows among baseline variables may be useful for documentation, but they are also extra causal assumptions. If those assumptions are not needed for the estimand, the grouped adjustment graph is usually easier to defend.

5. Sensitivity Analysis

```
library(tidyverse)
library(sensemakr)
library(EValue)
library(causaldata)
library(broom)
```

The previous chapters often treat identification as yes or no. Applied work is usually less clean. We may believe the adjustment set is good, but still worry about some remaining confounding. Sensitivity analysis asks a practical question: how strong would unmeasured confounding have to be to change the conclusion?

> **Related reading**: An extended treatment of the Cinelli-Hazlett robustness value and benchmark interpretations appears in the companion blog chapter on sensitivity analysis of *Topics on Econometrics and Causal Inference*.

Here I use three methods:

1. **Cinelli-Hazlett (2020) omitted-variable bias bounds** — for OLS regression estimates, quantifies how strong an omitted variable would need to be (in terms of partial R^2 with treatment and outcome) to bring the estimate to zero.
2. **E-values (VanderWeele-Ding 2017)** — for risk ratios, the minimum strength of association (on the risk-ratio scale) that an unmeasured confounder would need with both treatment and outcome to fully explain the observed association.
3. **Rosenbaum bounds** — for matched estimators, the magnitude of non-random treatment assignment within matched pairs that would render the test result insignificant.

5.1. The NHEFS smoking-cessation example

We use the National Health and Nutrition Examination Survey I Epidemiologic Follow-up Study (NHEFS) data — the same dataset used in the applied DAG-workflow chapter. The question: what is the effect of quitting smoking (`qsmk`) on weight gain over 10 years (`wt82_71`)? We start from the raw comparison.

```
nhefs <- causaldata::nhefs_complete |>
  filter(!is.na(wt82_71))

cat(sprintf("n = %d\n", nrow(nhefs)))
```

```
n = 1566
```

```
cat(sprintf("Treated (quit smoking) = %d\n", sum(nhefs$qsmk)))
```

```
Treated (quit smoking) = 403
```

```
cat(sprintf("Mean weight gain (treated):   %.2f kg\n",
            mean(nhefs$wt82_71[nhefs$qsmk == 1])))
```

```
Mean weight gain (treated):   4.53 kg
```

```
cat(sprintf("Mean weight gain (untreated): %.2f kg\n",
            mean(nhefs$wt82_71[nhefs$qsmk == 0])))
```

```
Mean weight gain (untreated): 1.98 kg
```

Of the 1,566 complete cases, 403 quit. Quitters gained 4.53 kg on average and continuing smokers 1.98 kg, a naive difference of 2.55 kg. But quitters and continuers differ on many baseline characteristics. We adjust for the standard set by OLS — sex, age, race, education, smoking intensity, years smoked, exercise, activity and baseline weight — and print the first three coefficients along with the treatment estimate:

```
ols <- lm(wt82_71 ~ qsmk + sex + age + race + education + smokeintensity +
                    smokeyrs + exercise + active + wt71,
          data = nhefs)

tidy(ols)[1:3, ] |> knitr::kable(digits = 3)
```

term	estimate	std.error	statistic	p.value
(Intercept)	16.090	1.508	10.673	0.000
qsmk	3.381	0.441	7.665	0.000
sex1	-1.429	0.456	-3.131	0.002

```
cat(sprintf("\nAdjusted ACE estimate: %.3f kg  (SE = %.3f)\n",
            coef(ols)["qsmk"], summary(ols)$coefficients["qsmk", 2]))
```

```
Adjusted ACE estimate: 3.381 kg  (SE = 0.441)
```

The adjusted estimate is 3.381 kg with a standard error of 0.441, larger than the naive 2.55 kg. Adjustment moved the answer away from zero, not toward it. But the identification assumption requires that we have controlled for *all* relevant confounders. What if we missed one — say, a propensity to gain weight that also correlates with quitting decisions? Sensitivity analysis addresses this directly.

5.2. Cinelli-Hazlett OVB bounds

`sensemakr` implements the Cinelli-Hazlett (2020) approach. The key inputs are:

- a fitted regression
- the treatment variable name
- one or more *benchmark covariates* whose strength serves as a reference for how strong a hypothetical unobserved confounder might be

```
sens <- sensemakr(
  model = ols,
  treatment = "qsmk",
  benchmark_covariates = c("smokeintensity", "smokeyrs"),
  kd = 1:3,                 # benchmark: 1×, 2×, 3× as strong as the benchmark
  ky = 1:3,
  q  = 1,                   # the "bias proportion" at which to flag a problem
  alpha = 0.05,
  reduce = TRUE
)

summary(sens)
```

```
Sensitivity Analysis to Unobserved Confounding

Model Formula: wt82_71 ~ qsmk + sex + age + race + education + smokeintensity +
    smokeyrs + exercise + active + wt71

Null hypothesis: q = 1 and reduce = TRUE
-- This means we are considering biases that reduce the absolute value of the current estimate
-- The null hypothesis deemed problematic is H0:tau = 0

Unadjusted Estimates of 'qsmk':
  Coef. estimate: 3.3812
  Standard Error: 0.4411
  t-value (H0:tau = 0): 7.6649

Sensitivity Statistics:
  Partial R2 of treatment with outcome: 0.0365
  Robustness Value, q = 1: 0.1767
  Robustness Value, q = 1, alpha = 0.05: 0.1347

Verbal interpretation of sensitivity statistics:

-- Partial R2 of the treatment with the outcome: an extreme confounder (orthogonal to the covar

-- Robustness Value, q = 1: unobserved confounders (orthogonal to the covariates) that explain
```

```
-- Robustness Value, q = 1, alpha = 0.05: unobserved confounders (orthogonal to the covariates)

Bounds on omitted variable bias:

--The table below shows the maximum strength of unobserved confounders with association with t

       Bound Label R2dz.x R2yz.dx Treatment Adjusted Estimate Adjusted Se
 1x smokeintensity 0.0143  0.0010      qsmk            3.3157      0.4442
 2x smokeintensity 0.0287  0.0020      qsmk            3.2492      0.4473
 3x smokeintensity 0.0430  0.0029      qsmk            3.1817      0.4504
       1x smokeyrs 0.0056  0.0015      qsmk            3.3299      0.4422
       2x smokeyrs 0.0112  0.0031      qsmk            3.2783      0.4431
       3x smokeyrs 0.0168  0.0046      qsmk            3.2264      0.4440
 Adjusted T Adjusted Lower CI Adjusted Upper CI
     7.4636            2.4443            4.1871
     7.2641            2.3718            4.1266
     7.0640            2.2982            4.0652
     7.5308            2.4626            4.1972
     7.3989            2.4092            4.1474
     7.2667            2.3555            4.0973
```

The summary tells us:

- **Robustness value (RV)**: the minimum strength of association (in terms of partial R^2) that an unmeasured confounder would need with both the treatment and the outcome to reduce the estimate to zero. Higher = more robust.
- **RV_q**: the strength needed to "explain away" a fraction q of the estimate (here $q = 1$ means complete explanation).
- **Benchmark comparisons**: how the RV compares to multiples of the observed covariates.

For this regression the partial R^2 of quitting with weight change is 3.65%. The robustness value at $q = 1$ is 17.67%: a confounder orthogonal to the covariates would have to explain more than 17.67% of the residual variance of both quitting and weight change to bring the estimate to zero. To leave it merely statistically indistinguishable from zero at the 5% level, 13.47% would be enough.

The benchmark rows put those numbers on a scale. Smoking intensity explains 1.43% of the residual variance of quitting and 0.10% of the residual variance of weight change. A confounder three times as strong on both margins would move the estimate from 3.381 to 3.182, with a 95% interval of $[2.30, 4.07]$. None of the measured covariates comes near the 17.67% that would be needed, so the estimate is robust to a confounder resembling the ones we already control for.

The same information is easier to read as a contour plot. The horizontal axis is the confounder's partial R^2 with the treatment, the vertical axis its partial R^2 with the outcome, and the contours give the adjusted estimate at each combination. The benchmark dots mark 1x, 2x and 3x smoking intensity and years smoked.

```
plot(sens)
```

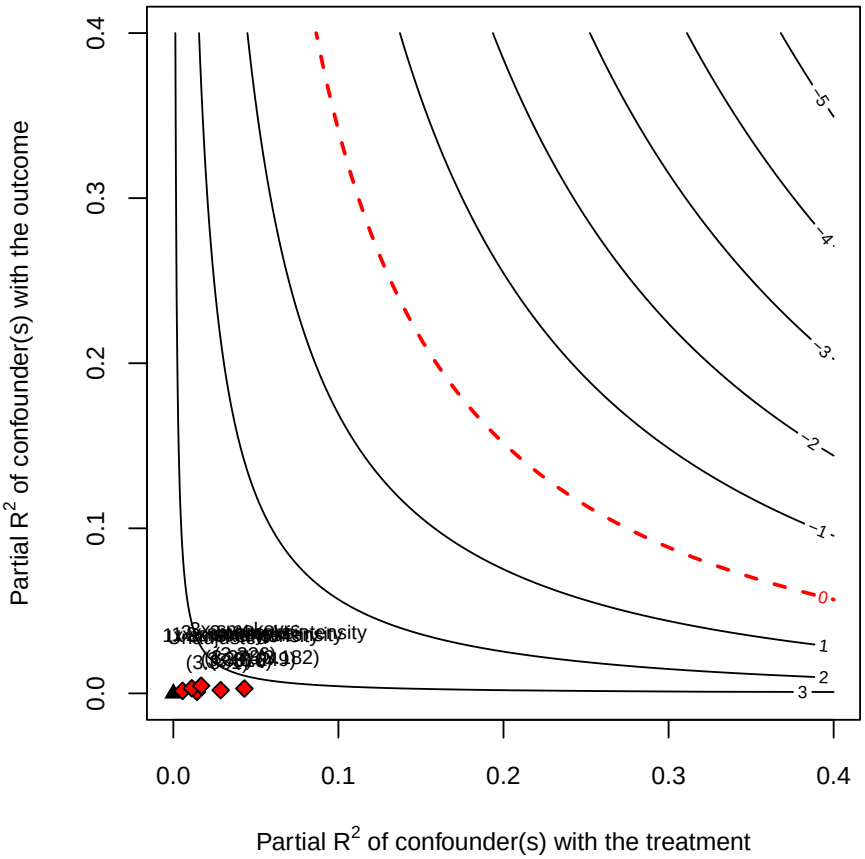


Figure 5.1.: Cinelli-Hazlett sensitivity contour. The dashed lines show benchmarks based on smokeintensity and smokeyrs at 1x, 2x, and 3x strength. Confounders falling in the shaded region would reduce the estimate to zero.

All six benchmark points sit below 0.05 on the treatment axis and below 0.005 on the outcome axis, in the region where the adjusted estimate stays above 3 kg. The contour that reaches zero is a long way out.

5.3. E-values for risk ratios

For binary outcomes and risk-ratio-type estimands, VanderWeele and Ding (2017) propose the **E-value**: the minimum strength of association on the risk-ratio scale that an unmeasured confounder would need to have with both the treatment and the outcome to fully explain the observed risk ratio.

To illustrate, we dichotomize weight change into any gain versus none, fit the risk ratio by Poisson regression with a log link on the same nine covariates, and take the confidence interval from robust standard errors:

```r
nhefs$gain <- as.numeric(nhefs$wt82_71 > 0)

rr_fit <- glm(gain ~ qsmk + sex + age + race + education + smokeintensity +
                     smokeyrs + exercise + active + wt71,
              data = nhefs, family = poisson(link = "log"))
rr_summary <- tidy(rr_fit, exp = TRUE, conf.int = TRUE)

rr_est <- exp(coef(rr_fit)["qsmk"])
# The Poisson likelihood is misspecified for a binary outcome (the Zou 2004
# log-binomial-via-Poisson trick), so the model-based variance is wrong.
# Use robust (sandwich/HC) standard errors for the RR confidence interval.
rr_ci  <- lmtest::coefci(rr_fit, parm = "qsmk",
                         vcov. = sandwich::vcovHC(rr_fit, type = "HC0"))
rr_lo  <- exp(rr_ci[1])
rr_hi  <- exp(rr_ci[2])

cat(sprintf("Adjusted RR (quit vs not): %.3f  [%.3f, %.3f]\n",
            rr_est, rr_lo, rr_hi))
```

```
Adjusted RR (quit vs not): 1.218  [1.135, 1.307]
```

```r
ev <- evalues.RR(est = rr_est, lo = rr_lo, hi = rr_hi)
print(ev)
```

```
            point    lower    upper
RR       1.217702 1.134911 1.306534
E-values 1.732578 1.526206       NA
```

The E-value reports two numbers:

- **E-value (point estimate)**: an unmeasured confounder would need to be associated with both treatment and outcome by risk ratios of at least this value, above and beyond the measured confounders, to fully explain away the observed effect.
- **E-value (confidence interval)**: the minimum strength needed to shift the lower confidence bound to 1.

The adjusted risk ratio is 1.218, with a 95% interval of $[1.135, 1.307]$. Quitters are about 22% more likely to gain any weight. The E-value for the point estimate is 1.73: an unmeasured confounder would need risk-ratio associations of at least 1.73 with both quitting and weight gain, beyond the nine measured covariates, to account for the whole association. The E-value for the confidence limit is 1.53, the strength that would push the lower bound to 1.

An E-value much greater than 1 suggests the result is hard to explain by unmeasured confounding. An E-value close to 1 means a modest unmeasured confounder could overturn the conclusion. At 1.73 this result sits in between: a confounder as strong as that is not implausible, but it would have to be unrelated to everything already in the model.

The useful part of the E-value is that it is on the risk-ratio scale. It can be explained without specifying a model for the unobserved confounder.

5.4. Rosenbaum bounds (matched estimators)

For matching estimators, **Rosenbaum bounds** (Rosenbaum 2002) provide a sensitivity analysis based on the maximum deviation from random assignment within matched pairs.

Suppose units in a matched pair are observationally identical, but one might still be more likely than the other to receive treatment due to unobserved factors. Let Γ bound the ratio of treatment odds between the two units of a pair: $\Gamma = 1$ means matched pairs are perfectly randomised (both units equally likely to be the treated one); $\Gamma = 2$ means hidden bias could make one unit's odds of treatment up to twice the other's.

Rosenbaum bounds compute the range of p-values for the matched test under each value of Γ. The **critical** Γ is the smallest value at which the test becomes insignificant.

Here is a worked example on the same NHEFS data. We fit the propensity score by logistic regression on the nine baseline covariates, match each of the 403 quitters to its nearest unmatched control on that score, take the paired differences in weight change, and compute the upper-bound Wilcoxon signed-rank p-value on a grid $\Gamma \in [1, 4]$ in steps of 0.25:

```
# Estimate propensity score
ps_fit <- glm(qsmk ~ sex + age + race + education + smokeintensity +
                     smokeyrs + exercise + active + wt71,
              data = nhefs, family = binomial)
nhefs$ps <- predict(ps_fit, type = "response")

# Simple 1:1 nearest-neighbour matching on propensity score
treated_idx <- which(nhefs$qsmk == 1)
control_idx <- which(nhefs$qsmk == 0)

set.seed(1)
matched_pairs <- data.frame(t_idx = integer(), c_idx = integer())
available_controls <- control_idx
for (ti in treated_idx) {
  if (length(available_controls) == 0) break
  dists <- abs(nhefs$ps[ti] - nhefs$ps[available_controls])
  best  <- which.min(dists)
  matched_pairs <- rbind(matched_pairs,
                         data.frame(t_idx = ti,
                                    c_idx = available_controls[best]))
  available_controls <- available_controls[-best]
}

# Compute paired differences in outcome
paired_diff <- nhefs$wt82_71[matched_pairs$t_idx] -
               nhefs$wt82_71[matched_pairs$c_idx]
cat(sprintf("N matched pairs: %d\n", nrow(matched_pairs)))
```

```
N matched pairs: 403
```

```r
cat(sprintf("Mean paired difference (matched ATT): %.3f kg\n",
            mean(paired_diff)))
```

```
Mean paired difference (matched ATT): 2.926 kg
```

```r
# Rosenbaum bound calculation: range of p-values for Wilcoxon signed-rank
# under each Gamma. We compute this directly rather than using a package
# to keep the dependency footprint small.
rosenbaum_pvalue <- function(diffs, gamma) {
  # The Wilcoxon signed-rank protocol discards exact-zero paired differences
  # before ranking (a zero carries no information about the sign) and
  # reduces the effective sample size accordingly; including them would
  # skew both the rank sum and its null variance.
  diffs <- diffs[diffs != 0]
  abs_d <- abs(diffs)
  signs <- sign(diffs)
  ranks <- rank(abs_d)
  # Under Gamma, P(positive sign) is bounded between 1/(1+Γ) and Γ/(1+Γ).
  # Upper-bound p-value: assume P(positive) = Γ/(1+Γ) (against null of zero)
  p_pos_upper <- gamma / (1 + gamma)
  # Test statistic: sum of positive ranks
  T_obs <- sum(ranks[signs > 0])
  n <- length(diffs)
  mu_upper  <- p_pos_upper * sum(ranks)
  var_upper <- p_pos_upper * (1 - p_pos_upper) * sum(ranks^2)
  z <- (T_obs - mu_upper) / sqrt(var_upper)
  1 - pnorm(z)
}

gammas <- seq(1, 4, by = 0.25)
pvals  <- sapply(gammas, function(g) rosenbaum_pvalue(paired_diff, g))

result <- tibble(Gamma = gammas, p_value_upper = pvals)
knitr::kable(result, digits = 4,
             caption = "Upper-bound p-values from Rosenbaum sensitivity analysis")
```

Table 5.2.: Upper-bound p-values from Rosenbaum sensitivity analysis

Gamma	p_value_upper
1.00	0.0000
1.25	0.0004
1.50	0.0334
1.75	0.2905
2.00	0.7106
2.25	0.9374

Gamma	p_value_upper
2.50	0.9921
2.75	0.9993
3.00	1.0000
3.25	1.0000
3.50	1.0000
3.75	1.0000
4.00	1.0000

Matching gives 403 pairs, one per quitter, with a mean paired difference of 2.926 kg. The upper-bound p-value is 0.0334 at $\Gamma = 1.50$ and 0.2905 at $\Gamma = 1.75$, so the crossing of 0.05 falls between those two grid points.

The **critical** Γ is the value at which the upper-bound p-value crosses 0.05. If it is around 2 or higher, the result is robust to moderate hidden bias. If it is close to 1, even small unmeasured confounding could overturn the conclusion. We read it off the grid:

```
sig_idx <- which(result$p_value_upper > 0.05)
critical_gamma <- if (length(sig_idx) > 0) result$Gamma[min(sig_idx)] else NA
if (is.na(critical_gamma)) {
  cat(sprintf("No Gamma up to %.2f pushed the p-value above 0.05 -- the result is robust across
} else {
  cat(sprintf("Critical Gamma (p > 0.05): %.2f\n", critical_gamma))
}
```

```
Critical Gamma (p > 0.05): 1.75
```

The critical Γ is 1.75. This is the first value *on the 0.25 grid* at which significance is lost; the true crossing lies between 1.50 and 1.75, and a finer grid would locate it more precisely. The convention in applied work is often to report the largest Γ at which significance is retained, which here is 1.50.

So hidden bias that made one member of a matched pair up to about 1.5 times more likely to quit than the other would leave the result significant, and bias beyond that would not.

The three analyses do not share a scale. Γ bounds an odds ratio within a matched pair, the E-value bounds a risk ratio, and the robustness value is a partial R^2. They also ask different questions: Γ and the E-value ask when the result stops being significant, the robustness value asks when the point estimate reaches zero. Reporting one of them is normal practice; the numbers should not be averaged or compared directly.

5.5. What about partial identification?

When sensitivity analysis shows the point estimate is fragile, partial identification is often the next step. Instead of one estimate, report bounds under weaker assumptions.

The Manski (1990) “no-assumption” bounds say:

$$\text{ATE} \in \Big[\ \mathbb{E}[Y|D{=}1]P(D{=}1) - (\mathbb{E}[Y|D{=}0]P(D{=}0) + K\,P(D{=}1)),\ \ (\mathbb{E}[Y|D{=}1]P(D{=}1) + K\,P(D{=}0)) - \mathbb{E}[Y|D{=}0]\Big] \tag{5.1}$$

Each potential-outcome mean is split by the law of total expectation, filling in the unobserved arm with its worst/best case. With outcomes bounded in $Y \in [0, K]$, this is a single interval of width exactly K — bounded, but often too wide to be useful. Tighter bounds come from adding structure to a specific problem: Lee (2009) bounds address sample selection/attrition (the outcome is observed only for selected units) by combining random assignment with monotone selection and trimming the over-selected arm. For IV settings, Mourifié and Wan (2017) provide a test of the LATE identifying assumptions (instrument validity plus monotonicity).

These bounds usually complement a point estimate. They are useful when the point estimate depends too much on an untestable assumption.

5.6. Summary

- Sensitivity analysis asks how strong hidden confounding must be to change the result.
- Cinelli-Hazlett bounds are useful for regression estimates.
- E-values are useful for risk ratios.
- Rosenbaum bounds are useful for matched pairs.
- Observational causal estimates should usually include at least one sensitivity analysis.

Part II.

Estimation

6. Estimation: RA, IPW, AIPW and IPWRA

Once the causal effect is identified, we need an estimator. This chapter covers four: regression adjustment (RA), inverse probability weighting (IPW), augmented IPW (AIPW), and inverse-probability-weighted regression adjustment (IPWRA).

6.1. Experimental Data

With experimental data the assumptions hold by design. A difference-in-means estimator suffices (under randomization ATE = ATT = ATU):

$$\hat{\tau} = \bar{Y}_1 - \bar{Y}_0 \tag{6.1}$$

A t-test or a regression of Y on W and X both recover the same estimate.

6.2. Adjustment Formula

The adjustment formula connects causal and statistical quantities:

$$\begin{aligned} E[Y(1) - Y(0)] &= E[Y(1)] - E[Y(0)] \\ &= E_X[E[Y(1)|X]] - E_X[E[Y(0)|X]] \\ &= E_X[E[Y(1)|W=1,X]] - E_X[E[Y(0)|W=0,X]] \\ &= E_X[E[Y|W=1,X]] - E_X[E[Y|W=0,X]] \end{aligned} \tag{6.2}$$

The steps are: linearity of expectation, iterated expectations, unconfoundedness plus overlap, and consistency.

6.3. Regression Adjustment

We estimate $E[Y \mid W = w, X = x]$ by regression.

We define a conditional mean function:

$$\begin{aligned} \mu_0(X) &\equiv E[Y|W=0,X] \\ &= E[Y(0)|W=0,X] \\ &= E[Y(0)|W=1,X] \end{aligned} \tag{6.3}$$

The RA estimators are:

$$\hat{\tau}_{ATT} = \frac{1}{n_1} \sum_{i:W_i=1} \{Y_i - \hat{\mu}_0(X_i)\} \tag{6.4}$$

$$\hat{\tau}_{ATE} = \frac{1}{n} \sum_{i=1}^{n} \{\hat{\mu}_1(X_i) - \hat{\mu}_0(X_i)\} \tag{6.5}$$

where $\hat{\mu}_0(X_i)$ is the predicted value of Y_i from a regression of Y on X for the control group.

Target:

$$\begin{aligned} \tau_{ATT} &= E[Y|W=1] - E[Y(0)|W=1] \\ &= E[Y|W=1] - E[\mu_0(X)|W=1] \\ &= E[Y|W=1] - (\alpha_0 + E[X|W=1]\beta_0) \end{aligned} \tag{6.6}$$

The last term is a linear form of $\mu_0(X)$. We can specify other forms, but the idea is to model it with some functional form. For parametric forms, we need to make sure extrapolation does not go out of control.

6.4. Linear RA

Regression Adjustment is basically an imputation estimator. While we observe $E[Y|W=1,X]$, we model $E[Y(0)|W=1]$, based on unconfoundedness and a functional form (say linear form). We estimate β_0 on the control sample, then get the expected values for the treated sample, for each value of X.

Implementation of linear RA is easy. We regress Y on X and W and their interaction. It's shown that we need to de-mean X to get the effect correct.

The data are the 401(k) file distributed with `hdm`, from the 1991 Survey of Income and Program Participation. There are 9,915 households. The outcome `net_tfa` is net total financial assets in dollars, the treatment `p401` is participation in a 401(k) plan, taken by 2,594 households, and we condition on household income `inc` and the reference person's `age`. Whether those two covariates suffice for unconfoundedness is doubtful, and the point here is the mechanics of the estimator rather than the substantive effect.

We de-mean income and age at their treated-group means, then regress net financial assets on participation, the two de-meaned covariates, and their interactions with participation.

```
data(pension)
#  for ATE, de-mean X by deducting the mean of X in the whole sample:
#  data <- pension %>% mutate(inc_dm=inc-mean(inc), age_dm=age-mean(age))
#  for ATT (used here), de-mean X by deducting the mean of X in the treated group:
data <- pension %>%
  # Reference the local (in-mutate) inc/age rather than pension$inc/pension$age
  # directly: hardcoding the parent object name here would silently keep using
  # the ORIGINAL full-sample means even if this code were reused on a
  # resampled/bootstrapped copy of the data assigned to `data`, producing
  # wrong point estimates and standard errors for that use case.
```

```
  mutate(inc_dm=inc-mean(inc[p401==1]), age_dm=age-mean(age[p401==1]))
lm_ra <- lm(net_tfa ~ p401*(inc_dm + age_dm),data=data)
summary(lm_ra)
```

```
Call:
lm(formula = net_tfa ~ p401 * (inc_dm + age_dm), data = data)

Residuals:
    Min      1Q  Median      3Q     Max
-508262  -17176   -3497    9223 1444086

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 2.410e+04  8.312e+02  28.998  < 2e-16 ***
p401        1.416e+04  1.397e+03  10.134  < 2e-16 ***
inc_dm      7.723e-01  3.009e-02  25.670  < 2e-16 ***
age_dm      7.977e+02  6.350e+01  12.561  < 2e-16 ***
p401:inc_dm 3.300e-01  5.133e-02   6.429 1.34e-10 ***
p401:age_dm 8.153e+02  1.333e+02   6.118 9.81e-10 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 57210 on 9909 degrees of freedom
Multiple R-squared:  0.1894,    Adjusted R-squared:  0.1889
F-statistic: 462.9 on 5 and 9909 DF,  p-value: < 2.2e-16
```

```
#coeftest(lm_ra, vcov=vcovHC(lm_ra, type='HC2'))[2,]
```

The coefficient on `p401` is 14,160 with a standard error of 1,397. Because we de-meaned at the treated-group means, that coefficient is the ATT: participants hold about $14,000 more in net financial assets than they would have without a 401(k), under the assumption that income and age are the only confounders. De-meaning at the whole-sample means instead would give the ATE.

Both interaction terms are large and significant, so the effect varies with the covariates. A household $10,000 above the treated-group mean income has an effect larger by $3,300, and each extra year of age adds $815. This is why the interactions are needed at all.

6.4.1. Questions on RA

1. Why do we have to do interaction of W and X? What if we don't do it?
2. What if we don't de-mean X?

Answers:

1. It has been shown (Słoczyński 2022) that with heterogeneous treatment effect, and unequal divide between treated and control, we need to do the interaction term. Otherwise, we'll get biased estimates, for τ_{ATT} or τ_{ATE}. The intuition is that we need to model the difference between treated and control, and the difference is not constant across X.
2. If we don't de-mean X, then the coefficient on W is not the average treatment effect. We can still retrieve the ATT or ATE by calculating the marginal effect of W though.

6.5. IPW

Under unconfoundedness and overlap,

$$\begin{aligned} E[Y(w)] &= E[E[Y|W=w,X]] \\ &= \sum_x (\sum_y yP(y|w,x))P(x) \\ &= \sum_x \sum_y yP(y|w,x)P(x)\frac{P(w|x)}{P(w|x)} \\ &= \sum_x \sum_y yP(y,w,x)\frac{1}{P(w|x)} \\ &= \sum_x E[\mathbb{1}(W=w,X=x)Y]\frac{1}{P(w|x)} \\ &= E[\frac{\mathbb{1}(W=w)Y}{P(w|X)}] \end{aligned} \tag{6.7}$$

The IPW equation means that the expected value of the potential outcome is the weighted average of the observed outcome, where the weight is the inverse of the propensity score $P(w|x)$. $\mathbb{1}(W=w)$ is an indicator function, which is 1 if $W=w$ and 0 otherwise.

$$\begin{aligned} \tau_{ATE} &= E[Y(1)] - E[Y(0)] \\ &= E[\frac{WY}{\pi(X)}] - E[\frac{(1-W)Y}{1-\pi(X)}] \end{aligned} \tag{6.8}$$

The sample estimators are:

$$\hat{\tau}_{ATE,IPW} = \frac{1}{N}\sum_i \frac{W_i Y_i}{\hat{\pi}(X_i)} - \frac{1}{N}\sum_i \frac{(1-W_i)Y_i}{1-\hat{\pi}(X_i)} \tag{6.9}$$

$$\hat{\tau}_{ATT,IPW} = \bar{Y}_1 - \frac{1}{N_1}\sum_i \frac{\hat{\pi}(X_i)}{1-\hat{\pi}(X_i)}(1-W_i)Y_i \tag{6.10}$$

For the ATT, control units are weighted by the odds $\hat{\pi}(X_i)/(1-\hat{\pi}(X_i))$ rather than $1/(1-\hat{\pi}(X_i))$: this reweights the controls to the covariate distribution of the treated (one can check $\mathrm{E}[\frac{\pi(X)}{1-\pi(X)}(1-W)Y] = P(W=1)\,\mathrm{E}[Y(0)|W=1]$).

6.5.1. IPW Intuition

IPW removes confounding X by creating a pseudo-population in which X is independent of W. The intuition is that we can create a pseudo-population in which X is independent of W, by weighting the observations by the inverse of the propensity score. Once X is independent of W, X is not a confounder anymore. We can then estimate the effect of W on Y by comparing the weighted average of Y for $W = 1$ and $W = 0$.

Suppose there are two types of people, one type has high probability to be treated, the other one has low probability to be treated. Without adjusting for the propensity score, the treated group will be dominated by the first type, and the control group will be dominated by the second type. The difference between the treated and control group will be due to the difference in the composition of the two groups, not due to the treatment effect. By weighting the observations by the inverse of the propensity score, we can create a pseudo-population in which the two groups have the same composition. The difference between the treated and control group will be due to the treatment effect.

6.6. Doubly Robust Estimators

We can model both outcome and treatment assignment, and use both models to estimate the treatment effect. The estimator is called doubly robust because it is consistent if *either* the outcome regression *or* the propensity/treatment model is correctly specified – we only need one of the two to be right, not both. (Note the precise statement: it is *not* consistent when both are misspecified, so "robust to either being misspecified" is the wrong reading.)

6.6.1. AIPW

$$\psi_1^{AIPW} = E[\frac{W(Y - \mu_1(X))}{\pi(X)} + \mu_1(X)] \tag{6.11}$$

$$\psi_0^{AIPW} = E[\frac{(1 - W)(Y - \mu_0(X))}{1 - \pi(X)} + \mu_0(X)] \tag{6.12}$$

Here $\mu_w(X) = E[Y|W = w, X]$ is the outcome-model nuisance function, while the scalar ψ_w^{AIPW} equals $E[Y(w)]$ as long as either the outcome model or the propensity score is correctly specified; the ATE is $\psi_1^{AIPW} - \psi_0^{AIPW}$.

We switch to the Cattaneo (2010) birth-weight data used in the Stata manuals: 4,642 births, outcome `bweight` in grams, treatment `mbsmoke` for maternal smoking during pregnancy (864 mothers), and covariates `prenatal1` (a prenatal visit in the first trimester), `mmarried`, `mage` (mother's age), `fbaby` (first baby) and `medu` (mother's education). Smokers' babies weigh 3,137.7 g on average and non-smokers' 3,412.9 g, a raw gap of −275.3 g.

The outcome model is a linear regression of birth weight on smoking interacted with prenatal care, marital status, mother's age and first-baby status. The propensity model is a logit of smoking on marital status, mother's age, first-baby status and education. We form the AIPW summand for each arm and average the difference.

```r
data <- read_dta(file="https://www.stata-press.com/data/r18/cattaneo2.dta")
mu <- lm(bweight ~ mbsmoke*(prenatal1 + mmarried + mage + fbaby), data=data)
pi <- glm( mbsmoke ~ mmarried + mage + fbaby + medu, data=data, family=binomial(link="logit"))
mm1 <- predict(mu, newdata=data %>% mutate(mbsmoke=1))
mm0 <- predict(mu, newdata=data %>% mutate(mbsmoke=0))
pi1 <- predict(pi, newdata=data, type="response")
data <- data %>%
  mutate(w=mbsmoke, y=bweight, m1=mm1, m0=mm0, pi1=pi1)
data2 <- data %>%
  mutate(mu1=(w*(y-m1)/pi1 + m1), mu0=((1-w)*(y-m0)/(1-pi1) + m0)) %>%
  mutate(tau=mu1-mu0)
data2 %>% summarise(mean(tau))
```

```
# A tibble: 1 x 1
  `mean(tau)`
        <dbl>
1       -234.
```

AIPW gives −234 g. Adjustment moves the estimate about 41 g toward zero from the raw −275 g, so part of the raw gap is composition: smokers differ from non-smokers in age, education and marital status.

6.6.2. IPWRA

The idea of IPWRA is to combine RA and IPW. Basically RA with IPW weights.

Implementation:

- Estimate propensity score. Get predicted probability of treated for each observation.
- Estimate outcome model with the IPW weights: two separate equations, one for treated (weights $1/\hat{\pi}(X)$) and one for control (weights $1/(1-\hat{\pi}(X))$). Equivalently, one weighted regression with full treatment–covariate interaction — the interacted design matrix is block-diagonal across the two groups, so the weighted normal equations decouple and the two routes give identical fits.
- Get predicted values for setting everyone treated. Get predicted values for setting everyone control. These are the two potential outcomes.
- Take difference. The average is the ATE.

We run this on the same birth-weight data and the same propensity model as AIPW, so the two estimates differ only in how the outcome and treatment models are combined.

```r
data <- read_dta(file="https://www.stata-press.com/data/r18/cattaneo2.dta")
pi <- glm( mbsmoke ~ mmarried + mage + fbaby + medu, data=data, family=binomial(link="logit"))
pi1 <- predict(pi, newdata=data, type="response")
data <- data %>%
  mutate(pi1=pi1)
```

```
data1 <- data %>% filter(mbsmoke==1)
data0 <- data %>% filter(mbsmoke==0)
mu1 <- lm(bweight ~ prenatal1 + mmarried + mage + fbaby, data=data1, weights=1/data1$pi1)
mu0 <- lm(bweight ~ prenatal1 + mmarried + mage + fbaby, data=data0, weights=1/(1-data0$pi1))
mm1 <- predict(mu1, newdata=data %>% mutate(mbsmoke=1))
mm0 <- predict(mu0, newdata=data %>% mutate(mbsmoke=0))
data <- data %>%
  mutate(w=mbsmoke, y=bweight, m1=mm1, m0=mm0, pi1=pi1)
data2 <- data %>%
  mutate(tau=m1-m0)
data2 %>% summarise(mean(tau))
```

```
# A tibble: 1 x 1
  `mean(tau)`
        <dbl>
1       -233.
```

IPWRA gives −233 g, within a gram of the AIPW estimate. The agreement is not automatic. Both are doubly robust and both use the same two nuisance models here, so they can only differ in how the models are combined. When the outcome and propensity models point in different directions, the two estimators separate.

7. Matching Estimators

```
library(tidyverse)
library(MatchIt)
library(cobalt)
library(WeightIt)
library(marginaleffects)
```

Matching is a way to make treated and control observations more comparable. For each treated observation, we look for controls with similar covariates. Then we compare outcomes in this matched sample.

This does not solve endogeneity. Matching only helps with selection on observables. If the treatment is selected on unobserved variables, matching will not fix the problem. The value of matching is that it makes the overlap problem visible. A regression can extrapolate quietly; matching often shows that there are no good controls for part of the treated sample.

In R, the main workflow is `MatchIt` for matching and `cobalt` for balance checking.

> **Related reading**: A longer treatment is in Matching and Weighting Part 1 of *Topics on Econometrics and Causal Inference*, which credits Noah Greifer and coauthors (the `MatchIt`/`WeightIt` package authors).

7.1. Assumptions

Matching needs the same assumptions as other selection-on-observables methods:

1. **SUTVA** — no interference, no hidden treatment versions.
2. **Ignorability** (unconfoundedness) — conditional on X, the treatment D is independent of the potential outcomes $(Y(0), Y(1))$.
3. **Overlap** (positivity) — every value of X has positive probability of being both treated and untreated.

Even if ignorability is true, the covariate distribution can be very different in the treated and control groups. Matching tries to repair that before estimating the effect. If there is no overlap, it should not pretend there is overlap.

7.2. The Lalonde example

I use the Lalonde job-training data because it is the standard example for matching. The treatment is job training. The outcome is 1978 earnings. The data set in `MatchIt` is the observational version, so the treated and control groups are not balanced at the start.

```
data("lalonde", package = "MatchIt")
glimpse(lalonde)
```

```
Rows: 614
Columns: 9
$ treat    <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
$ age      <int> 37, 22, 30, 27, 33, 22, 23, 32, 22, 33, 19, 21, 18, 27, 17, 1~
$ educ     <int> 11, 9, 12, 11, 8, 9, 12, 11, 16, 12, 9, 13, 8, 10, 7, 10, 13,~
$ race     <fct> black, hispan, black, black, black, black, black, black, blac~
$ married  <int> 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0~
$ nodegree <int> 1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1~
$ re74     <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0~
$ re75     <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0~
$ re78     <dbl> 9930.0460, 3595.8940, 24909.4500, 7506.1460, 289.7899, 4056.4~
```

There are 614 observations, 185 treated and 429 control. The treatment is `treat`; the outcome is `re78`, 1978 earnings in dollars; the covariates are `age`, `educ` (years of schooling), `race` (black, hispanic, white), `married`, `nodegree` (no high-school degree), and `re74` and `re75`, earnings in 1974 and 1975. The earnings variables matter most here, because they pile up at zero and differ sharply between the groups.

Before matching anything we check balance. `bal.tab` reports the standardized mean difference for each covariate and flags any above 0.1:

```
# Construct a pre-match MatchIt object for balance reporting
m.pre <- matchit(treat ~ age + educ + race + married + nodegree + re74 + re75,
                 data = lalonde, method = NULL,
                 distance = "glm")
bal.tab(m.pre, thresholds = c(m = 0.1))
```

```
Balance Measures
                Type Diff.Un     M.Threshold.Un
distance    Distance  1.7941
age          Contin. -0.3094 Not Balanced, >0.1
educ         Contin.  0.0550     Balanced, <0.1
race_black    Binary  0.6404 Not Balanced, >0.1
race_hispan   Binary -0.0827     Balanced, <0.1
race_white    Binary -0.5577 Not Balanced, >0.1
married       Binary -0.3236 Not Balanced, >0.1
nodegree      Binary  0.1114 Not Balanced, >0.1
```

```
re74         Contin. -0.7211 Not Balanced, >0.1
re75         Contin. -0.2903 Not Balanced, >0.1

Balance tally for mean differences
                    count
Balanced, <0.1          2
Not Balanced, >0.1      7

Variable with the greatest mean difference
 Variable Diff.Un     M.Threshold.Un
     re74 -0.7211 Not Balanced, >0.1

Sample sizes
    Control Treated
All     429     185
```

Seven of the nine contrasts exceed 0.1. Earnings in 1974 differ by -0.72 standard deviations, the black indicator by 0.64, the white indicator by -0.56, marital status by -0.32, age by -0.31, earnings in 1975 by -0.29. Only `educ` (0.055) and the hispanic indicator (-0.083) pass. The propensity score itself differs by 1.79 standard deviations. This sample is nowhere near balanced.

7.3. Distance measures

Matching needs a distance metric. The usual choices are:

1. **Propensity score**: estimate $\hat{e}(x) = P(D = 1 \mid X = x)$ and match on the fitted probability. This turns many covariates into one score.
2. **Mahalanobis distance**: match on the raw covariates using the squared distance $(x_i - x_j)'\Sigma^{-1}(x_i - x_j)$ (rankings are unchanged by the square root). This works better in low dimension.
3. **Hybrid**: impose a propensity-score caliper, then use Mahalanobis distance inside the caliper.

7.4. Matching methods

7.4.1. Nearest-neighbour matching on a propensity score

We fit the propensity score by logistic regression, take its linear index, and match each treated unit to the single closest control, without replacement.

```
m.nn <- matchit(treat ~ age + educ + race + married + nodegree + re74 + re75,
                data = lalonde,
                method = "nearest",
                distance = "glm",
                link = "linear.logit",
                ratio = 1)
```

```
m.nn
```

```
A `matchit` object
 - method: 1:1 nearest neighbor matching without replacement
 - distance: Propensity score
             - estimated with logistic regression and linearized
 - number of obs.: 614 (original), 370 (matched)
 - target estimand: ATT
 - covariates: age, educ, race, married, nodegree, re74, re75
```

All 185 treated units are matched, so 370 of the 614 observations are used and 244 controls are discarded. The target estimand is the ATT.

Now check balance again:

```
bal.tab(m.nn, thresholds = c(m = 0.1))
```

```
Balance Measures
                Type Diff.Adj        M.Threshold
distance    Distance   0.9192
age          Contin.   0.0718     Balanced, <0.1
educ         Contin.  -0.1290 Not Balanced, >0.1
race_black    Binary   0.3730 Not Balanced, >0.1
race_hispan   Binary  -0.1568 Not Balanced, >0.1
race_white    Binary  -0.2162 Not Balanced, >0.1
married       Binary  -0.0216     Balanced, <0.1
nodegree      Binary   0.0703     Balanced, <0.1
re74         Contin.  -0.0505     Balanced, <0.1
re75         Contin.  -0.0257     Balanced, <0.1

Balance tally for mean differences
                   count
Balanced, <0.1         5
Not Balanced, >0.1     4

Variable with the greatest mean difference
   Variable Diff.Adj        M.Threshold
 race_black    0.373 Not Balanced, >0.1

Sample sizes
          Control Treated
All           429     185
Matched       185     185
Unmatched     244       0
```

Five of nine now pass, up from two. The earnings histories are repaired: `re74` goes from −0.72 to −0.05 and `re75` from −0.29 to −0.03. Race is not: the black indicator is still 0.373 and the white indicator −0.216. And `educ` has got worse, from 0.055 to −0.129. This is what matching on a scalar score does: it balances the score, not the covariates. Two units with the same score can differ on individual covariates in offsetting ways, and nothing in the procedure prevents that.

7.4.2. Full matching

Full matching forms subclasses with treated and control observations inside each subclass. It can put one treated unit with several controls, or one control with several treated units. It uses all observations and then assigns weights.

```
m.full <- matchit(treat ~ age + educ + race + married + nodegree + re74 + re75,
                  data = lalonde,
                  method = "full",
                  distance = "glm",
                  link = "probit")

bal.tab(m.full, thresholds = c(m = 0.1))
```

```
Balance Measures
                Type Diff.Adj    M.Threshold
distance    Distance   0.0045 Balanced, <0.1
age          Contin.   0.0393 Balanced, <0.1
educ         Contin.  -0.0956 Balanced, <0.1
race_black    Binary   0.0043 Balanced, <0.1
race_hispan   Binary   0.0103 Balanced, <0.1
race_white    Binary  -0.0146 Balanced, <0.1
married       Binary   0.0259 Balanced, <0.1
nodegree      Binary   0.0504 Balanced, <0.1
re74         Contin.  -0.0009 Balanced, <0.1
re75         Contin.  -0.0091 Balanced, <0.1

Balance tally for mean differences
                   count
Balanced, <0.1        10
Not Balanced, >0.1     0

Variable with the greatest mean difference
 Variable Diff.Adj    M.Threshold
     educ  -0.0956 Balanced, <0.1

Sample sizes
                     Control Treated
All                   429.       185
Matched (ESS)          50.76     185
Matched (Unweighted)  429.       185
```

All ten contrasts now pass, the largest being `educ` at -0.096, and no observation is discarded. The cost is in the weights: the control effective sample size falls from 429 to 50.8, because full matching concentrates weight on the controls that resemble treated units.

The effective sample size is the number of equally weighted observations that would carry the same precision. An unweighted mean of m observations has variance σ^2/m; a weighted mean with weights summing to one has variance $\sigma^2 \sum_i w_i^2$. Setting the two equal gives $m = 1/\sum_i w_i^2$. So 50.8 means these 429 controls carry about as much information as 51 equally weighted ones. Balance improved and precision did not.

7.4.3. Mahalanobis matching

When the number of covariates is small, it is reasonable to match directly on the covariates with Mahalanobis distance.

```
m.maha <- matchit(treat ~ age + educ + race + married + nodegree + re74 + re75,
                  data = lalonde,
                  distance = "mahalanobis")
bal.tab(m.maha, thresholds = c(m = 0.1))
```

```
Balance Measures
               Type Diff.Adj        M.Threshold
age         Contin.   0.1269 Not Balanced, >0.1
educ        Contin.  -0.0430     Balanced, <0.1
race_black   Binary   0.3784 Not Balanced, >0.1
race_hispan  Binary   0.0000     Balanced, <0.1
race_white   Binary  -0.3784 Not Balanced, >0.1
married      Binary  -0.0595     Balanced, <0.1
nodegree     Binary   0.0486     Balanced, <0.1
re74        Contin.  -0.2476 Not Balanced, >0.1
re75        Contin.  -0.1322 Not Balanced, >0.1

Balance tally for mean differences
                   count
Balanced, <0.1         4
Not Balanced, >0.1     5

Variable with the greatest mean difference
   Variable Diff.Adj        M.Threshold
 race_black   0.3784 Not Balanced, >0.1

Sample sizes
          Control Treated
All           429     185
Matched       185     185
Unmatched     244       0
```

Mahalanobis matching balances only four of nine. The black and white indicators sit at ± 0.378, `re74` at -0.248, `re75` at -0.132, and `age` at 0.127. It does worse than the propensity score did on earnings. With a three-level race factor and two earnings variables that pile up at zero, the covariance matrix is a poor description of the covariate space, and the metric it defines is a poor guide to who resembles whom.

7.4.4. Coarsened Exact Matching (CEM)

CEM coarsens covariates into bins and then exact-matches on the binned values. This is very transparent: if a treated observation has no comparable control in the binned covariate space, it is dropped.

```
m.cem <- matchit(treat ~ age + educ + race + married + nodegree + re74 + re75,
                 data = lalonde,
                 method = "cem")
bal.tab(m.cem, thresholds = c(m = 0.1))
```

```
Balance Measures
               Type Diff.Adj    M.Threshold
age         Contin.   0.0493 Balanced, <0.1
educ        Contin.   0.0446 Balanced, <0.1
race_black   Binary   0.0000 Balanced, <0.1
race_hispan  Binary   0.0000 Balanced, <0.1
race_white   Binary   0.0000 Balanced, <0.1
married      Binary   0.0000 Balanced, <0.1
nodegree     Binary   0.0000 Balanced, <0.1
re74        Contin.  -0.0427 Balanced, <0.1
re75        Contin.  -0.0492 Balanced, <0.1

Balance tally for mean differences
                   count
Balanced, <0.1         9
Not Balanced, >0.1     0

Variable with the greatest mean difference
 Variable Diff.Adj    M.Threshold
      age   0.0493 Balanced, <0.1

Sample sizes
                     Control Treated
All                   429.       185
Matched (ESS)          41.29      65
Matched (Unweighted)   75.        65
Unmatched             354.       120
```

All nine contrasts pass, and the largest is `age` at 0.049. Read the sample sizes before celebrating: only 65 of the 185 treated units are matched, and 120 are dropped, along with 354 of the 429 controls. The control effective sample size is 41.3.

CEM can drop many observations. That is not necessarily a problem. It is often telling us that the original sample does not support the target comparison. But it does change the question. The estimand is now the average effect for the 65 treated units that had a comparable control, not the ATT for all 185, and those two need not be the same number.

7.5. Estimation after matching

After matching, we estimate the effect on the matched data. The matching weights come from `MatchIt`. With subclass matching, standard errors should be clustered by subclass: units within the same matched subclass are not independent draws – they were selected together specifically because they resemble each other on the matching covariates, and (for many-to-one or full matching) a single unit's outcome can appear, re-weighted, in the "observation" for more than one comparison. Treating them as independent (ordinary or heteroskedasticity-robust SEs) understates the true sampling variability; clustering by subclass accounts for that within-subclass correlation.

```
m.data <- match_data(m.full)

fit <- lm(re78 ~ treat * (age + educ + race + married + nodegree + re74 + re75),
          data    = m.data,
          weights = weights)

# Use marginaleffects for the average treatment effect with cluster-by-subclass SEs
avg_comparisons(fit,
                variables = "treat",
                vcov = ~subclass,
                newdata = subset(m.data, treat == 1))   # ATT
```

```
 Estimate Std. Error    z Pr(>|z|)   S 2.5 % 97.5 %
     1977        704 2.81  0.00501 7.6   596   3357

Term: treat
Type: response
Comparison: 1 - 0
```

The full-matching ATT is \$1,977 with a subclass-clustered standard error of \$704, a 95% interval of $[596, 3357]$, and $p = 0.005$. Training raised 1978 earnings for the trained.

The point is not that matching mechanically produces the "right" answer. The point is that after improving balance, the estimate is much less driven by obvious covariate differences.

7.6. Balance plots

`cobalt::love.plot` is the easiest way to see the balance change. It plots standardized mean differences before and after matching.

```
love.plot(m.full,
          stats = "mean.diffs",
          thresholds = c(m = 0.1),
          abs = TRUE,
          var.order = "unadjusted",
          title = "Balance plot - Full matching")
```

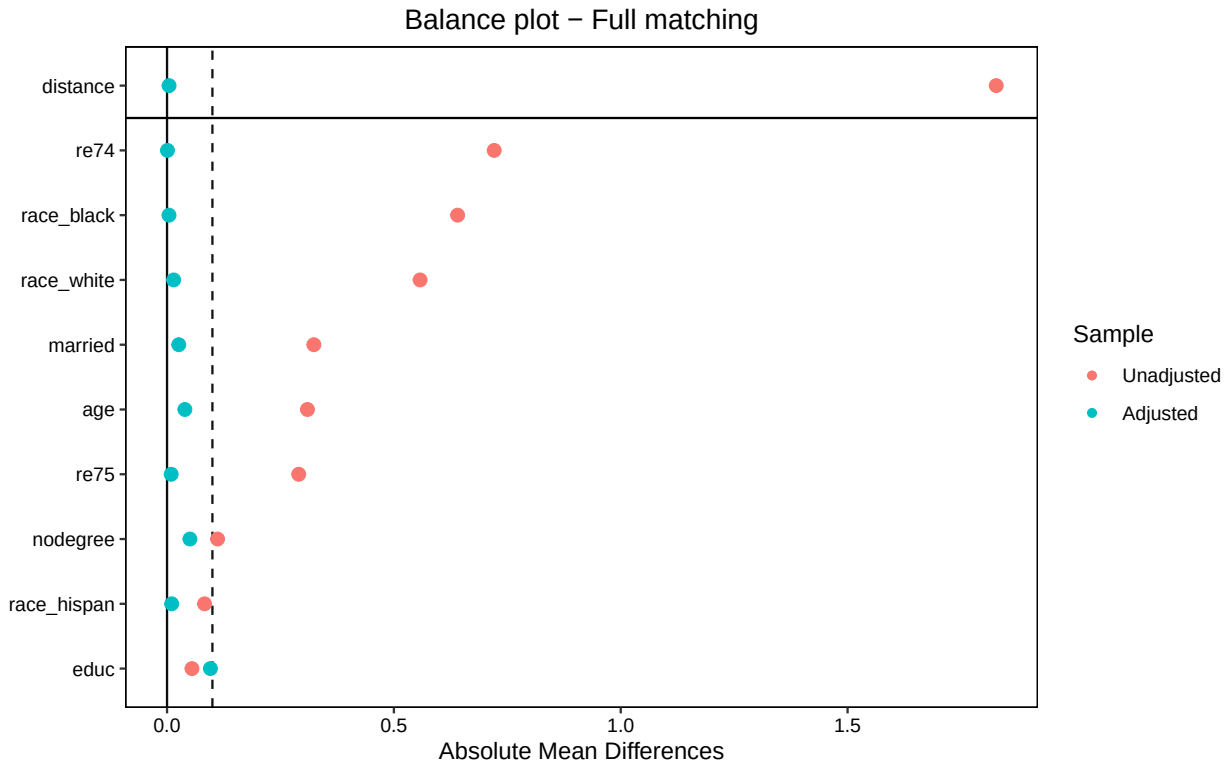


Figure 7.1.: Balance before and after full matching, as absolute standardized mean differences.

Each variable appears twice, unadjusted and adjusted, with the variables ordered by their unadjusted difference and differences plotted in absolute value. Every point moves toward zero except `educ`, which starts at 0.055 and ends at 0.096 — it was already balanced, and full matching traded a little of it for the rest. `re74`, the worst offender before matching at −0.72, moves furthest, ending at −0.0009.

A useful rule is that standardized mean differences should be below 0.1, or below 0.05 if we want to be stricter.

7.7. Matching with replacement vs without

By default a control observation can be used only once. With replacement, the same good control can be used several times. This often improves balance when overlap is weak, but the effective sample size becomes smaller.

```
m.repl <- matchit(treat ~ age + educ + race + married + nodegree + re74 + re75,
                  data = lalonde,
                  method = "nearest",
                  distance = "glm",
                  replace = TRUE)
bal.tab(m.repl, thresholds = c(m = 0.1))
```

```
Balance Measures
                Type Diff.Adj        M.Threshold
distance    Distance   0.0044     Balanced, <0.1
age          Contin.   0.2395 Not Balanced, >0.1
educ         Contin.  -0.0161     Balanced, <0.1
race_black    Binary   0.0054     Balanced, <0.1
race_hispan   Binary  -0.0054     Balanced, <0.1
race_white    Binary   0.0000     Balanced, <0.1
married       Binary   0.0595     Balanced, <0.1
nodegree      Binary   0.0054     Balanced, <0.1
re74         Contin.  -0.0493     Balanced, <0.1
re75         Contin.   0.0087     Balanced, <0.1

Balance tally for mean differences
                   count
Balanced, <0.1         9
Not Balanced, >0.1     1

Variable with the greatest mean difference
 Variable Diff.Adj        M.Threshold
      age   0.2395 Not Balanced, >0.1

Sample sizes
                     Control Treated
All                   429.       185
Matched (ESS)          46.31     185
Matched (Unweighted)   82.       185
Unmatched             347.         0
```

With replacement, nine of ten contrasts pass, against five of nine without. `age` is the exception, at 0.240. Only 82 distinct controls are used, and their effective sample size is 46.3. Compared with the 1:1 match without replacement, which used 185 distinct controls, the same good controls are being reused.

If matching without replacement cannot balance the sample, matching with replacement is a reasonable next step.

7.8. ATE, ATT, ATC — choose carefully

`MatchIt` usually targets the ATT. That means we are asking what treatment did for the treated units. For ATE, use `estimand = "ATE"` and a method that keeps the whole sample, such as full matching or weighting. For ATC, use `estimand = "ATC"`. These are not just software options; they are different causal questions.

7.9. When matching fails

Matching fails in predictable ways:

- High-dimensional matching is hard. Propensity scores reduce dimension, but they do not create overlap.
- If some values of X have no treated or no control units, no matching method can recover the missing comparison.
- Propensity-score matching depends on the propensity-score model. If that model is bad, the matching can be bad too.
- Matching does not address hidden confounding. Rosenbaum bounds, discussed in the Sensitivity Analysis chapter, ask how strong hidden bias would have to be to change the conclusion.

7.10. Matching vs weighting

Matching and weighting do similar jobs. Matching chooses comparable observations. Weighting keeps observations but changes their contribution so the treated and control covariate distributions become closer.

When overlap is good, both approaches often give similar answers. When overlap is bad, both approaches become fragile. Then the honest choices are to change the target population, report the lack of overlap, or use a doubly-robust estimator such as AIPW or TMLE while still being clear about the support problem.

7.11. Weighting Alternatives

`WeightIt` implements several weighting methods. The companion blog chapter on weighting has more detail. Here I use the same Lalonde data so the comparison is easy.

7.11.1. Inverse probability of treatment weighting (IPW)

The baseline method is IPW. Estimate the propensity score, then weight by the inverse probability of receiving the observed treatment.

```
w_ipw <- weightit(treat ~ age + educ + race + married + nodegree + re74 + re75,
                  data = lalonde, estimand = "ATT", method = "glm")
bal.tab(w_ipw, stats = c("m"), thresholds = c(m = 0.05))
```

```
Balance Measures
                Type Diff.Adj         M.Threshold
prop.score  Distance  -0.0205     Balanced, <0.05
age          Contin.   0.1188 Not Balanced, >0.05
educ         Contin.  -0.0284     Balanced, <0.05
race_black    Binary  -0.0022     Balanced, <0.05
race_hispan   Binary   0.0002     Balanced, <0.05
race_white    Binary   0.0021     Balanced, <0.05
married       Binary   0.0186     Balanced, <0.05
nodegree      Binary   0.0184     Balanced, <0.05
re74         Contin.  -0.0021     Balanced, <0.05
re75         Contin.   0.0110     Balanced, <0.05

Balance tally for mean differences
                    count
Balanced, <0.05         9
Not Balanced, >0.05     1

Variable with the greatest mean difference
 Variable Diff.Adj         M.Threshold
      age   0.1188 Not Balanced, >0.05

Effective sample sizes
           Control Treated
Unadjusted  429.       185
Adjusted     99.82     185
```

Note the stricter 0.05 threshold from here on. IPW balances nine of ten; `age` fails at 0.119. The control effective sample size is 99.8 out of 429, better than any of the matching methods above.

7.11.2. Covariate balancing propensity score (CBPS)

CBPS estimates the propensity score while also trying to balance covariate means. So the propensity-score model is not judged only by likelihood; it is also judged by balance.

```
w_cbps <- weightit(treat ~ age + educ + race + married + nodegree + re74 + re75,
                   data = lalonde, estimand = "ATT", method = "cbps")
bal.tab(w_cbps, stats = c("m"), thresholds = c(m = 0.05))
```

```
Balance Measures
                Type Diff.Adj     M.Threshold
prop.score  Distance   -0.018 Balanced, <0.05
age          Contin.    0.000 Balanced, <0.05
educ         Contin.   -0.000 Balanced, <0.05
race_black    Binary   -0.000 Balanced, <0.05
race_hispan   Binary   -0.000 Balanced, <0.05
race_white    Binary    0.000 Balanced, <0.05
married       Binary   -0.000 Balanced, <0.05
nodegree      Binary   -0.000 Balanced, <0.05
re74         Contin.   -0.000 Balanced, <0.05
re75         Contin.   -0.000 Balanced, <0.05

Balance tally for mean differences
                    count
Balanced, <0.05        10
Not Balanced, >0.05     0

Variable with the greatest mean difference
 Variable Diff.Adj     M.Threshold
     re74       -0 Balanced, <0.05

Effective sample sizes
           Control Treated
Unadjusted  429.       185
Adjusted     98.46     185
```

Every covariate mean difference is zero to three decimals, and the control effective sample size is 98.5.

7.11.3. Entropy balancing

Entropy balancing chooses weights so that the weighted control group matches the treated group on covariate means. It is useful because the balance constraint is explicit.

```
w_ebal <- weightit(treat ~ age + educ + race + married + nodegree + re74 + re75,
                   data = lalonde, estimand = "ATT", method = "ebal")
bal.tab(w_ebal, stats = c("m"), thresholds = c(m = 0.05))
```

```
Balance Measures
                Type Diff.Adj     M.Threshold
```

```
age         Contin.       -0 Balanced, <0.05
educ        Contin.       -0 Balanced, <0.05
race_black   Binary        0 Balanced, <0.05
race_hispan  Binary        0 Balanced, <0.05
race_white   Binary       -0 Balanced, <0.05
married      Binary       -0 Balanced, <0.05
nodegree     Binary        0 Balanced, <0.05
re74        Contin.       -0 Balanced, <0.05
re75        Contin.       -0 Balanced, <0.05

Balance tally for mean differences
                     count
Balanced, <0.05          9
Not Balanced, >0.05      0

Variable with the greatest mean difference
 Variable Diff.Adj     M.Threshold
     re74       -0 Balanced, <0.05

Effective sample sizes
           Control Treated
Unadjusted  429.       185
Adjusted     98.46     185
```

The standardized mean differences are essentially zero because entropy balancing imposes mean balance directly. The effective sample size is 98.5.

That is the same table CBPS produced, and it is not a coincidence: the two sets of control weights are numerically identical here, agreeing to twelve decimal places. Just-identified CBPS for the ATT solves the exact mean-balance moment conditions with a logistic link, which makes the control weights proportional to $\exp(x'\lambda)$. Entropy balancing minimises the Kullback-Leibler divergence from uniform weights subject to the same exact mean-balance constraints, and its solution has the same $\exp(x'\lambda)$ form. Same constraints, same functional form, same weights. The two methods are motivated differently and arrive at one answer.

7.11.4. Energy balancing

Energy balancing targets the whole covariate distribution, not just means. It is more ambitious than entropy balancing, and usually more computationally expensive.

```r
w_energy <- weightit(treat ~ age + educ + race + married + nodegree + re74 + re75,
                     data = lalonde, estimand = "ATT", method = "energy")
bal.tab(w_energy, stats = c("m"), thresholds = c(m = 0.05))
```

```
Balance Measures
               Type Diff.Adj     M.Threshold
```

```
age         Contin.  -0.0016 Balanced, <0.05
educ        Contin.   0.0106 Balanced, <0.05
race_black   Binary   0.0060 Balanced, <0.05
race_hispan  Binary  -0.0008 Balanced, <0.05
race_white   Binary  -0.0053 Balanced, <0.05
married      Binary  -0.0011 Balanced, <0.05
nodegree     Binary   0.0050 Balanced, <0.05
re74        Contin.  -0.0021 Balanced, <0.05
re75        Contin.   0.0226 Balanced, <0.05

Balance tally for mean differences
                    count
Balanced, <0.05         9
Not Balanced, >0.05     0

Variable with the greatest mean difference
 Variable Diff.Adj     M.Threshold
     re75   0.0226 Balanced, <0.05

Effective sample sizes
           Control Treated
Unadjusted  429.       185
Adjusted     41.82     185
```

All nine contrasts pass, the largest being `re75` at 0.023 — close to zero but not exactly zero, because energy balancing does not impose mean balance as a constraint. The control effective sample size is 41.8, less than half of entropy balancing's 98.5. That is the price of targeting the whole covariate distribution rather than its means.

7.11.5. Estimation with weighted-aware regression

After computing weights, use `lm_weightit()` instead of plain `lm` if we want standard errors that account for the estimated weights.

```
fit_ebal <- lm_weightit(
  re78 ~ treat * (age + educ + race + married + nodegree + re74 + re75),
  data     = lalonde,
  weightit = w_ebal
)

avg_comparisons(fit_ebal, variables = "treat",
                newdata = subset(lalonde, treat == 1))   # ATT
```

```
 Estimate Std. Error    z Pr(>|z|)   S 2.5 % 97.5 %
     1273        770 1.65   0.0983 3.3  -236   2783
```

```
Term: treat
Type: probs
Comparison: 1 - 0
```

The entropy-balanced ATT is \$1,273 with a sandwich standard error of \$770, a 95% interval of $[-236, 2783]$, and $p = 0.098$. The interval includes zero.

7.11.6. When to use each weighting method

Method	Strengths	Weaknesses
IPW (glm)	Familiar; well-studied	Sensitive to PS misspecification; extreme weights
CBPS	Balance-constrained PS; robust to model misspecification	Slower; can fail with many covariates
Entropy balancing	Exact mean balance; doubly robust for ATT	Balances means only, not distributions
Energy balancing	Balances entire distribution	Computationally heavier; less mature theory

For applied work, entropy balancing is often a good default because it is simple and the balance diagnostics are easy to explain. Energy balancing is useful when matching the whole covariate distribution is important.

7.11.7. Comparing matching to weighting

The matching and weighting examples here target the same ATT, so put the two estimates next to each other rather than reporting whichever came last:

```
cmp <- rbind(
  transform(as.data.frame(
    avg_comparisons(fit, variables = "treat", vcov = ~subclass,
                    newdata = subset(m.data, treat == 1))),
    method = "Full matching (subclass-clustered SE)"),
  transform(as.data.frame(
    avg_comparisons(fit_ebal, variables = "treat",
                    newdata = subset(lalonde, treat == 1))),
    method = "Entropy balancing (sandwich SE)")
)

cmp[, c("method", "estimate", "std.error", "conf.low", "conf.high", "p.value")] |>
  knitr::kable(digits = c(0, 0, 0, 0, 0, 3),
               caption = "Two routes to the same ATT on the Lalonde data")
```

Table 7.2.: Two routes to the same ATT on the Lalonde data

method	estimate	std.error	conf.low	conf.high	p.value
Full matching (subclass-clustered SE)	1977	704	596	3357	0.005
Entropy balancing (sandwich SE)	1273	770	-236	2783	0.098

The gap is instructive. The two point estimates differ by roughly one standard error, and one interval excludes zero while the other does not — a reader who sees only one of these tables would draw a different conclusion about whether the programme worked. Note also that the ranking on balance does not settle it: entropy balancing imposes *exact* mean balance (every standardized difference above is at most 0.023), whereas full matching leaves `educ` at -0.096, yet it is entropy balancing that gives the smaller estimate and the wider interval. Better mean balance is not the same thing as a more precise or more credible answer.

This is model dependence, not a bug in either method. In an applied paper I would report balance for each method, not just the final coefficient, and I would report both estimates. If several reasonable methods give similar balance and similar estimates, the result is more credible. If they differ sharply, as here, the problem is usually weak overlap or model dependence, and the honest summary says so instead of picking the number that crosses the significance line.

For more details on `MatchIt` and the Stata `teffects` commands, see the matching blog chapter and treatment-effects in Stata.

7.12. Summary

- Matching is for selection on observables. It does not fix unobserved confounding.
- The main diagnostic is covariate balance, not the treatment-effect coefficient.
- Propensity-score matching is simple, but full matching or weighting often balances better.
- If there is no overlap, change the estimand or restrict the sample. Do not hide the support problem.

8. Nonparametric Causal Methods

Related reading: For a hands-on TMLE simulation comparing super-learner libraries and assessing model misspecification, see the companion blog chapter on TMLE in *Topics on Econometrics and Causal Inference*.

8.1. Why Do We Need Nonparametric Methods?

- Parametric models are often mis-specified
- Nonparametric models are more flexible; less assumptions
- What kind of nonparametrics are good?

Considerations:

- Plug-in (g-computation) estimators are easy.
- The problem is not the decomposition into $E[Y(0)]$ and $E[Y(1)]$ itself: with correctly specified models and regularity, a plug-in estimator can be efficient. The real issue is that a flexible/regularized nuisance estimate carries a first-order *regularization bias* into the plug-in target, and the plug-in lacks the augmentation (correction) term that would make the estimator first-order insensitive to nuisance error.
- So they can be consistent, but the convergence rate inherits the (often slow) nuisance rate, which is especially bad in high dimensions.
- Ideally we remove that first-order sensitivity using an orthogonal influence function, and get $\sqrt{n}$ efficiency.
- Influence-function-based (one-step/AIPW/TMLE) estimators are shown to be efficient and orthogonal to first-order nuisance error.

8.2. Influence Function Based Estimators

Estimators based on efficient influence function are often $\sqrt{n}$ consistent, meaning they converge to the truth at the $1/\sqrt{n}$ rate. This means the estimation error not only goes to 0 as n goes to infinity, but does so as fast as $1/\sqrt{n}$ goes to 0. Why does this matter? Because we have limited sample size, when we are based on asymptotics, the faster the bias goes to 0 the smaller sample size we need.

If we have an estimator such that

$$\sqrt{n}(\hat{\psi} - \psi) = \sqrt{n}P_n(\phi) + o_P(1) \to N(0, var(\phi)) \tag{8.1}$$

Then this estimator is $\sqrt{n}$ consistent and asymptotically normal; and ϕ is the influence function. The efficient influence function is the one with variance at the lower bound (similar to parametric case for Cramer-Rao lower bound).

Unfortunately there is no universal formula for efficient influence function for every problem. We need to derive it for each problem. For example, ATE has an IF based estimator, but ATT needs a different formula; and LATE has a different formula too, etc. Deriving them is hard!

Take the mean potential outcome under treatment, $\psi_1 = E\{E(Y|X, A = 1)\} = E[Y(1)]$ (the ATE is the difference $\psi_1 - \psi_0$ of the two arm means, and its EIF is the difference of the two arm-specific EIFs below). The efficient influence function for ψ_1 is

$$\phi(Z; P) = \frac{A}{\pi(X)}\{Y - \mu(X)\} + \mu(X) - \psi_1 \tag{8.2}$$

where $\pi(X) = P(A = 1|X)$ the propensity score model, $\mu(X) = E(Y|X, A = 1)$, the outcome model.

For example, an IF-based bias-corrected estimator

$$\hat{\psi} = P_n[\frac{A}{\hat{\pi}(X)}\{Y - \hat{\mu}(X)\} + \hat{\mu}(X)] \tag{8.3}$$

where P_n means sample mean. This is the familiar AIPW estimator. It is known to be doubly robust.

Since we know the influence function, we can estimate the asymptotic variance. The uncentered summand $\frac{A}{\hat{\pi}(X)}\{Y - \hat{\mu}(X)\} + \hat{\mu}(X)$ differs from ϕ only by the constant ψ_1, so the two have the same variance. We can use any estimators for the nuisance parameters; the sample mean of the summand is then $\hat{\psi}_1$, its sample variance estimates $var(\phi)$, and the standard error of $\hat{\psi}_1$ is $sd(\hat{\phi})/\sqrt{n}$ (differencing the two arms gives the ATE).

In this estimator, both $\hat{\mu}$ and $\hat{\pi}$ can be done using any estimator, such as machine learning. One caveat: $\sqrt{n}$ inference with flexible ML nuisances requires either Donsker-type complexity restrictions or sample splitting (cross-fitting), where the nuisance models are fit on one fold and evaluated on another. The manual estimators below use in-sample predictions for simplicity; the `npcausal` calls cross-fit with `nsplits=2`, and the DoubleML section below returns to sample splitting as a key ingredient.

8.2.1. Simulation Setup

The aim is to run one estimand through five estimators on a single data set — a plug-in outcome regression, AIPW computed by hand, AIPW from `npcausal`, TMLE, and DoubleML — and compare them against a truth we can compute exactly.

We simulate $n = 1000$ people with four covariates: $w_1, w_2 \sim$ Bernoulli(0.5), $w_3 \sim U(0, 4)$ and $w_4 \sim U(0, 5)$, the last two rounded to three decimals. Treatment is $A \sim$ Bernoulli($\Lambda(-0.4 + 0.2w_2 + 0.15w_3 + 0.2w_4 + 0.15w_2w_4)$). Both potential outcomes are Bernoulli with the same logit index, $Y(d) \sim$ Bernoulli($\Lambda(-1 + d - 0.1w_1 + 0.3w_2 + 0.25w_3 + 0.2w_4 + 0.15w_2w_4)$), and we observe $Y = AY(1) + (1 - A)Y(0)$.

Treatment shifts the logit index by exactly 1, so the individual effect $\Lambda(\text{lin} + 1) - \Lambda(\text{lin})$ varies across people even though the shift is constant. Note also that w_1 enters the outcome but not the treatment, and that both the propensity and the outcome carry a w_2w_4 interaction. Neither nuisance function is linear in the covariates, which is why flexible estimators are worth using here.

```
set.seed(1)
n=1000
w1 <- rbinom(n, size=1, prob=0.5)
w2 <- rbinom(n, size=1, prob=0.5)
w3 <- round(runif(n, min=0, max=4), digits=3)
w4 <- round(runif(n, min=0, max=5), digits=3)
A  <- rbinom(n, size=1, prob= plogis(-0.4 + 0.2*w2 + 0.15*w3 + 0.2*w4 + 0.15*w2*w4))
Y.1 <- rbinom(n, size=1, prob= plogis(-1 + 1 -0.1*w1 + 0.3*w2 + 0.25*w3 + 0.2*w4 + 0.15*w2*w4)
Y.0 <- rbinom(n, size=1, prob= plogis(-1 + 0 -0.1*w1 + 0.3*w2 + 0.25*w3 + 0.2*w4 + 0.15*w2*w4)
Y <- Y.1*A + Y.0*(1 - A)
W<-data.frame(cbind(w1,w2,w3,w4))
data <-  data.frame(w1, w2, w3, w4, A, Y, Y.1, Y.0)
True_Psi <- mean(data$Y.1-data$Y.0);
cat(" True_Psi:", True_Psi)
```

```
 True_Psi: 0.21
```

The line above reports `mean(Y.1 - Y.0)` for this particular draw. That is a legitimate quantity — the *sample* average treatment effect — but it is not a fixed benchmark, and it is easy to mistake for one. The potential outcomes are binary, so at $n = 1000$ this realized average has a sampling standard deviation of about 0.020: two runs of the same DGP under different seeds can differ by 0.04 without anything being wrong. (The R and Julia editions of this chapter print 0.21 and 0.184 for exactly this reason.) Estimator standard errors in this chapter are around 0.03, so a noisy target of that size would swamp the comparison we are trying to make.

The *population* ATE is a definite number, and since the DGP is fully specified we can integrate it out rather than simulate it:

```
# w1, w2 ~ Bernoulli(0.5); w3 ~ U(0,4); w4 ~ U(0,5). Average the true individual
# effect plogis(lin0 + 1) - plogis(lin0) over that covariate distribution.
pop_ate <- mean(sapply(list(c(0,0), c(0,1), c(1,0), c(1,1)), function(w)
  integrate(function(w4) sapply(w4, function(u4)
    integrate(function(w3) {
      lin0 <- -1 - 0.1*w[1] + 0.3*w[2] + 0.25*w3 + 0.2*u4 + 0.15*w[2]*u4
      plogis(lin0 + 1) - plogis(lin0)
    }, 0, 4)$value / 4), 0, 5)$value / 5))

cat(sprintf("Realized-sample ATE (this draw): %.4f\n", True_Psi))
```

```
Realized-sample ATE (this draw): 0.2100
```

```
cat(sprintf("Exact population ATE:           %.4f\n", pop_ate))
```

```
Exact population ATE:           0.2028
```

The realized draw is 0.2100 and the population value is 0.2028. Compare the estimators below against 0.2028.

All of them need estimates of the outcome regression μ and the propensity score π. We estimate both with the same super-learner library: MARS (`SL.earth`), a generalised additive model, the lasso, logistic regression with pairwise interactions, the sample mean, a random forest, and gradient boosting. The super learner takes a cross-validated convex combination of these, so no single functional form has to be right.

```
SL.library <- c("SL.earth","SL.gam","SL.glmnet","SL.glm.interaction", "SL.mean","SL.ranger", "S
```

8.2.2. Plug-in Estimator from Outcome Regression

> ⚠ Warning
>
> **For illustration only – do not use in-sample ML nuisance predictions in applied work.** The manual `mu1hat/mu0hat/pi1hat` estimators below reuse the same data to fit and evaluate the outcome-regression and propensity models (no cross-fitting). With flexible machine-learning nuisance estimators this causes overfitting bias that invalidates the estimator's asymptotic normality and the validity of standard-error calculations based on it (Chernozhukov et al., 2018). Always use sample splitting / cross-fitting with ML nuisances – as the `npcausal` calls below do (`nsplits=2`), and as DoubleML does later in this chapter.

We fit the outcome regression on all four covariates and treatment, then predict each unit's outcome twice, once with A set to 1 and once with A set to 0, and average the difference. This is the plug-in, or g-computation, estimator: no propensity score anywhere.

```
set.seed(1)
mufit <- SuperLearner(Y=data$Y, X=data %>% dplyr::select(w1,w2,w3,w4,A),
                                    SL.library=SL.library, family="binomial")
# predict the PO Y^1
mu1hat<- predict(mufit, newdata=data %>% dplyr::select(w1,w2,w3,w4,A) %>% mutate(A=1))$pred
mu0hat<- predict(mufit, newdata=data %>% dplyr::select(w1,w2,w3,w4,A) %>% mutate(A=0))$pred
mean(mu1hat-mu0hat)
```

```
[1] 0.1824593
```

The plug-in gives 0.1825 against the true 0.2028. It comes with no standard error, and this is the point of the section: the super learner's regularisation bias passes straight into the average, and there is no correction term to remove it.

8.2.3. AIPW ATE

We now add the augmentation term. We fit the propensity score with the same library, form the two arm summands $\frac{A}{\hat{\pi}}(Y-\hat{\mu}_1)+\hat{\mu}_1$ and $\frac{1-A}{1-\hat{\pi}}(Y-\hat{\mu}_0)+\hat{\mu}_0$, and average their difference. The standard

error is the sample standard deviation of that difference divided by $\sqrt{n}$, the influence-function standard error. The printed vector is the estimate followed by the lower and upper 95% bounds.

```
set.seed(123)
pifit <- SuperLearner(Y=data$A, X=data %>% dplyr::select(w1,w2,w3,w4),
                      SL.library=SL.library, family="binomial")
pi1hat <- pifit$SL.predict
pi0hat <- 1- pi1hat
IF.1<-((data$A/pi1hat)*(data$Y-mu1hat)+mu1hat)
IF.0<-(((1-data$A)/pi0hat)*(data$Y-mu0hat)+mu0hat)
IF<-IF.1-IF.0
aipw.1<-mean(IF.1);aipw.0<-mean(IF.0)
aipw.manual<-aipw.1-aipw.0
ci.lb<-mean(IF)-qnorm(.975)*sd(IF)/sqrt(length(IF))
ci.ub<-mean(IF)+qnorm(.975)*sd(IF)/sqrt(length(IF))
res.manual.aipw<-c(aipw.manual,ci.lb, ci.ub)
res.manual.aipw
```

```
[1] 0.1893510 0.1291976 0.2495044
```

AIPW gives 0.1894 with a 95% interval of $[0.1292, 0.2495]$, which covers the true 0.2028. The augmentation term moved the estimate from 0.1825 to 0.1894 and supplied an interval the plug-in could not.

8.2.4. AIPW ATE Using npcausal

The same estimator from a package, with one change that matters: `nsplits=2` cross-fits the nuisances, so each unit's $\hat{\mu}$ and $\hat{\pi}$ come from a model fitted without it. The output reports both arm means and their difference.

```
library(npcausal)
set.seed(1)
aipw<- ate(y=Y, a=A, x=W, nsplits=2, sl.lib=SL.library)
```

```
  |
  |                                                                      |   0%
  |
  |=========                                                             |  12%
  |
  |==================                                                    |  25%
  |
  |==========================                                            |  38%
  |
  |==================================                                    |  50%
```

```
  |
  |=========================================                             |  62%
  |
  |=================================================                     |  75%
  |
  |=========================================================             |  88%
  |
  |=====================================================================| 100%
     parameter       est         se     ci.ll     ci.ul pval
1      E{Y(0)} 0.5624540 0.02695873 0.5096149 0.6152931    0
2      E{Y(1)} 0.7567899 0.01739370 0.7226983 0.7908815    0
3 E{Y(1)-Y(0)} 0.1943359 0.03160068 0.1323985 0.2562732    0
```

Cross-fitting gives $E[Y(0)] = 0.5625$, $E[Y(1)] = 0.7568$, and a difference of 0.1943 with a standard error of 0.0316 and an interval of $[0.1324, 0.2563]$. That is closer to the true 0.2028 than the in-sample version, and the standard error is the one to trust.

8.2.5. AIPW ATT

The ATT has its own efficient influence function. With $p = P(A = 1)$ and $\mu_0(X) = E[Y|X, A = 0]$,

$$\phi_{ATT}(Z) = \frac{1}{p}\left\{A(Y - \mu_0(X)) - (1 - A)\frac{\pi(X)}{1 - \pi(X)}(Y - \mu_0(X))\right\} - \frac{A}{p}\psi_{ATT}. \tag{8.4}$$

Treated units contribute their outcome-model residual; control units enter with the odds weight $\pi(X)/(1 - \pi(X))$, which reweights them to the covariate distribution of the treated. Note the correction term uses the CONTROL units — an estimator computed on the treated subsample alone would silently reduce to regression adjustment.

So we fit μ_0 on the controls only, predict it for everyone, and build the summand above from the propensity score already estimated. The printed vector is again the estimate and the two 95% bounds.

```
# npcausal (attached above) exports its own SL.ranger, whose fit object is
# incompatible with SuperLearner's predict method; pin SuperLearner's wrapper
SL.ranger <- SuperLearner::SL.ranger
data2 <- data %>%
  dplyr::mutate(pi0hat=as.numeric(pi0hat), pi1hat=as.numeric(pi1hat))
set.seed(1)
data0 <- data2 %>% dplyr::filter(A==0)
mufit <- SuperLearner(Y=data0$Y, X=data0 %>% dplyr::select(w1,w2,w3,w4),
                                      SL.library=SL.library, family="binomial")
# predict mu0 for ALL units, treated and control
mu0hat_all <- as.numeric(predict(mufit, newdata=data2 %>% dplyr::select(w1,w2,w3,w4))$pred)
p1 <- mean(data2$A)
resid0 <- data2$Y - mu0hat_all
# uncentered IF-based ATT estimator
```

```
phi <- (data2$A*resid0 -
        (1-data2$A)*(data2$pi1hat/data2$pi0hat)*resid0)/p1
aipw.manual.att <- mean(phi)
# centered EIF for the variance
phi_c <- phi - (data2$A/p1)*aipw.manual.att
ci.lb<-aipw.manual.att-qnorm(.975)*sd(phi_c)/sqrt(length(phi_c))
ci.ub<-aipw.manual.att+qnorm(.975)*sd(phi_c)/sqrt(length(phi_c))
res.manual.aipw.att<-c(aipw.manual.att,ci.lb, ci.ub)
res.manual.aipw.att
```

```
[1] 0.1817775 0.1204417 0.2431132
```

The ATT comes out at 0.1818 with an interval of $[0.1204, 0.2431]$.

8.2.6. AIPW ATT Using npcausal

```
library(npcausal)
set.seed(1)
aipw<- att(y=Y, a=A, x=W, nsplits=2, sl.lib=SL.library)
```

```
  |
  |                                                                      |   0%
  |
  |==================                                                    |  25%
  |
  |===================================                                   |  50%
  |
  |====================================================                  |  75%
  |
  |======================================================================| 100%
      parameter       est         se      ci.ll     ci.ul  pval
1      E(Y|A=1) 0.7668232 0.01686602 0.73376577 0.7998806 0.000
2   E{Y(0)|A=1} 0.5938218 0.05052032 0.49480194 0.6928416 0.000
3 E{Y-Y(0)|A=1} 0.1730014 0.05289367 0.06932981 0.2766730 0.001
```

Cross-fitted, the ATT is 0.1730 with a standard error of 0.0529 and an interval of $[0.0693, 0.2767]$.

The ATT is a different estimand from the ATE, and here they genuinely differ: the treatment probability rises with w_2, w_3 and w_4, and the individual effect varies with those same covariates, so averaging over the treated is not the same as averaging over everyone. The two estimates are 0.1730 and 0.1943, and with standard errors of 0.0529 and 0.0316 the data cannot separate them. Note that the ATT standard error is two-thirds larger, because only the treated units contribute to the average.

8.3. TMLE

TMLE is a variant of IF based estimator. It is also a doubly robust estimator. See Mark van der Laan and his coauthors for the recent development of TMLE.

TMLE targets the same ATE with the same nuisance library, so we should expect it to agree with `npcausal`.

```
library(tmle)
TMLE<- tmle(Y=data$Y,A=data$A,W=W, family="binomial", Q.SL.library=SL.library, g.SL.library=SL

TMLE$estimates$ATE
```

```
$psi
[1] 0.1894847

$var.psi
[1] 0.0009250159

$CI
[1] 0.1298742 0.2490951

$pvalue
[1] 4.659488e-10

$bs.var
[1] NA

$bs.CI.twosided
[1] NA NA

$bs.CI.onesided.lower
[1] -Inf   NA

$bs.CI.onesided.upper
[1]  NA Inf
```

TMLE gives 0.1893 with an interval of $[0.1288, 0.2498]$, against AIPW's 0.1894 and `npcausal`'s 0.1943. The three agree to well within a standard error, which is what the theory predicts: they share an efficient influence function and differ only in how they solve its estimating equation.

8.4. DoubleML

The R package DoubleML implements the double/debiased machine learning framework of Chernozhukov et al. (2018). It provides functionalities to estimate parameters in causal models based on

machine learning methods. The double machine learning framework consists of three key ingredients: Neyman orthogonality, High-quality machine learning estimation and Sample splitting.

They consider a partially linear model:

$$y_i = \theta d_i + g_0(x_i) + \eta_i \tag{8.5}$$

$$d_i = m_0(x_i) + v_i \tag{8.6}$$

This model is quite general, except it does not allow interaction of d and x; therefore no heterogeneous treatment effect across x. But "DoubleML" implements more than partially linear model, it actually allows for heterogeneous treatment effects, in models such as interactive regression model.

The basic idea of doubleML is to use any machine learning algorithm to estimate outcome equation ($l_0(X) = E(Y|X)$) and treatment equation ($m_0(X) = E(D|X)$). Then get the residuals, namely $\tilde{Y} = Y - \hat{l}_0(X)$ and $\tilde{D} = D - \hat{m}_0(X)$.

Then regress $\tilde{Y}$ on $\tilde{D}$. Based on FWL theorem, you get $\hat{\theta}$.

An important component here is to specify a Neyman-orthogonal score function. In the case of PLR, the "partialling-out" score (the DoubleML default, matching the residual-on-residual regression above) is

$$\psi(Z; \theta, \eta) = (Y - l(X) - \theta(D - m(X)))(D - m(X)) \tag{8.7}$$

The estimator $\hat{\theta}$ solves the equation that the sample mean of this score function is 0.

The estimator's variance comes from the usual sandwich formula built on the score, $\hat{\sigma}^2 = \hat{J}^{-1}\,\widehat{E[\psi^2]}\,\hat{J}^{-1}$ with $\hat{J} = -\frac{1}{n}\sum_i (D_i - \hat{m}(X_i))^2$.

We set up the data object first, naming the outcome, the treatment and the four covariates:

```
library(DoubleML)
dml_data = DoubleMLData$new(data,
                            y_col = "Y",
                            d_cols = "A",
                            x_cols = c("w1","w2","w3","w4"))
print(dml_data)
```

```
================= DoubleMLData Object ==================


------------------ Data summary      ------------------
Outcome variable: Y
Treatment variable(s): A
Covariates: w1, w2, w3, w4
Instrument(s):
Selection variable:
No. Observations: 1000
```

Both nuisance functions are estimated by a random forest with 500 trees, depth at most 5, and a minimum node size of 2. DoubleML cross-fits them and solves the partialling-out score.

```
library(mlr3)
library(mlr3learners)
# suppress messages from mlr3 package during fitting
lgr::get_logger("mlr3")$set_threshold("warn")
learner = lrn("regr.ranger", num.trees=500,  max.depth=5, min.node.size=2)
ml_l = learner$clone()
ml_m = learner$clone()
learner = lrn("regr.glmnet")
ml_g_sim = learner$clone()
ml_m_sim = learner$clone()
set.seed(123)
obj_dml_plr = DoubleMLPLR$new(dml_data, ml_l=ml_l, ml_m=ml_m)
obj_dml_plr$fit()
obj_dml_plr$summary()
```

```
Estimates and significance testing of the effect of target variables
  Estimate. Std. Error t value Pr(>|t|)
A   0.19257    0.03136    6.14 8.24e-10 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

DoubleML returns 0.19257 with a standard error of 0.03136, in line with AIPW's 0.1894 and TMLE's 0.1893.

One caveat on what this number is. The partially linear model has no treatment-covariate interaction, so its θ is not the ATE when effects are heterogeneous, and in this DGP they are: the individual effect $\Lambda(\text{lin}+1)-\Lambda(\text{lin})$ depends on the covariates. By the Frisch-Waugh-Lovell argument behind the partialling-out score, θ is the average of the conditional effects weighted by the conditional variance of treatment, $\text{Var}(A \mid X)$, so people whose treatment is hardest to predict count most. It equals the ATE only when the effect is constant. Here the two are close, 0.193 against 0.203, but that is a property of this DGP and not a general result.

9. Heterogeneous Treatment Effects with Machine Learning

```
library(tidyverse)
library(grf)
library(policytree)
library(ggplot2)
```

The estimands chapter defined the conditional average treatment effect

$$\text{CATE}(x) = \mathbb{E}[Y(1) - Y(0) \mid X = x] \tag{9.1}$$

as the effect of treatment for units with covariates $X = x$. In the simulation we could compute it from known potential outcomes. In real data we do not observe both $Y(0)$ and $Y(1)$, so CATE has to be estimated from (X, D, Y).

I use four kinds of tools:

- **Meta-learners**: S-, T-, X-, R-, and DR-learners.
- **Causal forests**: random forests designed for CATE estimation.
- **BLP and CLAN**: simple summaries and tests of heterogeneity.
- **Policy trees**: rules for deciding who should be treated.

9.1. The data-generating process

We simulate $n = 5000$ units with five covariates, each drawn independently from $U(0, 1)$. Treatment is $D \sim \text{Bernoulli}(\Lambda(-0.3 + 1.5X_2))$. The CATE function is the one from the estimands chapter, $\tau(x) = 1 + 2x_1$. The potential outcomes are $Y(0) = 0.5X_2 + \varepsilon$ with $\varepsilon \sim N(0, 1)$ and $Y(1) = Y(0) + \tau(X)$, and we observe $Y = DY(1) + (1 - D)Y(0)$.

Two covariates do the work and three are irrelevant. X_1 modifies the effect but has nothing to do with selection. X_2 drives both treatment and the outcome, so it is a backdoor confounder, and it is observed. X_3, X_4 and X_5 enter neither equation. The population ATE is $\int_0^1 (1 + 2x)\, dx = 2$.

The aim is to hand six estimators the same data and see which recover $\tau(x)$, judged by the correlation between estimated and true individual effects.

```r
set.seed(42)
n <- 5000
p <- 5      # 5 covariates: only X1 is the effect modifier; X2..X5 are noise

X <- matrix(runif(n * p), n, p)
colnames(X) <- paste0("X", 1:p)
# Treatment depends on the OBSERVED covariate X2 (a backdoor confounder we
# adjust for), so unconfoundedness given X holds and the estimators are
# consistent for the true ATE/CATE.
ps  <- plogis(-0.3 + 1.5 * X[, 2])
D   <- rbinom(n, 1, ps)
tau <- 1 + 2 * X[, 1]          # CATE varies along X1 only
Y0  <- 0.5 * X[, 2] + rnorm(n)
Y1  <- Y0 + tau
Y   <- ifelse(D == 1, Y1, Y0)

cat(sprintf("n = %d, true ATE = %.3f\n", n, mean(tau)))
```

```
n = 5000, true ATE = 2.006
```

```r
cat(sprintf("True CATE at X1 = 0.2: %.2f\n", 1 + 2 * 0.2))
```

```
True CATE at X1 = 0.2: 1.40
```

```r
cat(sprintf("True CATE at X1 = 0.8: %.2f\n", 1 + 2 * 0.8))
```

```
True CATE at X1 = 0.8: 2.60
```

The realized ATE in this draw is 2.006 against a population value of 2, and the true CATE runs from 1.40 at $X_1 = 0.2$ to 2.60 at $X_1 = 0.8$. Because X_2 is observed and every estimator below conditions on it, unconfoundedness holds and all of them are consistent for the true CATE and ATE. Any differences we see are finite-sample behaviour, not bias from a missing variable.

9.2. Meta-learners

A meta-learner is a recipe that turns a regression method into a CATE estimator. The regression method can be a random forest, boosting, neural network, or a simple linear model.

9.2.1. S-learner ("single")

Fit one regression of Y on (D, X), then predict the difference between $D = 1$ and $D = 0$ at each x:

$$\hat{\tau}^S(x) = \hat{\mu}(1, x) - \hat{\mu}(0, x). \tag{9.2}$$

```
# Use a flexible learner - random forest from grf::regression_forest
XD_train <- cbind(D = D, X)
fit_S <- regression_forest(XD_train, Y, num.trees = 500)

# Counterfactual predictions at D = 1 and D = 0 for every X
mu1 <- predict(fit_S, cbind(D = rep(1, n), X))$predictions
mu0 <- predict(fit_S, cbind(D = rep(0, n), X))$predictions
tau_S <- mu1 - mu0

cor_S <- cor(tau_S, tau)
cat(sprintf("S-learner CATE correlation with truth: %.3f\n", cor_S))
```

```
S-learner CATE correlation with truth: 0.970
```

The S-learner's estimates correlate 0.970 with the true individual effects. It is simple, and its weakness is that the model may treat D as a minor predictor of Y and shrink the treatment effect toward zero.

9.2.2. T-learner ("two")

Fit two separate regressions: $\hat{\mu}_1$ on treated units, $\hat{\mu}_0$ on controls. Then $\hat{\tau}^T(x) = \hat{\mu}_1(x) - \hat{\mu}_0(x)$.

```
fit_T1 <- regression_forest(X[D == 1, ], Y[D == 1], num.trees = 500)
fit_T0 <- regression_forest(X[D == 0, ], Y[D == 0], num.trees = 500)

mu1_T <- predict(fit_T1, X)$predictions
mu0_T <- predict(fit_T0, X)$predictions
tau_T <- mu1_T - mu0_T

cor_T <- cor(tau_T, tau)
cat(sprintf("T-learner CATE correlation with truth: %.3f\n", cor_T))
```

```
T-learner CATE correlation with truth: 0.968
```

The T-learner correlates 0.968, essentially the same as the S-learner. Giving treatment and control separate outcome models can help with heterogeneity, but it can also extrapolate badly when the treated and control covariate distributions do not overlap.

9.2.3. X-learner (Künzel et al. 2019)

The X-learner starts from the T-learner. It imputes missing potential outcomes, creates pseudo-treatment effects, and then smooths those pseudo-effects as functions of X.

```
# Step 1: T-learner fits (already done above)
# Step 2: Pseudo-outcomes
D1_pseudo <- Y[D == 1] - predict(fit_T0, X[D == 1, ])$predictions   # observed - imputed Y(0)
D0_pseudo <- predict(fit_T1, X[D == 0, ])$predictions - Y[D == 0]   # imputed Y(1) - observed

# Step 3: Regress pseudo-outcomes on X in each arm
fit_X1 <- regression_forest(X[D == 1, ], D1_pseudo, num.trees = 500)
fit_X0 <- regression_forest(X[D == 0, ], D0_pseudo, num.trees = 500)

tau_X1 <- predict(fit_X1, X)$predictions
tau_X0 <- predict(fit_X0, X)$predictions

# Step 4: Weighted combination - use propensity score as the weight
ps_fit <- regression_forest(X, D, num.trees = 500)
ps_hat <- predict(ps_fit, X)$predictions
ps_hat <- pmin(pmax(ps_hat, 0.02), 0.98)   # trim

tau_X <- ps_hat * tau_X0 + (1 - ps_hat) * tau_X1

cor_X <- cor(tau_X, tau)
cat(sprintf("X-learner CATE correlation with truth: %.3f\n", cor_X))
```

```
X-learner CATE correlation with truth: 0.984
```

The X-learner correlates 0.984, the best of the five meta-learners here. The extra smoothing step is what buys the improvement: regressing the pseudo-effects on X imposes that the effect is a function of the covariates, which is true by construction in this DGP. The X-learner is also useful when one treatment arm is much smaller than the other.

9.2.4. R-learner (Nie and Wager 2021)

The R-learner partials out the main effects of X from both Y and D. It then estimates how the residualized treatment predicts the residualized outcome:

$$\tilde{Y}_i = \frac{Y_i - \hat{m}(X_i)}{D_i - \hat{e}(X_i)}, \quad \text{weights} = (D_i - \hat{e}(X_i))^2, \tag{9.3}$$

where $\hat{m}(x) = \mathbb{E}[Y \mid X = x]$ and $\hat{e}(x) = \mathbb{E}[D \mid X = x]$ are cross-fitted nuisance estimates. Regressing $\tilde{Y}$ on X with these weights gives the CATE estimate.

```r
# Cross-fitting: split sample into 2 folds, fit nuisances on one, predict on the other
set.seed(7)
folds <- sample(rep(1:2, length.out = n))

m_hat <- numeric(n)
e_hat <- numeric(n)
for (k in 1:2) {
  train <- folds != k
  test  <- folds == k
  m_fit <- regression_forest(X[train, ], Y[train], num.trees = 500)
  e_fit <- regression_forest(X[train, ], D[train], num.trees = 500)
  m_hat[test] <- predict(m_fit, X[test, ])$predictions
  e_hat[test] <- predict(e_fit, X[test, ])$predictions
}
e_hat <- pmin(pmax(e_hat, 0.02), 0.98)

# R-learner pseudo-outcome and weights
pseudo_R <- (Y - m_hat) / (D - e_hat)
weights_R <- (D - e_hat)^2

# Regress pseudo-outcomes on X with weights - use a regression forest
fit_R <- regression_forest(X, pseudo_R, sample.weights = weights_R, num.trees = 500)
tau_R <- predict(fit_R, X)$predictions

cor_R <- cor(tau_R, tau)
cat(sprintf("R-learner CATE correlation with truth: %.3f\n", cor_R))
```

```
R-learner CATE correlation with truth: 0.930
```

The R-learner correlates 0.930, the weakest here. Errors in the nuisance models have only second-order effects on the target, which is the theoretical appeal, but the pseudo-outcome divides by $D_i - \hat{e}(X_i)$, and that denominator is near zero for any unit whose treatment is well predicted. Those units get enormous pseudo-outcomes. The weights $(D_i - \hat{e}(X_i))^2$ are designed to offset exactly that, and they do so in a weighted least-squares sense, but the regression target is still much noisier than the T- or X-learner's.

9.2.5. DR-learner (Kennedy 2020)

The DR-learner uses an AIPW-style pseudo-outcome and then regresses it on X:

$$\tilde{Y}_i^{DR} = \hat{\mu}_1(X_i) - \hat{\mu}_0(X_i) + \frac{D_i(Y_i - \hat{\mu}_1(X_i))}{\hat{e}(X_i)} - \frac{(1 - D_i)(Y_i - \hat{\mu}_0(X_i))}{1 - \hat{e}(X_i)}. \tag{9.4}$$

```
mu1_cf <- numeric(n); mu0_cf <- numeric(n); e_cf <- numeric(n)
for (k in 1:2) {
  train <- folds != k
  test  <- folds == k
  mu1_fit <- regression_forest(X[train & D == 1, , drop = FALSE], Y[train & D == 1],
                               num.trees = 500)
  mu0_fit <- regression_forest(X[train & D == 0, , drop = FALSE], Y[train & D == 0],
                               num.trees = 500)
  e_fit   <- regression_forest(X[train, ], D[train], num.trees = 500)
  mu1_cf[test] <- predict(mu1_fit, X[test, ])$predictions
  mu0_cf[test] <- predict(mu0_fit, X[test, ])$predictions
  e_cf[test]   <- predict(e_fit,   X[test, ])$predictions
}
e_cf <- pmin(pmax(e_cf, 0.02), 0.98)

pseudo_DR <- (mu1_cf - mu0_cf) +
             D * (Y - mu1_cf) / e_cf -
             (1 - D) * (Y - mu0_cf) / (1 - e_cf)

fit_DR <- regression_forest(X, pseudo_DR, num.trees = 500)
tau_DR <- predict(fit_DR, X)$predictions

cor_DR <- cor(tau_DR, tau)
cat(sprintf("DR-learner CATE correlation with truth: %.3f\n", cor_DR))
```

```
DR-learner CATE correlation with truth: 0.931
```

The DR-learner correlates 0.931. Its pseudo-outcome has no random denominator, but it does carry the two inverse-probability terms, and like the R-learner it cross-fits on two folds, so each nuisance model is trained on 2,500 observations rather than 5,000. Both properties cost accuracy at this sample size. The doubly-robust construction pays off when the nuisance models are wrong, and here they are not.

9.3. Causal forests

Causal forests are random forests built for treatment-effect heterogeneity. They split the data to find treatment-effect differences, not just outcome differences. The `grf` implementation also uses honest sample splitting for inference.

```
cf <- causal_forest(X = X, Y = Y, W = D, num.trees = 2000)

tau_cf <- predict(cf)$predictions
cor_cf <- cor(tau_cf, tau)
cat(sprintf("Causal forest CATE correlation with truth: %.3f\n", cor_cf))
```

```
Causal forest CATE correlation with truth: 0.983
```

```
# Doubly robust ATE estimate
ate_cf <- average_treatment_effect(cf)
cat(sprintf("\nCausal forest AIPW ATE: %.3f  (SE = %.3f, true = %.3f)\n",
            ate_cf["estimate"], ate_cf["std.err"], mean(tau)))
```

```
Causal forest AIPW ATE: 1.994  (SE = 0.031, true = 2.006)
```

The causal forest correlates 0.983 with the truth, matching the X-learner, and its doubly-robust ATE is 1.994 with a standard error of 0.031 against a true 2.006. Use `predict(cf, estimate.variance = TRUE)` when pointwise confidence intervals are needed.

9.3.1. Comparing all estimators

The plot puts each estimator's predicted CATE against X_1, one panel per estimator, with a loess fit through the points and the true $\tau(x) = 1 + 2x_1$ as the dashed red line. What to look for is whether the blue curve tracks the red line and how tightly the points cluster around it.

```
comparison <- tibble(
  X1     = X[, 1],
  true   = tau,
  S      = tau_S,
  Tlearn = tau_T,
  X      = tau_X,
  R      = tau_R,
  DR     = tau_DR,
  CF     = tau_cf
) |>
  pivot_longer(cols = c(S, Tlearn, X, R, DR, CF),
               names_to = "Estimator", values_to = "Predicted")

ggplot(comparison, aes(x = X1, y = Predicted)) +
  geom_point(alpha = 0.1, size = 0.4) +
  geom_smooth(method = "loess", se = FALSE, colour = "steelblue", linewidth = 1) +
  geom_abline(aes(intercept = 1, slope = 2), linetype = "dashed",
              colour = "firebrick") +
  facet_wrap(~ Estimator, ncol = 3) +
  labs(x = expression(X[1]),
       y = "Estimated CATE",
       caption = "Red dashed line = true  (x) = 1 + 2*X1") +
  theme_minimal()
```

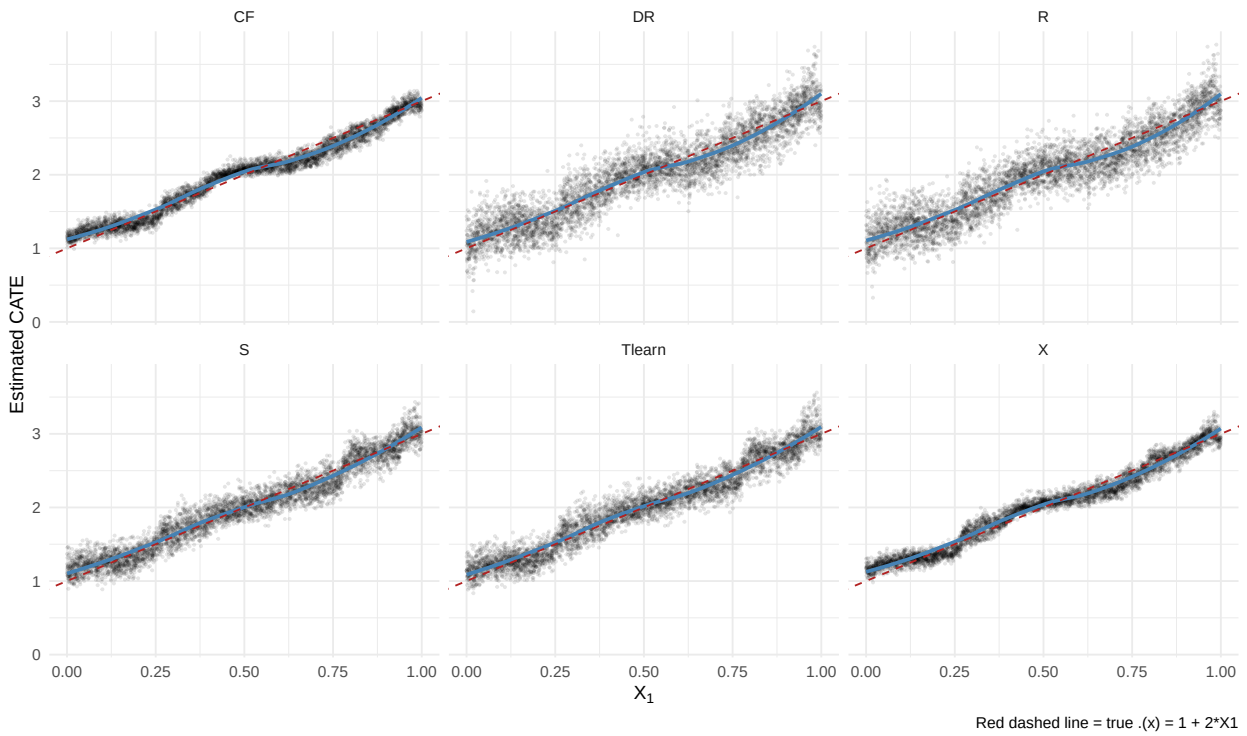


Figure 9.1.: Estimated CATE vs true (x) = 1 + 2*X1 for each estimator. Closer to the dashed line = better.

All six recover the upward slope. Ranked by correlation with the truth, the X-learner (0.984) and the causal forest (0.983) come first, the S- and T-learners next (0.970 and 0.968), and the R- and DR-learners last (0.930 and 0.931). All six are consistent here, so this ranking is about finite-sample noise. It would change under a DGP where the nuisance models are misspecified, which is the case the doubly-robust constructions are built for.

9.4. Best linear projection (BLP)

Even if CATE is nonlinear, we often want a regression-style summary: which covariates are associated with larger effects? The best linear projection of CATE onto X is

$$\mathrm{BLP}(X) = \arg\min_{\beta_0,\beta} \mathbb{E}\left[(\tau(X) - \beta_0 - X'\beta)^2\right], \tag{9.5}$$

which `grf` estimates with valid standard errors:

```
blp <- best_linear_projection(cf, A = X)
print(blp)
```

```
Best linear projection of the conditional average treatment effect.
```

```
Confidence intervals are cluster- and heteroskedasticity-robust (HC3):

             Estimate Std. Error t value Pr(>|t|)
(Intercept)  1.077587   0.119928  8.9853   <2e-16 ***
X1           2.008322   0.104710 19.1798   <2e-16 ***
X2          -0.051931   0.106412 -0.4880   0.6256
X3          -0.064522   0.102963 -0.6266   0.5309
X4           0.044172   0.105384  0.4192   0.6751
X5          -0.115495   0.104588 -1.1043   0.2695
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The coefficient on X_1 is 2.008 with a standard error of 0.105, against a true slope of 2. The intercept is 1.078 against a true 1. None of X_2 through X_5 is significant, with coefficients between -0.115 and 0.044 and standard errors around 0.105. The projection finds the one effect modifier and correctly reports nothing for the other four.

9.5. Variable importance

`grf` also reports variable importance. It is a rough diagnostic for which variables the forest uses to find heterogeneity.

```r
vi <- variable_importance(cf)
vi_df <- tibble(variable = paste0("X", 1:p), importance = as.numeric(vi)) |>
  arrange(desc(importance))
print(vi_df)
```

```
# A tibble: 5 x 2
  variable importance
  <chr>         <dbl>
1 X1            0.725
2 X4            0.0953
3 X5            0.0703
4 X3            0.0565
5 X2            0.0533
```

X_1 takes 0.725 of the importance. The other four sit between 0.053 and 0.095, well below the 0.2 each would receive if the forest split at random. The forest found the effect modifier without being told which one it was.

9.6. Policy trees: who should we treat?

A CATE estimate is not yet a policy. If treatment has a cost, the policy question is who should be treated. A policy tree turns estimated treatment effects into a simple treatment rule.

```
# double_robust_scores() returns an n x 2 matrix of AIPW scores,
# one column per action: Gamma_i(control), Gamma_i(treated)
Gamma <- double_robust_scores(cf)
cost <- 2     # cost of treating one unit, in outcome units

# Net value of each action: control stays as is, treating pays the cost.
# With tau(x) = 1 + 2*x1 in [1, 3], the optimal rule is: treat iff x1 > 0.5.
Gamma[, "treated"] <- Gamma[, "treated"] - cost

# Fit a depth-2 policy tree (shallow for interpretability)
tree <- policy_tree(X, Gamma, depth = 2)
print(tree)
```

```
policy_tree object
Tree depth:  2
Actions:  1: control 2: treated
Variable splits:
(1) split_variable: X1  split_value: 0.626569
  (2) split_variable: X1  split_value: 0.42175
    (4) * action: 1
    (5) * action: 2
  (3) split_variable: X1  split_value: 0.644418
    (6) * action: 1
    (7) * action: 2
```

```
# Recommended action for each unit (1 = control, 2 = treated)
actions <- predict(tree, X)
cat(sprintf("\nTreatment rate under policy tree: %.1f%% (optimal: %.1f%%)\n",
            mean(actions == 2) * 100, mean(X[, 1] > 0.5) * 100))
```

```
Treatment rate under policy tree: 56.5% (optimal: 50.5%)
```

```
# Value of the learned rule vs treating everyone, both evaluated with
# the same doubly-robust scores
welfare_tree <- mean(ifelse(actions == 2, Gamma[, "treated"], Gamma[, "control"]))
welfare_all  <- mean(Gamma[, "treated"])
cat(sprintf("Estimated welfare: policy tree %.3f vs treat-everyone %.3f (gain %.3f)\n",
            welfare_tree, welfare_all, welfare_tree - welfare_all))
```

```
Estimated welfare: policy tree 0.525 vs treat-everyone 0.261 (gain 0.264)
```

The tree splits on X_1 at every node, which is the true effect modifier, and treats 56.5% of units against an optimal 50.5%. Its estimated welfare is 0.525 against 0.261 for treating everyone, a gain

of 0.264 per unit. The rule beats treat-everyone because with a cost of 2 and $\tau(x) = 1 + 2x_1$, units with $x_1 < 0.5$ have $\tau(x) < 2$ and are not worth treating.

The point of the shallow tree is interpretability. It gives up some flexibility in exchange for a rule that can be inspected.

Inspect the printed tree rather than summarising it, though, because at depth 2 both branches split X_1 again: the fitted rule treats $X_1 \in (0.422, 0.627]$ and $X_1 > 0.644$, but not the sliver between them. Since $\tau(x) = 1 + 2x_1$ is monotone, the optimal rule is a *single* threshold at $x_1 = 0.5$, so that gap is estimation noise and not a discovery. The effective lower cut of 0.42 rather than 0.50 is what produces the 56.5% treatment rate against an optimal 50.5%. A depth-1 tree is the better-specified model for this DGP, and the practical lesson is to fit the shallower tree too: a depth-2 structure should not be read as evidence of a genuinely non-monotone rule when a depth-1 tree does as well.

For an extended walkthrough of policy trees with IPW and AIPW losses, see the companion blog chapter on `policytree`. For a comparison of CATE estimators across software ecosystems (including Stata 19's new `cate` command), see the Stata CATE blog chapter.

9.7. GATES: sorted group ATEs

GATES (group average treatment effects) is a simple way to report heterogeneity (Chernozhukov et al. 2018). Sort observations by predicted CATE, split them into bins, and estimate the ATE in each bin. If the bin ATEs differ, the heterogeneity is not just a graph. (The same paper's CLAN — classification analysis — is the natural companion: compare average *covariates* between the most- and least-affected bins; here that comparison would show high X_1 in the top quintile and low X_1 in the bottom one.)

```
# Sort by predicted CATE, split into 5 quintiles
nq <- 5
quintile <- cut(tau_cf, breaks = quantile(tau_cf, probs = seq(0, 1, 1/nq)),
                include.lowest = TRUE, labels = FALSE)

# Compute AIPW ATE within each quintile
get_ate <- function(idx) {
  ate <- average_treatment_effect(cf, subset = idx)
  c(est = ate["estimate"], se = ate["std.err"])
}

clan <- t(sapply(1:nq, function(q) get_ate(quintile == q)))
clan_df <- as_tibble(clan) |>
  mutate(quintile = 1:nq,
         lo = est.estimate - 1.96 * se.std.err,
         hi = est.estimate + 1.96 * se.std.err)

knitr::kable(clan_df[, c("quintile", "est.estimate", "se.std.err", "lo", "hi")],
             col.names = c("Quintile", "ATE", "SE", "95% LB", "95% UB"),
             digits = 3,
             caption = "GATES: quintile-specific ATEs sorted by predicted CATE. Heterogeneity
```

Table 9.1.: GATES: quintile-specific ATEs sorted by predicted CATE. Heterogeneity is evident if quintile estimates differ.

Quintile	ATE	SE	95% LB	95% UB
1	1.263	0.070	1.126	1.400
2	1.517	0.067	1.386	1.649
3	2.106	0.065	1.979	2.234
4	2.274	0.068	2.141	2.408
5	2.810	0.068	2.677	2.943

The quintile ATEs rise monotonically from 1.263 to 2.810, and the standard errors are all about 0.068, so the top and bottom quintiles are more than twenty standard errors apart. The heterogeneity is not a graphical artefact. The range also sits inside the true $[1, 3]$, as it must, and the quintile means bracket the ATE of 2. One caveat: the quintiles are formed from the same forest's (out-of-bag) predictions used to estimate the group ATEs, which mitigates but does not fully remove the bias from grouping and estimating on the same data; for formal inference use a held-out fold or `grf::rank_average_treatment_effect()`.

9.8. Causal forests with panel data

With panel data, unit effects can be correlated with treatment and covariates. A cross-sectional causal forest can then be biased. Two practical fixes are:

1. **Demean by fixed effects** before running the causal forest.
2. **Pass a `clusters` argument** to `causal_forest` so that observations from the same unit are kept in the same training fold during honest sample-splitting.

To show what each fix does and does not do, we simulate a panel of 200 firms observed over 5 periods, 1,000 observations in all. Each firm draws a unit effect $\alpha_i \sim N(0, 1.5^2)$ and a firm-level covariate $V_{1,i} \sim N(0, 1)$, and the observed covariate adds within-firm noise, $V_{1,it} = V_{1,i} + N(0, 0.3^2)$. Treatment is $W_{it} \sim \text{Bernoulli}(\Lambda(0.3V_{1,i} + 0.5\alpha_i))$, so the unit effect drives treatment. The CATE is $\tau_{it} = 0.5 + V_{1,it}$, and the outcome is $Y_{it} = \alpha_i + V_{1,it} + \tau_{it}W_{it} + N(0, 1)$.

The unit effect raises both the treatment probability and the outcome, and the forest never sees it. That is textbook fixed-effect confounding. The true ATE is 0.566 in this draw.

```
set.seed(2024)
n_firms <- 200
n_t     <- 5
n_total <- n_firms * n_t

# Panel data with a unit effect that drives BOTH treatment and the outcome
# (classic fixed-effect confounding; unit_fe is unobserved to the forest)
firm_id  <- rep(1:n_firms, each = n_t)
unit_fe  <- rnorm(n_firms, sd = 1.5)[firm_id]
V1_firm  <- rnorm(n_firms, sd = 1.0)[firm_id]   # firm-level covariate
```

```
V1       <- V1_firm + rnorm(n_total, sd = 0.3)  # within-firm variation
W        <- rbinom(n_total, 1, plogis(0.3 * V1_firm + 0.5 * unit_fe))
# True CATE varies linearly with V1
tau_panel <- 0.5 + 1.0 * V1
Y_panel  <- unit_fe + V1 + tau_panel * W + rnorm(n_total)

df_panel <- tibble(firm = firm_id, V1 = V1, W = W, Y = Y_panel)
glimpse(df_panel)
```

```
Rows: 1,000
Columns: 4
$ firm <int> 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 5, 5,~
$ V1   <dbl> 0.60231981, 0.23871074, 0.05916058, -0.24663844, 0.28957601, -0.1~
$ W    <int> 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1,~
$ Y    <dbl> 3.99219997, 3.99833174, 1.50594149, 2.51156617, 2.37409880, 0.254~
```

A naive causal forest ignores the firm effects. Because `unit_fe` raises both the treatment probability and the outcome, and is not in X, the naive estimate is badly biased upward:

```
X_panel <- as.matrix(df_panel[, "V1", drop = FALSE])

cf_naive <- causal_forest(X = X_panel, Y = df_panel$Y, W = df_panel$W,
                          num.trees = 1000)
ate_naive <- average_treatment_effect(cf_naive)

cat(sprintf("Naive panel CF ATE: %.3f  (SE = %.3f)\n",
            ate_naive["estimate"], ate_naive["std.err"]))
```

```
Naive panel CF ATE: 1.706  (SE = 0.127)
```

```
cat(sprintf("True ATE: %.3f\n", mean(tau_panel)))
```

```
True ATE: 0.566
```

The naive forest returns 1.706 with a standard error of 0.127 against a true 0.566. It is off by three times the truth, and about nine standard errors away.

Adding `clusters = firm` keeps a firm's observations together when `grf` does the honest sample split. Note what this does and does not do: it makes the honest splitting and the standard errors respect the panel structure, but it does *not* remove the unobserved-unit-effect confounding — the estimate stays close to the naive one:

```
cf_clust <- causal_forest(X = X_panel, Y = df_panel$Y, W = df_panel$W,
                          clusters = df_panel$firm,
                          num.trees = 1000)
ate_clust <- average_treatment_effect(cf_clust)

cat(sprintf("Clustered panel CF ATE: %.3f  (SE = %.3f)\n",
            ate_clust["estimate"], ate_clust["std.err"]))
```

```
Clustered panel CF ATE: 1.741  (SE = 0.163)
```

Clustering gives 1.741 with a standard error of 0.163. The estimate moved slightly further from the truth, not closer, and the standard error grew. This is the point: `clusters=` is about honesty and inference, never about confounding.

What removes the confounding is the fixed-effect style adjustment: demean Y, W, and V_1 by firm before fitting, which sweeps out `unit_fe`. A caveat on interpretation: with a demeaned binary treatment W_{dm}, the GRF target is *not* generally mean(τ_{panel}). It is a within-unit, partialled-out contrast whose implicit weighting depends on the distribution of demeaned treatment values. Read the number below as a within-firm residualized treatment effect, not as a drop-in fixed-effect estimate of the same CATE/ATE target as the cross-sectional forest:

```
df_dm <- df_panel |>
  group_by(firm) |>
  mutate(Y_dm = Y - mean(Y),
         W_dm = W - mean(W),
         V1_dm = V1 - mean(V1)) |>
  ungroup()

cf_fe <- causal_forest(X = as.matrix(df_dm[, "V1_dm"]),
                       Y = df_dm$Y_dm,
                       W = df_dm$W_dm,
                       clusters = df_dm$firm,
                       num.trees = 1000)
ate_fe <- average_treatment_effect(cf_fe)
cat(sprintf("Within-transform CF ATE: %.3f  (SE = %.3f)\n",
            ate_fe["estimate"], ate_fe["std.err"]))
```

```
Within-transform CF ATE: 0.577  (SE = 0.103)
```

The within transform gives 0.577 with a standard error of 0.103, against a true 0.566. Sweeping out the firm means removed the confounding that clustering did not touch.

The clustered, within-transformed causal forest is a heuristic analogue to a fixed-effect regression with heterogeneous effects, but it does not target the same estimand as a textbook within estimator: properly identified heterogeneous panel effects need stronger assumptions or a dedicated panel/DML construction with an orthogonal score. See the companion blog chapter on causal forests in panel data for an extended simulation study comparing the panel CF to fixed-effect regression and random-effect regression under linear and nonlinear DGPs.

9.9. Summary

- Meta-learners turn outcome-regression tools into CATE estimators.
- Causal forests are the main `grf` estimator for heterogeneous effects.
- BLP and GATES are useful because they summarize heterogeneity in a way that looks like familiar regression output.
- Policy trees translate CATE estimates into treatment rules.
- In applied work, compare at least two CATE estimators and always report simple heterogeneity summaries, not only a CATE scatterplot.

10. Continuous and Multivalued Treatments

```
library(tidyverse)
library(WeightIt)
library(causaldrf)
library(lmtp)
library(SuperLearner)
```

So far most examples use a binary treatment. Many treatments are not binary: dose of a drug, hours in a program, dollars spent, or a treatment with several levels. In those cases "the treatment effect" is not one number. We usually want a dose-response curve,

$$\mu(d) = \mathbb{E}[Y(d)], \quad d \in \mathcal{D}. \tag{10.1}$$

I use three approaches:

1. **Generalized propensity score** (GPS; Hirano-Imbens 2004), which models the conditional density of treatment given covariates.
2. **Doubly-robust dose-response estimation** (in the spirit of Kennedy et al. 2017), which combines a GPS with an outcome model.
3. **Longitudinal modified treatment policies** [LMTP; Díaz et al. (2023)], which are useful when treatments are time-varying or when fixed treatment levels create positivity problems.

> **Related reading**: The LMTP material below is condensed from the longer treatment in Longitudinal modified treatment policy (LMTP) in *Topics on Econometrics and Causal Inference*, which follows Nicholas Williams's tutorials at beyondtheate.com.

10.1. The continuous-treatment setup

The aim is to recover a known dose-response curve from data in which the dose is chosen partly on covariates that also move the outcome.

We simulate $n = 2000$ units with two covariates, $X_1, X_2 \sim N(0,1)$, independent of each other. The dose is $D = 2 + 0.5X_1 + 0.5X_2 + \nu$ with $\nu \sim N(0,1)$. The dose-response curve is $\mu(d) = 1 + 2d - 0.15d^2$, and the outcome is $Y = \mu(D) + 0.3X_1 - 0.3X_2 + \varepsilon$ with $\varepsilon \sim N(0,1)$.

```r
set.seed(42)
n  <- 2000
X1 <- rnorm(n)
X2 <- rnorm(n)
# Continuous treatment, depends on X1, X2
D  <- 2 + 0.5 * X1 + 0.5 * X2 + rnorm(n, sd = 1.0)
# Dose-response: nonlinear in D
true_mu <- function(d) 1.0 + 2.0 * d - 0.15 * d^2
Y  <- true_mu(D) + 0.3 * X1 - 0.3 * X2 + rnorm(n)

df <- tibble(Y = Y, D = D, X1 = X1, X2 = X2)
cat(sprintf("Treatment range: [%.2f, %.2f]\n", min(D), max(D)))
```

```
Treatment range: [-1.82, 6.39]
```

```r
cat(sprintf("True  (d=2) = %.2f,  (d=4) = %.2f,  (d=6) = %.2f\n",
            true_mu(2), true_mu(4), true_mu(6)))
```

```
True  (d=2) = 4.40,  (d=4) = 6.60,  (d=6) = 7.60
```

The observed doses run from -1.82 to 6.39, and the true curve gives $\mu(2) = 4.40$, $\mu(4) = 6.60$ and $\mu(6) = 7.60$.

The dose-response curve is concave, with clearly diminishing returns. It is worth being precise about the shape rather than calling it hump-shaped, because the vertex sits at $d = 2/0.3 \approx 6.67$, just *beyond* the largest dose in the sample. Over the observed range the curve is therefore increasing throughout — its slope $2 - 0.3d$ is still positive (about 0.08) at the maximum observed dose — so the downturn is extrapolation, not something these data identify. The estimation problem is to recover the curvature that is in range when the observed dose is confounded by X.

10.2. Naive regression vs adjusted regression

We fit three models and compare their implied dose-response curves against the truth: Y on D alone, Y on D and the covariates, and Y on a quadratic in D and the covariates. All three are evaluated at $X_1 = X_2 = 0$ over $d \in [0, 10]$.

One feature of this DGP is worth stating before reading the plot. X_1 and X_2 are confounders in the graph, since each raises the dose and each moves the outcome. But their linear bias cancels: each raises D by 0.5, while they enter Y with $+0.3$ and -0.3, so the omitted-variable term $\mathrm{Cov}(D,\, 0.3X_1 - 0.3X_2)/\mathrm{Var}(D)$ is $0.3(0.5) - 0.3(0.5) = 0$ exactly. The naive slope is 1.421 and the adjusted slope 1.409. Adjusting for X changes almost nothing here, and the one problem left for the linear models is functional form.

```r
# Linear fit, ignoring covariates
linear_naive <- lm(Y ~ D, data = df)
# Linear fit with covariate adjustment
linear_adj   <- lm(Y ~ D + X1 + X2, data = df)
# Nonlinear fit with covariate adjustment
poly_adj     <- lm(Y ~ poly(D, 2) + X1 + X2, data = df)

ds <- seq(0, 10, by = 0.5)

predict_mu <- function(fit, ds) {
  data.frame(d = ds, X1 = 0, X2 = 0) |>
    rename(D = d) -> nd
  predict(fit, newdata = nd)
}

# True dose-response
df_true <- tibble(d = ds, mu = true_mu(ds))

df_fits <- bind_rows(
  tibble(d = ds, mu = predict_mu(linear_naive, ds), method = "Linear (naive)"),
  tibble(d = ds, mu = predict_mu(linear_adj,   ds), method = "Linear (adj X)"),
  tibble(d = ds, mu = predict_mu(poly_adj,     ds), method = "Quadratic (adj X)")
)
```

```r
ggplot() +
  geom_line(data = df_fits, aes(d, mu, colour = method), linewidth = 1.0) +
  geom_line(data = df_true, aes(d, mu), colour = "black",
            linetype = "dashed", linewidth = 1.0) +
  labs(x = "Dose D", y = "Expected outcome E[Y(d)]",
       colour = "Method",
       caption = "Black dashed line = true  (d) = 1 + 2d - 0.15 d²") +
  theme_minimal()
```

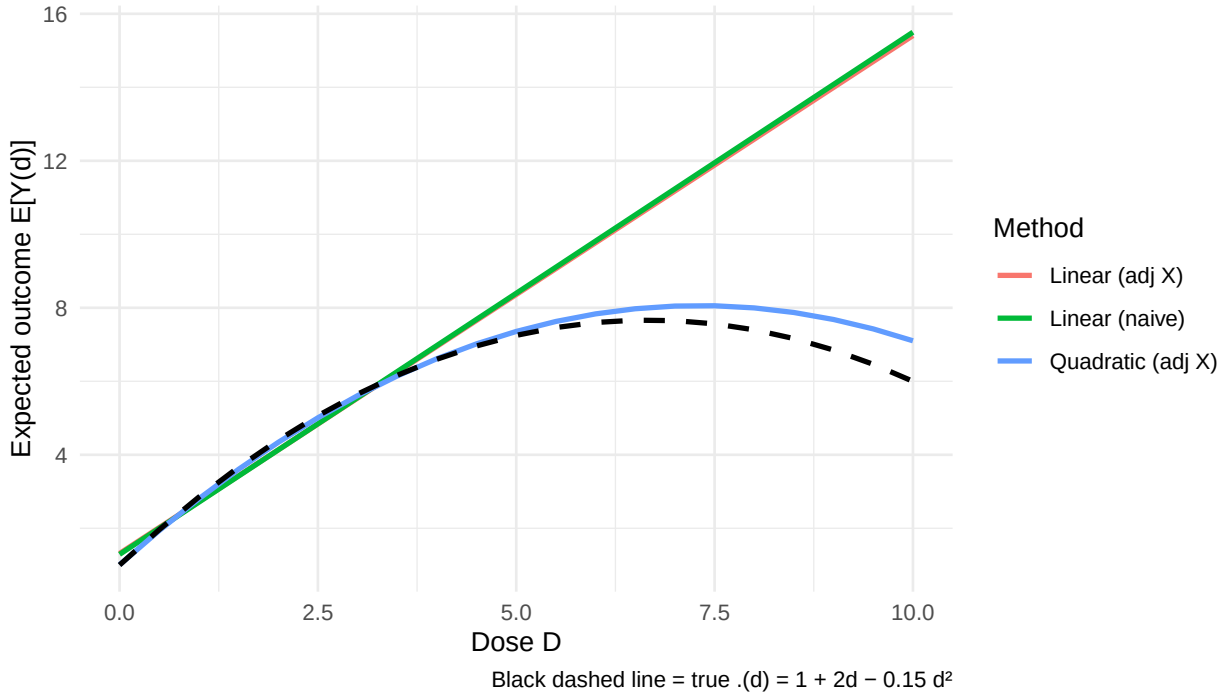


Figure 10.1.: Fitted vs true dose-response. Linear fits miss the curvature; the quadratic-adjusted fit comes closer.

The two linear fits lie almost on top of each other, as the cancellation above implies, and both run straight through the curvature. At $d = 2$ they give 4.13 against a true 4.40; at $d = 6$ they give 9.82 and 9.77 against a true 7.60. Their common slope of about 1.41 is close to the average of the true slope $2 - 0.3d$ over the observed doses, which is 1.403 — a straight line through a concave curve recovers the average derivative, not the curve.

The quadratic adjusted fit gives 4.34, 6.61 and 7.83 at $d = 2, 4, 6$ against the true 4.40, 6.60 and 7.60. It is better here because the true curve is quadratic and we fitted a quadratic. In real applications the functional form is not known, which is what motivates the next two sections.

10.3. Generalized propensity score (GPS)

The **generalized propensity score** is the conditional density of the treatment given covariates:

$$r(d, x) = f_{D|X}(d \mid x). \tag{10.2}$$

Under unconfoundedness, the GPS plays the same role as the propensity score for binary treatment. In practice we often start with a normal model for $D \mid X$: $D \mid X \sim \mathcal{N}(\beta_0 + \beta' X, \sigma^2)$.

We fit D on X_1 and X_2 by OLS and read the GPS off the normal density at each unit's own dose. The weights printed below are a diagnostic only: they are the marginal density of D divided by the GPS, trimmed at the 99th percentile.

```r
# Fit GPS model: D ~ X with Normal residuals
gps_model <- lm(D ~ X1 + X2, data = df)
d_pred    <- predict(gps_model)
sigma_e   <- sigma(gps_model)

# GPS for each unit's observed treatment
gps <- dnorm(df$D, mean = d_pred, sd = sigma_e)
df$gps <- gps

# IPW-style weights for continuous treatment: 1 / GPS
# Stabilise by multiplying by the marginal density f_D(D)
# NOTE: sw_cont is computed only as an overlap/positivity diagnostic.
# The Hirano-Imbens estimator below does NOT weight by it: the GPS
# enters as a regressor in the outcome model.
marginal_D <- density(df$D)
fD <- approx(marginal_D$x, marginal_D$y, xout = df$D)$y
df$sw_cont <- pmin(fD / gps, quantile(fD / gps, 0.99))

cat(sprintf("Weight summary: min=%.3f, mean=%.3f, max=%.3f\n",
            min(df$sw_cont), mean(df$sw_cont), max(df$sw_cont)))
```

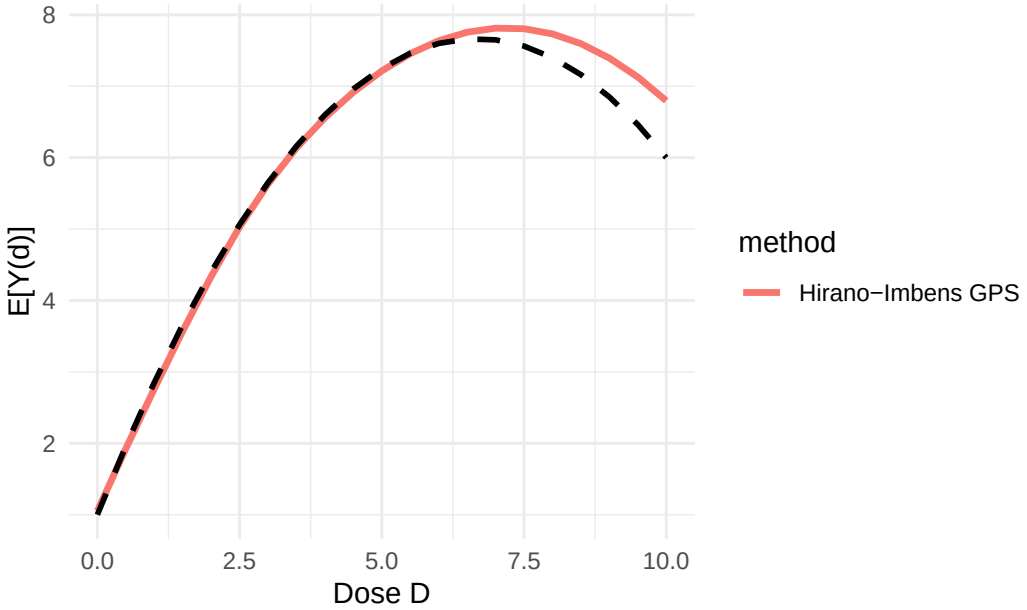
```
Weight summary: min=0.039, mean=0.947, max=4.390
```

The weights run from 0.039 to 4.390 with a mean of 0.947. No unit carries an extreme weight, so overlap along the dose is good and the estimators below are not resting on a handful of observations.

The Hirano-Imbens estimator has two steps. First regress Y on flexible functions of D and the estimated GPS. Then, for each dose d, average the predicted outcome over the sample. The plot compares the resulting curve against the truth.

```r
# Stage 2: regress Y on flexible function of (D, GPS)
hi_fit <- lm(Y ~ poly(D, 2) + poly(gps, 2) + I(D * gps), data = df)

# Dose-response at each d: average over the empirical distribution of GPS
# at that d.
estimate_dose <- function(d) {
  # Predict GPS at this d for each unit
  gps_at_d <- dnorm(d, mean = d_pred, sd = sigma_e)
  nd <- data.frame(D = d, gps = gps_at_d)
  mean(predict(hi_fit, newdata = nd))
}

ds <- seq(0, 10, by = 0.5)
hi_dose <- sapply(ds, estimate_dose)

df_hi <- tibble(d = ds, mu = hi_dose, method = "Hirano-Imbens GPS")
```

```r
ggplot() +
  geom_line(data = df_hi, aes(d, mu, colour = method), linewidth = 1.2) +
  geom_line(data = df_true, aes(d, mu), colour = "black",
            linetype = "dashed", linewidth = 1.0) +
  labs(x = "Dose D", y = "E[Y(d)]",
       caption = "Dashed black = truth; coloured = Hirano-Imbens GPS estimator") +
  theme_minimal()
```



The GPS curve tracks the concavity that the linear fits missed. It is fitted with a quadratic in D and in the GPS, so it is not assumption-free either, but the assumption is now about how the outcome depends on the dose and the score rather than about a straight line.

10.4. Doubly-robust dose-response estimation

A kernel-localized AIPW estimator — in the spirit of Kennedy et al. (2017), who develop a more elaborate pseudo-outcome version with formal guarantees — is the continuous-treatment analogue of AIPW. It combines:

- An outcome model $\hat{\mu}(d, x) = \mathbb{E}[Y \mid D = d, X = x]$
- A generalised propensity score $\hat{r}(d \mid x)$

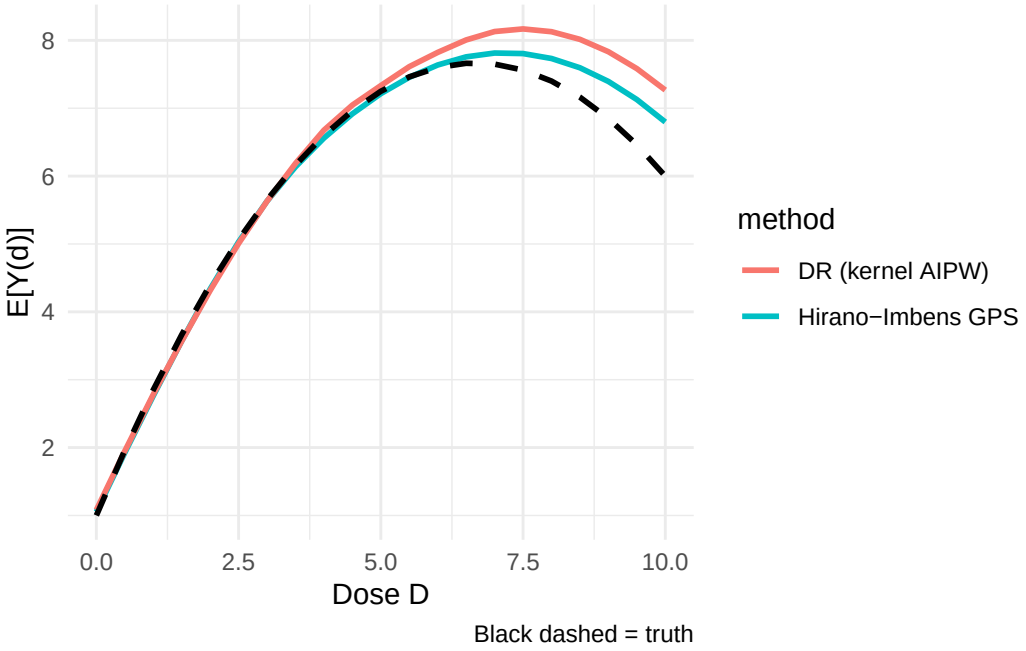
Then the dose-response at d is estimated by

$$\hat{\mu}_{DR}(d) = \frac{1}{n} \sum_i \left[\hat{\mu}(d, X_i) + K_h(D_i - d) \cdot \frac{Y_i - \hat{\mu}(D_i, X_i)}{\hat{r}(D_i \mid X_i)} \right], \tag{10.3}$$

where K_h is a kernel around dose d. The bandwidth h controls how much nearby observed doses contribute to the estimate. (The code below uses the weight-normalized — Hájek — version of the correction term, dividing the weights by their mean; the population limit is unchanged.)

Using the GPS already estimated above, the implementation is short. The outcome model is a quadratic in D with both covariates and their interactions with D; the kernel is Gaussian with bandwidth $h = 0.5$. The plot puts this curve next to the GPS curve and the truth.

```
# Outcome model ^(D, X): flexible polynomial
mu_fit <- lm(Y ~ D + I(D^2) + X1 + X2 + I(D * X1) + I(D * X2), data = df)
df$mu_at_obs <- predict(mu_fit)

# DR estimator at dose d with bandwidth h
dr_estimate <- function(d, h = 0.5) {
  # Counterfactual prediction at D = d for each unit
  mu_at_d <- predict(mu_fit, newdata = transform(df, D = d))
  # Kernel weight on observed treatment near d
  kernel <- dnorm(df$D - d, sd = h)
  weight <- kernel / df$gps
  weight <- weight / mean(weight)  # normalise
  # DR correction
  correction <- weight * (df$Y - df$mu_at_obs)
  mean(mu_at_d) + mean(correction)
}

ds <- seq(0, 10, by = 0.5)
dr_dose <- sapply(ds, dr_estimate)

df_drf <- tibble(d = ds, mu = dr_dose, method = "DR (kernel AIPW)")

ggplot() +
  geom_line(data = df_hi,  aes(d, mu, colour = method), linewidth = 1.0) +
  geom_line(data = df_drf, aes(d, mu, colour = method), linewidth = 1.0) +
  geom_line(data = df_true, aes(d, mu), colour = "black",
            linetype = "dashed", linewidth = 1.0) +
  labs(x = "Dose D", y = "E[Y(d)]",
       caption = "Black dashed = truth") +
  theme_minimal()
```



Read this as an illustrative kernel-AIPW construction rather than a turnkey doubly robust theorem. For continuous treatments, a formal doubly robust dose-response result – consistency if either the outcome model or the GPS is correct – requires additional bandwidth, nuisance-convergence-rate, smoothness, and orthogonalization conditions (see Kennedy et al. on continuous-treatment DR estimators). The self-normalized kernel correction shown here conveys the intuition but does not by itself establish those guarantees. As usual, the bandwidth trades bias against variance.

The `causaldrf` package provides ready-made versions of these estimators, including `hi_est`, `iptw_est`, `aipwee_est`, `gam_est`, `nw_est`, and `bart_est`.

10.5. Longitudinal Modified Treatment Policies (LMTP)

Fixed-dose interventions can be unrealistic. For example, there may be almost no observations near $D = 10$, so estimating $Y(10)$ requires extrapolation. LMTP avoids this by defining interventions as changes to the observed treatment, such as "shift everyone's dose down by one unit." This keeps the counterfactual treatment closer to the observed support.

10.5.1. A two-period example

We simulate $N = 1500$ units over two periods. The first covariate is $L_1 \sim U(0, 1)$ and the first treatment is a coin flip, $A_1 \sim \text{Bernoulli}(0.5)$. The second covariate is $L_2 \sim N(0.25L_1, 0.5^2)$, and the second treatment depends on both the first treatment and the current covariate, $A_2 \sim \text{Bernoulli}(\Lambda(0.5 - 0.2A_1 + 0.1L_2))$. The outcome is $Y \sim N(A_2 + L_2, 0.3^2)$.

The truths follow from the DGP without simulation. L_2 does not depend on A_1, so setting both treatments to 0 leaves $E[Y] = E[L_2] = 0.25 \cdot E[L_1] = 0.125$. Setting both to 1 gives $1+0.125 = 1.125$.

And A_1 affects Y only through A_2, which the policy fixes, so the contrast between the two policies is exactly 1.

```
set.seed(1)
N    <- 1500
L_1  <- runif(N, 0, 1)
A_1  <- rbinom(N, 1, 0.5)
L_2  <- rnorm(N, 0.25 * L_1, 0.5)
A_2  <- rbinom(N, 1, plogis(0.5 - 0.2 * A_1 + 0.1 * L_2))
Y    <- rnorm(N, A_2 + L_2, 0.3)

lmtp_data <- data.frame(L_1, A_1, L_2, A_2, Y)
head(lmtp_data)
```

```
        L_1 A_1        L_2 A_2           Y
1 0.2655087   0  0.4913989   0 -0.42997401
2 0.3721239   1 -0.3696255   0 -0.76396535
3 0.5728534   0  0.5900039   0  0.79312079
4 0.9082078   1 -0.2434529   1  1.23628221
5 0.2016819   0  0.3198965   0  0.02620015
6 0.8983897   0  0.1336102   1  1.00441306
```

For simplicity the worked example uses *static* policies (set every treatment to 0, then to 1) on a binary treatment — `lmtp` handles these in the same framework. A genuinely *modified* policy on a continuous dose would instead return a shift of the observed value, e.g. `function(data, trt) data[[trt]] - 1`, which is what keeps the counterfactual on the observed support. Here is the set-to-zero policy:

```
shift_zero <- function(data, trt) {
  # Set treatment to 0 everywhere - for comparison to "never treat"
  a <- data[[trt]]
  rep(0, length(a))
}
```

Then fit the LMTP estimator with `lmtp::lmtp_tmle`:

```
# TMLE-based LMTP estimator with super-learners for nuisance functions
# (using SL.glm to keep it fast; in practice use richer SL libraries)
lmtp_fit <- lmtp_tmle(
  data       = lmtp_data,
  trt        = c("A_1", "A_2"),
  outcome    = "Y",
  baseline   = NULL,
  time_vary  = list(c("L_1"), c("L_2")),
  shift      = shift_zero,
  outcome_type = "continuous",
```

```
  learners_outcome = c("SL.glm"),
  learners_trt     = c("SL.glm"),
  folds = 2
)

# tidy() gives the estimate, SE, and CI as a data frame
# (the styled console print does not render cleanly in the book)
tidy(lmtp_fit)
```

```
# A tibble: 1 x 4
  estimate std.error conf.low conf.high
     <dbl>     <dbl>    <dbl>     <dbl>
1    0.118    0.0252   0.0681     0.167
```

The set-to-zero policy gives 0.118 with a standard error of 0.0252 and a 95% interval of $[0.068, 0.167]$, which covers the true 0.125.

The useful part of `lmtp` is that it handles the time-ordering of covariates, treatments, and outcomes: L_2 is a post-treatment covariate for A_1 and a pre-treatment covariate for A_2, and the estimator has to respect that ordering rather than conditioning on L_2 throughout.

10.5.2. Contrasts between policies

We can also compare two policies, such as "set to 0" versus "set to 1":

```
shift_one <- function(data, trt) {
  a <- data[[trt]]
  rep(1, length(a))
}

lmtp_one <- lmtp_tmle(
  data       = lmtp_data,
  trt        = c("A_1", "A_2"),
  outcome    = "Y",
  baseline   = NULL,
  time_vary  = list(c("L_1"), c("L_2")),
  shift      = shift_one,
  outcome_type = "continuous",
  learners_outcome = c("SL.glm"),
  learners_trt     = c("SL.glm"),
  folds = 2
)

contrast <- lmtp_contrast(lmtp_one, ref = lmtp_fit)
as.data.frame(contrast$estimates)
```

```
     shift       ref  estimate  std.error  conf.low conf.high       p.value
1 1.103091 0.1176079 0.9854826 0.03489166 0.9170962  1.053869 1.683133e-175
```

The contrast is the difference between the two policy-specific mean outcomes. Treat-always gives 1.103 against a true 1.125, treat-never gives 0.118 against a true 0.125, and their difference is 0.9855 with a standard error of 0.0349 and a 95% interval of $[0.917, 1.054]$. The true contrast is exactly 1, and the interval covers it.

10.6. Choosing among the three

Method	Strengths	Weaknesses
GPS (Hirano-Imbens)	Conceptually transparent; widely understood	Parametric assumptions on the GPS and the outcome-GPS regression; sensitive to their misspecification
Doubly-robust (kernel AIPW)	Robust to misspecification of one nuisance; attains nonparametric oracle rates	More complex; nuisance models need careful tuning; sensitive to bandwidth
LMTP	Handles continuous + multivalued + time-varying uniformly; respects positivity	More setup; requires defining a shift function

In practice:

- For a simple cross-sectional continuous treatment with good overlap, GPS or the Kennedy DR estimator is a natural start.
- For time-varying treatments or policy shifts, use LMTP.
- For multivalued treatments, use a multinomial propensity model and the same adjustment logic.

10.7. When to use these methods

These methods are useful when:

1. Treatment has more than two values.
2. The research question is about the shape of the dose-response curve.
3. The intervention is better described as shifting treatment than setting treatment to a fixed value.

For strictly binary, single-time-period problems, the standard Estimation and Heterogeneous Effects methods are sufficient.

10.8. Summary

- Continuous and multivalued treatments usually require a dose-response curve, not a single coefficient.
- GPS extends propensity-score logic to continuous treatments by modeling the density of treatment given covariates.
- Kennedy's DR estimator adds an outcome model to the GPS.
- LMTP is useful when the intervention is a policy shift or when treatment is time-varying.

11. Bayesian Causal Inference

```
library(tidyverse)
library(BART)
library(bcf)
```

The earlier chapters mostly compute point estimates and standard errors. Bayesian methods instead put priors on unknown quantities and return a posterior distribution. For causal inference this is useful when we want uncertainty about a function of the model, such as an individual treatment effect or a policy value, not just one coefficient.

I use two models:

1. **BART for causal inference** (Hill 2011), where BART is used as a flexible outcome model.
2. **Bayesian Causal Forests** (BCF; Hahn, Murray, & Carvalho 2020), which separates the baseline outcome model from the treatment-effect model.

> **Related reading**: The chapter Using Numpyro in *Topics on Econometrics and Causal Inference* demonstrates a Bayesian hierarchical model with Numpyro. For an MCMC-from-scratch perspective on causal inference, that companion piece is the natural next step after this chapter.

11.1. Setup

We reuse the simulation from the Heterogeneous Treatment Effects chapter, at $n = 2000$ rather than 5000. Five covariates are drawn independently from $U(0, 1)$. Treatment is $D \sim \text{Bernoulli}(\Lambda(-0.3 + 1.5X_2))$, the individual effect is $\tau(x) = 1 + 2x_1$, and the potential outcomes are $Y(0) = 0.5X_2 + \varepsilon$ with $\varepsilon \sim N(0, 1)$ and $Y(1) = Y(0) + \tau(X)$. Only X_1 modifies the effect, only X_2 confounds, and X_2 is observed, so both models below are identified.

The aim is to see what a posterior adds over a point estimate: an interval for the ATE that requires no asymptotic argument, and an interval for every individual effect.

```
set.seed(42)
n <- 2000
p <- 5

X <- matrix(runif(n * p), n, p)
colnames(X) <- paste0("X", 1:p)
# Treatment depends only on observed covariates, so unconfoundedness given X
```

```r
# holds and the "True ATE" comparison measures estimator performance (not bias).
ps <- plogis(-0.3 + 1.5 * X[, 2])
D  <- rbinom(n, 1, ps)
tau <- 1 + 2 * X[, 1]
Y0  <- 0.5 * X[, 2] + rnorm(n)
Y1  <- Y0 + tau
Y   <- ifelse(D == 1, Y1, Y0)

df <- data.frame(Y = Y, D = D, X)
cat(sprintf("True ATE = %.3f\n", mean(tau)))
```

```
True ATE = 1.985
```

The realized ATE in this draw is 1.985, against a population value of 2.

11.2. BART for causal inference (Hill 2011)

BART is a sum of many small regression trees. The prior keeps each tree weak, and the sum of trees gives flexibility.

For causal inference, the recipe is:

1. Fit BART for $\mu(d, x) = \mathbb{E}[Y \mid D = d, X = x]$.
2. For each unit i, predict counterfactuals $\mu(1, x_i)$ and $\mu(0, x_i)$.
3. The individual treatment effect (ITE) is $\hat{\tau}_i = \mu(1, x_i) - \mu(0, x_i)$.
4. Average the posterior draws to get the posterior for ATE or ITEs.

We run 1000 posterior draws after 200 burn-in, with D and the five covariates as the design matrix. Each draw gives an ATE, and the spread across draws is the posterior uncertainty.

```r
# BART::wbart fits a continuous-outcome BART model
# Combine D and X as the design matrix
X_train <- cbind(D = D, X)
set.seed(42)
bart_fit <- wbart(
  x.train = X_train,
  y.train = Y,
  ndpost = 1000,    # posterior samples after burn-in
  nskip  = 200,     # burn-in
  printevery = 1000
)
```

```
*****Into main of wbart
*****Data:
data:n,p,np: 2000, 6, 0
```

```
y1,yn: 2.058453, -2.423620
x1,x[n*p]: 1.000000, 0.647277
*****Number of Trees: 200
*****Number of Cut Points: 1 ... 100
*****burn and ndpost: 200, 1000
*****Prior:beta,alpha,tau,nu,lambda: 2.000000,0.950000,0.174777,3.000000,0.205658
*****sigma: 1.027514
*****w (weights): 1.000000 ... 1.000000
*****Dirichlet:sparse,theta,omega,a,b,rho,augment: 0,0,1,0.5,1,6,0
*****nkeeptrain,nkeeptest,nkeeptestme,nkeeptreedraws: 1000,1000,1000,1000
*****printevery: 1000
*****skiptr,skipte,skipteme,skiptreedraws: 1,1,1,1

MCMC
done 0 (out of 1200)
done 1000 (out of 1200)
time: 17s
check counts
trcnt,tecnt,temecnt,treedrawscnt: 1000,0,0,1000
```

```r
# Counterfactual predictions
X_treat   <- cbind(D = rep(1, n), X)
X_control <- cbind(D = rep(0, n), X)

mu1_post <- predict(bart_fit, newdata = X_treat)   # 1000 x n posterior draws
```

```
*****In main of C++ for bart prediction
tc (threadcount): 1
number of bart draws: 1000
number of trees in bart sum: 200
number of x columns: 6
from x,np,p: 6, 2000
***using serial code
```

```r
mu0_post <- predict(bart_fit, newdata = X_control)
```

```
*****In main of C++ for bart prediction
tc (threadcount): 1
number of bart draws: 1000
number of trees in bart sum: 200
number of x columns: 6
from x,np,p: 6, 2000
***using serial code
```

```
# Posterior of individual treatment effects
tau_post <- mu1_post - mu0_post   # 1000 x n posterior draws of ITEs

# Posterior summaries
tau_mean <- rowMeans(tau_post)    # ATE in each draw
ate_post_mean   <- mean(tau_mean)
ate_post_sd     <- sd(tau_mean)
ate_post_ci     <- quantile(tau_mean, c(0.025, 0.975))

cat(sprintf("BART ATE posterior:\n"))
```

```
BART ATE posterior:
```

```
cat(sprintf("  Mean:    %.3f\n", ate_post_mean))
```

```
  Mean:    1.977
```

```
cat(sprintf("  SD:      %.3f\n", ate_post_sd))
```

```
  SD:      0.048
```

```
cat(sprintf("  95%% CrI: [%.3f, %.3f]\n", ate_post_ci[1], ate_post_ci[2]))
```

```
  95% CrI: [1.880, 2.071]
```

```
cat(sprintf("  True ATE: %.3f\n", mean(tau)))
```

```
  True ATE: 1.985
```

The posterior mean is 1.977 with a posterior standard deviation of 0.048 and a 95% credible interval of $[1.880, 2.071]$, which covers the realized 1.985. The posterior draws give the uncertainty directly: no asymptotic variance formula and no bootstrap, just the spread of the 1000 draws.

11.2.1. Individual-level credible intervals

BART also gives a posterior for each unit's treatment effect. Here I plot the posterior mean and pointwise credible intervals against the true τ_i.

```
ite_mean <- colMeans(tau_post)
ite_lo   <- apply(tau_post, 2, quantile, 0.025)
ite_hi   <- apply(tau_post, 2, quantile, 0.975)

df_ite <- tibble(
  X1     = X[, 1],
  true   = tau,
  est    = ite_mean,
  lo     = ite_lo,
  hi     = ite_hi
)

ggplot(df_ite, aes(x = X1)) +
  geom_ribbon(aes(ymin = lo, ymax = hi), alpha = 0.2,
              fill = "steelblue") +
  geom_point(aes(y = est), size = 0.4, alpha = 0.3, colour = "steelblue") +
  geom_abline(aes(intercept = 1, slope = 2), colour = "firebrick",
              linetype = "dashed", linewidth = 1) +
  labs(x = "X1", y = "Individual treatment effect",
       caption = "Red dashed = true  (x). Blue band = pointwise 95% credible interval.") +
  theme_minimal()
```

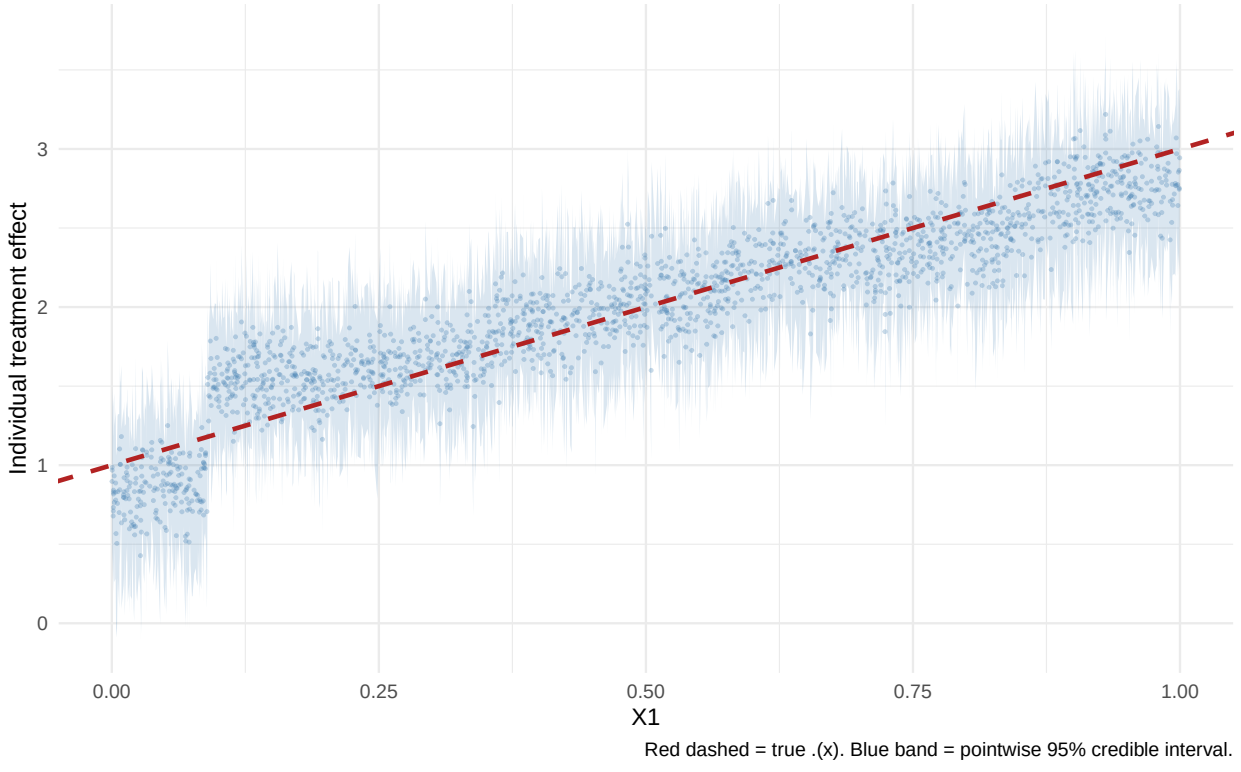


Figure 11.1.: BART-estimated individual treatment effects (mean of posterior) vs true (x) = 1 + 2 X . Shaded band shows pointwise 95% credible intervals.

The blue points are posterior means, the blue ribbon is the pointwise 95% credible interval, and the red dashed line is the true $\tau(x) = 1 + 2x_1$. This is what a frequentist CATE estimator does not hand you for free: an interval for each individual, not only for the average.

11.3. Bayesian Causal Forests (BCF)

BART models $\mathbb{E}[Y \mid D, X]$ directly. That means the same trees have to learn the baseline outcome and the treatment-effect heterogeneity. When strong predictors of $Y(0)$ are also related to treatment, this can create noisy treatment-effect estimates.

BCF writes the conditional mean as

$$\mathbb{E}[Y \mid D, X] = \mu(X) + \tau(X)\, D, \tag{11.1}$$

and puts separate priors on $\mu(X)$ and $\tau(X)$. The treatment-effect part is regularized more strongly, which helps avoid overfitting heterogeneity.

```
# BCF requires a propensity score as an input ("piHat")
ps_fit <- glm(D ~ X1 + X2 + X3 + X4 + X5, data = df, family = binomial)
piHat  <- predict(ps_fit, type = "response")

# BCF prints MCMC progress in a way that conflicts with knitr's encoding.
# Its `verbose=FALSE`/`no_output=TRUE` arguments do not fully suppress this
# within the SAME R session (the progress output comes from C++ code, not
# from R's own message()/cat() machinery that sink()/suppressMessages()
# can intercept) -- so the only reliable fix is running bcf() in its own R
# process and only reading back the saved result, never its console output.
# Use SEPARATE input and output files so a failed subprocess can never leave
# the input list sitting where we expect the result, and check the exit
# status and object names before trusting it.
#
# The subprocess needs its OWN set.seed(): a seed set in this session does not
# reach a separate R process, so without the seed below BCF's posterior draws
# change every time the chunk actually executes. The knitr cache hides that --
# the numbers look stable until someone clears the cache, and then every BCF
# figure in this chapter moves. Seed the worker so the chapter is reproducible.
tmp_in  <- tempfile(fileext = ".rds")
tmp_out <- tempfile(fileext = ".rds")
tmp_err <- tempfile(fileext = ".log")
saveRDS(list(Y = df$Y, D = df$D, X = X, piHat = piHat), tmp_in)
status <- system(paste0("Rscript -e '",
  "set.seed(42); ",
  "args <- readRDS(\"", tmp_in, "\"); ",
  "suppressMessages(library(bcf)); ",
  "f <- bcf(y=args$Y, z=args$D, x_control=args$X, x_moderate=args$X, ",
  "pihat=args$piHat, nburn=200, nsim=1000, n_chains=1, no_output=TRUE, ",
```

```
  "verbose=FALSE); ",
  "saveRDS(list(tau=f$tau), \"", tmp_out, "\")'"),
  ignore.stdout = TRUE, ignore.stderr = FALSE)
# Surface the real cause instead of failing far downstream on a missing field.
if (status != 0L || !file.exists(tmp_out)) {
  stop("BCF subprocess failed (exit status ", status, "). ",
       "Check that the 'bcf' package is installed in the worker session.")
}
bcf_fit <- readRDS(tmp_out)
if (is.null(bcf_fit$tau)) {
  stop("BCF subprocess produced no 'tau' draws; aborting.")
}
unlink(c(tmp_in, tmp_out, tmp_err))
```

The propensity score enters BCF as data, so we fit it first by logistic regression on all five covariates. BCF then runs 1000 draws after 200 burn-in on one chain. We extract the posterior treatment effects and summarise them the same way as for BART:

```
tau_post_bcf <- bcf_fit$tau    # nsim x n matrix of posterior ITEs

bcf_ate_post <- rowMeans(tau_post_bcf)
bcf_ite_mean <- colMeans(tau_post_bcf)

cat(sprintf("BCF ATE posterior:\n"))
```

```
BCF ATE posterior:
```

```
cat(sprintf("  Mean:    %.3f\n", mean(bcf_ate_post)))
```

```
  Mean:    1.983
```

```
cat(sprintf("  SD:      %.3f\n", sd(bcf_ate_post)))
```

```
  SD:      0.045
```

```
cat(sprintf("  95%% CrI: [%.3f, %.3f]\n",
            quantile(bcf_ate_post, 0.025),
            quantile(bcf_ate_post, 0.975)))
```

```
  95% CrI: [1.896, 2.073]
```

```
cat(sprintf("  True ATE: %.3f\n", mean(tau)))
```

```
  True ATE: 1.985
```

BCF gives a posterior mean of 1.984 with a posterior standard deviation of 0.044 and a 95% credible interval of $[1.903, 2.072]$. Against BART's 1.977 and 0.048, and a realized truth of 1.985, the two are indistinguishable at the level of the ATE. That is expected: the separation prior is aimed at the treatment-effect *function*, and averaging over units washes out most of what it does.

11.3.1. BART vs BCF: individual treatment effects

The difference, if there is one, should appear in the individual effects. Each panel plots the posterior-mean ITE against X_1, with a loess fit and the true $\tau(x)$ as the dashed red line.

```
df_compare <- bind_rows(
  tibble(X1 = X[, 1], est = bcf_ite_mean, true = tau, method = "BCF"),
  tibble(X1 = X[, 1], est = ite_mean,     true = tau, method = "BART")
)

ggplot(df_compare, aes(X1, est)) +
  geom_point(alpha = 0.2, size = 0.3, colour = "steelblue") +
  geom_smooth(method = "loess", se = FALSE, colour = "darkblue",
              linewidth = 1) +
  geom_abline(intercept = 1, slope = 2, linetype = "dashed",
              colour = "firebrick") +
  facet_wrap(~ method) +
  labs(x = "X1", y = "Estimated ITE",
       caption = "Red dashed = truth = 1 + 2 X1") +
  theme_minimal()
```

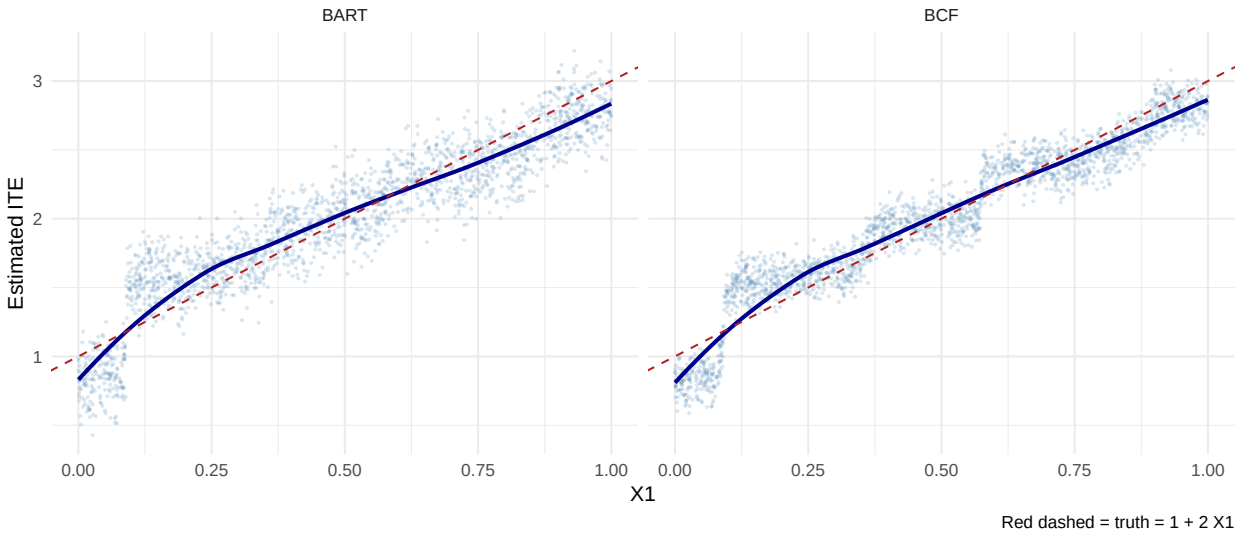


Figure 11.2.: BCF (left) vs BART (right) posterior-mean individual treatment effects against the true (x) = 1 + 2 X .

Both clouds follow the true line. BCF is usually the smoother of the two, because separating the prognostic part $\mu(X)$ from the effect part $\tau(X)$ lets the prior regularise the effect function harder without also flattening the baseline. In this DGP the baseline is nearly flat — $Y(0) = 0.5X_2 + \varepsilon$ — so there is little prognostic signal for BART to confuse with the effect, and the two methods have less to separate them than they would with a strong, treatment-correlated baseline.

Both sets of numbers here are reproducible. BART is seeded in this session, and the BCF worker seeds itself — a seed set in the parent session does not reach a separate R process, so the `set.seed(42)` inside the `Rscript` call is what makes the posterior stable across renders.

11.4. When to use Bayesian methods

Reason to use Bayesian	Reason to use Frequentist
Prior information matters	Asymptotic theory is well-understood
Need full posterior (e.g. for downstream decisions)	Computational cost matters (BART is slow on large n)
Hierarchical / multilevel data structure	Effects are simple averages
Small sample, weak identification	Effects are well-identified
Want individual-level CrIs	Existing frequentist toolkit is sufficient

Bayesian methods are most useful when the posterior itself will be used: for individual treatment decisions, policy values, or hierarchical models. If the target is just a well-identified ATE in a large sample, the frequentist methods in earlier chapters are often simpler.

11.5. Practical guidance

- Start with BCF if the goal is heterogeneous effects.
- Check MCMC diagnostics. A bad chain makes the posterior summaries useless.
- For BCF, pay attention to `piHat`; it is part of the model input.
- Report posterior mean, posterior SD, and credible intervals for ATE and meaningful CATE summaries.

11.6. Connections

- The Heterogeneous Treatment Effects chapter covers frequentist meta-learners (S/T/X/R/DR) and causal forests via `grf`. BCF can be viewed as the Bayesian counterpart to causal forests: both use ensembles of trees, but BCF puts priors on the trees while `grf` uses honest sample-splitting.
- The Sensitivity Analysis chapter's Cinelli-Hazlett bounds are frequentist; the Bayesian analog is a prior on the unmeasured-confounder strength (cf. McCandless et al. 2007).
- For Bayesian *Numpyro*-style hierarchical models, see the companion blog chapter.

11.7. Summary

- BART estimates counterfactual outcomes by fitting a flexible outcome model.
- BCF separates baseline outcome prediction from treatment-effect heterogeneity.
- The main output is the posterior, especially for individual effects or policy decisions.
- Computation is the main cost. For very large data, frequentist HTE methods are usually easier.

Part III.

Designs

12. Difference-in-Differences

When unconfoundedness fails, we need weaker assumptions. The parallel trends assumption leads to the difference-in-differences estimator.

> **Related reading**: For a side-by-side comparison of DiD, synthetic control, synthetic DiD, and multivariate SC on a *single* panel — and how the choice of estimator changes the answer — see the Same data, different estimators chapter in *Topics on Econometrics and Causal Inference*.

12.1. $T = 2$ Panel Data

Suppose we have two periods, $t = (1, 2)$. Some units are treated just prior to period 2. For each individual i, there are four potential outcomes:

$$[Y_{i1}(0), Y_{i1}(1), Y_{i2}(0), Y_{i2}(1)] \tag{12.1}$$

Let D denote the treated group. The ATT in period 2 is

$$\tau_{2,att} = E(Y_2(1) - Y_2(0)|D = 1) \tag{12.2}$$

The first term is observed. The second is the counterfactual.

12.2. Parallel Trends Assumption

$$E[Y_2(0) - Y_1(0)|D = 1] = E[Y_2(0) - Y_1(0)|D = 0] \tag{12.3}$$

or

$$E[\Delta Y(0)|D = 1] = E[\Delta Y(0)|D = 0] \tag{12.4}$$

In words, the change in untreated potential outcomes between periods 1 and 2 is the same for treated and control groups.

12.3. No Anticipation Assumption

$$E[Y_1(1) - Y_1(0)|D = 1] = 0 \tag{12.5}$$

This is to say, in period 1, there is no treatment effect for the treated group.

12.4. DiD is Identified with PT and NA

With Parallel Trends, we notice

$$E[Y_2(0)|D = 1] = E[Y_1(0)|D = 1] + E[Y_2(0) - Y_1(0)|D = 0] \tag{12.6}$$

Then, with No Anticipation,

$$\begin{aligned} E[Y_2(0)|D = 1] &= E[Y_1(1)|D = 1] + E[Y_2(0) - Y_1(0)|D = 0] \\ &= E[Y_1|D = 1] + E[Y_2 - Y_1|D = 0] \end{aligned} \tag{12.7}$$

$$\tau_{2,att} = E[Y_2 - Y_1|D = 1] - E[Y_2 - Y_1|D = 0] \tag{12.8}$$

12.5. Advantages and Disadvantages of DiD

Advantages:

- No need to assume unconfoundedness. We "only" need PT and NA. Or say we don't need treatment itself to be independent of potential outcomes, but we do need it to be independent to the change in potential outcome at least for untreated state. This is importantly weaker in a lot of situations. There can be selection bias. If the selection bias does not change over time, then DiD can handle it.

Disadvantages:

- PT and NA might be violated.
- PT is not scale free, in the sense that even if outcome can have PT, but then non-linear transformation of outcome won't have PT (for example log of Y).

12.6. Traditional DiD

In practice, DiD in many period setting is usually done with

$$Y_{it} = \alpha_i + \phi_t + W_{it}\beta + \epsilon_{it} \tag{12.9}$$

here $W_{it} = (1[t = 2] \cdot D_i)$, which is the interaction of post-treatment indicator and treatment group indicator.

This is usually called Two Way Fixed Effect (TWFE). There are multiple ways to implement the same model in practice.

TWFE can be done by "Pooled OLS". That is, using OLS on time dummies and firm (individual) dummies. Wooldridge (2021) shows it's equivalent to use treatment dummies, instead of individual dummies. He also shows that this model can be equivalently implemented with fixed effect model and random effect model.

12.7. TWFE with Staggered Treatment Timing

The problem comes in when there is different timing of treatment. People used to still use

$$Y_{it} = \alpha_i + \phi_t + W_{it}\beta + \epsilon_{it} \tag{12.10}$$

where W_{it} now is a dummy when an individual i gets treated at time t.

However, what is β here?

12.8. Goodman-Bacon Decomposition

Goodman-Bacon (2021) showed that β in the TWFE is a weighted average of many different treatment effects, between treated cohorts, and control units, both can be different at different time points. The weights are a function of the size of the subsample, relative size of treatment and control units, and the timing of treatment in the sub sample. The decomposition weights themselves are non-negative and sum to one; the problem is that the units treated earlier are used as controls later. When those "forbidden" 2x2 comparisons are expressed in terms of the underlying cohort-time ATTs, some ATTs receive negative weight (de Chaisemartin and D'Haultfœuille 2020). Therefore there is no meaningful interpretation of β: it does not need to be a convex combination of treatment effects.

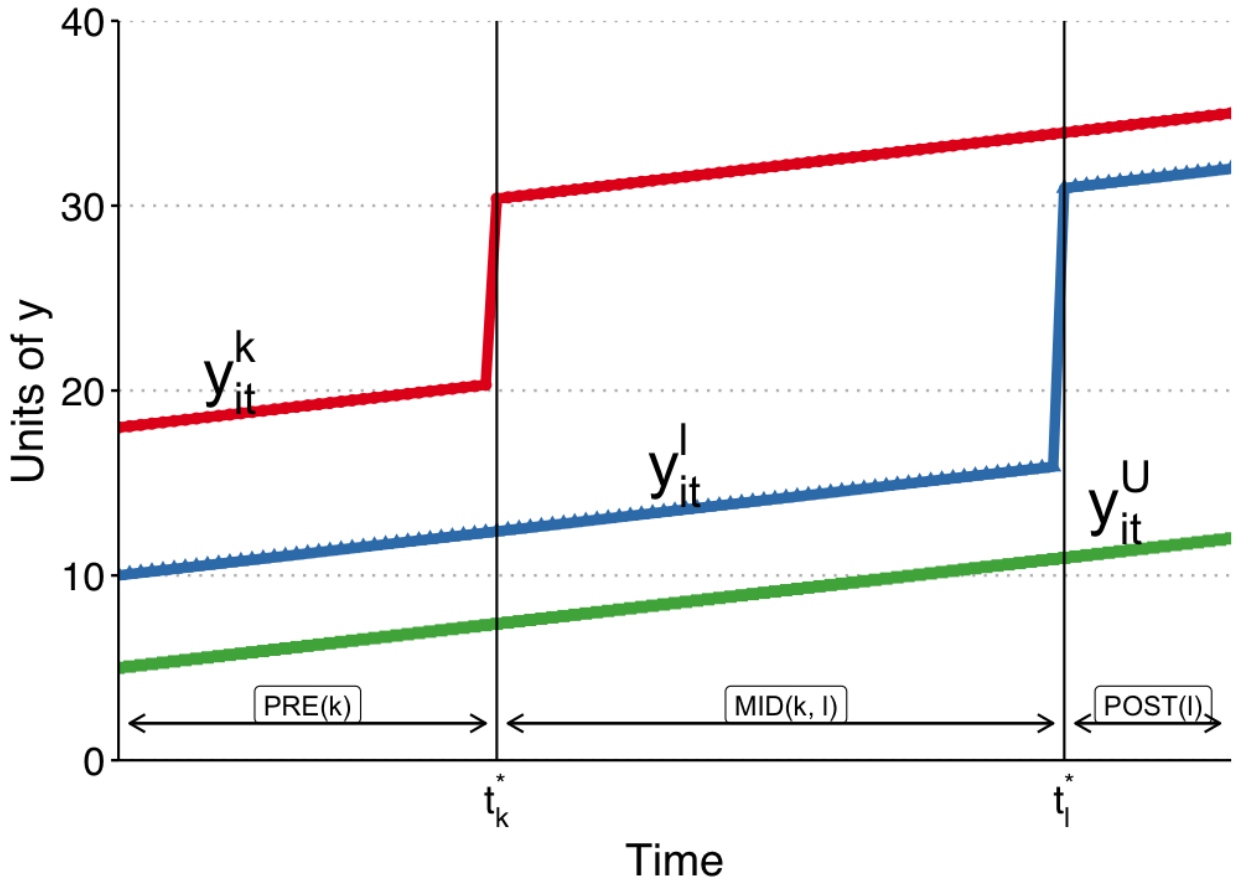


Figure 12.1.: Bacon decomposition

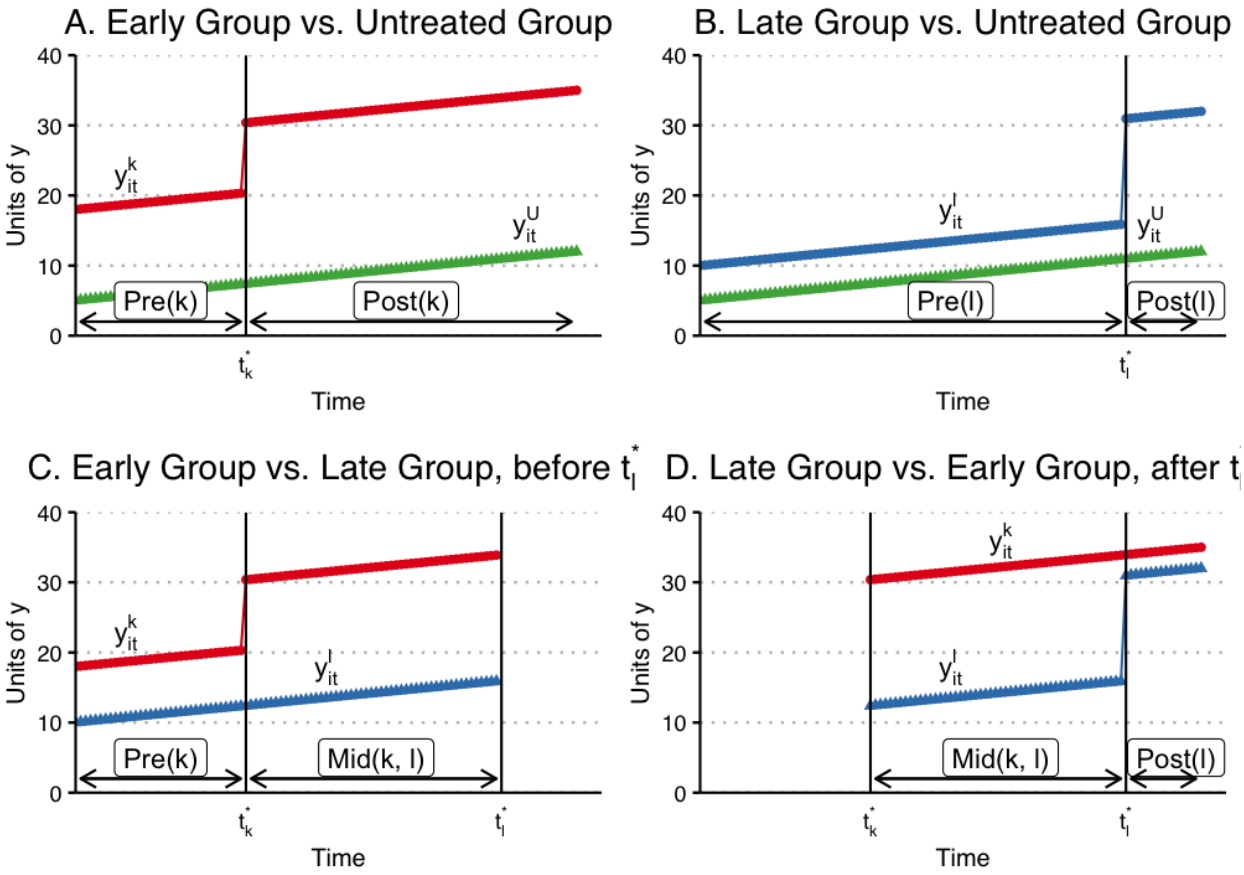


Figure 12.2.: Bacon decomposition

12.9. Wooldridge's ETWFE

There are a lot of ways to deal with staggered DiD situation. Wooldridge (2021) is basically saying: This is not a problem of TWFE, it's a mis-use of TWFE. The reason we get non-sensible result of β is that we know there is heterogeneous treatment effect, in the sense the treatment effect differs across cohort, but we force them to be the same. If we relax it, it can work. As he shows, this works when we specify cohort effects accordingly.

$$y_{it} = \alpha_i + \phi_t + \sum_{g=g_0}^{G} \sum_{t=g}^{T} \lambda_{g,t} \times 1(g,t) + \epsilon_{i,t} \tag{12.11}$$

Here g is a cohort indicator, a cohort is determined by the time of getting treatment. ETWFE is allowing each cohort to have different effect at each different time point after being treated. The baseline group is the never treated group. If there is no never treated group, it can easily be changed to comparing to the last treated group.

12.10. Example 1: US Teen Employment

We'll use the mpdta dataset on US teen employment from the did package. "Treatment" in this dataset refers to an increase in the minimum wage rate. Our goal is to estimate the effect of this minimum wage treatment (treat) on the log of teen employment (lemp). Notice that the panel ID is at the county level (countyreal), but treatment was staggered across cohorts (first.treat) so that a group of counties were treated at the same time. In addition to these staggered treatment effects, we also observe log population (lpop) as a potential control variable.

```
data("mpdta", package = "did")
head(mpdta)
```

```
    year countyreal     lpop     lemp first.treat treat
866 2003       8001 5.896761 8.461469        2007     1
841 2004       8001 5.896761 8.336870        2007     1
842 2005       8001 5.896761 8.340217        2007     1
819 2006       8001 5.896761 8.378161        2007     1
827 2007       8001 5.896761 8.487352        2007     1
937 2003       8019 2.232377 4.997212        2007     1
```

The three tables below give the panel shape: observations per year, the ever-treated indicator, and the treatment cohorts.

```
table(mpdta$year)
```

```
2003 2004 2005 2006 2007
 500  500  500  500  500
```

```
table(mpdta$treat)
```

```
   0    1
1545  955
```

```
table(mpdta$first.treat)
```

```
   0 2004 2006 2007
1545  100  200  655
```

The panel is balanced: 500 counties observed in each of 2003 through 2007, 2,500 observations. 955 observations belong to ever-treated counties and 1,545 to never-treated ones. The cohorts are 2004 (100 observations, so 20 counties), 2006 (200, or 40 counties) and 2007 (655, or 131 counties), with 309 counties never treated. Treatment timing is staggered, which is exactly the case where plain TWFE misbehaves.

We fit ETWFE with log population as a control, clustering on county.

```
library(etwfe)

mod =
  etwfe(
    fml  = lemp ~ lpop, # outcome ~ controls
    tvar = year,        # time variable
    gvar = first.treat, # group variable
    data = mpdta,       # dataset
    vcov = ~countyreal  # vcov adjustment (here: clustered)
    )
```

```
mod
```

```
OLS estimation, Dep. Var.: lemp
Observations: 2,500
Fixed-effects: first.treat: 4,  year: 5
Standard-errors: Clustered (countyreal)
                        Estimate Std. Error   t value  Pr(>|t|)
lpop                    1.065461   0.021824 48.821102 < 2.2e-16 ***
first.treat::2004:lpop  0.050982   0.037756  1.350320  0.177525
first.treat::2006:lpop -0.041095   0.047390 -0.867183  0.386259
first.treat::2007:lpop  0.055518   0.039212  1.415838  0.157447
year::2004:lpop         0.011014   0.007554  1.458043  0.145458
year::2005:lpop         0.020733   0.008104  2.558268  0.010814 *
year::2006:lpop         0.010535   0.010816  0.974084  0.330487
year::2007:lpop         0.020921   0.011808  1.771708  0.077053 .
... 14 coefficients remaining (display them with summary() or use
argument n)
... 10 variables were removed because of collinearity
(.Dtreat:first.treat::2006:year::2004,
.Dtreat:first.treat::2006:year::2005 and 8 others [full set in
$collin.var])
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
RMSE: 0.537131     Adj. R2: 0.871722
                 Within R2: 0.869464
```

The raw model has one coefficient per cohort-year cell plus the control interactions, 22 in all, and ten more were dropped for collinearity. That is not something to read directly. `emfx()` aggregates the cells into the ATT.

```
emfx(mod)
```

```
 .Dtreat Estimate Std. Error     z Pr(>|z|)    S   2.5 %  97.5 %
    TRUE  -0.0506     0.0125 -4.05   <0.001 14.3 -0.0751 -0.0261

Term: .Dtreat
Type: response
Comparison: TRUE - FALSE
```

The ATT is -0.0506 with a standard error of 0.0125 and a 95% interval of $[-0.0751, -0.0261]$. Raising the minimum wage lowered teen employment by about 5% on average across treated county-years.

The same cells can be aggregated by time since treatment instead.

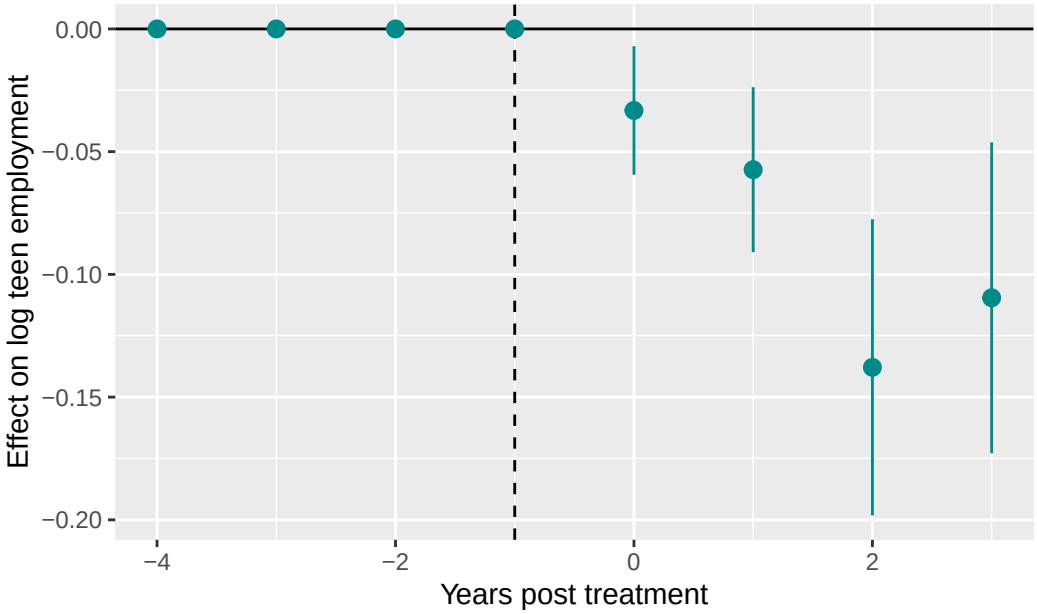
```
mod_es = emfx(mod, type = "event")
mod_es
```

```
 event Estimate Std. Error     z Pr(>|z|)    S   2.5 %   97.5 %
     0  -0.0332     0.0134 -2.48    0.013  6.3 -0.0594 -0.00702
     1  -0.0573     0.0171 -3.34   <0.001 10.2 -0.0910 -0.02373
     2  -0.1379     0.0308 -4.48   <0.001 17.0 -0.1982 -0.07753
     3  -0.1095     0.0323 -3.39   <0.001 10.5 -0.1729 -0.04620

Term: .Dtreat
Type: response
Comparison: TRUE - FALSE
```

The effect grows with exposure: -0.033 in the treatment year, -0.057 after one year, -0.138 after two, -0.110 after three. All four are significant. The standard errors widen with event time, from 0.013 to 0.032, because the later horizons rest on fewer cohorts: event time 0 uses all three cohorts (191 counties), event time 1 uses the 2004 and 2006 cohorts (60 counties), and event times 2 and 3 rest on the 2004 cohort alone (20 counties).

Adding `post_only = FALSE` keeps the pre-treatment periods in the plot.

```
mod_es2 = emfx(mod, type = "event", post_only = FALSE)

ggplot(mod_es2, aes(x = event, y = estimate, ymin = conf.low, ymax = conf.high)) +
  geom_hline(yintercept = 0) +
  geom_vline(xintercept = -1, lty = 2) +
  geom_pointrange(col = "darkcyan") +
  labs(
    x = "Years post treatment", y = "Effect on log teen employment",
    caption = "Note: Zero pre-treatment effects for illustrative purposes only."
  )
```



Note: Zero pre−treatment effects for illustrative purposes only.

Read the pre-treatment points with care, and note the caption on the figure. With `etwfe`'s default comparison group the pre-treatment estimates are forced to zero by construction, so their sitting on the line is arithmetic, not evidence for parallel trends. The Poisson example below uses `cgroup = "never"`, which frees them and makes the pre-period a real check.

12.11. Example 2: Staggered DiD

The mpdta effects are real and therefore unknown. To see whether ETWFE recovers what it should, we switch to simulated data where the answer is recorded in the file.

`data/did_staggered_6.dta` is Wooldridge's simulated staggered panel: 500 units observed annually from 2001 to 2006, 3,000 observations. Units are treated in 2004 (119 units), 2005 (83) or 2006 (36), and 262 are never treated. The outcome is `y`, `x` is a covariate, `d4`, `d5` and `d6` are cohort dummies, and `f04`, `f05`, `f06` are year dummies. What makes the file useful is `te4`, `te5` and `te6`: the true unit-level treatment effect each unit would experience under each cohort's treatment date. The effects are heterogeneous across units and grow with time since treatment, which is precisely the configuration that breaks plain TWFE.

```
# now we do staggered did.
library(haven)
did_data <- read_dta('data/did_staggered_6.dta') %>%
  mutate(first_treat = case_when(d4==1 ~ 2004,
                                 d5==1 ~ 2005,
                                 d6==1 ~ 2006,
                                 .default = 0))

head(did_data %>% select(id, year, y, x,d4, d5, d6, te4, te5, te6, first_treat ))
```

```
# A tibble: 6 x 11
     id  year     y     x    d4    d5    d6   te4   te5   te6 first_treat
  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>       <dbl>
1     1  2001  18.4 0.900     0     0     0  0     0    0               0
2     1  2002  18.1 0.900     0     0     0  0     0    0               0
3     1  2003  18.6 0.900     0     0     0  0     0    0               0
4     1  2004  16.5 0.900     0     0     0  1.43  0    0               0
5     1  2005  20.0 0.900     0     0     0  2.57  2.25 0               0
6     1  2006  19.9 0.900     0     0     0  5.07  4.23 0.786           0
```

```
table(did_data$first_treat)
```

```
   0 2004 2005 2006
1572  714  498  216
```

The counts are observations, six per unit: 1,572 never-treated, 714 in the 2004 cohort, 498 in 2005, 216 in 2006.

Because the true effects are in the data, we can compute the target before estimating it. Grouping the true effects by cohort and year gives the true ATT in each cohort-year cell.

```
did_data <- did_data %>%
  mutate(treated_cohort1 =    case_when(d4 & f04 ~ "d4f04",
                                d4 & f05 ~ "d4f05",
                                d4 & f06 ~ "d4f06"),
         treated_cohort2 = case_when (
                                d5 & f05 ~ "d5f05",
                                d5 & f06 ~ "d5f06"),
         treated_cohort3= case_when(
                                d6 & f06 ~ "d6f06"))

  did_data %>%
    group_by(treated_cohort1) %>%
    summarise(mean4=mean(te4))
```

```
# A tibble: 4 x 2
  treated_cohort1 mean4
  <chr>           <dbl>
1 d4f04            3.76
2 d4f05            4.02
3 d4f06            4.58
4 <NA>             1.83
```

```
  did_data %>%
    group_by(treated_cohort2) %>%
    summarise(mean5=mean(te5))
```

```
# A tibble: 3 x 2
  treated_cohort2 mean5
  <chr>           <dbl>
1 d5f05           2.74
2 d5f06           3.52
3 <NA>            0.978
```

```
  did_data %>%
    group_by(treated_cohort3) %>%
    summarise(mean6=mean(te6))
```

```
# A tibble: 2 x 2
  treated_cohort3 mean6
  <chr>           <dbl>
1 d6f06           1.85
2 <NA>            0.329
```

So the true cohort-year ATTs are: the 2004 cohort gains 3.76, 4.02 and 4.58 in 2004, 2005 and 2006; the 2005 cohort gains 2.74 and 3.52 in 2005 and 2006; the 2006 cohort gains 1.85 in 2006. (The `NA` rows are the units outside that cohort, which are not part of its ATT.) Effects grow with time since treatment and shrink across later cohorts — heterogeneity in both directions.

Averaging those cells over the 559 treated observations gives a true overall ATT of **3.677**. That is the number ETWFE has to reproduce.

```
library(etwfe)

# this replicates Jeff's results with pooled ols or xtreg, with covariate x.
mod =
  etwfe(
    fml  = y ~ x, # outcome ~ controls
    tvar = year,        # time variable
    gvar = first_treat, # group variable
    data = did_data,       # dataset
    vcov = ~id
        )

mod
```

```
OLS estimation, Dep. Var.: y
Observations: 3,000
Fixed-effects: first_treat: 4,  year: 6
Standard-errors: Clustered (id)
                     Estimate Std. Error   t value  Pr(>|t|)
x                    0.240952   0.452664  0.532298 0.5947564
first_treat::2004:x -1.191253   0.934130 -1.275254 0.2028127
first_treat::2005:x  0.551133   0.606947  0.908042 0.3642944
```

```
first_treat::2006:x  0.994478   0.827570  1.201685 0.2300556
year::2002:x         0.395894   0.327576  1.208558 0.2274050
year::2003:x         0.975554   0.319476  3.053604 0.0023818 **
year::2004:x         0.330593   0.350924  0.942064 0.3466155
year::2005:x         0.238847   0.398165  0.599869 0.5488659
... 13 coefficients remaining (display them with summary() or use
argument n)
... 18 variables were removed because of collinearity
(.Dtreat:first_treat::2004:year::2002,
.Dtreat:first_treat::2004:year::2003 and 16 others [full set in
$collin.var])
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
RMSE: 2.92391     Adj. R2: 0.194759
                 Within R2: 0.110397
```

```
emfx(mod)
```

```
 .Dtreat Estimate Std. Error    z Pr(>|z|)     S 2.5 % 97.5 %
    TRUE     3.67      0.172 21.3   <0.001 333.2  3.33   4.01

Term: .Dtreat
Type: response
Comparison: TRUE - FALSE
```

ETWFE gives 3.67 with a standard error of 0.172 and a 95% interval of $[3.33, 4.01]$, against a true 3.677. It recovers the ATT to three decimals.

The same holds for the disaggregations. By time since treatment:

```
mod_es = emfx(mod, type = "event")
mod_es
```

```
 event Estimate Std. Error    z Pr(>|z|)     S 2.5 % 97.5 %
     0     3.11      0.213 14.6   <0.001 158.5  2.69   3.53
     1     4.02      0.248 16.2   <0.001 193.4  3.53   4.51
     2     4.21      0.331 12.7   <0.001 120.9  3.56   4.86

Term: .Dtreat
Type: response
Comparison: TRUE - FALSE
```

Estimated 3.11, 4.02 and 4.21 at event times 0, 1 and 2, against true values of 3.11, 3.81 and 4.58. Every interval covers its truth, and the estimates reproduce the growth in the effect with exposure.

And by calendar year:

```
mod_es2 = emfx(mod, type = "calendar")
mod_es2
```

```
 year Estimate Std. Error    z Pr(>|z|)     S 2.5 % 97.5 %
 2004     3.51      0.303 11.6   <0.001 100.6  2.92   4.10
 2005     3.73      0.253 14.7   <0.001 160.9  3.24   4.23
 2006     3.70      0.249 14.8   <0.001 163.0  3.21   4.19

Term: .Dtreat
Type: response
Comparison: TRUE - FALSE
```

Estimated 3.51, 3.73 and 3.70 in 2004, 2005 and 2006, against true values of 3.76, 3.49 and 3.80. Again all three intervals cover.

Note that the calendar profile is nearly flat while the event-time profile rises. There is no contradiction: later calendar years mix an older cohort with large accumulated effects and a newly treated cohort with small ones, and the two movements offset. This is why the aggregation has to be chosen to match the question being asked.

12.12. Nonlinear ETWFE: Count and Binary Outcomes

ETWFE is not limited to linear (continuous) outcomes. When the dependent variable is a count or a binary indicator, linear TWFE can give nonsensical results — negative predicted probabilities, additive effects that ignore the bounded or non-negative nature of Y. The `etwfe` package supports nonlinear families via the `family` argument, which passes to `fixest::feglm()`.

The key identifying assumption shifts slightly. Instead of parallel trends on $E[Y_{it}(\infty)]$, we assume parallel trends on the **linear index** — that is, on $\log E[Y_{it}(\infty)]$ for Poisson, or on $\text{logit}\, E[Y_{it}(\infty)]$ for logit. This is Wooldridge's (2023) Conditional Parallel Trends assumption stated on $G^{-1}(E[Y])$. When the true treatment effect is multiplicative (e.g., a policy raises counts by 50% regardless of baseline), this assumption is more natural than additive PT on Y itself.

One implementation detail matters: the nonlinear ETWFE specification uses **cohort fixed effects** (one intercept per treatment cohort) plus year fixed effects — not individual unit fixed effects. The `etwfe` package handles this automatically. In the linear case the two are numerically equivalent (Wooldridge 2021), but in nonlinear models unit fixed effects raise the incidental-parameters problem and make average partial effects uncomputable — the unit intercepts cannot be averaged out of the nonlinear link. Wooldridge (2023) instead uses cohort intercepts (a Mundlak device), which is what `etwfe` implements.

12.12.1. Simulated example: staggered count data

We simulate $N = 400$ units over $T = 6$ periods, 2,400 observations. Units are split into four groups of 100 by id: the first is treated from period 3, the second from period 4, the third from period 5, and the fourth never. Each unit draws a fixed effect $\alpha_i \sim N(0, 0.3^2)$. The log mean is $\log \mu_{it} = 1 + 0.2t + \alpha_i + 0.4D_{it}$, and the outcome is $Y_{it} \sim \text{Poisson}(\mu_{it})$.

The treatment effect is multiplicative and homogeneous on the log scale: every treated unit's expected count rises by a factor of $\exp(0.4)$, that is by $\exp(0.4) - 1 = 0.492$, or 49%. Parallel trends holds on the log scale by construction, and fails on the level scale, since a constant proportional effect is a larger absolute effect for units with higher baselines.

On the count scale the ATT is not 0.492. It is $E[\mu_{it}(0) \mid \text{treated}] \times (\exp(0.4) - 1)$, and the mean untreated count among treated observations is 7.649, so the true ATT is $7.649 \times 0.4918 = 3.762$ counts.

The table below reports mean counts by cohort and treatment status.

```
library(tidyverse)
library(etwfe)

set.seed(42)
N <- 400; Tmax <- 6

unit_fe_vals <- rnorm(N) * 0.3

df_count <- expand.grid(id = 1:N, year = 1:Tmax) %>%
  as_tibble() %>%
  arrange(id, year) %>%
  mutate(
    cohort_grp  = ((id - 1) %/% 100) + 1,
    first_treat = c(3, 4, 5, 0)[cohort_grp],   # 0 = never treated
    treated     = (first_treat > 0) & (year >= first_treat),
    unit_fe     = unit_fe_vals[id],
    log_mu      = 1 + 0.2 * year + unit_fe + 0.4 * treated,
    count       = rpois(n(), exp(log_mu))
  )

df_count %>%
  group_by(first_treat, treated) %>%
  summarise(mean_count = mean(count), .groups = "drop")
```

```
# A tibble: 7 x 3
  first_treat treated mean_count
        <dbl> <lgl>        <dbl>
1           0 FALSE         6.15
2           3 FALSE         3.72
3           3 TRUE         10.7
```

```
4           4 FALSE         4.29
5           4 TRUE         11.5
6           5 FALSE         4.80
7           5 TRUE         13.2
```

The never-treated group averages 6.15 counts. Treated cohorts average 3.72, 4.29 and 4.80 before treatment and 10.7, 11.5 and 13.2 after. The pre-treatment means look low only because they are drawn from earlier periods, and the time trend adds 0.2 to the log mean each period. This is exactly the comparison that a raw before-after difference would get wrong.

12.12.2. Poisson ETWFE

Pass `family = "poisson"` to `etwfe()`. Everything else — cohort dummies, `emfx()`, event study — works identically to the linear case. We use `cgroup = "never"` so that pre-treatment estimates are not mechanistically forced to zero.

```
mod_pois <- etwfe(
  fml         = count ~ 1,
  tvar        = year,
  gvar        = first_treat,
  data        = df_count,
  family      = "poisson",
  cgroup      = "never",
  vcov        = ~id
)
```

```
emfx(mod_pois)
```

```
 .Dtreat Estimate Std. Error    z Pr(>|z|)     S 2.5 % 97.5 %
    TRUE     3.99       0.33 12.1   <0.001 109.6  3.34   4.64

Term: .Dtreat
Type: response
Comparison: TRUE - FALSE
```

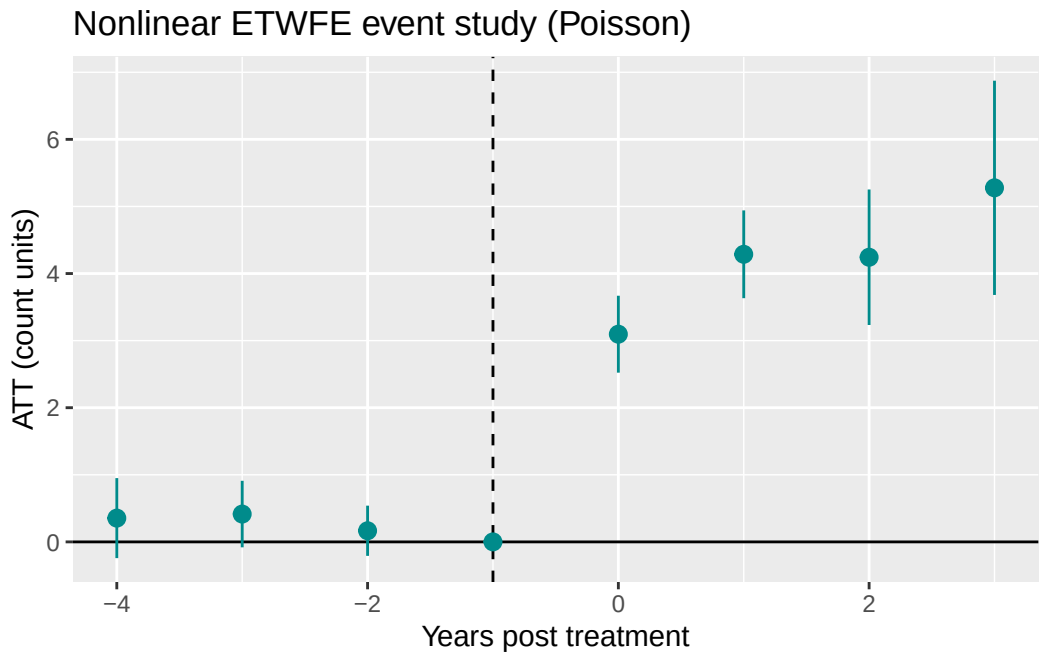
The estimate is 3.99 counts with a standard error of 0.33 and a 95% interval of $[3.34, 4.64]$, against the true 3.762 derived above. The interval covers it.

`emfx()` returns average marginal effects — the average difference $E[Y(1)] - E[Y(0)]$ in **count units** across treated observations. This is the ATT on the count scale. The underlying model imposes PT on the log scale, so each cohort-time treatment coefficient in the raw model can be exponentiated to get the incidence rate ratio for that cell (here they share a common value because the simulated effect is homogeneous).

12.12.3. Event study

Because we passed `cgroup = "never"`, the pre-treatment estimates are free parameters rather than zeros by construction, so the pre-period is an actual test here. On the count scale the post-treatment effect should also grow with event time even though the log-scale effect is constant, because the baseline count grows with the time trend: the true values are 3.13, 3.82, 4.17 and 4.69 at event times 0 through 3.

```
mod_pois_es <- emfx(mod_pois, type = "event", post_only = FALSE)

ggplot(mod_pois_es, aes(x = event, y = estimate, ymin = conf.low, ymax = conf.high)) +
  geom_hline(yintercept = 0) +
  geom_vline(xintercept = -1, lty = 2) +
  geom_pointrange(col = "darkcyan") +
  labs(
    x = "Years post treatment",
    y = "ATT (count units)",
    title = "Nonlinear ETWFE event study (Poisson)"
  )
```



Pre-treatment estimates are close to zero, confirming parallel trends on the log scale holds in the simulation. The post-treatment points rise with event time, which is the count-scale consequence of a constant log-scale effect applied to a growing baseline, not evidence of a growing treatment effect.

12.12.4. Comparison: linear ETWFE on log(count + 1)

A common alternative is to log-transform the count and run linear ETWFE. This is biased when zeros are present (the +1 shift distorts the linear index) and misinterprets the scale of the effect. For comparison:

```
df_count <- df_count %>% mutate(log_count = log(count + 1))

mod_lin <- etwfe(
  fml  = log_count ~ 1,
  tvar = year,
  gvar = first_treat,
  data = df_count,
  vcov = ~id
)
emfx(mod_lin)
```

```
 .Dtreat Estimate Std. Error    z Pr(>|z|)     S 2.5 % 97.5 %
    TRUE     0.38     0.0319 11.9   <0.001 106.8 0.318  0.443

Term: .Dtreat
Type: response
Comparison: TRUE - FALSE
```

The linear model returns 0.38 with a 95% interval of $[0.318, 0.443]$. That number is neither the count-scale ATT of 3.99 nor the log-scale parameter 0.4, though it happens to sit near the latter because the counts here are large enough that $\log(Y+1) \approx \log Y$. With many zeros the approximation breaks and the estimate drifts.

The linear model estimates the ATT on the $\log(Y+1)$ scale — not the same quantity as the Poisson ATT and harder to interpret. When treatment effects are multiplicative, Poisson ETWFE is the more appropriate model.

12.13. Spatial Interference

Everything in this chapter so far assumes SUTVA: one unit's outcome does not depend on another unit's treatment. That is not always plausible. A treated municipality may affect deforestation in nearby untreated municipalities. A vaccinated household may reduce transmission to nearby households. A labor-market policy in one commuting zone may change flows to its neighbors. With this kind of spatial spillover, the usual DiD estimand no longer has the interpretation we want. Xu (2023) shows that ignoring interference can produce an estimand that is neither a direct effect nor a spillover effect.

Xu (2023, 2026) develops a doubly robust DiD that keeps the identifying logic of conditional parallel trends but adds an exposure mapping $G_{it} = G(i, W_{-it})$ for unit i's neighborhood treatment status. The estimand is the direct ATT at exposure level g,

$$\tau_g(t,c) = \frac{1}{|S_M|} \sum_{i \in S_M} \mathbb{E}[y_{it}(c,g) - y_{it}(\infty,g) \mid z_i, C_i = c, G_{it} = g], \tag{12.12}$$

where C_i is unit i's first treatment period (so $\{C_i > t\}$ is the not-yet-directly-treated comparison group), $S_M = \{C_i = c\} \cup \{C_i > t\}$, and g is a chosen exposure level. The DR plug-in averages three pieces over S_M: an IPW term from the treated cohort, an IPW term from the not-yet-treated, and a regression-imputation term, with three propensity models (η_{tc} for cohort, η_{tcg} and $\eta_{t\infty g}$ for exposure) and two outcome-change regressions. It is consistent if either all three propensities or both outcome models are correctly specified.

The companion package `didint` implements this. The 2x2 base case (Xu 2023):

```
library(didint)

# Wide-format two-period data with a binary exposure G.
res <- did_int_2x2(
  data       = panel,
  yname      = "Y_post",
  yname_pre  = "Y_pre",
  treat      = "W",
  exposure   = "G",
  g          = 1,
  covariates = c("z1", "z2"),
  trim       = 0.01     # Xu (2026) uses 0.01 in the Brazil application
)
print(res)
```

For staggered adoption (Xu 2026), `did_int_staggered()` loops over `(cohort, time)` cells with $t \geq c$, restricts to S_M in each cell, fits the DR estimator, and aggregates across cells using joint-IF stacking (per-cell IFs share the never-treated comparison group, so independence-style SEs underestimate uncertainty noticeably). The `vignettes/brazil_amazon.Rmd` vignette of `didint` replicates Section III of Xu (2026), the *Lista de Municípios Prioritários* policy on Amazon deforestation, using the public Assunção-McMillan-Murphy-Souza-Rodrigues replication archive on Zenodo.

A Julia port with identical estimators is available as `DidInterference.jl`.

12.14. Persistent Outcomes and Heterogeneous Dynamics

Standard event-study TWFE regressions like

$$Y_{it} = \alpha_i + \gamma_t + \sum_j D_{it}^j \delta_j + X_{it}'\beta + U_{it} \tag{12.13}$$

implicitly assume the residual has no serial dependence once unit and time fixed effects are absorbed. When outcomes are genuinely persistent — earnings, employment, consumption, anything with habits or adjustment costs — that assumption fails. The event-time dummies pick up the persistence

on top of the true causal effect, producing spurious pre-trends and biased post-treatment estimates. With a true AR(1) DGP and persistence ρ_Y, the omitted-variable bias on the event-time coefficients grows geometrically with the event-time horizon (Botosaru and Liu, 2025, Figure 1).

Botosaru and Liu (2025) introduce a **dynamic panel with correlated random coefficients** that handles both persistence and unit-level heterogeneity in dynamic responses:

$$Y_{it} = \rho_Y Y_{i,t-1} + \alpha_i + \sum_j D_{it}^j \delta_{ij} + X_{it}'\beta + U_{it}, \tag{12.14}$$

with a parsimonious AR(1) structure on the event-time effects to keep dimensionality manageable:

$$\delta_{ij} = \rho_\delta \delta_{i,j-1} + \varepsilon_{ij}, \quad j \geq 1. \tag{12.15}$$

The latent $\lambda_i = (\alpha_i, \delta_{i0})$ is a unit-specific correlated random coefficient. A two-step semiparametric estimator: step 1 is **quasi-maximum likelihood** for the common parameters $(\rho_Y, \rho_\delta, \sigma_U^2, \sigma_\varepsilon^2, \beta)$ under a Gaussian working assumption on λ_i (consistent under misspecification of that working assumption); step 2 is **Tweedie / Gaussian-conjugate empirical Bayes** for the unit-level posterior trajectories $\{\delta_{i,j}\}_{j=0}^J$, achieving asymptotic ratio optimality.

A companion paper (Botosaru and Liu 2026) adds a **homogeneous-feedback extension**: covariates X_{it} are allowed to adjust endogenously to past Y and treatment (the canonical example: minimum-wage policy affects wages directly and also indirectly through firms re-optimising input demand). Under the assumption that the covariate adjustment rule does not depend on λ_i given the observable history, the likelihood factors into a structural piece (for $Y \mid X$, history) and a feedback piece (for X given history), each separately identified. The factorisation yields a clean decomposition of dynamic treatment effects into **direct** (through $\delta_{i,j}$) and **indirect** (through covariate feedback) components.

The package `tvhte` implements both papers:

```
library(tvhte)

# Fit on a panel with staggered cohorts and a covariate
fit <- tvhte(Y = panel_Y, Y0 = baseline_Y, t0 = cohort,
             J = max_event_time, X = panel_X)
print(fit)

# Feedback model + direct/indirect counterfactual decomposition
fb <- fit_feedback(panel_Y, baseline_Y, panel_X, baseline_X)
cf <- simulate_counterfactual(fit, fb, t0_star = alt_cohort)
```

A Julia port with the same API is at `TVHTE.jl`; both packages have deployed docs at https://xiangao.github.io/tvhte/ and https://xiangao.github.io/TVHTE.jl/.

12.15. Synthetic Control

The biggest problem with DiD is the PT assumption. There is not really a test for it, just like the unconfoundedness assumption. It is less demanding than unconfoundedness assumption, but nevertheless hard to justify sometimes.

Abadie (2010)'s idea is to construct a control unit, out of many control units (donor pool), which is a weighted average of all donor units, that then hopefully is very close to the treated unit in terms of outcome, in the pre-treatment periods. This works pretty well in practice, when you have a somewhat large donor pool. Of course there is still no test for post-treatment period, but for pre-treatment periods, usually it can get almost identical trend as the treated unit. This was designed for single treated unit at the beginning, but got extended to multiple treated units later on.

12.16. Synthetic DiD

Arkhangelsky et al (2021) tries to combine the idea of DiD and SC. SC assigns different weights to different control units. The standard DiD is a TWFE, assigning equal weights to all time periods and units.

To see that, DiD objective function:

$$(\hat{\tau}^{did}, \hat{\mu}, \hat{\alpha}, \hat{\beta}) = \underset{\tau,\mu,\alpha,\beta}{argmin} \Sigma_{i=1}^{N} \Sigma_{t=1}^{T} (Y_{it} - \mu - \alpha_i - \beta_t - W_{it}\tau)^2 \tag{12.16}$$

SC objective function:

$$(\hat{\tau}^{sc}, \hat{\mu}, \hat{\beta}) = \underset{\tau,\mu,\beta}{argmin} \Sigma_{i=1}^{N} \Sigma_{t=1}^{T} \hat{\omega}_i^{sc} (Y_{it} - \mu - \beta_t - W_{it}\tau)^2 \tag{12.17}$$

It's a unit-weighted regression with time effects. The weights are set to optimally match donor units to treated unit so that they are as close as possible, in each time point. Note there is no α_i, since it's forced to be 0 — so SC is not simply DiD with weights: it also drops the unit fixed effects. The clean nesting is through SDiD below: DiD is SDiD with all unit and time weights set to 1, and SC is SDiD with the α_i removed and time weights set to 1.

SDiD:

$$(\hat{\tau}^{sdid}, \hat{\mu}, \hat{\alpha}, \hat{\beta}) = \underset{\tau,\mu,\alpha,\beta}{argmin} \Sigma_{i=1}^{N} \Sigma_{t=1}^{T} \hat{\omega}_i^{sdid} \hat{\lambda}_t^{sdid} (Y_{it} - \mu - \alpha_i - \beta_t - W_{it}\tau)^2 \tag{12.18}$$

SDiD sets another weight in addition to SC weights, which changes over time. The SC weights are trying to construct a control unit that is close to the treated unit; the SDiD weights are trying to put more weights on pre-treatment periods that are more similar to post-treatment periods.

12.16.1. Example: California Proposition 99

The `california_prop99` panel is annual per-capita cigarette sales in packs for 39 US states from 1970 to 2000. California raised its cigarette tax in 1989, so there is one treated unit, 38 controls, 19 pre-treatment years and 12 post-treatment years. The estimand is the effect of Proposition 99 on California's cigarette consumption.

```
library(synthdid)
data('california_prop99')
setup = panel.matrices(california_prop99)
tau.hat = synthdid_estimate(setup$Y, setup$N0, setup$T0)
```

```
summary(tau.hat)
```

```
$estimate
[1] -15.60383

$se
     [,1]
[1,]   NA

$controls
               estimate 1
Nevada              0.124
New Hampshire       0.105
Connecticut         0.078
Delaware            0.070
Colorado            0.058
Illinois            0.053
Nebraska            0.048
Montana             0.045
Utah                0.042
New Mexico          0.041
Minnesota           0.039
Wisconsin           0.037
West Virginia       0.034
North Carolina      0.033
Idaho               0.031
Ohio                0.031
Maine               0.028
Iowa                0.026

$periods
     estimate 1
1988      0.427
1986      0.366
1987      0.206
```

```
$dimensions
          N1           N0 N0.effective           T1           T0 T0.effective
       1.000       38.000       16.388       12.000       19.000        2.783
```

The point estimate, −15.6, is the headline number from Arkhangelsky et al. (2021). Note that the `se` entry comes back `NA`, and that is not a glitch to skim past: `summary()` defaults to a **jackknife** standard error, which deletes one treated unit at a time. California is the *only* treated unit here, so deleting it leaves nothing to estimate and the jackknife is undefined. The bootstrap SE fails for the same reason. With a single treated unit the usable option is the placebo method, which builds the reference distribution by pretending each control state was treated:

```
set.seed(12345)
cat(sprintf("N0 = %d control states, T0 = %d pre-periods, N1 = %d treated unit\n",
            setup$N0, setup$T0, nrow(setup$Y) - setup$N0))
```

```
N0 = 38 control states, T0 = 19 pre-periods, N1 = 1 treated unit
```

```
se_placebo <- as.numeric(synthdid_se(tau.hat, method = "placebo"))
cat(sprintf("SDiD estimate %.2f, placebo SE %.2f, 95%% CI [%.2f, %.2f]\n",
            as.numeric(tau.hat), se_placebo,
            as.numeric(tau.hat) - 1.96 * se_placebo,
            as.numeric(tau.hat) + 1.96 * se_placebo))
```

```
SDiD estimate -15.60, placebo SE 8.37, 95% CI [-32.01, 0.80]
```

Two cautions on that interval. The placebo standard error is a permutation quantity, so it moves with the seed — report the seed, or average over several. And it is justified under the assumption that the treated unit is exchangeable with the controls, which is a real assumption about California, not a technicality. The interval here comfortably includes zero, which is worth stating plainly: the much-cited −15.6 is not statistically distinguishable from no effect with 38 control states and one treated unit. Note also that `summary(tau.hat, se.method = "placebo")` does *not* fix the `NA` — the `se` field stays empty; `synthdid_se()` has to be called directly.

The summary output is also worth reading past the point estimate. The unit weights are concentrated: Nevada takes 0.124, New Hampshire 0.105, Connecticut 0.078, and the effective number of controls is 16.4 out of 38. The time weights put 0.427 on 1988, 0.366 on 1986 and 0.206 on 1987, giving an effective 2.8 pre-periods out of 19. Both are the point of the method — weight the donors and the pre-periods that resemble the target — and both are the reason the inference is hard.

Running the same panel through all three estimators shows what the weighting does:

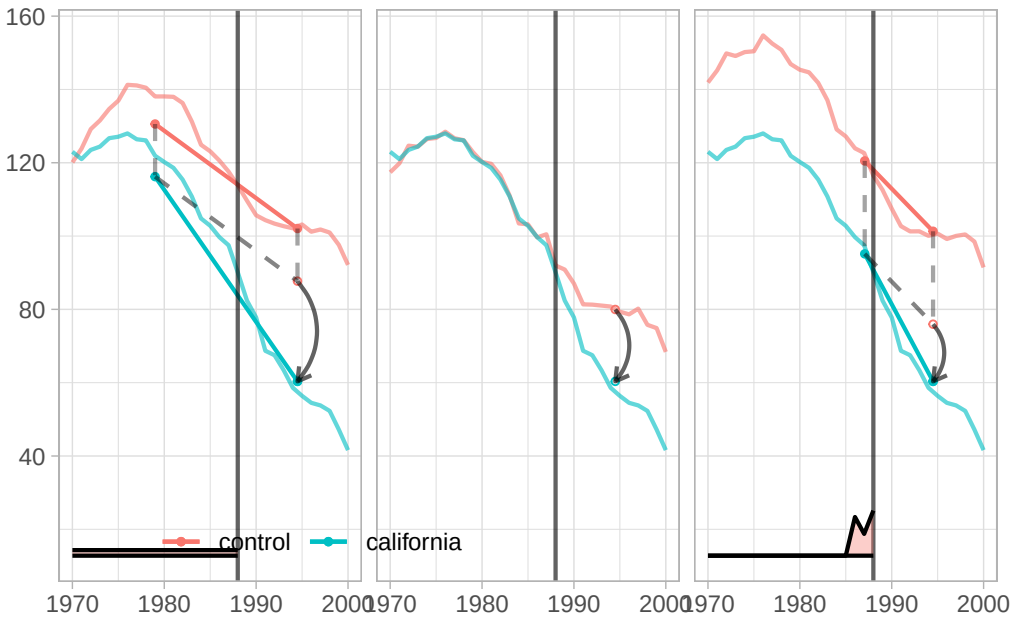
```
tau.sc   = sc_estimate(setup$Y, setup$N0, setup$T0)
tau.did  = did_estimate(setup$Y, setup$N0, setup$T0)
estimates = list(tau.did, tau.sc, tau.hat)
names(estimates) = c('Diff-in-Diff', 'Synthetic Control', 'Synthetic Diff-in-Diff')
print(unlist(estimates))
```

```
      Diff-in-Diff      Synthetic Control Synthetic Diff-in-Diff
         -27.34911              -19.61966              -15.60383
```

Plain DiD gives −27.35, synthetic control −19.62, and synthetic DiD −15.60. The three differ by more than the placebo standard error of 8.37, so the choice of estimator matters more here than the sampling uncertainty within any one of them. DiD weights every control state and every pre-period equally, which is why it moves furthest: states unlike California, and pre-periods unlike the late 1980s, all count in full.

```
plot <- synthdid_plot(estimates, facet.vertical=FALSE,
                control.name='control', treated.name='california',
                lambda.comparable=TRUE, se.method = 'none',
                trajectory.linetype = 1, line.width=.75, effect.curvature=-.4,
                trajectory.alpha=.7, effect.alpha=.7,
                diagram.alpha=1, onset.alpha=.7) +
    theme(legend.position=c(.26,.07), legend.direction='horizontal',
          legend.key=element_blank(), legend.background=element_blank(),
          strip.background=element_blank(), strip.text.x = element_blank())
```

```
plot
```



Each panel shows California's trajectory against the weighted control trajectory, with the estimated effect drawn as the gap after 1989. The time weights λ_t are plotted along the bottom, so the pre-periods that actually enter the comparison are visible: for synthetic DiD they concentrate on 1986 to 1988, the years just before the tax change. The synthetic control and synthetic DiD panels track California closely before 1989; the DiD panel does not, which is the visual counterpart to its larger estimate.

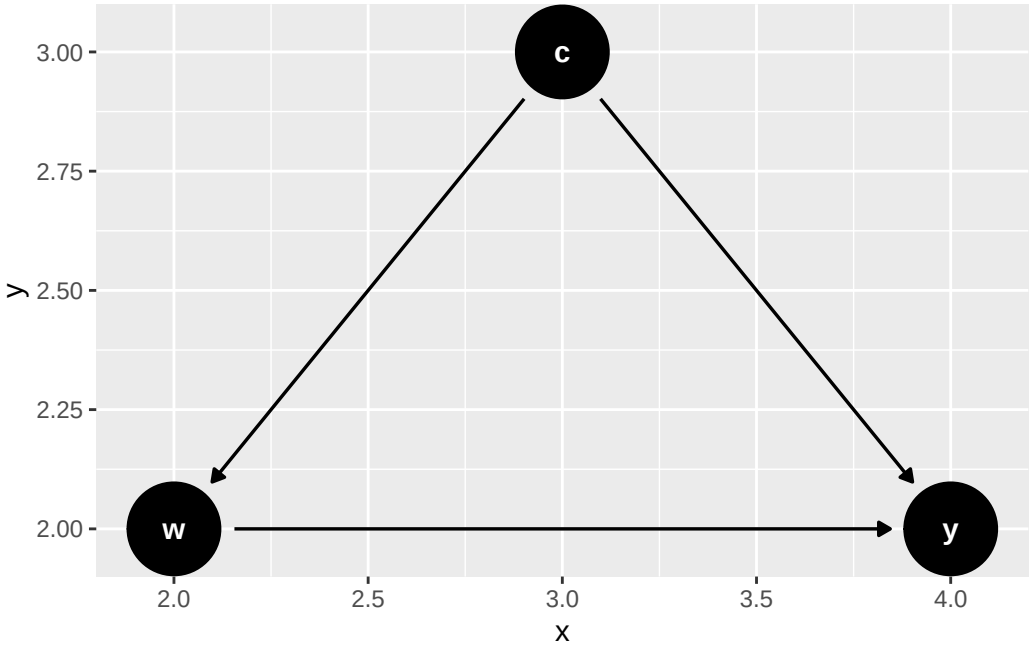
13. IV & Regression Discontinuity

With an unobserved confounder, unconfoundedness fails. Even a randomized experiment can face this problem through noncompliance: subjects are randomly assigned but some do not comply, so the actually-treated group is self-selected and that selection may be correlated with the outcome.

13.1. Instrumental Variables

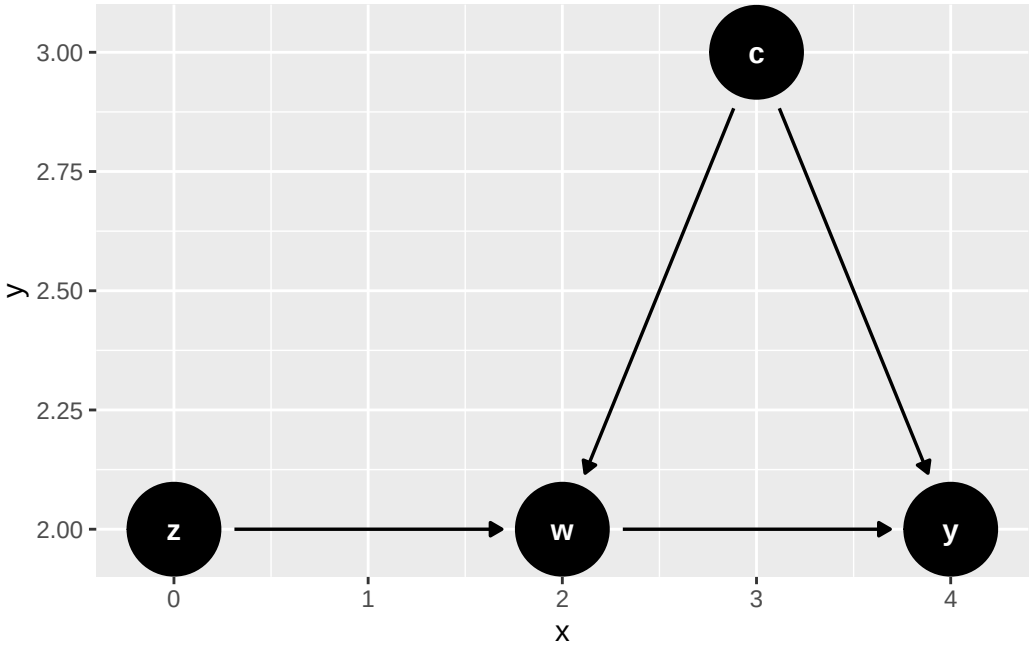
13.1.1. Uncontrolled Confounder

The graph below is the problem: c is an unobserved common cause of the treatment w and the outcome y. There is no adjustment set, because c is not measured.



13.1.2. Why Does IV Work?

Adding z gives a variable that moves w and reaches y only through w. The arrow $z \to w$ is the relevance condition, and the *absence* of any arrow from z to y or to c is the exclusion restriction and the exogeneity condition. The missing arrows carry the assumptions.



13.1.3. IV Under Potential Outcomes

- W has two potential outcomes W(1) and W(0), as a function of Z.
- Y has two potential outcomes Y(1) and Y(0), as a function of W.

Intention to Treat (ITT) is the average treatment effect of the treatment assignment Z. It is the difference between the average potential outcome under treatment and the average potential outcome under control. It is the average treatment effect of the treatment assignment Z, regardless of whether the subject complies with the assignment.

There are four possible groups of subjects in the case of binary treatment and binary instrument. The groups are defined by the *potential* treatment status $(W(0), W(1))$ — how the subject would take treatment under each value of the instrument — not by the observed (Z, W):

- $W(0) = 0, W(1) = 0$: never-taker
- $W(0) = 0, W(1) = 1$: complier
- $W(0) = 1, W(1) = 0$: defier
- $W(0) = 1, W(1) = 1$: always-taker

Table 13.1.: compliance groups

	W(1)=0	W(1)=1
W(0)=0	N	C
W(0)=1	D	A

These four groups (strata) have different causal mechanisms. The compliers and defiers are the groups whose *treatment status* responds to the assignment Z — compliers in the same direction as

the assignment, defiers in the opposite direction. The always-takers and never-takers take the same treatment regardless of Z.

Unfortunately we cannot identify the compliance group by looking at the data.

Z	W	G
0	0	C, N
0	1	A, D
1	0	N, D
1	1	C, A

13.1.4. Assumptions

- Exclusion restriction: $Y(w,1) = Y(w,0)$
- Exogeneity of instrument: $Z \perp [W(0), W(1), Y(0), Y(1)]$
- Monotonicity (no defiers): $W(0) \leq W(1)$

13.1.5. LATE Identification

Since we excluded defiers, and the other two groups (always-takers and never-takers) are not affected by the treatment assignment Z. The only effect we can identify is the treatment effect on the compliers. This is called the Local Average Treatment Effect (LATE).

Below we write potential outcomes indexed by the instrument, $Y(Z = z)$, rather than by treatment status, $Y(d)$. The exclusion restriction is exactly what licenses this: since $Y(w,1) = Y(w,0)$, Y depends on Z only through W, so $Y(Z = z) = Y(W(z), z) = Y(W(z))$ in the usual treatment-indexed notation. For a complier $(W(1) = 1, W(0) = 0)$, this means $Y(Z = 1) = Y(1)$ and $Y(Z = 0) = Y(0)$, so $E[Y(Z = 1) - Y(Z = 0) \mid G = C]$ below is exactly $E[Y(1) - Y(0) \mid G = C]$, the LATE in treatment-indexed notation.

$$
\begin{aligned}
&E[Y(Z=1) - Y(Z=0) | G = C] \\
&= \frac{E[Y(Z=1) - Y(Z=0)]}{P(W(1)=1, W(0)=0)} \\
&= \frac{E[Y|Z=1] - E[Y|Z=0]}{1 - P(W=0|Z=1) - P(W=1|Z=0)} \text{ (monotonicity rules out D; the A and N shares subtract)} \\
&= \frac{E[Y|Z=1] - E[Y|Z=0]}{P(W=1|Z=1) - P(W=1|Z=0)} \\
&= \frac{E[Y|Z=1] - E[Y|Z=0]}{E[W|Z=1] - E[W|Z=0]}
\end{aligned}
\tag{13.1}
$$

13.1.6. Parametric Models

With a binary instrument this is exactly the ratio of two OLS coefficients — no linearity assumption is needed, since a regression on a binary regressor just computes the two group means. This is sometimes called the Wald estimator.

$$\begin{aligned}\tau_{LATE} &= \frac{E[Y|Z=1] - E[Y|Z=0]}{E[W|Z=1] - E[W|Z=0]} \\ &= \frac{Cov(Y,Z)}{Cov(W,Z)}\end{aligned} \tag{13.2}$$

Why do covariates help? It is important to separate two assumptions here. Conditioning on covariates X can make the *as-if-random* (conditional independence) assumption more credible: the instrument may be unconfounded only within levels of X. But the *exclusion restriction* – that the instrument has no direct effect on the outcome – is a substantive structural assumption that covariates do not generally repair. Adding X removes a direct $Z \to Y$ path only if that path runs entirely through X, or if exclusion is conditional by design. So covariates help with conditional independence, not with exclusion per se.

13.1.7. Example

We simulate $n = 10{,}000$ observations. Three variables (w, m, z) are drawn jointly normal with mean zero and unit variances, so their covariances are also their correlations: $\text{corr}(w,m) = 0.36$, $\text{corr}(w,z) = 0.64$, and $\text{corr}(m,z) = 0$. A fourth variable $u \sim N(0,1)$ is independent of everything. The outcome is $y = w + m + u$.

So m is the omitted confounder, correlated with the treatment w but not with the instrument z, and the true coefficient on w is 1. Because all variances are 1, we can predict the omitted-variable bias exactly: regressing y on w alone gives $1 + \text{Cov}(m,w) = 1.36$.

We fit three models: OLS of y on w, OLS of y on w and m, and 2SLS using z as the instrument for w.

```
## DGP: data$y <- data$w + data$m + data$u
library(sem)
set.seed(66)
nobs=10000
nDim = 3
sdww = 1
sdzz=1
sdmm=1
## here we have three variables w, m, z.
## m is the omitted variable; w and m are correlated; z is the instrument, which is correlated
## with unit variances, the covariances below ARE the correlations
crwm=.36
crmz=0
crwz=.64
```

```
covarMat = matrix( c(sdww^2, crwm, crwz, crwm, sdmm^2, crmz,  crwz, crmz, sdzz^2 ) , nrow=nDim
data  = data.frame(MASS::mvrnorm(n=nobs, mu=rep(0,nDim), Sigma=covarMat ))
names(data) <- c('w','m','z')
data$u <- rnorm(nobs,0,1)
# dgp
data$y <- data$w +  data$m + data$u
lm <- lm(y~w, data=data)
lm.full <- lm(y~ w + m, data=data)
tsls.model <- sem::tsls(y ~ w , ~ z , data=data)
```

```
# lm is biased
summary(lm)$coefficients
```

```
                Estimate Std. Error    t value  Pr(>|t|)
(Intercept) -0.003580657 0.01372253 -0.2609326 0.7941498
w            1.358930425 0.01396918 97.2805995 0.0000000
```

```
# lm.full is good
summary(lm.full)$coefficients
```

```
               Estimate  Std. Error    t value  Pr(>|t|)
(Intercept) 0.001833551 0.009973474  0.1838428 0.8541405
w           0.992438925 0.010868032 91.3172630 0.0000000
m           1.013403057 0.010723435 94.5035837 0.0000000
```

```
# tsls is good.
summary(tsls.model)$coefficients
```

```
               Estimate Std. Error    t value  Pr(>|t|)
(Intercept) 0.006008287 0.01424525  0.4217749 0.6731984
w           0.972416527 0.02312558 42.0493890 0.0000000
```

OLS on w alone gives 1.3589, matching the predicted 1.36 to three decimals. OLS controlling for m gives 0.9924 on w and 1.0134 on m, both close to the true 1, which is what we would do if the confounder were observed. 2SLS gives 0.9724 with a standard error of 0.0231, recovering the truth without ever using m.

Note the cost of instrumenting: the 2SLS standard error is 0.0231 against 0.0109 for the OLS that controls for m, twice as large. The instrument uses only the part of w that z explains, and here that is 41% of its variance.

13.1.8. Control Function Approach

Two-stage least squares can be implemented equivalently via the **control function (CF) approach**. Understanding why makes the extension to nonlinear models (next chapter) transparent.

Algebraic motivation. Because `w` is endogenous, decompose the structural error:

$$\varepsilon = \rho v + \eta, \qquad v \equiv w - E[w \mid z], \qquad E(z\eta) = 0,\ E(zv) = 0. \tag{13.3}$$

If we *knew* v, controlling for it directly would eliminate the confounding channel. We don't know v, but the first-stage residual $\hat{v} = w - \hat{w}$ is a consistent estimate. Substituting into the structural equation:

$$y = \alpha + \beta w + \rho\,\hat{v} + \text{error}. \tag{13.4}$$

OLS on this **augmented** second stage gives a consistent $\hat{\beta}$ — identical to 2SLS by the Frisch–Waugh–Lovell (FWL) theorem. The coefficient $\hat{\rho}$ on $\hat{v}$ doubles as a **Durbin–Wu–Hausman (DWH) endogeneity test**: under H_0 (w is exogenous), $\rho = 0$.

```
# Manual 2SLS, explicitly
stage1     <- lm(w ~ z, data = data)
data$w_hat <- fitted(stage1)
tsls_manual <- lm(y ~ w_hat, data = data)

# Control function: first-stage residual v_hat in the augmented regression
data$v_hat <- residuals(stage1)
lm_cf      <- lm(y ~ w + v_hat, data = data)

# Compare ^_w from manual 2SLS and CF - should be identical by FWL
coef_tsls <- coef(tsls_manual)["w_hat"]
coef_cf   <- coef(lm_cf)["w"]
sprintf("Manual 2SLS   beta_w = %.8f", coef_tsls)
```

```
[1] "Manual 2SLS   beta_w = 0.97241653"
```

```
sprintf("Control funct beta_w = %.8f", coef_cf)
```

```
[1] "Control funct beta_w = 0.97241653"
```

```
# DWH endogeneity test: t-stat on v_hat
rho_summary <- summary(lm_cf)$coefficients["v_hat", ]
sprintf("DWH test on v_hat: coef = %.4f  se = %.4f  t = %.3f  p = %.4g",
        rho_summary["Estimate"], rho_summary["Std. Error"],
        rho_summary["t value"], rho_summary["Pr(>|t|)"])
```

```
[1] "DWH test on v_hat: coef = 0.6366  se = 0.0279  t = 22.827  p = 1.797e-112"
```

Both routes give $\hat{\beta}_w = 0.97241653$, agreeing to all eight printed digits, which confirms FWL. The coefficient on $\hat{v}$ is 0.6366 with a standard error of 0.0279, so $t = 22.8$ and $H_0 : \rho = 0$ is rejected: `w` is endogenous. That coefficient also has a value we can derive. With $\varepsilon = m + u$ and $v = w - 0.64z$, we have $\mathrm{Cov}(\varepsilon, v) = \mathrm{Cov}(m, w) = 0.36$ and $\mathrm{Var}(v) = 1 - 0.64^2 = 0.5904$, so $\rho = 0.36/0.5904 = 0.610$. The estimate is within one standard error of it. One caveat: the OLS standard errors from the augmented regression are not valid for $\hat{\beta}$ ($\hat{v}$ is a generated regressor; the DWH test is fine because under H_0 the first-stage estimation error does not matter). Use 2SLS standard errors, or bootstrap the two stages jointly as we do in the Poisson IV chapter.

> **Note**
>
> **Why FWL fails in nonlinear models.** FWL is a property of linear projections; once we replace OLS with Poisson or Probit, the estimator is no longer a linear projection and the partialling-out logic does not apply. In a nonlinear second stage, $\hat{v}$ must enter *inside* the link function (e.g. $\exp(\mathbf{x}\beta + \rho\hat{v})$), which requires a structural assumption beyond instrument exogeneity. This is the subject of the next chapter.

13.2. When 2SLS with Covariates Is *Actually* LATE

The LATE identification above is clean because we have no covariates. The Wald estimator gives us LATE, full stop. But in practice we usually add covariates to defend conditional exogeneity:

$$Y = \beta W + \gamma^\top X + U, \qquad W = \pi Z + \delta^\top X + V. \tag{13.5}$$

Does 2SLS on this still give us a LATE? Angrist and Pischke (2009) say it gives a weighted average of covariate-specific LATEs. Blandhol et al. (2025) show that this is generally not true.

13.2.1. Why the Linear Specification Hides an Assumption

By Frisch-Waugh-Lovell, the IV estimand is

$$\beta_{iv} = \frac{E[Y\tilde{Z}]}{E[W\tilde{Z}]}, \qquad \tilde{Z} = Z - L[Z \mid X], \tag{13.6}$$

where $L[Z \mid X]$ is the **linear** projection of Z on X. So $\tilde{Z}$ is the part of Z that a linear regression on X cannot explain.

What if the true $E[Z \mid X]$ is not linear in X? Then $L[Z \mid X] \neq E[Z \mid X]$, and the residual instrument $\tilde{Z}$ still picks up some of the conditional mean. The IV estimand then mixes treatment effects across compliance groups. Blandhol et al. (2025) prove that β_{iv} is a non-negatively weighted average of conditional LATEs only if the rich covariates condition holds:

$$L[Z \mid X] = E[Z \mid X]. \tag{13.7}$$

This is a parametric assumption about $E[Z \mid X]$. It is implicit in the linear-in-X specification, and it is rarely defended. When it fails, β_{iv} picks up treatment effects for always-takers as well as compliers, and some always-taker terms enter with negative weight.

> ⚠ Warning
>
> "2SLS with covariates estimates LATE" is true only if X enters saturated (one dummy per cell), or the true $E[Z \mid X]$ happens to be linear in X.

13.2.2. A Simulation

We build a DGP where $E[Z \mid X]$ is strongly nonlinear in X, so the linear-in-X specification has to fail, and then compare four estimators against the true LATE.

We draw $n = 8000$ observations with a single covariate $X \sim U(-1, 1)$. The instrument is $Z \sim \text{Bernoulli}(p_Z(X))$ with $p_Z(X) = \Lambda(0.3 + X + 2\sin(2.5\pi X))$ — the sine term is what a linear projection cannot reproduce. Potential treatments come from one uniform draw U compared against two thresholds, $T_0 = \mathbb{1}\{U < \Lambda(-1.6 + X)\}$ and $T_1 = \mathbb{1}\{U < \min(\Lambda(-0.4 + X) + 0.2,\ 0.95)\}$, so $T_1 \geq T_0$ and monotonicity holds by construction. Observed treatment is $W = ZT_1 + (1 - Z)T_0$. The effect is heterogeneous in X, $\tau(X) = 0.5 + 1.5X$, and the potential outcomes are $Y(0) = 1 + 2X + 0.8X^2 + \varepsilon$ with $\varepsilon \sim N(0, 1)$ and $Y(1) = Y(0) + \tau(X)$.

Two features matter. X enters both the instrument propensity and the treatment thresholds, so Z is exogenous only conditional on X. And τ depends on X, so which units the instrument moves determines the answer.

```r
library(ivreg)          # ivreg()
library(DoubleML)       # PLIV / debiased ML
library(mlr3)
library(mlr3learners)
library(lmtest)         # resettest()

set.seed(20260522)
n <- 8000

# Single covariate on [-1, 1]
X <- runif(n, -1, 1)

# True conditional mean of Z is wildly nonlinear in X.
# A linear projection L[Z|X] cannot reproduce this.
pZ_true <- plogis(0.3 + 1.0 * X + 2.0 * sin(2.5 * pi * X))
Z       <- rbinom(n, 1, pZ_true)

# Potential treatment states with monotonicity (T1 >= T0).
```

```
# X enters the propensity, so X confounds W <-> Y.
U   <- runif(n)
p0  <- plogis(-1.6 + 1.0 * X)            # P(always-taker | X)
p1  <- pmin(plogis(-0.4 + 1.0 * X) + 0.2, 0.95)  # P(AT or CP | X)
T0  <- as.integer(U < p0)
T1  <- as.integer(U < p1)
W   <- ifelse(Z == 1, T1, T0)            # observed treatment

G   <- ifelse(T1 == 1 & T0 == 1, "AT",
        ifelse(T1 == 0 & T0 == 0, "NT", "CP"))

# Heterogeneous treatment effect - varies with X
tau <- 0.5 + 1.5 * X
Y0  <- 1.0 + 2.0 * X + 0.8 * X^2 + rnorm(n, 0, 1)
Y1  <- Y0 + tau
Y   <- ifelse(W == 1, Y1, Y0)

# Truth (only knowable in simulation)
true_LATE <- mean(tau[G == "CP"])
prop.table(table(G))
```

```
G
      AT       CP       NT
0.184000 0.429875 0.386125
```

```
cat(sprintf("True unconditional LATE = %.3f\n", true_LATE))
```

```
True unconditional LATE = 0.615
```

The compliance shares are 18.4% always-takers, 43.0% compliers and 38.6% never-takers, and the true LATE, the mean of τ over compliers, is 0.615.

We now compare four estimators against it: the unconditional Wald ratio, 2SLS with X entered linearly, 2SLS with a degree-7 polynomial in X, and DDML PLIV with random-forest nuisance learners.

```
# (a) Wald - biased: Z is not unconditionally exogenous because X confounds.
wald <- (mean(Y[Z == 1]) - mean(Y[Z == 0])) /
        (mean(W[Z == 1]) - mean(W[Z == 0]))

# (b) TSLS with X entered linearly - the textbook spec.
tsls_lin  <- ivreg(Y ~ W + X | Z + X)
b_lin     <- coef(tsls_lin)["W"]

# (c) TSLS with a rich basis (polynomial up to degree 7) - closer to saturation.
tsls_poly <- ivreg(Y ~ W + poly(X, 7) | Z + poly(X, 7))
```

```r
b_poly    <- coef(tsls_poly)["W"]

# (d) DDML PLIV via the DoubleML package with random-forest nuisance learners.
lgr::get_logger("mlr3")$set_threshold("warn")
dml_df   <- data.frame(Y = Y, W = W, Z = Z, X = X)
dml_data <- DoubleMLData$new(dml_df, y_col = "Y", d_cols = "W",
                             x_cols = "X", z_cols = "Z")
rf <- lrn("regr.ranger", num.trees = 500, max.depth = 5, min.node.size = 2)
set.seed(20260522)
dml_pliv <- DoubleMLPLIV$new(dml_data,
                             ml_l = rf$clone(),
                             ml_m = rf$clone(),
                             ml_r = rf$clone(),
                             n_folds = 5)
dml_pliv$fit()
b_pliv <- as.numeric(dml_pliv$coef)

se_lin  <- summary(tsls_lin)$coefficients["W", "Std. Error"]
se_poly <- summary(tsls_poly)$coefficients["W", "Std. Error"]
se_pliv <- as.numeric(dml_pliv$se)

results <- data.frame(
  Estimator = c("True LATE",
                "Wald (no covariates)",
                "TSLS, X linear",
                "TSLS, poly(X, 7)",
                "DDML PLIV (DoubleML + ranger)"),
  Estimate  = c(true_LATE, wald, b_lin, b_poly, b_pliv),
  SE        = c(NA, NA, se_lin, se_poly, se_pliv)
)
print(results, row.names = FALSE, digits = 3)
```

```
                    Estimator Estimate     SE
                    True LATE    0.615     NA
          Wald (no covariates)    1.839     NA
                TSLS, X linear    0.486 0.0609
              TSLS, poly(X, 7)    0.497 0.0695
 DDML PLIV (DoubleML + ranger)    0.510 0.0719
```

The linear-in-X 2SLS does not recover the true LATE — and neither do the more flexible versions. Two things are going on, and the standard errors in the table help separate them. First, the estimands differ: for this DGP the linear-projection 2SLS converges to 0.55, and the polynomial 2SLS and DDML PLIV converge to approximately $\beta_{rich} = 0.58$ (computable by integrating the weight formula below), all below the population LATE of 0.615 — a real but modest gap that persists at any sample size. Second, sampling noise: with SEs near 0.06, this particular draw happens to land about 1.5 SEs below its own estimand, which is why the estimates cluster near

0.49–0.51 rather than 0.55–0.58. The Wald estimator is far off for a different reason entirely, at 1.839 against a truth of 0.615: Z is not unconditionally exogenous, and Wald has no way to use X.

What are the polynomial 2SLS and DDML PLIV estimating, if not LATE? PLIV targets β_{rich}, a weighted average of conditional LATEs with weights proportional to $\text{Var}(Z \mid X) \cdot P(\text{complier} \mid X)$; the polynomial 2SLS targets nearly the same quantity (a degree-7 polynomial cannot fully reproduce this $E[Z \mid X]$, so its estimand is a misspecified-projection variant sitting just below β_{rich}). These are non-negatively weighted, so weakly causal. But they are not the unconditional LATE we usually want.

13.2.3. Diagnostic: RESET on Z ~ X

Rich covariates says $E[Z \mid X]$ is linear in X. That is exactly the null of Ramsey's RESET test (Ramsey 1969). We can apply it to a regression of Z on X and see whether higher-order terms of X are jointly significant.

```
reset_z <- resettest(lm(Z ~ X), power = 2:3, type = "regressor")
reset_z
```

```
    RESET test

data:  lm(Z ~ X)
RESET = 117.02, df1 = 2, df2 = 7996, p-value < 2.2e-16
```

It rejects strongly here, RESET = 117.0 on 2 and 7,996 degrees of freedom with $p < 2.2 \times 10^{-16}$, which is the right answer given that we put a sine wave in $E[Z \mid X]$. The same test on real data is cheap, and it tells us whether the LATE interpretation is defensible before we report a 2SLS coefficient.

13.2.4. What to Do Instead

Blandhol et al. (2025) give four steps for empirical work.

1. **Reconsider the covariates.** If Z is unconditionally exogenous, drop them. A kitchen-sink set of controls makes rich covariates *less* likely.
2. **Run RESET on `Z ~ X`.** If it does not reject, the linear-IV-as-LATE interpretation is defensible.
3. **If RESET rejects, report DDML PLIV alongside 2SLS.** That is what the `DoubleML` chunk above does. DDML PLIV targets β_{rich}, a non-negatively weighted average of conditional LATEs.

4. **For a binary instrument, also estimate the unconditional LATE.** With instrument propensity score weighting (Słoczyński 2024),

$$\hat{\beta}_{late} = \frac{\sum_i Y_i[Z_i/\hat{p}(X_i) - (1-Z_i)/(1-\hat{p}(X_i))]}{\sum_i W_i[Z_i/\hat{p}(X_i) - (1-Z_i)/(1-\hat{p}(X_i))]}, \tag{13.8}$$

where $\hat{p}(X) = P(Z = 1 \mid X)$ is estimated nonparametrically. Stata has `kappalate` (Słoczyński, Uysal and Wooldridge), which implements this and related weighting estimators of the LATE. In R it is straightforward to build once $\hat{p}(X)$ is in hand.

13.2.5. A Note on the FRDD Example

The fuzzy-RDD example at the end of this chapter is 2SLS with race, state of birth, and quarter-of-birth fixed effects entered as covariates, and its 2SLS estimate sits above the local `rdrobust` one. That section attributes the gap mostly to precision. Rich covariates is a second candidate explanation for the difference in point estimates, and the RESET test on the above-cutoff instrument against those covariates is the right first thing to run.

13.2.6. Bottom Line

"2SLS with covariates estimates LATE" needs the rich covariates condition. That condition is a parametric assumption on $E[Z \mid X]$ that researchers rarely defend, and a simple RESET test often rejects it. The honest alternatives are DDML PLIV for β_{rich} and IPSW (or DDML) for the unconditional LATE. Even when rich covariates holds, β_{rich} can be quite different from the unconditional LATE we usually care about.

13.3. Regression Discontinuity Design

RDD (regression discontinuity design) is a quasi-experimental design that is used to estimate causal effects of interventions when assignment to the intervention is determined by whether a subject's value on an observed covariate exceeds a threshold. The idea is that the assignment is as good as random, so we can estimate the causal effect of the intervention by comparing the outcomes of subjects who are just above and just below the threshold.

13.3.1. Sharp RDD

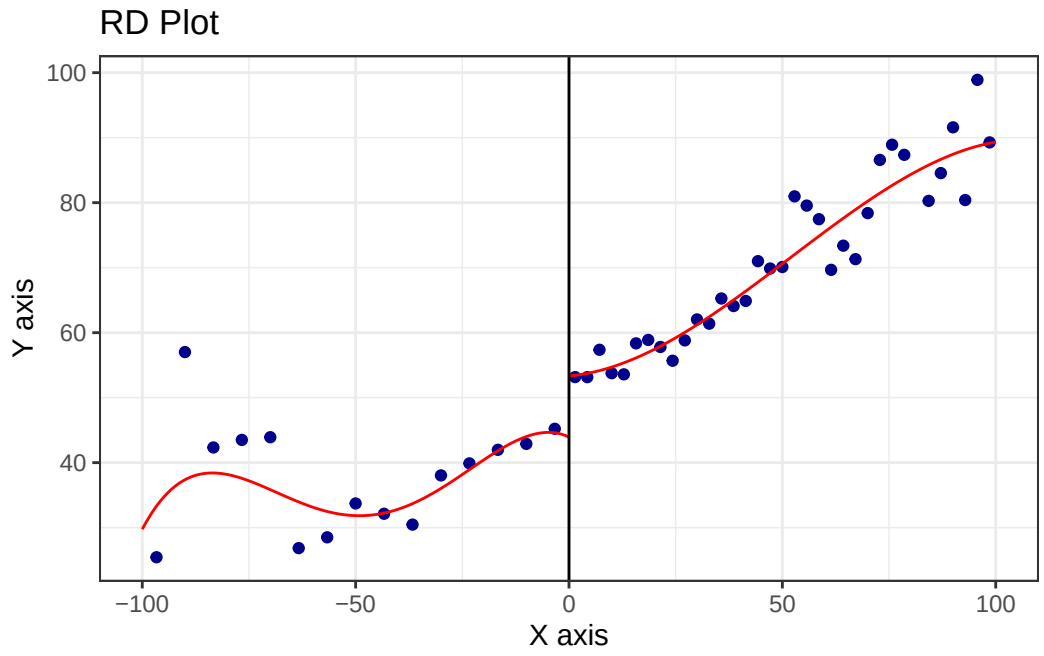
We have a continuous variable X, called the running variable, which determines the binary treatment W. The treatment is assigned according to a threshold c. The outcome variable Y is a function of X and W.

$$W = \mathbb{1}(X > c) \tag{13.9}$$

Lee (2008) studies the effect of incumbency advantage in elections. His identification strategy is based on the discontinuity generated by the rule that the party with a majority vote share wins. The forcing variable X_i is the difference in vote share between the Democratic and Republican parties in one election, with the threshold $c = 0$. The outcome variable Y_i is vote share at the second election.

`rdplot` bins the running variable and plots bin means with a polynomial fit on each side of the cutoff. A visible jump at zero is the design working.

```
library(rdrobust)
data(rdrobust_RDsenate)
rdplot(x=rdrobust_RDsenate$margin, y=rdrobust_RDsenate$vote)
```



The `rdrobust_RDsenate` data are 1,297 US Senate races. The running variable `margin` is the Democratic margin of victory in an election and `vote` is the Democratic vote share in the next election for the same seat. The bin means step up discontinuously at a margin of zero.

13.3.2. Identification of SRDD

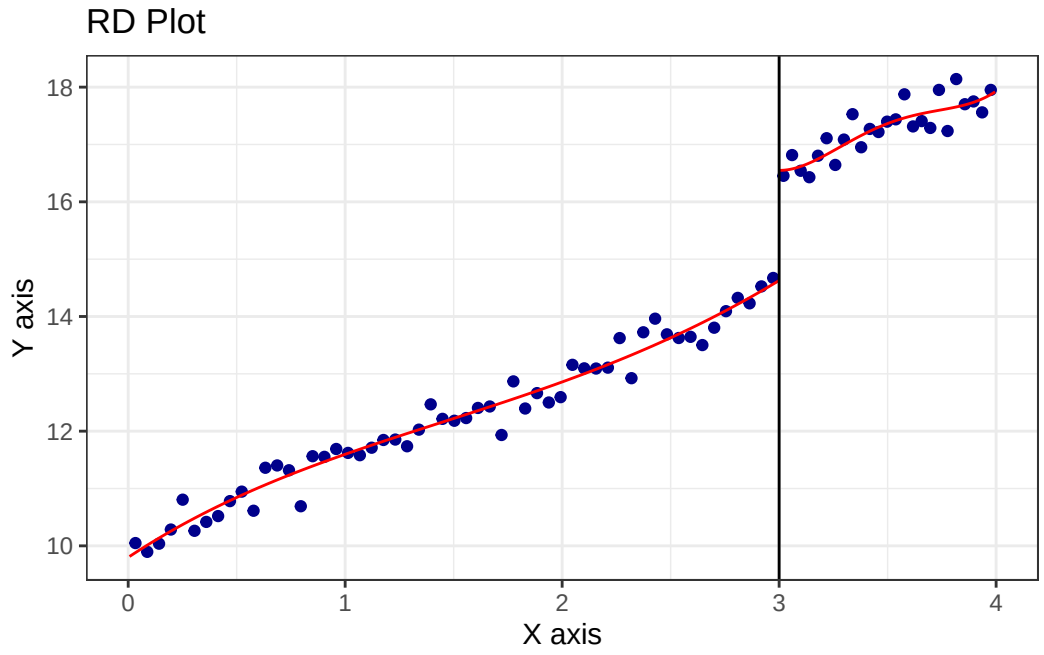
$$\tau_{RD} = \lim_{x \to c^+} E[Y|X = x] - \lim_{x \to c^-} E[Y|X = x] \tag{13.10}$$

In words, the treatment effect is the difference in the outcome from the right side and from the left side. This is the same as the difference in the outcome at the threshold, as if the treatment is assigned randomly.

13.3.3. Estimation of SRDD

Before the real data, a simulation where we know the answer. We draw $n = 1000$ students with GPA $\sim U(0, 4)$ and outcome $Y = 10 + 1.5\,\text{GPA} + 2 \cdot \mathbb{1}\{\text{GPA} > 3\} + \varepsilon$ with $\varepsilon \sim N(0, 1)$. The cutoff is 3 and the true jump is 2. The running variable also has a direct effect on the outcome, with slope 1.5, which is what the local regression on each side has to absorb.

```
GPA <- runif(1000, 0, 4)
# Treatment is W = 1(GPA > 3), so the +2 jump must sit on the right
# (above-cutoff) side to match the right-minus-left estimand above.
future_success <- 10 + 1.5 * GPA + 2 * (GPA > 3) + rnorm(1000)
rdplot(x=GPA, y=future_success, c=3)
```




```
# estimate the sharp RDD model
rdd_gpa <- rdrobust(x=GPA, y=future_success, c=3)
summary(rdd_gpa)
```

```
Call: rdrobust

Sharp RD estimates using local polynomial regression.

Number of Obs.                 1000
BW type                       mserd
Kernel                   Triangular
VCE method                       NN

                              Left         Right
```

```
Number of Obs.                  734          266
Eff. Number of Obs.             103           99
Order est. (p)                    1            1
Order bias (q)                    2            2
BW est. (h)                   0.407        0.407
BW bias (b)                   0.611        0.611
rho (h/b)                     0.666        0.666
Unique Obs.                     734          266

=============================================================================
                   Point    Robust Inference
                Estimate         z     P>|z|      [ 95% C.I. ]
-----------------------------------------------------------------------------
     RD Effect     1.827     5.333     0.000     [1.154 , 2.494]
=============================================================================
```

The estimated jump is 1.827 with a robust 95% interval of $[1.154, 2.494]$, covering the true 2. Note how little data the estimate uses: the MSE-optimal bandwidth is 0.407, so of the 1,000 observations only 103 on the left and 99 on the right are effective. A local estimator pays for its weak assumptions with a small effective sample, which is why the interval is 1.3 wide for a jump of 2.

13.3.4. Nonparametric Estimation

Now the same estimator on the Senate data, with the cutoff at a margin of zero.

```
rdd_house <- rdrobust(x=rdrobust_RDsenate$margin, y=rdrobust_RDsenate$vote, c=0)
summary(rdd_house)
```

```
Call: rdrobust

Sharp RD estimates using local polynomial regression.

Number of Obs.                 1297
BW type                       mserd
Kernel                   Triangular
VCE method                       NN

                              Left        Right
Number of Obs.                 595          702
Eff. Number of Obs.            360          323
Order est. (p)                   1            1
Order bias (q)                   2            2
BW est. (h)                 17.754       17.754
BW bias (b)                 28.028       28.028
rho (h/b)                    0.633        0.633
Unique Obs.                    595          665
```

```
==============================================================================
                     Point    Robust Inference
                  Estimate         z     P>|z|      [ 95% C.I. ]
------------------------------------------------------------------------------
     RD Effect     7.414     4.311     0.000     [4.094 , 10.919]
==============================================================================
```

The incumbency effect is 7.414 percentage points with a robust 95% interval of $[4.094, 10.919]$. Barely winning a Senate seat raises the party's vote share in the next election for that seat by about 7 points. The bandwidth is 17.75 margin points, leaving 360 observations on the left and 323 on the right out of 1,297.

13.3.5. Fuzzy RDD

In fuzzy RDD, the treatment is assigned according to a threshold c, but there exist non-compliance. This is similar to IV case.

$$\tau_{FRD} = \frac{\lim_{x \to c^+} E[Y|X = x] - \lim_{x \to c^-} E[Y|X = x]}{\lim_{x \to c^+} E[W|X = x] - \lim_{x \to c^-} E[W|X = x]} \tag{13.11}$$

For example, if food stamp eligibility is given to all households below a certain income, but not all households receive the food stamps. In other words, income does not solely determine the assignment. Cutoff point increases the probability of treatment but doesn't completely determine treatment.

FRDD is IV.

13.3.6. FRDD Example

Fetter (2013)'s main question of interest is how much of the increase in the home ownership rate in the midcentury US was due to mortgage subsidies given out by the government.

We're using the running variable quarter of birth (qob), which has been centered on the quarter of birth you'd need to be to be eligible for a mortgage subsidy for fighting in the Korean War (qob_minus_kw). This determines whether you were a veteran of either the Korean War or World War II (vet_wwko).

```
vet <- causaldata::mortgages
# Create an "above-cutoff" variable as the instrument
vet <- vet %>% mutate(above = qob_minus_kw > 0)
# Impose a bandwidth of 12 quarters on either side
vet <- vet %>%  filter(abs(qob_minus_kw) < 12)
# Note: the running variable qob_minus_kw is exogenous, so it goes in the
# control block (not the endogenous block). Listing it in both the
# endogenous and instrument sets makes newer fixest error ("endogenous
# variable fully explained"). The endogenous regressors are veteran status
```

```
# and its interaction with the running variable, instrumented by being above
# the cutoff and that interaction.
m <- feols(home_ownership ~
    nonwhite + qob_minus_kw | # exogenous controls incl. the running variable
    bpl + qob | # fixed effect controls
    vet_wwko + qob_minus_kw:vet_wwko ~ # endogenous: treatment and its RDD slope interaction
    above + qob_minus_kw:above, # instruments: above-cutoff and its interaction
    se = 'hetero', # heteroskedasticity-robust SEs
    data = vet)
summary(m)
```

```
TSLS estimation: Second stage
|- D.V.    : home_ownership
|- Endo.   : vet_wwko, vet_wwko:qob_minus_kw
|- Instr.  : above, above:qob_minus_kw
Dep. Var.: home_ownership
Observations: 56,901
Fixed-effects: bpl: 52,  qob: 4
Standard-errors: Heteroskedasticity-robust
                           Estimate Std. Error   t value   Pr(>|t|)
fit_vet_wwko                0.170119   0.045918   3.70483 2.1173e-04 ***
fit_vet_wwko:qob_minus_kw -0.002874   0.002641  -1.08829 2.7647e-01
nonwhite                  -0.190429   0.006893 -27.62760  < 2.2e-16 ***
qob_minus_kw              -0.007146   0.001776  -4.02406 5.7278e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
RMSE: 0.441907     Adj. R2: 0.066481
                 Within R2: 0.051235
F-test (1st stage), vet_wwko             : stat =   210.46260, p < 2.2e-16 , on 3 and 56,841 D
F-test (1st stage), vet_wwko:qob_minus_kw: stat = 1,240.72000, p < 2.2e-16 , on 3 and 56,841 D
                               Wu-Hausman: stat =     6.75359, p = 0.001168, on 2 and 56,840 D
                                   Sargan: stat =     0.00014, p = 0.990622, on 1 DoF.
```

The 2SLS estimate is 0.1701 with a heteroskedasticity-robust standard error of 0.0459: veteran status raises home ownership by 17 percentage points. The sample is 56,901 people within 12 quarters of the cutoff, with birthplace and quarter-of-birth fixed effects. The first-stage F statistics are 210 and 1,241, so the instruments are strong, and the Wu-Hausman test rejects exogeneity of veteran status at $p = 0.001$. The interaction of veteran status with the running variable is not significant, so there is little evidence that the slope differs across the cutoff.

Now the same design estimated locally with `rdrobust`, passing the covariates as a model matrix.

```
controls <- vet %>%
    select(nonwhite, bpl, qob) %>%
    mutate(qob = factor(qob))
conmatrix <- model.matrix(~., data = controls)
m <- rdrobust(vet$home_ownership,
```

```
              vet$qob_minus_kw,
              fuzzy = vet$vet_wwko,
              c = 0,
              covs = conmatrix)
m$Estimate
```

```
         tau.us     tau.bc     se.us   se.rb
[1,] 0.09594601 0.03340165 0.1793835 0.22143
```

`rdrobust` returns a conventional estimate of 0.0959 with a standard error of 0.1794, and a bias-corrected estimate of 0.0334 with a robust standard error of 0.2214. The point estimates are well below the 2SLS 0.1701.

The gap is worth reading carefully, because it is easy to overstate. The two point estimates differ by a factor of nearly two, but the `rdrobust` standard error is 0.179, so its 95% interval runs from about −0.26 to 0.45 and comfortably contains the 2SLS estimate. The two are not statistically distinguishable. What separates them is precision: 2SLS has a standard error of 0.046 because it uses all 56,901 observations and a linear specification, while `rdrobust` uses a local window around the cutoff and gets an interval four times wider — so wide that it cannot reject zero.

That is the real trade-off. 2SLS buys precision with a functional-form assumption over the full bandwidth; the local estimator drops that assumption and pays for it in variance. It is a genuine sensitivity to model choice, but here it shows up as a difference in what can be concluded rather than as a contradiction between two estimates.

14. Shift-Share Instrumental Variables

Shift-share, or Bartik, instruments combine local industry shares with industry-level shocks. The idea is simple: a location with a large baseline share in an industry is more exposed to shocks in that industry. The difficult part is not constructing the instrument. The difficult part is stating what is assumed exogenous: the shares, the shocks, or both.

14.1. The construction

For each location ℓ and industry k, let $s_{\ell k}$ be the share of local employment in industry k at baseline, and let g_k be a national growth rate (or trade shock) for that industry. The shift-share instrument is

$$B_\ell \;=\; \sum_{k=1}^{K} s_{\ell k}\, g_k. \tag{14.1}$$

This is "shift-share" because it combines a local share with an industry-level shift. Bartik (1991) used local industry mix to predict local employment growth. Autor, Dorn, and Hanson (2013) used local industry shares interacted with industry-level Chinese import growth in other high-income countries.

The first-stage regression is then

$$\Delta L_\ell \;=\; \pi_0 + \pi_1 B_\ell + X_\ell'\gamma + \epsilon_\ell, \tag{14.2}$$

with B_ℓ instrumenting for the observed local shock in a 2SLS for the outcome of interest.

14.2. Two identification views

There are two ways to justify a shift-share instrument. They put the exogeneity assumption on different objects.

14.2.1. Share view (GPSS 2020)

Treat the shares $s_{\ell k}$ as the source of identifying variation, with the shocks g_k acting as weights. The instrument is valid if shares are exogenous to the outcome (conditional on controls). Goldsmith-Pinkham, Sorkin, and Swift (2020) (henceforth GPSS) prove that the shift-share IV is numerically equivalent to a GMM combination of K just-identified IVs, one per industry share:

$$\hat{\beta}^{SS} = \sum_{k=1}^{K} \hat{\alpha}_k \, \hat{\beta}_k, \tag{14.3}$$

where $\hat{\beta}_k$ is the just-identified IV using share $s_{\cdot k}$ alone and $\hat{\alpha}_k$ is the **Rotemberg weight**. This gives an important diagnostic: which industry shares are driving the estimate?

14.2.2. Shock view (BHJ 2022)

Treat the shocks g_k as the source of identifying variation. Shares are exposure weights. Under this view, shocks should be uncorrelated with the second-stage unobservables, conditional on industry-level controls. Borusyak, Hull, and Jaravel (2022) (henceforth BHJ) show how to rewrite the regression at the shock level.

The two views are not mutually exclusive. In an application, we should be clear which one is being defended and report diagnostics for both.

14.3. Inference

Standard OLS or cluster-robust standard errors on the regional regression **underestimate uncertainty** because the same industry shocks appear in many regions, inducing cross-region correlation that region-level clustering does not capture. Two corrections are now standard:

- **Cluster on shocks (BHJ)**: equivalent to running the regression at the industry-shock level. Implemented via shock-level reweighting.
- **AKM SE** (Adão, Kolesár, and Morales 2019): explicit formula for SE accounting for shock-level correlation. Available in the `ShiftShareSE` R package.

14.4. Simulation: build intuition

The aim is to see the shift-share IV recover a known effect that OLS misses, and then to break the identifying assumption on purpose and see which diagnostic catches it.

We use 500 regions and 20 industries. Each region has a vector of industry shares $s_{\ell k}$ summing to one, concentrated so that a few industries dominate any given region. Each industry draws a shock $g_k \sim N(0,1)$, independently of the shares and of everything at the region level. The instrument is $B_\ell = \sum_k s_{\ell k} g_k$.

There is one region-level confounder $u_\ell \sim N(0,1)$. The observed local shock and the outcome are

$$X_\ell = B_\ell + 0.3u_\ell + \varepsilon_\ell, \qquad Y_\ell = 0.5X_\ell + u_\ell + \eta_\ell, \tag{14.4}$$

with ε and η independent noise of standard deviation about 0.3 and 0.5. The true effect is $\beta = 0.5$.

Because u enters both X and Y, OLS is biased, and we can say by how much: $\mathrm{Cov}(X, u)/\mathrm{Var}(X) = 0.3/0.315 = 0.95$, so OLS should return about $0.5 + 0.95 = 1.45$.

The data are generated once and stored as CSVs so that this book and its Julia companion produce identical numbers.

```
n_region   <- 500
n_industry <- 20
beta_true  <- 0.5

df     <- read_csv("data/shift_share_sim.csv", show_col_types = FALSE)
shares <- as.matrix(read_csv("data/shift_share_shares.csv", show_col_types = FALSE))
# This simulation is constructed to satisfy the Borusyak et al. (2022)
# shock-exogeneity assumption: the industry-level shocks are drawn
# independently of the region-level confounder `u` and of the shares
# themselves. The shares are allowed to be endogenous (correlated with
# region characteristics) -- it is specifically the shocks that must be
# "as good as randomly assigned" for this identification argument, which is
# a different (and in this simulation, the maintained) assumption from the
# Goldsmith-Pinkham et al. (2020) shares-exogeneity view discussed above.
shocks <- read_csv("data/shift_share_shocks.csv", show_col_types = FALSE)$shock
u      <- df$u
head(df)
```

```
# A tibble: 6 x 5
  region      X      Y       B      u
   <dbl>  <dbl>  <dbl>   <dbl>  <dbl>
1      1 -0.518  0.117 -0.138  -0.308
2      2  0.213 -1.07   0.547  -0.539
3      3  0.652  0.892  0.351   1.37
4      4  0.286  0.861  0.0385  1.10
5      5 -0.334 -0.330  0.421  -0.112
6      6  0.376 -1.73   1.17   -1.75
```

A naive OLS suffers from the confounder `u`:

```
ols  <- feols(Y ~ X, data = df)
ivss <- feols(Y ~ 1 | X ~ B, data = df)
etable(ols, ivss, headers = c("OLS", "Shift-share IV"),
       digits = 3, digits.stats = 3, fitstat = ~ . + ivf)
```

```
                                  ols             ivss
                                  OLS   Shift-share IV
Dependent Var.:                     Y                Y

Constant              -0.154** (0.047)    0.077 (0.059)
X                      1.45*** (0.075) 0.531*** (0.131)
--------------------- --------------- ----------------
S.E. type                         IID              IID
Observations                      500              500
R2                              0.428            0.025
Adj. R2                         0.427            0.022
F-test (1st stage), X             --             370.5
---
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

OLS gives 1.45 with a standard error of 0.075, matching the 1.45 predicted above and nearly three times the true effect of 0.5. The shift-share IV gives 0.531 with a standard error of 0.131, within a quarter of a standard error of the truth. The first-stage F is 370.5, so the instrument is strong.

Note the price again: the IV standard error is 0.131 against 0.075 for OLS, and the second-stage R^2 is 0.025 against 0.428. A biased estimator can fit well.

14.4.1. Rotemberg weights

The GPSS decomposition says the shift-share IV is a weighted average of K just-identified IVs (one per industry share). Compute the weights:

```
# Rotemberg weight for industry k (using centered moments to match the
# demeaned 2SLS regression):
#   alpha_k = g_k * Cov(s_{.k}, X) / sum_j g_j * Cov(s_{.j}, X)
# Each industry's just-identified IV estimate uses s_{.k} as instrument:
#   beta_k = Cov(s_{.k}, Y) / Cov(s_{.k}, X)
# GPSS prove the shift-share IV estimate equals sum_k alpha_k * beta_k.

cov_sk_X <- sapply(1:n_industry, function(k) cov(shares[, k], df$X))
cov_sk_Y <- sapply(1:n_industry, function(k) cov(shares[, k], df$Y))
denom    <- sum(shocks * cov_sk_X)

alpha  <- shocks * cov_sk_X / denom
beta_k <- cov_sk_Y / cov_sk_X

rotemberg <- tibble(industry = 1:n_industry,
                    shock    = shocks,
                    weight   = alpha,
                    beta_k   = beta_k)

kable(rotemberg, digits = 3,
```

```
    caption = paste0("Rotemberg decomposition. Sum of alpha*beta_k = ",
                     round(sum(alpha * beta_k), 3),
                     " (= shift-share IV estimate of ",
                     round(coef(ivss)["fit_X"], 3), ")."))
```

Table 14.1.: Rotemberg decomposition. Sum of alpha*beta_k = 0.531 (= shift-share IV estimate of 0.531).

industry	shock	weight	beta_k
1	1.250	0.054	0.604
2	-0.959	0.048	0.244
3	-0.212	0.005	2.576
4	0.745	0.002	-10.225
5	2.663	0.387	0.469
6	-1.068	0.082	1.025
7	0.100	-0.001	-1.627
8	-0.718	0.025	-0.689
9	1.181	0.058	0.853
10	-0.197	0.004	1.188
11	0.741	0.023	-0.112
12	0.051	-0.001	-0.055
13	-0.442	0.014	0.611
14	0.270	0.003	-0.013
15	0.631	0.003	-3.240
16	2.271	0.204	0.746
17	-0.212	0.001	0.174
18	-0.115	0.000	-39.532
19	0.020	0.000	1.234
20	-1.183	0.089	0.554

The weighted sum reproduces the 2SLS estimate exactly: $\sum_k \alpha_k \hat{\beta}_k = 0.531$, which is the shift-share IV coefficient. That is the GPSS equivalence, and it holds numerically, not approximately.

The weights are concentrated. Industry 5 carries 0.387 and industry 16 carries 0.204, so those two supply nearly 60% of the identifying variation, and the next three (industries 20, 6 and 9) bring the total past 80%. Under the share view, the exogeneity of those few industries' shares is the assumption the estimate rests on. Notice that the two dominant industries are also the two with the largest shocks, 2.66 and 2.27: α_k is proportional to $g_k \text{Cov}(s_{\cdot k}, X)$, so a big shock buys influence.

The $\hat{\beta}_k$ column is more surprising and worth reading carefully. Industry 18 gives -39.5, industry 4 gives -10.2, industry 15 gives -3.2. These are not near the true 0.5, and it would be easy to read them as evidence of severe treatment-effect heterogeneity. They are not. Each $\hat{\beta}_k = \text{Cov}(s_{\cdot k}, Y)/\text{Cov}(s_{\cdot k}, X)$ is a just-identified IV using one share as the instrument, and when $\text{Cov}(s_{\cdot k}, X)$ is near zero that ratio explodes. It is a weak-instrument artefact, one industry at a time.

The decomposition is immune to it by construction. Since

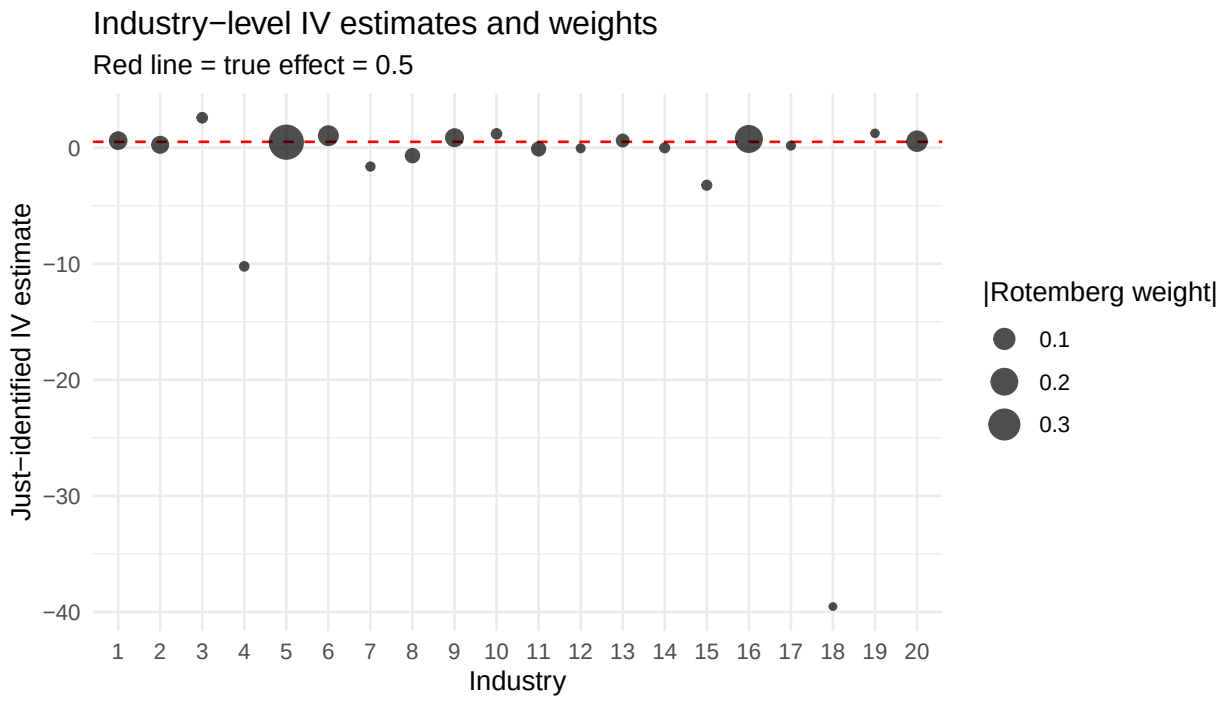
$$\alpha_k \hat{\beta}_k = \frac{g_k \mathrm{Cov}(s_{\cdot k}, X)}{\sum_j g_j \mathrm{Cov}(s_{\cdot j}, X)} \cdot \frac{\mathrm{Cov}(s_{\cdot k}, Y)}{\mathrm{Cov}(s_{\cdot k}, X)} = \frac{g_k \mathrm{Cov}(s_{\cdot k}, Y)}{\sum_j g_j \mathrm{Cov}(s_{\cdot j}, X)}, \tag{14.5}$$

the troublesome $\mathrm{Cov}(s_{\cdot k}, X)$ cancels. Every industry with an exploding $\hat{\beta}_k$ has a weight near zero for exactly the same reason its $\hat{\beta}_k$ blew up. Industry 18 carries a weight of 0.000 and industry 4 a weight of 0.002.

Read the two columns together, then. Among the industries that carry real weight, the estimates are 0.469, 0.746, 0.554, 1.025, 0.853 and 0.604 — close enough to 0.5 to be sampling noise, which is what we should see in a simulation with a homogeneous effect. In real data, if industries with *large* weights give very different estimates, the shift-share IV is averaging over genuine heterogeneity and that should be reported.

The plot below makes the pairing visible: each industry's just-identified IV estimate on the vertical axis, with point size proportional to its Rotemberg weight. The outlying estimates are the smallest points.

```
ggplot(rotemberg, aes(x = factor(industry), y = beta_k, size = abs(weight))) +
  geom_hline(yintercept = beta_true, linetype = "dashed", color = "red") +
  geom_point(alpha = 0.7) +
  scale_size_continuous(name = "|Rotemberg weight|") +
  labs(x = "Industry", y = "Just-identified IV estimate",
       title = "Industry-level IV estimates and weights",
       subtitle = paste("Red line = true effect =", beta_true)) +
  theme_minimal()
```



14.4.2. What goes wrong when shares are endogenous

Now break the share-exogeneity assumption on purpose. We introduce a second region-level confounder v_ℓ, add $0.15 v_\ell$ to industry 1's share (renormalising the rows to sum to one), and let v enter the outcome with coefficient 0.6:

$$Y_\ell = 0.5 X_\ell + u_\ell + 0.6 v_\ell + \eta_\ell. \tag{14.6}$$

Everything else is unchanged. Industry 1's share is now correlated with the second-stage error, so the instrument built from it is invalid, and we ask which diagnostic notices.

```
# Re-do shares so industry 1's share covaries with a new confounder v
v <- read_csv("data/shift_share_bad_v.csv", show_col_types = FALSE)$v
shares_bad <- shares
shares_bad[, 1] <- pmax(0.01, shares[, 1] + 0.15 * v)
shares_bad <- shares_bad / rowSums(shares_bad)   # renormalise to sum to 1

bad_noise <- read_csv("data/shift_share_bad_noise.csv", show_col_types = FALSE)
B_bad <- as.numeric(shares_bad %*% shocks)
X_bad <- B_bad + 0.3 * u + bad_noise$noise_x
Y_bad <- beta_true * X_bad + u + 0.6 * v + bad_noise$noise_y

df_bad <- tibble(X = X_bad, Y = Y_bad, B = B_bad)
ivss_bad <- feols(Y ~ 1 | X ~ B, data = df_bad)
cat("True beta:", beta_true, "\n")
```

```
True beta: 0.5
```

```
cat("Shift-share IV with bad share 1:", round(coef(ivss_bad)["fit_X"], 3), "\n")
```

```
Shift-share IV with bad share 1: 0.798
```

```
# Rotemberg weights recomputed under the bad shares
cov_sk_X_bad <- sapply(1:n_industry, function(k) cov(shares_bad[, k], X_bad))
alpha_bad    <- shocks * cov_sk_X_bad / sum(shocks * cov_sk_X_bad)
cat("Rotemberg weight of industry 1:", round(alpha_bad[1], 3),
    "-- rank", rank(-abs(alpha_bad))[1], "of", n_industry, "\n")
```

```
Rotemberg weight of industry 1: 0.11 -- rank 3 of 20
```

```
# Leave-one-industry-out: rebuild the instrument without industry 1
B_loo    <- as.numeric(shares_bad[, -1] %*% shocks[-1])
ivss_loo <- feols(Y ~ 1 | X ~ B, data = tibble(X = X_bad, Y = Y_bad, B = B_loo))
cat("Shift-share IV without industry 1:", round(coef(ivss_loo)["fit_X"], 3), "\n")
```

```
Shift-share IV without industry 1: 0.48
```

The estimate moves from 0.531 to 0.798 against a true 0.5, so one endogenous share out of twenty is enough to inflate the estimate by 60%. Note which diagnostic catches it. Rotemberg weights measure *influence* — whose exogeneity the estimate leans on — not endogeneity: the offending industry carries a weight of only 0.11, third largest of twenty, yet it alone moves the estimate from 0.5 to 0.8. The leave-one-industry-out check is what isolates it: rebuilding the instrument without industry 1 restores the estimate to 0.48.

14.5. Empirical: the China shock (Autor, Dorn, Hanson 2013)

The standard shift-share application is the China-shock study of Autor, Dorn, and Hanson (2013) (henceforth ADH). Commuting zones are exposed differently to Chinese import competition because their baseline industry mixes differ. The instrument is

$$\text{IV}_\ell \;=\; \sum_k \frac{L_{\ell k,\,1990}}{L_{\ell,\,1990}} \, \frac{\Delta M_k^{other}}{L_{k,\,1990}}, \tag{14.7}$$

where the shocks ΔM_k^{other} are growth in Chinese imports into other high-income countries. This leave-one-out construction removes US-specific demand from the shock. In the actual ADH design the employment shares are lagged one census behind the outcome period (e.g. 1980 employment for the 1990–2000 stack), precisely to mitigate share endogeneity; we date everything to 1990 here to keep the notation light.

```r
# David Dorn distributes the replication data at
# https://www.ddorn.net/data.htm - files needed:
#   workfile_china.dta  (commuting zone data, 1990-2007)
#   industry_shares.csv (czone-industry shares)
#   industry_imports.csv (industry-level imports)

library(haven)
library(fixest)

cz   <- read_dta("workfile_china.dta")
fit  <- feols(d_sh_empl_mfg ~ 1 | d_tradeusch_pw ~ d_tradeotch_pw_lag,
              data = cz,
              cluster = ~ statefip)
summary(fit)
```

Modern best-practice extensions of the basic ADH regression. These blocks are schematic — they do not run here, so check argument names against the current package documentation before adapting them:

```r
# 1. Rotemberg weights via the bartik.weight package (Goldsmith-Pinkham)
# devtools::install_github("paulgp/bartik-weight")
library(bartik.weight)
rw <- bw(cz, master = master_spec, y = "d_sh_empl_mfg",
         x = "d_tradeusch_pw", weight = "timepwt48",
         G = G_growth, Z = Z_shares)

# Plot the Rotemberg weights to see which industries drive the estimate
plot(rw)

# 2. Adão-Kolesár-Morales standard errors
# devtools::install_github("kolesarm/ShiftShareSE")
library(ShiftShareSE)
ivreg_ss(d_sh_empl_mfg ~ d_tradeusch_pw + controls,
         X = "d_tradeotch_pw_lag",
         data = cz, W = shock_weights, region_cvar = "czone",
         method = "akm0")

# 3. BHJ shock-level inference: collapse to shock (industry-period) level
# devtools::install_github("borusyak/shift-share")
library(ssaggregate)
shocks_data <- ssaggregate(data = cz, vars = c("d_sh_empl_mfg",
                                               "d_tradeusch_pw"),
                           shock = "d_tradeotch_pw_lag", weights = "timepwt48",
                           l = "czone", n = "industry", t = "year",
                           s = "share")
# Now regress at shock level - interpretation: shock-level IV
feols(d_sh_empl_mfg ~ 1 | d_tradeusch_pw ~ shock, data = shocks_data)
```

For a new shift-share paper, I would expect Rotemberg weights, AKM standard errors, and a BHJ shock-level version or an explanation for why it is not appropriate.

14.6. Summary

- Shift-share IV combines local shares and industry shocks.
- The GPSS view puts exogeneity on shares; the BHJ view puts it on shocks.
- Rotemberg weights show which industry shares drive the 2SLS estimate.
- Region-level clustering is usually too optimistic. Use AKM standard errors or shock-level inference when possible.
- In R, `fixest`, `bartik.weight`, `ShiftShareSE`, and `ssaggregate` cover most of the workflow.

15. IV in Poisson with Fixed Effects

Poisson regression specifies the conditional mean as an exponential function. As a quasi-MLE, it needs the mean to be correct, not the full count distribution. If a regressor is endogenous, the IV strategy has to respect that nonlinear mean. I consider two cases: a continuous endogenous variable and a binary endogenous variable.

15.1. The Control Function Framework

15.1.1. Linear second stage — CF equals 2SLS

In the previous chapter we saw that adding the first-stage residual $\hat{v}_2$ to an OLS second stage recovers the same coefficient as 2SLS. The algebra is:

1. **First stage**: $y_2 = \mathbf{z}\pi + v_2$, estimated by OLS. Residual $\hat{v}_2 = y_2 - \hat{y}_2$ satisfies $E(\mathbf{z}'\hat{v}_2) = 0$.
2. **Structural error decomposition**: write $\varepsilon_1 = \rho v_2 + \eta_1$, where $E(\mathbf{z}'\eta_1) = 0$ and $E(v_2\,\eta_1) = 0$ by construction.
3. **Augmented second stage**: substitute to get $y_1 = \mathbf{z}_1\delta_1 + \alpha_1 y_2 + \rho\,\hat{v}_2 + \eta_1$.

OLS on this equation is the **control function estimator**. The Frisch–Waugh–Lovell (FWL) theorem guarantees it is numerically identical to 2SLS — no extra assumption beyond $E(\mathbf{z}'\varepsilon_1) = 0$ is required.

15.1.2. Nonlinear second stage — CF diverges from 2SLS

When the conditional mean is $\exp(\mathbf{x}\beta)$ (Poisson), a direct IV/GMM approach solves moment conditions in the instruments without modifying the exponential mean. The form of the moments matters. The additive moments $E[\mathbf{z}'(y_1 - \exp(\mathbf{x}\beta))] = 0$ require an additive structural error; when unobserved heterogeneity sits *inside* $\exp(\cdot)$ — the natural case for counts — the valid moments are the multiplicative ones of Mullahy (1997), $E[\mathbf{z}'(y_1 e^{-\mathbf{x}\beta} - 1)] = 0$. The **control function approach** instead imposes the stronger *conditional mean assumption* (Wooldridge 2010, sec. 18.5; 2014):

$$\boxed{E[y_1 \mid \mathbf{z}_1,\, y_2,\, v_2] = \exp(\mathbf{z}_1\delta_1 + \alpha_1 y_2 + \rho v_2).} \tag{15.1}$$

This says the unobservable enters the conditional mean **linearly inside** $\exp(\cdot)$. Under this assumption, substituting $\hat{v}_2$ for v_2 and running Poisson QMLE gives a consistent estimate of α_1.

Three practical consequences:

Property	Linear (OLS/2SLS)	Nonlinear (Poisson CF)
Extra assumption beyond IV exogeneity	None — FWL applies	CF mean assumption required
CF vs. 2SLS/GMM numerically	**Identical**	**Different** — FWL fails
SE from second stage alone	Valid only under $\rho = 0$ — use 2SLS SEs	**Invalid** — bootstrap needed
Test of endogeneity	t-stat on $\hat{v}_2$ (DWH)	Same — t-stat on $\hat{v}_2$

> **Note**
>
> **Intuition for the CF assumption.** The decomposition $\varepsilon_1 = \rho v_2 + \eta_1$ is exactly the same as in the linear case. What changes is where the decomposition gets applied: in the linear model it enters additively outside any transformation; in Poisson it must enter inside $\exp(\cdot)$ to be absorbed by the quasi-MLE score. If the true $\rho \neq 0$, omitting $\hat{v}_2$ from the exponential mean leaves a multiplicative endogeneity term $e^{\rho v_2}$ correlated with y_2, biasing $\hat{\alpha}_1$ upward or downward depending on the sign of ρ.

15.2. Part 1 — Continuous endogenous variable

15.2.1. Data generating process

We simulate a panel where `time` (continuous, e.g. minutes on site) is endogenous: it is correlated with an unobserved factor `e` that also affects `visits`. The excluded instrument is `phone` (binary, e.g. phone-access indicator). Fixed effects are `ad` (20 groups) and `female`.

There are 20 ad groups of 250 observations each, $n = 5000$. Each group draws a fixed effect $\alpha_g \sim N(0, 0.5^2)$. At the individual level, female $\sim$ Bernoulli(0.5), phone $\sim$ Bernoulli(0.4), frfam $\sim U(0,1)$, and the unobservable is $e \sim N(0,1)$. The endogenous regressor is

$$\text{time} = 1.5\,\text{phone} + 0.5\,\text{frfam} + \alpha_g + e, \tag{15.2}$$

and the count is visits $\sim$ Poisson(λ) with

$$\lambda = \exp(0.5 + 0.8\,\text{time} + 0.4\,\text{frfam} + \alpha_g + 0.3\,\text{female} + 0.5\,e). \tag{15.3}$$

The true α_1 is 0.8. The same e appears in `time` and in the log mean, which is the endogeneity, and `phone` appears only in `time`, which is the exclusion restriction. Note that e enters the log mean *linearly inside* $\exp(\cdot)$: the control-function assumption holds here by construction, so this simulation tests the estimator rather than the assumption.

```r
set.seed(42)
n_ad <- 20; obs_per <- 250; n <- n_ad * obs_per

fe_ad  <- rnorm(n_ad) * 0.5
fe_grp <- rep(1:n_ad, each = obs_per)
female <- as.integer(runif(n) < 0.5)
phone  <- as.integer(runif(n) < 0.4)
frfam  <- runif(n)
e      <- rnorm(n)

time <- 1.5 * phone + 0.5 * frfam + fe_ad[fe_grp] + e

true_alpha <- 0.8
lambda <- exp(0.5 + true_alpha * time + 0.4 * frfam +
              fe_ad[fe_grp] + 0.3 * female + 0.5 * e)
visits <- rpois(n, lambda)

df1 <- data.frame(
  visits = visits, time = time, phone = phone,
  frfam = frfam, female = female, ad = fe_grp
)

cat(sprintf("n = %d   groups = %d   visits mean = %.2f\n", n, n_ad, mean(visits)))
```

```
n = 5000   groups = 20   visits mean = 26.43
```

15.2.2. Naive Poisson — endogeneity bias

Ignoring the endogeneity of `time` gives a biased estimate of α_1:

```r
naive1 <- fepois(visits ~ time + frfam | ad + female, data = df1)
etable(naive1, title = "Naive Poisson - time is endogenous")
```

```
                           naive1
Dependent Var.:            visits

time             1.139*** (0.0025)
frfam           0.2202*** (0.0092)
Fixed-Effects:  ------------------
ad                            Yes
female                        Yes
--------------- ------------------
S.E. type                     IID
Observations                5,000
Squared Cor.              0.95297
```

```
Pseudo R2                   0.91842
BIC                        32,823.9
---
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The naive estimate is 1.139 against a true 0.8, an upward bias of 0.34. The sign follows the callout above: $\rho = 0.5 > 0$, so omitting the control function leaves a multiplicative $e^{0.5e}$ term positively correlated with `time`. The magnitude is roughly predictable too. Conditioning on `frfam`, `ad` and `female` but not `phone`, the surviving variation in `time` is $1.5\,\text{phone} + e$ with variance $1.5^2(0.4)(0.6) + 1 = 1.54$, so a linear approximation puts the bias near $0.5 \times 1/1.54 = 0.32$, close to the 0.34 observed.

15.2.3. Control function approach

The two-step CF procedure:

1. **First stage** — project `time` on `phone`, `frfam`, and fixed effects via `feols`. The residual $\hat{v}_2 = \text{time} - \widehat{\text{time}}$ satisfies $E(\mathbf{z}'\hat{v}_2) = 0$ by construction. A linear first stage is appropriate here because `time` is continuous. **This does not carry over when y_2 is binary** — see the derivation in Part 2, where Wooldridge is explicit that the linear reduced-form/independent-error setup cannot be assumed once y_2 is discrete (Wooldridge 2010, sec. 18.5; Imbens and Wooldridge 2007).
2. **Second stage** — Poisson QMLE with $\hat{v}_2$ added as a regressor inside the exponential mean. The CF assumption $E[y_1 \mid \mathbf{z}_1, y_2, v_2] = \exp(\mathbf{z}_1\delta_1 + \alpha_1 y_2 + \rho v_2)$ holds exactly here because the DGP specifies $0.5\,e$ linearly inside $\exp(\cdot)$.

Note that unlike 2SLS in a linear model, $\hat{v}_2$ does *not* replace y_2 — both enter together.

```r
# Step 1: linear projection
fs1       <- feols(time ~ phone + frfam | ad + female, data = df1)
df1$v2hat <- residuals(fs1)

# Step 2: Poisson CF
m1_cf <- fepois(visits ~ time + v2hat + frfam | ad + female, data = df1)
etable(m1_cf, title = "CF: linear projection first stage")
```

```
                              m1_cf
Dependent Var.:              visits

time              0.7801*** (0.0041)
v2hat             0.5150*** (0.0050)
frfam             0.3775*** (0.0093)
Fixed-Effects:    ------------------
ad                               Yes
female                           Yes
----------------- ------------------
```

```
S.E. type                      IID
Observations                 5,000
Squared Cor.               0.99614
Pseudo R2                  0.94394
BIC                       22,627.7
---
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
cat(sprintf("time coefficient: %.4f  (true %.1f)\n", coef(m1_cf)["time"], true_alpha))
```

```
time coefficient: 0.7801  (true 0.8)
```

The coefficient on `time` is 0.7801, within 2.5 percent of the true $\alpha_1 = 0.8$, against the naive 1.139. The coefficient on `v2hat` is 0.5150, which is ρ, and the DGP sets $\rho = 0.5$ — the control function recovers the endogeneity parameter as well as the structural one. Its t-statistic of 103 is the Hausman-type test, and it rejects exogeneity decisively.

15.2.4. Bootstrap standard errors

The second-stage standard errors are incorrect because first-stage estimation error propagates. A cluster bootstrap (by `ad` group) corrects this. `parLapply` with a PSOCK cluster parallelises the 500 replications safely inside Quarto's rendering environment:

```
bootstrap_cf_continuous <- function(df, B = 500) {
  clusters <- unique(df$ad)
  K        <- length(clusters)

  boot_one <- function(b) {
    library(fixest)
    set.seed(b)
    drawn <- sample(clusters, K, replace = TRUE)
    df_b  <- do.call(rbind, lapply(seq_along(drawn), function(i) {
      sub         <- df[df$ad == drawn[i], ]
      sub$ad_boot <- i
      sub
    }))
    fs         <- feols(time ~ phone + frfam | ad_boot + female, data = df_b)
    df_b$v2hat <- residuals(fs)
    ss         <- fepois(visits ~ time + v2hat + frfam | ad_boot + female, data = df_b)
    coef(ss)["time"]
  }

  n_cores <- max(1L, detectCores() - 1L)
  cl      <- makePSOCKcluster(n_cores)
  clusterExport(cl, c("df", "clusters", "K"), envir = environment())
  results <- parLapply(cl, seq_len(B), boot_one)
```

```
  stopCluster(cl)
  unlist(results)
}

boot1    <- bootstrap_cf_continuous(df1, 500)
coef1    <- coef(m1_cf)["time"]
naive_se <- se(m1_cf)["time"]

cat(sprintf("time coefficient:   %.4f  (true %.1f)\n", coef1, true_alpha))
```

```
time coefficient:   0.7801  (true 0.8)
```

```
cat(sprintf("Naive 2nd-stage SE: %.4f\n", naive_se))
```

```
Naive 2nd-stage SE: 0.0041
```

```
cat(sprintf("Bootstrap SE:       %.4f\n", sd(boot1)))
```

```
Bootstrap SE:       0.0079
```

```
cat(sprintf("Bootstrap 95%% CI: [%.4f, %.4f]\n",
            quantile(boot1, 0.025), quantile(boot1, 0.975)))
```

```
Bootstrap 95% CI: [0.7637, 0.7930]
```

The bootstrap standard error is 0.0079 against the naive second-stage 0.0041, almost twice as large, confirming that ignoring first-stage uncertainty understates uncertainty by about half here. Note that the 95% interval here happens to just exclude the true 0.8: a Monte Carlo over fresh seeds of this design centers the CF estimator on 0.80 (mean 0.802, sd 0.009), so this draw is simply an unlucky one — and with only 20 clusters the bootstrap interval is itself noisy.

15.3. Part 2 — Binary endogenous variable

When $y_2 \in \{0, 1\}$, the linear-projection first stage from Part 1 loses its justification, and two probit-based alternatives become the theoretically grounded routes instead.

15.3.1. Data generating process

We use a clean probit DGP for the binary endogenous variable with a fresh structural error `e2` independent of Part 1. The endogeneity coefficient `= 0.2` is kept small so the CF assumption holds approximately well and estimates are close to the true value.

The covariates `phone`, `frfam`, `female` and the group effects α_g are carried over from Part 1, and we draw a new $e_2 \sim N(0,1)$. The binary endogenous variable is a probit threshold,

$$\text{time\_hi} = \mathbb{1}\{1.5\,\text{phone} + 0.4\,\text{frfam} + e_2 \geq 0\}, \tag{15.4}$$

and the count is visits $\sim$ Poisson(λ_2) with

$$\lambda_2 = \exp(0.5 + 0.8\,\text{time\_hi} + 0.4\,\text{frfam} + \alpha_g + 0.3\,\text{female} + 0.2\,e_2). \tag{15.5}$$

The true α_1 is again 0.8, and $\rho = 0.2$. Because the latent index is exactly a probit, the first stage in Approach B is correctly specified by construction, which isolates the approximation error from any misspecification.

```
set.seed(123)
e2      <- rnorm(n)

# Binary endogenous variable: clean probit latent index
time_hi <- as.integer(1.5 * phone + 0.4 * frfam + e2 >= 0)

# Poisson outcome - true coefficient 0.8, endogeneity via  *e2 ( =0.2)
rho2    <- 0.2
lambda2 <- exp(0.5 + true_alpha * time_hi + 0.4 * frfam +
               fe_ad[fe_grp] + 0.3 * female + rho2 * e2)
visits2 <- rpois(n, lambda2)

df2 <- data.frame(
  visits = visits2, time_hi = time_hi, phone = phone,
  frfam = frfam, female = female, ad = fe_grp, e2 = e2
)

cat(sprintf("time_hi mean:      %.3f\n", mean(time_hi)))
```

```
time_hi mean:      0.729
```

```
cat(sprintf("Cor(time_hi, e2):  %.3f  (endogeneity)\n", cor(time_hi, e2)))
```

```
Cor(time_hi, e2):  0.622  (endogeneity)
```

```
cat(sprintf("visits mean:       %.2f\n", mean(visits2)))
```

```
visits mean:       6.18
```

73% of observations have `time_hi = 1`, the correlation between `time_hi` and e_2 is 0.622, and mean visits are 6.18, well below Part 1's 26.4 — a binary regressor cannot generate the exponential variation that a continuous, unbounded `time` does.

15.3.2. Why Approach A's assumption breaks down for binary y_2

Part 1's control function rests on writing $y_2 = \mathbf{z}\pi + v_2$ with v_2 independent of $\mathbf{z}$ (Assumption (3.25) in Imbens and Wooldridge (2007); eq. 18.39 in Wooldridge (2010), sec. 18.5). That is a genuine assumption about the *reduced form*, not an automatic consequence of OLS: the population linear projection $\mathbf{z}\pi$ always exists and its residual is always uncorrelated with $\mathbf{z}$ ($E(\mathbf{z}'v_2) = 0$), but *independence* of v_2 from $\mathbf{z}$ is much stronger, and it is what the Poisson control-function argument actually needs.

For a binary y_2, $v_2 = y_2 - \mathbf{z}\pi$ is degenerate given $\mathbf{z}$: conditional on $\mathbf{z}$, v_2 takes only the two values $1-\mathbf{z}\pi$ (when $y_2 = 1$) and $-\mathbf{z}\pi$ (when $y_2 = 0$), with $\text{Var}(v_2 \mid \mathbf{z}) = p(\mathbf{z})[1-p(\mathbf{z})]$ a function of $\mathbf{z}$ itself. Independence from $\mathbf{z}$ is not just approximately violated — it fails by construction. Wooldridge makes this explicit in the *exact* setting of this chapter (Poisson/exponential mean, binary EEV — **e**ndogenous **e**xplanatory **v**ariable): starting from the same equations (3.23)–(3.25) used for the continuous case, he writes

> "If y_2 is a binary model following a probit, then a CF approach due to Terza (1998) can be used… **We can no longer assume (3.24) and (3.25).** Instead, replace (3.24) $y_2 = 1[\mathbf{z}\pi_2 + v_2 \geq 0]$ **and still adopt (3.25).** In fact, we assume (r_1, v_2) is jointly normal." (Imbens and Wooldridge 2007, sec. 3.3; nearly identical language in Wooldridge 2010, sec. 18.5: "it would be very unusual for a discrete variable to be expressible as a linear equation with additive error independent of $\mathbf{z}$")

The closing clause is the one to read twice. (3.25) — independence of the unobservables from $\mathbf{z}$ — is not discarded; it is *re-asserted*, now about the latent v_2 of the threshold model. Only (3.24), the linear reduced form, is replaced. So the move is not "drop the independence assumption" but "assert it about a different object," which is exactly the distinction the next paragraph turns on.

So Approach A is not a weaker-but-valid alternative to Approach B for binary y_2 — its own first-stage assumption cannot hold. What survives is a **local approximation** argument, derived below, plus its unconditional validity as a Hausman-type exogeneity test (the t-statistic on $\hat{v}_2$ stays valid under $H_0 : \rho = 0$ regardless of how v_2 was constructed, because under the null there is no endogeneity left to approximate).

Does Approach B avoid this by not being two-valued given z? No — it is two-valued too, and that is not the distinction. Approach B's generalized residual $\hat{r}_2 = y_2\lambda(\mathbf{z}\hat{\pi}_2) - (1-y_2)\lambda(-\mathbf{z}\hat{\pi}_2)$ is, for fixed $\mathbf{z}$, a function of $y_2 \in \{0, 1\}$ alone — exactly as degenerate given $\mathbf{z}$ as Approach A's v_2. What differs is *which* object the independence assumption is asserted about. Approach A asserts it directly of the observed, two-valued $v_2 = y_2 - \mathbf{z}\pi_2$, and that fails because both the two values themselves and the probabilities of landing on them move with $\mathbf{z}$ — nothing about

v_2 is **z**-invariant. Terza's setup instead asserts independence of the **continuous latent probit error** v_2 in $y_2 = 1[\mathbf{z}\pi_2 + v_2 \geq 0]$, where $v_2 \mid \mathbf{z} \sim N(0,1)$ — the same assumption every probit MLE already makes, and one that is genuinely **z**-invariant. The generalized residual is then the *correct conditional expectation* of a transform of that continuous error given the observed binary y_2 (worked out explicitly below); it comes out two-valued given **z** because it is a statistic derived from something continuous and conditioned on a coarse observation of it, not because independence of **z** failed anywhere in its derivation. Discreteness of the *derived* residual and independence-of-**z** of the *underlying* error are separate properties — Approach A conflates them, Approach B does not.

15.3.3. The exact correction (Terza 1998) and Approach B as its local approximation

Terza (1998), exposited in Wooldridge (2010) (sec. 18.5) and Imbens and Wooldridge (2007) (sec. 3.3, eqs. 3.32–3.36), derives the *exact* control function when y_2 follows a probit, $y_2 = 1[\mathbf{z}\pi_2 + v_2 \geq 0]$, and (c_1, v_2) — the Poisson heterogeneity and the probit error — are jointly normal. Here ρ is the *coefficient* in the linear projection of c_1 on v_2, i.e. $E(\exp(c_1) \mid v_2) = \exp(\rho v_2)$ — **not** the correlation between them. With v_2 standard normal the two differ by a factor of σ_{c_1}, since $\rho = \mathrm{Corr}(c_1, v_2)\,\sigma_{c_1}$, and the distinction is not cosmetic: this chapter's DGP sets $c_1 = 0.2\,e_2$ with $v_2 = e_2$, so the *correlation* is exactly 1 while the ρ that governs the approximation below is 0.2. Reading ρ as a correlation is also what $h(\cdot)$ forbids — it is ρ in the coefficient sense that makes $\exp(\rho^2/2)$ the lognormal correction and puts ρ on the same scale as the probit index. It is **not** "add the generalized residual linearly inside $\exp(\cdot)$" (that is Approach B as coded below). The exact result is multiplicative:

$$E(y_1 \mid \mathbf{z}, y_2) = \exp(\mathbf{z}_1\delta_1 + \alpha_1 y_2) \cdot h(y_2, \mathbf{z}\pi_2, \rho) \tag{15.6}$$

$$h(y_2, \mathbf{z}\pi_2, \rho) = \exp(\rho^2/2)\left\{ y_2 \frac{\Phi(\rho + \mathbf{z}\pi_2)}{\Phi(\mathbf{z}\pi_2)} + (1 - y_2)\frac{1 - \Phi(\rho + \mathbf{z}\pi_2)}{1 - \Phi(\mathbf{z}\pi_2)} \right\} \tag{15.7}$$

estimated by plugging in the first-stage probit's $\hat{\pi}_2$ and then jointly QMLE-fitting $(\delta_1, \alpha_1, \rho)$ against this mean function — a genuinely nonlinear second stage, not a linear regressor addition.

Why Approach B (linear addition of the generalized residual) is a good approximation, and exactly how good. Taylor-expand h in ρ around $\rho = 0$. Since $\frac{d}{d\rho}\exp(\rho^2/2)\big|_{\rho=0} = 0$, only the Φ-ratio terms contribute to first order:

$$\frac{\partial}{\partial\rho}\frac{\Phi(\rho + \mathbf{z}\pi_2)}{\Phi(\mathbf{z}\pi_2)}\bigg|_{\rho=0} = \frac{\phi(\mathbf{z}\pi_2)}{\Phi(\mathbf{z}\pi_2)} = \lambda(\mathbf{z}\pi_2), \qquad \frac{\partial}{\partial\rho}\frac{1 - \Phi(\rho + \mathbf{z}\pi_2)}{1 - \Phi(\mathbf{z}\pi_2)}\bigg|_{\rho=0} = -\lambda(-\mathbf{z}\pi_2) \tag{15.8}$$

where $\lambda(\cdot) = \phi(\cdot)/\Phi(\cdot)$ is the inverse Mills ratio. Since $h(y_2, \mathbf{z}\pi_2, 0) = 1$,

$$h(y_2, \mathbf{z}\pi_2, \rho) \approx 1 + \rho \cdot \underbrace{\left[y_2\lambda(\mathbf{z}\pi_2) - (1 - y_2)\lambda(-\mathbf{z}\pi_2) \right]}_{= \hat{r}_2,\ \text{the generalized residual}} \tag{15.9}$$

and $\exp(x_1\beta_1) \cdot h(\cdot) \approx \exp(x_1\beta_1) \cdot (1 + \rho\,\hat{r}_2) \approx \exp(x_1\beta_1 + \rho\,\hat{r}_2)$ for small $\rho\,\hat{r}_2$ — recovering exactly Approach B's construction (adding $\hat{r}_2$ linearly inside $\exp(\cdot)$). So **Approach B is the first-order-in-ρ local approximation to Terza's exact correction**, not the exact correction itself; the approximation error is $O(\rho^2)$. This is consistent with the DGP above (small $\rho = 0.2$, kept deliberately small so this approximation stays tight) and with a direct large-n check of both A and

B run against this chapter's own DGP: the probability limits are $\hat{\alpha}_1^A \to 0.839$ and $\hat{\alpha}_1^B \to 0.822$ against the true 0.8 — both biased, B roughly half of A's bias, matching the theoretical ranking (B is a valid local approximation to the exact correction; A has no such derivation behind it at all for binary y_2) but neither exact.

15.3.4. Why use Approach B instead of the exact correction?

If Terza's $h(\cdot)$ is exact and Approach B is only its local approximation, why does anyone use the approximation? Four reasons, not just convenience:

1. **The exact correction still needs a nonlinear second stage — it does not drop into standard FE-Poisson software.** Even in Terza's own procedure, stage 2 fits $\exp(\mathbf{x}_1\beta_1) \cdot h(y_2, \mathbf{z}\hat{\pi}_2, \rho)$, and $h(\cdot)$ is not reducible to exp(linear index) — the Φ-ratios and the $\exp(\rho^2/2)$ term make it genuinely nonlinear in ρ. Approach B's second stage, by contrast, has *exactly* the same functional form as ordinary Poisson — exp(linear index) — with one extra regressor ($\hat{r}_2$) added to the index. It runs in `fepois`/`ppmlhdfe`/`xtpoisson` unmodified; the exact correction needs custom nonlinear-optimization code, and — per the callout on Approach C above — combining a genuinely nonlinear correction with the within-group demeaning that makes FE-Poisson tractable with many fixed effects is not straightforward. That cost scales badly exactly where FE-Poisson is most needed: many high-dimensional fixed effects.

2. **Joint nonlinear estimation is more efficient but harder to converge.** Wooldridge says this explicitly for the closely analogous binary-y_1, binary-y_2 case (Wooldridge 2010, sec. 15.7.3): *"Maximum likelihood estimation has some decided advantages over two-step procedures. First, MLE is more efficient than any two-step procedure… The drawback with the MLE is computational. Sometimes it can be difficult to get the iterations to converge, as $\hat{\rho}_1$ sometimes tends toward -1 or 1."* The same tension — efficiency versus numerical fragility — applies to fitting Terza's $h(\cdot)$.

3. **The efficiency gain often is not worth it when ρ is modest.** The verified check above already showed A and B within a few percent of the truth at $\rho = 0.2$ (0.839 / 0.822 vs. 0.8). When the approximation error is that small next to ordinary sampling noise, the extra machinery buys little against real implementation risk.

4. **Approach B is more diagnostically transparent.** The coefficient on $\hat{r}_2$ is a plain Hausman-type t-test for exogeneity, sitting next to the coefficient of interest in a regression table that looks like any other. A rewritten joint likelihood with a different functional form does not give the same "how much did this correction move my estimate" legibility.

This is the standard control-function tradeoff (2SRI generally, not just this Poisson case): give up some efficiency for a second stage that is numerically simple, compatible with existing FE software, and easy to diagnose.

15.3.5. Three approaches: practical recap

	First stage	What enters 2nd stage	Status for binary y_2
A	`feols` (OLS)	Residuals $\hat{v}_2$	**No valid first-stage assumption** — v_2 independent of **z** fails by construction; usable only as an approximation, and as a Hausman-type exogeneity test
B	`feglm` (probit)	Gen. residuals $\hat{r}_2$	First-order-in-ρ local approximation to Terza's exact correction; requires probit correctly specified
C	Any model	Fitted values $\hat{p}_2$	Distribution-free (Mullahy (1997)) — robust to misspecification of the model for $\Pr(y_2 = 1 \mid \mathbf{z})$

None of the three recovers the *exact* Terza (1998) correction as coded — that requires fitting the nonlinear $h(\cdot)$ mean function jointly (previous subsection). A and B additionally require the second-stage Poisson conditional mean to be correctly specified; C's validity does not depend on how well the first-stage model for y_2 fits.

15.3.6. Approach A — Linear projection CF

```
fs_a      <- feols(time_hi ~ phone + frfam | ad + female, data = df2)
df2$v2hat <- residuals(fs_a)

m_a <- fepois(visits ~ time_hi + v2hat + frfam | ad + female, data = df2)
etable(m_a, title = "Approach A: OLS residual as control function")
```

```
                              m_a
Dependent Var.:            visits

time_hi          0.8613*** (0.0321)
v2hat            0.2674*** (0.0318)
frfam            0.4168*** (0.0199)
Fixed-Effects:   ------------------
ad                             Yes
female                         Yes
```

```
--------------- ------------------
S.E. type                      IID
Observations                 5,000
Squared Cor.               0.73423
Pseudo R2                  0.41908
BIC                       22,269.3
---
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Approach A gives 0.8613 with a coefficient of 0.2674 on `v2hat`.

15.3.7. Approach B — Probit generalized residuals

```
fs_b <- feglm(time_hi ~ phone + frfam | ad + female,
              data = df2, family = binomial(probit))

# Generalized residual: r̂ = y· ( )/Φ( ) − (1−y)· ( )/(1−Φ( ))
eta      <- predict(fs_b, type = "link")
df2$gr2  <- ifelse(df2$time_hi == 1,
                   dnorm(eta) / pnorm(eta),
                   -dnorm(eta) / (1 - pnorm(eta)))

m_b <- fepois(visits ~ time_hi + gr2 + frfam | ad + female, data = df2)
etable(m_b, title = "Approach B: probit generalized residual")
```

```
                             m_b
Dependent Var.:           visits

time_hi         0.8544*** (0.0329)
gr2             0.1664*** (0.0200)
frfam           0.4162*** (0.0199)
Fixed-Effects:  ------------------
ad                            Yes
female                        Yes
--------------- ------------------
S.E. type                      IID
Observations                 5,000
Squared Cor.               0.73424
Pseudo R2                  0.41905
BIC                       22,270.4
---
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Approach B gives 0.8544, slightly closer to the truth than A's 0.8613, with a coefficient of 0.1664 on the generalized residual. That coefficient is an estimate of ρ, whose true value is 0.2; it is not exact because Approach B is itself only a first-order approximation.

15.3.8. Does the first stage have to be probit? Logit works too — with a caveat

Nothing about the local-approximation argument above is specific to the normal distribution. The generalized residual is defined for *any* correctly-specified binary-response GLM (Gourieroux et al. 1987), and for a **logit** first stage it collapses to something particularly simple. With $\Lambda(\cdot)$ the logistic CDF and $\lambda(\eta) = \Lambda(\eta)[1-\Lambda(\eta)]$ its density,

$$\hat{r}_2 = y_2 \cdot \frac{\lambda(\hat{\eta})}{\Lambda(\hat{\eta})} - (1-y_2) \cdot \frac{\lambda(\hat{\eta})}{1-\Lambda(\hat{\eta})} = y_2[1-\Lambda(\hat{\eta})] - (1-y_2)\Lambda(\hat{\eta}) = y_2 - \Lambda(\hat{\eta}) = y_2 - \hat{p}_2, \quad (15.10)$$

i.e. **the logit generalized residual is just the plain response residual** $y_2 - \hat{p}_2$ — no Mills-ratio machinery needed: `logit` first stage, then `y2 - predict(fit, type="response")`, is the right generalized residual, not a shortcut.

```
fs_b_logit <- feglm(time_hi ~ phone + frfam | ad + female,
                    data = df2, family = binomial(logit))
df2$phat_logit <- predict(fs_b_logit, type = "response")
df2$gr2_logit  <- df2$time_hi - df2$phat_logit   # algebraically = generalized residual

m_b_logit <- fepois(visits ~ time_hi + gr2_logit + frfam | ad + female, data = df2)
etable(m_b_logit, title = "Approach B with a logit first stage")
```

```
                        m_b_logit
Dependent Var.:            visits

time_hi          0.8625*** (0.0320)
gr2_logit        0.2668*** (0.0317)
frfam            0.4157*** (0.0199)
Fixed-Effects:  ------------------
ad                             Yes
female                         Yes
--------------- ------------------
S.E. type                      IID
Observations                 5,000
Squared Cor.               0.73428
Pseudo R2                  0.41908
BIC                       22,269.2
---
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
cat(sprintf("Probit-based Approach B: alpha1 = %.4f\n", coef(m_b)["time_hi"]))
```

```
Probit-based Approach B: alpha1 = 0.8544
```

```
cat(sprintf("Logit-based  Approach B: alpha1 = %.4f\n", coef(m_b_logit)["time_hi"]))
```

```
Logit-based  Approach B: alpha1 = 0.8625
```

The logit first stage gives 0.8625 against the probit's 0.8544, a difference of 0.008 on estimates whose standard errors are 0.032. The choice of link is not what matters here.

Two important limits, though:

1. **The *exact* Terza (1998) correction is probit-specific, not logit.** The closed form $h(\cdot)$ derived above comes from the truncated-moment properties of the *bivariate normal* — $E[\exp(\rho v_2) \mid v_2 > \mathbf{z}\pi_2]$ has that clean Φ-ratio form only because v_2 is normal. A logistic v_2 has no analogous tractable joint distribution with the Poisson heterogeneity, and no such exact result appears anywhere in Wooldridge (2010), Imbens and Wooldridge (2007), or Terza (1998) for a logit reduced form. So logit gives you Approach B (the local, $O(\rho^2)$-error approximation) exactly as validly as probit — but it does not give you a logit analog of the exact correction, because none has been worked out in this literature.
2. **As a fitted-probability *instrument* (Approach C), probit and logit are both fine.** From Imbens and Wooldridge (2007): *"Obtain the fitted probabilities, $\Phi(\mathbf{z}_i\hat{\pi}_2)$, from the first stage probit, and then use these as IVs for y_{i2}. This method is fully robust to misspecification of the probit model; the standard errors need not be adjusted for the first-stage probit (asymptotically)."* Nothing in that argument privileges the normal link: a logit fitted probability is simply another function of $\mathbf{z}$, and any function of $\mathbf{z}$ is a valid instrument. That robustness (correct even if the link is wrong) is specific to using the fitted value as an *instrument*, not as a *residual* — it does not carry over to Approaches A or B, where the second-stage conditional mean still has to be exactly right.

15.3.9. Approach C — Fitted values as instrument (IV/GMM Poisson)

Any function of the exogenous $\mathbf{z}$ is a valid instrument. Probit fitted probabilities are used here as the instrument for `time_hi`. We implement IV/GMM Poisson via `gmm::gmm()` with the multiplicative moment conditions $E[\mathbf{z}'(y_1 e^{-\mathbf{x}\beta} - 1)] = 0$ (Mullahy 1997). The additive moments $E[\mathbf{z}'(y_1 - \exp(\mathbf{x}\beta))] = 0$ would be inconsistent here: the DGP puts its unobservables (the `ad` effects, `female`, and $\rho_2 e_2$) inside $\exp(\cdot)$, so the structural error is multiplicative — running the additive version on these data gives $\hat{\alpha}_1 \approx 0.63$, a bias no intercept shift can absorb:

```
# Probit without FE for fitted probabilities (instrument)
fs_c     <- glm(time_hi ~ phone + frfam, family = binomial(probit), data = df2)
df2$phat <- fitted(fs_c)

# Multiplicative moment conditions (Mullahy 1997): Z'(y*exp(-X*beta) - 1) = 0
# X = [time_hi, frfam, 1],  Z = [phat, frfam, 1]
gmm_moments <- function(beta, x) {
  y      <- x[, "visits"]
  th     <- x[, "time_hi"]
  fr     <- x[, "frfam"]
```

```
  ph     <- x[, "phat"]
  eta    <- pmin(pmax(beta[1] * th + beta[2] * fr + beta[3], -30), 30)
  resid  <- y * exp(-eta) - 1
  cbind(ph * resid, fr * resid, resid)
}

gmm_data <- as.matrix(df2[, c("visits", "time_hi", "frfam", "phat")])
m_c      <- gmm(gmm_moments, x = gmm_data,
                t0 = c(0, 0, log(mean(df2$visits))))
coef_c   <- coef(m_c)
se_c     <- sqrt(diag(vcov(m_c)))

cat(sprintf("\nApproach C - IV/GMM Poisson (no FE, for illustration)\n"))
```

```
Approach C - IV/GMM Poisson (no FE, for illustration)
```

```
cat(sprintf("  time_hi:  coef = %7.4f   SE = %7.4f   z = %6.3f\n",
            coef_c[1], se_c[1], coef_c[1] / se_c[1]))
```

```
  time_hi:  coef =  0.7844   SE =  0.0656   z = 11.950
```

```
cat(sprintf("  frfam:    coef = %7.4f   SE = %7.4f   z = %6.3f\n",
            coef_c[2], se_c[2], coef_c[2] / se_c[2]))
```

```
  frfam:    coef =  0.4576   SE =  0.0370   z = 12.369
```

GMM gives 0.7844 with a standard error of 0.0656 on `time_hi`, and 0.4576 on `frfam` against a true 0.4. The point estimate is the closest of the three to 0.8, and the standard error is twice A's and B's — the trade-off the comparison section below takes apart.

> **Note**
>
> **Approach C with FE** requires demeaning the regressor and instrument matrices via within-group centering before solving the Poisson moment conditions. Because Poisson is nonlinear this is not straightforward. Approaches A and B both run unmodified in `feols`/`feglm` + `fepois`, so either is practical when many fixed effects are present; of the two, prefer B (see the takeaway below).

15.3.10. A vs C — not equivalent in Poisson

In a linear model, using $\hat{y}_2$ as an instrument (2SLS) is numerically identical to adding $\hat{v}_2 = y_2 - \hat{y}_2$ as a regressor — the Frisch-Waugh-Lovell theorem. **In Poisson this equivalence breaks**, because Approach A puts $\hat{v}_2$ inside $\exp(\cdot)$ while Approach C uses $\hat{y}_2$ in the moment conditions without modifying the exponential mean.

```
coef_a <- coef(m_a)["time_hi"]
coef_b <- coef(m_b)["time_hi"]

cat(sprintf("Approach A  (CF, OLS residual, with FE):   ^ = %.4f\n", coef_a))
```

```
Approach A  (CF, OLS residual, with FE):   ^ = 0.8613
```

```
cat(sprintf("Approach B  (CF, probit GR, with FE):      ^ = %.4f\n", coef_b))
```

```
Approach B  (CF, probit GR, with FE):      ^ = 0.8544
```

```
cat(sprintf("Approach C  (IV/GMM Poisson, no FE):       ^ = %.4f\n", coef_c[1]))
```

```
Approach C  (IV/GMM Poisson, no FE):       ^ = 0.7844
```

```
cat(sprintf("True    = %.1f\n", true_alpha))
```

```
True    = 0.8
```

The three land at 0.8613, 0.8544 and 0.7844 against a true 0.8. They differ despite using the same exclusion restriction, which is the point: in a linear model FWL would force A and C to agree exactly, and here they are 0.077 apart. 2SLS $\neq$ CF for Poisson.

15.3.11. Bootstrap for Approach A (binary EEV)

```
bootstrap_cf_binary <- function(df, B = 500) {
  clusters <- unique(df$ad)
  K        <- length(clusters)

  boot_one <- function(b) {
    library(fixest)
    set.seed(b)
    drawn <- sample(clusters, K, replace = TRUE)
    df_b  <- do.call(rbind, lapply(seq_along(drawn), function(i) {
      sub         <- df[df$ad == drawn[i], ]
      sub$ad_boot <- i
      sub
    }))
    fs         <- feols(time_hi ~ phone + frfam | ad_boot + female, data = df_b)
    df_b$v2hat <- residuals(fs)
    ss         <- fepois(visits ~ time_hi + v2hat + frfam | ad_boot + female, data = df_b)
    coef(ss)["time_hi"]
```

```
  }

  n_cores <- max(1L, detectCores() - 1L)
  cl      <- makePSOCKcluster(n_cores)
  clusterExport(cl, c("df", "clusters", "K"), envir = environment())
  results <- parLapply(cl, seq_len(B), boot_one)
  stopCluster(cl)
  unlist(results)
}

boot2     <- bootstrap_cf_binary(df2, 500)
naive_se2 <- se(m_a)["time_hi"]

cat(sprintf("Approach A bootstrap (binary EEV, 500 reps, cluster by ad)\n"))
```

```
Approach A bootstrap (binary EEV, 500 reps, cluster by ad)
```

```
cat(sprintf("  Point estimate:     %.4f  (true %.1f)\n", coef_a, true_alpha))
```

```
  Point estimate:     0.8613  (true 0.8)
```

```
cat(sprintf("  Naive 2nd-stage SE: %.4f\n", naive_se2))
```

```
  Naive 2nd-stage SE: 0.0321
```

```
cat(sprintf("  Bootstrap SE:       %.4f\n", sd(boot2)))
```

```
  Bootstrap SE:       0.0331
```

```
cat(sprintf("  Bootstrap 95%% CI:  [%.4f, %.4f]\n",
            quantile(boot2, 0.025), quantile(boot2, 0.975)))
```

```
  Bootstrap 95% CI:  [0.8025, 0.9308]
```

The bootstrap standard error is 0.0331 against the naive 0.0321 — almost identical here, unlike Part 1 where the bootstrap was twice the naive figure. The reason is that the binary first stage carries much less estimation uncertainty into the second stage than the continuous one did. The 95% interval $[0.803, 0.931]$ excludes the true 0.8, which is the approximation bias discussed above showing up in inference, not a coverage failure of the bootstrap.

15.3.12. Comparison across all methods

```r
naive_p2    <- fepois(visits ~ time_hi + frfam | ad + female, data = df2)
coef_naive2 <- coef(naive_p2)["time_hi"]

methods <- c(
  "True value",
  "Naive Poisson (binary y )",
  "A: Linear CF (with FE)",
  "B: Probit GR CF (with FE)",
  "C: IV/GMM Poisson (no FE)"
)
ests <- c(true_alpha, coef_naive2, coef_a, coef_b, coef_c[1])

cat("\n  Binary EEV: coefficient on time_hi            \n")
```

```
  Binary EEV: coefficient on time_hi
```

```r
for (i in seq_along(methods)) {
  cat(sprintf("  %-38s  ^  = %6.4f\n", methods[i], ests[i]))
}
```

```
  True value                              ^  = 0.8000
  Naive Poisson (binary y )             ^  = 1.0832
  A: Linear CF (with FE)                  ^  = 0.8613
  B: Probit GR CF (with FE)               ^  = 0.8544
  C: IV/GMM Poisson (no FE)               ^  = 0.7844
```

The naive Poisson gives 1.0832 against a true 0.8, so ignoring the endogeneity overstates the effect by 35%. Approaches A and B (with FE) recover the true value closely, at 0.8613 and 0.8544; a small upward bias from $\rho = 0.2$ and the binary EEV is expected — see the derivation above (A has no valid first-stage assumption for binary y_2; B is correct only to first order in ρ). Approach C happens to land closest in this draw, and its near-centring is not luck: the omitted `ad` and `female` effects are independent multiplicative heterogeneity, absorbed by the intercept under the multiplicative moments ($\beta_0 \to \beta_0 + \log E[e^c]$), so omitting the fixed effects costs no slope *bias* at all. Mullahy's moments are *consistent* for this DGP, where A and B are only approximately unbiased: over 10 seeds at this n, C averages 0.7985 against A's 0.8323 and B's 0.8185, and at $n = 10^6$ it sits at 0.8015 (sd 0.0013) against plims of 0.839 and 0.822. The catch is variance rather than bias. Without the fixed effects C is about three times as noisy (sd 0.068 at $n = 5000$, against 0.024 and 0.026), so at this sample size its RMSE is the *worst* of the three — 0.065, against A's 0.039 and B's 0.031 — and a single draw can easily leave it further from 0.8 than either CF estimator. With many groups, Approach B remains the practical choice, understood as a first-order approximation whose small bias buys real precision rather than as a theoretically exact method.

15.3.13. Takeaway for the binary case

- **Use Approach B when you need fixed effects.** Among the estimators that drop into FE-Poisson unmodified it is the theoretically motivated choice — the first-order-in-ρ local approximation to Terza's (1998) exact correction — and its error is *derived*, not assumed: $O(\rho^2)$, shrinking as the endogeneity shrinks.
- **Approach C trades bias for variance — decide which one binds.** Mullahy's multiplicative moments are *consistent* here, not merely accurate to first order, whenever the multiplicative heterogeneity is independent of $\mathbf{z}$. But consistency is not accuracy at a given sample size. Over 10 seeds at this chapter's $n = 5000$, C is nearly unbiased (mean 0.7985) and yet almost three times as noisy as B (sd 0.068 against 0.026), so its RMSE is the *worst* of the three: 0.065, against A's 0.039 and B's 0.031. B's small derived bias buys a great deal of precision. Prefer C once n is large enough that its variance is not the binding constraint — and note that it does not accommodate high-dimensional fixed effects either way.
- **Approach A can still be a reasonable numerical approximation** — this chapter's own simulation lands it close to the truth at $\rho = 0.2$ — but it has no theoretical justification for binary y_2 at all: its first-stage independence assumption fails by construction (see above). Treat it as a fallback, not a first choice, and do not lean on it when ρ is not known to be small.
- **Approach B works equally well with a probit or a logit first stage.** Both are valid instances of the same generalized-residual construction. Terza's *exact* correction happens to be worked out only for probit (it relies on bivariate normality), but the local-approximation argument that justifies Approach B does not require normality specifically — only a correctly-specified single-index binary model, which probit and logit both are.

Part IV.

Longitudinal Causal Inference

16. G-Methods for Time-Varying Treatments

```
library(tidyverse)
library(ltmle)
library(ipw)
library(survey)
library(broom)
```

Most chapters so far use one treatment decision. Many real problems are longitudinal. A patient receives medication at several visits. A worker can enter training in more than one year. Treatment changes over time, and covariates respond to past treatment. Those covariates then affect future treatment. Standard regression adjustment can fail in this setting.

> **Related reading**: The g-estimation with time-varying covariates chapter of *Topics on Econometrics and Causal Inference* derives the identification result with sequential ignorability and applies the parametric g-formula to a worked binary-treatment example. The LMTP framework (for continuous and modified-policy interventions) is covered in Longitudinal modified treatment policy (LMTP) and in the Continuous Treatments chapter.

The main tools are:

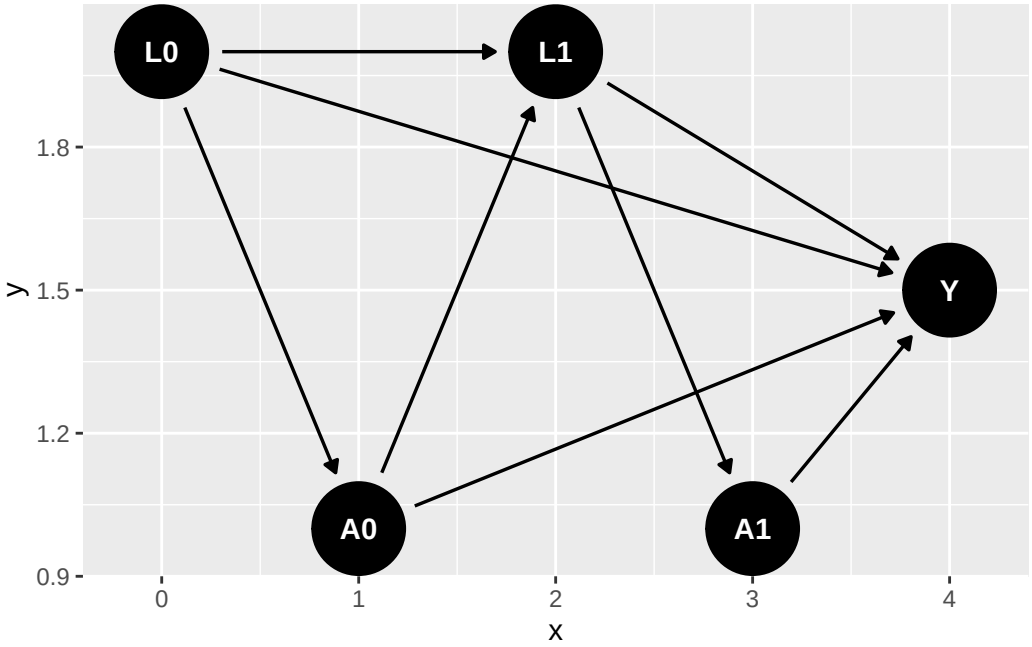
1. **G-formula**: simulate outcomes under a treatment regime.
2. **IPTW**: reweight observations by the probability of their treatment history.
3. **Marginal structural models**: fit a treatment-history regression in the weighted population.
4. **Longitudinal TMLE**: combine outcome and treatment-history models.

16.1. The time-varying confounding problem

Consider a two-period example:

- L_0: baseline covariate (e.g. health status at study entry)
- A_0: treatment at time 0 (e.g. medication)
- L_1: covariate at time 1, influenced by A_0 (e.g. health status after first dose)
- A_1: treatment at time 1, influenced by L_1
- Y: outcome at the end of follow-up (e.g. mortality)

The DAG:



L_1 is the problem variable. It confounds the effect of A_1 on Y, so we would like to adjust for it. But it is also affected by A_0, so adjusting for it blocks part of the effect of A_0. Regression without L_1 is confounded; regression with L_1 over-controls.

16.1.1. Simulating the failure of regression

We simulate $n = 5000$ units over two periods. The baseline covariate is $L_0 \sim N(0, 1)$, and the first treatment is $A_0 \sim \text{Bernoulli}(\Lambda(-0.5 + 0.8L_0))$. The time-varying confounder responds to the first treatment, $L_1 = 0.5L_0 + 1.0A_0 + \nu$ with $\nu \sim N(0, 1)$, and the second treatment responds to it, $A_1 \sim \text{Bernoulli}(\Lambda(-0.5 + 0.8L_1))$. The outcome is $Y = 0.5A_0 + 0.5A_1 + 0.5L_1 + 0.5L_0 + \varepsilon$ with $\varepsilon \sim N(0, 1)$.

The truth follows from the coefficients. A_1 has a total effect of 0.5. A_0 has a direct effect of 0.5 plus an indirect effect through L_1 of $1.0 \times 0.5 = 0.5$, so its total effect is 1.0. Always treating versus never treating is therefore worth $1.0 + 0.5 = 1.5$.

We fit two regressions that a reader might reach for first: one omitting L_1 and one including it.

```
set.seed(1)
n <- 5000

# Baseline covariate
L0 <- rnorm(n)
# Treatment at t=0 depends on L0
A0 <- rbinom(n, 1, plogis(-0.5 + 0.8 * L0))
# Time-varying confounder L1: affected by A0 (and L0)
L1 <- 0.5 * L0 + 1.0 * A0 + rnorm(n)
```

```
# Treatment at t=1 depends on L1
A1 <- rbinom(n, 1, plogis(-0.5 + 0.8 * L1))
# Outcome: each treatment has a direct effect of 0.5 on Y.
# A0 ALSO affects Y indirectly through L1 (A0 -> L1 -> Y): the L1 path
# contributes 1.0 * 0.5 = 0.5, so A0's TOTAL effect is 0.5 + 0.5 = 1.0,
# while A1's total effect is 0.5. Always-vs-never total effect = 1.5.
Y <- 0.5 * A0 + 0.5 * A1 + 0.5 * L1 + 0.5 * L0 + rnorm(n)

df <- tibble(L0, A0, L1, A1, Y)

cat("True total effect of always-treated vs never-treated: 1.5\n\n")
```

```
True total effect of always-treated vs never-treated: 1.5
```

```
# Naive regression 1: ignore L1
naive_no_L1 <- lm(Y ~ A0 + A1 + L0, data = df)
cat(sprintf("Naive (no L1): A0=%.3f, A1=%.3f, total=%.3f\n",
            coef(naive_no_L1)["A0"], coef(naive_no_L1)["A1"],
            coef(naive_no_L1)["A0"] + coef(naive_no_L1)["A1"]))
```

```
Naive (no L1): A0=0.979, A1=0.834, total=1.813
```

```
# Naive regression 2: include L1
naive_with_L1 <- lm(Y ~ A0 + A1 + L0 + L1, data = df)
cat(sprintf("Naive (with L1): A0=%.3f, A1=%.3f, total=%.3f\n",
            coef(naive_with_L1)["A0"], coef(naive_with_L1)["A1"],
            coef(naive_with_L1)["A0"] + coef(naive_with_L1)["A1"]))
```

```
Naive (with L1): A0=0.545, A1=0.503, total=1.048
```

Neither regression recovers the true total effect of 1.5. Omitting L_1 gives 1.813, too high; including it gives 1.048, too low. The truth is between them, and no choice between these two specifications reaches it.

The two failures have different sources:

- Excluding L_1 leaves confounding from $L_1 \to A_1$ that biases A_1's coefficient, 0.834 against a true 0.5. It also inflates A_0's coefficient to 0.979 for the same reason.
- Including L_1 blocks the $A_0 \to L_1 \to Y$ pathway, so A_0's coefficient now measures only the direct effect, not the total effect. It gives 0.545 against a true direct effect of 0.5, and A_1 gives 0.503 against a true 0.5 — both correct, but for the direct effects, and the direct effects do not sum to the total.

This is exactly the setting g-methods were designed for.

16.2. Parametric g-formula

The g-formula writes the mean outcome under a treatment regime as:

$$\mathbb{E}[Y(\bar{a})] = \sum_{\bar{l}} \mathbb{E}[Y \mid \bar{a}, \bar{l}] \prod_{t=0}^{K} f(l_t \mid \bar{a}_{t-1}, \bar{l}_{t-1}), \tag{16.1}$$

where $\bar{a} = (a_0, a_1, \ldots, a_K)$ is a treatment regime and $\bar{l}$ is the covariate history. In practice, we fit models for the time-varying covariates and the outcome, then simulate the data under the treatment regime.

```
# Step 1: Fit models for each time-varying variable
# Model L1 | L0, A0
mod_L1 <- lm(L1 ~ L0 + A0, data = df)
# Model Y | L0, A0, L1, A1
mod_Y  <- lm(Y  ~ L0 + A0 + L1 + A1, data = df)

# Step 2: Simulate counterfactual outcomes under each treatment regime
# Regime 1: always treat (A0=1, A1=1)
# Regime 2: never treat (A0=0, A1=0)

# Because both mod_L1 and mod_Y are linear, E[Y | do(A0=a0,A1=a1)] could be
# obtained by a deterministic plug-in instead: substitute the conditional
# mean E[L1|L0,A0=a0] directly into mod_Y, without ever drawing rnorm(). We
# simulate a full stochastic draw of L1 here instead, because that approach
# generalizes directly to non-linear systems (e.g. a binary L1 fit with
# logistic regression, or an interaction between L1 and A1 in mod_Y) where
# E[Y | do(...)] is NOT just mod_Y evaluated at the plugged-in conditional
# mean of L1 -- you genuinely need to integrate Y's model over the
# distribution of L1, which Monte Carlo simulation does automatically.
g_compute <- function(a0, a1) {
  L1_sim <- predict(mod_L1, newdata = transform(df, A0 = a0)) +
            rnorm(n, 0, sigma(mod_L1))   # simulate from L1 | L0, A0=a0
  Y_sim  <- predict(mod_Y, newdata = transform(df, A0 = a0, L1 = L1_sim, A1 = a1))
  mean(Y_sim)
}

# Monte Carlo: simulate many times to average over L1 distribution
set.seed(99)
B <- 100
E_Y11 <- mean(replicate(B, g_compute(1, 1)))
E_Y00 <- mean(replicate(B, g_compute(0, 0)))

g_effect <- E_Y11 - E_Y00
cat(sprintf("G-formula estimate (always treat vs never treat): %.3f\n", g_effect))
```

```
G-formula estimate (always treat vs never treat): 1.548
```

```
cat("True total effect: 1.5\n")
```

```
True total effect: 1.5
```

The g-formula gives 1.548 against a true 1.5, where the two regressions gave 1.813 and 1.048.

The important step is simulating L_1 under the counterfactual treatment. We do not condition on the observed L_1, because observed L_1 is partly a consequence of observed treatment.

16.2.1. What if we wanted the effect of treating only at t=0?

The g-formula can handle other regimes too. For example, treat only at $t = 0$:

```
set.seed(99)
E_Y10 <- mean(replicate(B, g_compute(1, 0)))
cat(sprintf("Effect of treating only at t=0:  %.3f  (true = 1.0)\n",
            E_Y10 - E_Y00))
```

```
Effect of treating only at t=0:  1.045  (true = 1.0)
```

This recovers the true effect of 1.0 from treating only at $t = 0$ — the direct effect of A_0 (0.5) plus its indirect effect through L_1 (0.5). Note this differs from A_0's *direct* effect alone (0.5): the regime $(1, 0)$ lets A_0 act through the mediator L_1.

16.3. IPTW for time-varying treatments

IPTW reweights observations by the inverse probability of their observed treatment history. The stabilized weight is:

$$SW_i = \prod_{t=0}^{K} \frac{f(A_t \mid \bar{A}_{t-1})}{f(A_t \mid \bar{A}_{t-1}, \bar{L}_t)}. \tag{16.2}$$

The numerator uses treatment history only. The denominator also conditions on the covariate history up to and including L_t — the covariates measured just before A_t. (Skipping L_t would leave the $L_t \to A_t$ confounding intact in the pseudo-population.) Stabilized weights usually have lower variance than unstabilized weights.

```
# Estimate the propensity-score components at each time
# Numerator: P(A0) and P(A1 | A0)
n0_fit <- glm(A0 ~ 1, data = df, family = binomial)
n1_fit <- glm(A1 ~ A0, data = df, family = binomial)

# Denominator: P(A0 | L0) and P(A1 | A0, L0, L1)
```

```
d0_fit <- glm(A0 ~ L0, data = df, family = binomial)
d1_fit <- glm(A1 ~ A0 + L0 + L1, data = df, family = binomial)

# Probability of observed A at each time
p_n0 <- predict(n0_fit, type = "response")
p_n1 <- predict(n1_fit, type = "response")
p_d0 <- predict(d0_fit, type = "response")
p_d1 <- predict(d1_fit, type = "response")

w_t0_num <- ifelse(df$A0 == 1, p_n0,           1 - p_n0)
w_t0_den <- ifelse(df$A0 == 1, p_d0,           1 - p_d0)
w_t1_num <- ifelse(df$A1 == 1, p_n1,           1 - p_n1)
w_t1_den <- ifelse(df$A1 == 1, p_d1,           1 - p_d1)

# Stabilized weight (product over time)
sw <- (w_t0_num * w_t1_num) / (w_t0_den * w_t1_den)

cat(sprintf("Weight summary: min=%.3f, mean=%.3f, max=%.3f\n",
            min(sw), mean(sw), max(sw)))
```

```
Weight summary: min=0.267, mean=0.999, max=14.792
```

```
# Trim extreme weights (standard practice)
sw_trimmed <- pmin(sw, quantile(sw, 0.99))
```

The weights run from 0.267 to 14.792 with a mean of 0.999. A mean of essentially one is what stabilized weights should give; the maximum of 14.8 is the part to watch, since a handful of observations carrying fifteen times their share will dominate the variance. That is what the trimming step addresses.

16.4. Marginal structural models

With the stabilized weights, the marginal structural model is a weighted regression of Y on treatment history:

```
# MSM: linear in cumulative treatment.
# Use self-documenting column names so the trimmed vs untrimmed weights
# are never confused in the survey design formulas below.
df$sw_trimmed <- sw_trimmed
design <- svydesign(ids = ~1, weights = ~sw_trimmed, data = df)
msm_fit <- svyglm(Y ~ A0 + A1, design = design)

tidy(msm_fit) |> knitr::kable(digits = 3,
                              caption = "MSM coefficients with stabilized weights")
```

Table 16.1.: MSM coefficients with stabilized weights

term	estimate	std.error	statistic	p.value
(Intercept)	-0.057	0.032	-1.744	0.081
A0	1.108	0.049	22.696	0.000
A1	0.545	0.048	11.439	0.000

```
cat(sprintf("\nMSM total effect (always vs never): %.3f  (true = 1.5)\n",
            coef(msm_fit)["A0"] + coef(msm_fit)["A1"]))
```

```
MSM total effect (always vs never): 1.652  (true = 1.5)
```

```
# For comparison: the untrimmed weights
df$sw_untrimmed <- sw
msm_untrimmed <- svyglm(Y ~ A0 + A1,
                        design = svydesign(ids = ~1, weights = ~sw_untrimmed, data = df))
cat(sprintf("Untrimmed MSM total effect: %.3f\n",
            coef(msm_untrimmed)["A0"] + coef(msm_untrimmed)["A1"]))
```

```
Untrimmed MSM total effect: 1.560
```

In the weighted population, A_0 gets 1.108 and A_1 gets 0.545, against true total effects of 1.0 and 0.5. Note what changed: with L_1 in the regression, A_0's coefficient was 0.545, its *direct* effect. Weighting instead of conditioning lets A_0 keep its effect through L_1, and the coefficient roughly doubles.

The weighted population is constructed so treatment is no longer associated with past covariates. That is why the MSM targets the total effect. Note the trimmed total of 1.652 sits above the truth of 1.5, while the untrimmed 1.560 is closer: the weight models here are correctly specified, so the untrimmed weights are consistent, and capping the top 1% of weights reintroduces a bit of confounding bias in exchange for lower variance. Trimming is a bias–variance trade, not a free lunch.

16.5. Longitudinal TMLE

The `ltmle` package implements longitudinal TMLE. It combines outcome models and treatment models; `summary()` reports the TMLE estimate by default (the IPTW estimate is available via `summary(fit, estimator = "iptw")`, and g-computation by calling `ltmle()` with `gcomp = TRUE`).

```
# ltmle expects a specific column order: covariates, treatments, outcome
ltmle_data <- df[, c("L0", "A0", "L1", "A1", "Y")]

ltmle_fit <- ltmle(
```

```
  data    = ltmle_data,
  Anodes  = c("A0", "A1"),
  Lnodes  = c("L0", "L1"),
  Ynodes  = "Y",
  abar    = c(1, 1),       # the regime: always treated
  estimate.time = FALSE
)

summary(ltmle_fit)
```

```
Estimator:  tmle
Call:
ltmle(data = ltmle_data, Anodes = c("A0", "A1"), Lnodes = c("L0",
    "L1"), Ynodes = "Y", abar = c(1, 1), estimate.time = FALSE)

   Parameter Estimate:  1.5106
    Estimated Std Err:  0.03981
              p-value:  <2e-16
    95% Conf Interval: (1.4325, 1.5886)
```

Read the printed 1.5106 carefully. With a single `abar`, `ltmle` returns the counterfactual **mean** $E[Y(1,1)]$, not a treatment effect. It sits next to a true total effect of 1.5 and looks like a correctly recovered effect, but that is a coincidence of this DGP: $E[Y(0,0)] = 0$ here, because with both treatments set to zero the outcome reduces to $0.5L_1 + 0.5L_0 + \varepsilon$ with $L_1 = 0.5L_0 + \nu$, and L_0 is mean zero. When the never-treated mean is zero, the mean under treatment and the effect are the same number. Change the intercept of the outcome model and they would not be.

Comparing the TMLE estimate with its IPTW and g-computation counterparts (via the options above) is a useful check on how much the answer depends on the modeling choices.

To estimate a *contrast* — the always-treated vs never-treated effect — specify both regimes:

```
ltmle_contrast <- ltmle(
  data    = ltmle_data,
  Anodes  = c("A0", "A1"),
  Lnodes  = c("L0", "L1"),
  Ynodes  = "Y",
  abar    = list(treatment = c(1, 1), control = c(0, 0)),
  estimate.time = FALSE
)

summary(ltmle_contrast)
```

```
Estimator:  tmle
Call:
ltmle(data = ltmle_data, Anodes = c("A0", "A1"), Lnodes = c("L0",
    "L1"), Ynodes = "Y", abar = list(treatment = c(1, 1), control = c(0,
```

```
    0)), estimate.time = FALSE)

Treatment Estimate:
   Parameter Estimate:  1.5106
    Estimated Std Err:  0.03981
              p-value:  <2e-16
    95% Conf Interval: (1.4325, 1.5886)

Control Estimate:
   Parameter Estimate:  -0.026342
    Estimated Std Err:  0.030169
              p-value:  <2e-16
    95% Conf Interval: (-0.085472, 0.032788)

Additive Treatment Effect:
   Parameter Estimate:  1.5369
    Estimated Std Err:  0.047618
              p-value:  <2e-16
    95% Conf Interval: (1.4436, 1.6302)
```

Now all three quantities are printed. The treated mean is 1.5106, the control mean is -0.0263, and the additive treatment effect is 1.5369 with a standard error of 0.0476 and a 95% interval of $[1.444, 1.630]$, which covers the true 1.5. The control mean confirms the point above: it is essentially zero, which is why the single-regime call looked like an effect estimate.

One oddity in that output: the control arm reports $p < 2 \times 10^{-16}$ beside a 95% interval of $[-0.085, 0.033]$ that contains zero. The two disagree because that p-value is not testing the counterfactual mean against zero. Read the interval.

Collecting the estimates of the always-versus-never effect: the g-formula gives 1.548, the MSM 1.652 trimmed and 1.560 untrimmed, and LTMLE 1.537, against a truth of 1.5. The two naive regressions gave 1.813 and 1.048. Every g-method lands within about 0.15 of the truth; neither regression comes close.

16.6. Choosing among the approaches

Estimator	Strengths	Weaknesses
G-formula	Handles arbitrary regimes; transparent	Sensitive to model misspecification on L_t and Y
IPTW + MSM	Simple, intuitive; valid with correct propensity model	Variance can be large with extreme weights
LTMLE	Doubly robust; efficient	More complex; needs care with super-learners

In practice:

- Use the g-formula when the covariate and outcome models are credible.
- Use IPTW + MSM when the treatment-history model is more credible.
- Use LTMLE when available and when the extra modeling work is justified.

16.7. When to reach for these methods

Use g-methods when:

1. There is a time-varying confounder L_t that is affected by past treatment A_{t-1} and itself affects subsequent treatment A_t.
2. The research question concerns a *sustained* or *dynamic* treatment regime, not a single decision at a single time.

This includes many clinical follow-up studies and longitudinal studies of education or labor-market policies.

For a one-shot treatment, the methods from the Estimation and Nonparametric Causal Methods chapters are sufficient. The added complexity of g-methods is only justified when the time-varying confounder problem is present.

16.8. Summary

- Standard regression fails when a time-varying confounder is itself affected by past treatment.
- G-formula simulates outcomes under treatment regimes.
- IPTW + MSM reweights the population and then fits a weighted treatment history model.
- LTMLE combines the outcome and treatment-model approaches.
- Disagreement across estimators is useful information, not just a nuisance.

Part V.

Survival & Time-to-Event

17. Survival Analysis with Causal Inference

```
library(tidyverse)
library(survival)
library(survminer)
library(riskRegression)
```

When the outcome is time-to-event, such as death, readmission, job exit, or machine failure, ordinary regression is not enough. Some observations are censored. We know the event had not happened by the end of follow-up, but we do not know the event time. Dropping those observations or treating the observed follow-up time as the event time creates bias.

I cover:

1. Cox proportional hazards models with covariate adjustment.
2. Inverse probability of censoring weighting (IPCW).
3. Restricted mean survival time (RMST) differences.
4. Doubly robust survival estimators in `riskRegression`.

17.1. The challenge of censored outcomes

Censoring means we observe $\min(T_i, C_i)$ and $\delta_i = \mathbb{1}\{T_i \leq C_i\}$, where T_i is the event time and C_i is the censoring time. The main quantity of interest is the survival function

$$S(t) = P(T > t) \tag{17.1}$$

the probability of surviving past time t. Treatment effects are usually reported as:

- **Hazard ratio (HR)**: $HR = h_1(t)/h_0(t)$ (the ratio of hazards under treatment vs control). Easy to estimate but hard to interpret causally (Hernán 2010).
- **Restricted mean survival difference (RMST)**: $\int_0^\tau [S_1(t) - S_0(t)]dt$ — the difference in mean survival up to time τ. Causally interpretable as a "years gained" quantity.
- **Survival probability difference**: $S_1(t) - S_0(t)$ at a specified time. Reports a direct probability difference.

17.2. Setup: simulated survival data

The data are simulated once and shared with the Julia companion, so both books report the same numbers. The generator is `data/gen_survival.R`.

We draw $n = 1500$ people with age $\sim N(60, 10^2)$ and sex $\sim$ Bernoulli(0.5). Treatment is trt $\sim$ Bernoulli($\Lambda(-1.0 + 0.03(\text{age} - 60) + 0.4\,\text{sex})$), so older people and one sex are more likely to be treated. Event times follow a Weibull proportional-hazards model with shape $k = 1.4$ and scale $\lambda_0 = 0.11$,

$$S(t \mid X) = \exp\!\left(-(\lambda_0 t)^k e^{\eta}\right), \qquad \eta = -0.4\,\text{trt} + 0.03(\text{age} - 60) + 0.3\,\text{sex}, \tag{17.2}$$

so the true log hazard ratio for treatment is -0.4 and the true hazard ratio is $e^{-0.4} = 0.670$. Censoring is exponential dropout with mean 40 years, capped by administrative end of follow-up at 12 years, and we observe $\min(T, C)$ with an event indicator.

Because the model is fully specified, the other estimands follow by integration. Over the population distribution of age and sex, the true five-year risk is 0.295 under treatment and 0.403 under control, a **risk difference of** -0.108, and the true restricted mean survival to five years is 4.341 years under treatment against 4.070 under control, an **RMST difference of 0.271 years**. Note that treatment is confounded in the direction that hides its benefit: age raises both the chance of treatment and the hazard, so the treated are sicker at baseline.

```
df <- read_csv("data/survival_sim.csv", show_col_types = FALSE)
n <- nrow(df)
cat(sprintf("n = %d, events = %d (%.1f%%), censored = %d\n",
            n, sum(df$event), 100 * mean(df$event), sum(1 - df$event)))
```

```
n = 1500, events = 1010 (67.3%), censored = 490
```

Of the 1,500 people, 1,010 have an observed event (67.3%) and 490 are censored.

17.3. Kaplan-Meier survival curves

The Kaplan-Meier estimator is the basic nonparametric estimate of $S(t)$:

```
km_fit <- survfit(Surv(time, event) ~ trt, data = df)
ggsurvplot(km_fit, data = df,
           pval = TRUE,
           risk.table = TRUE,
           xlab = "Time (years)",
           legend.labs = c("Control", "Treatment"))
```

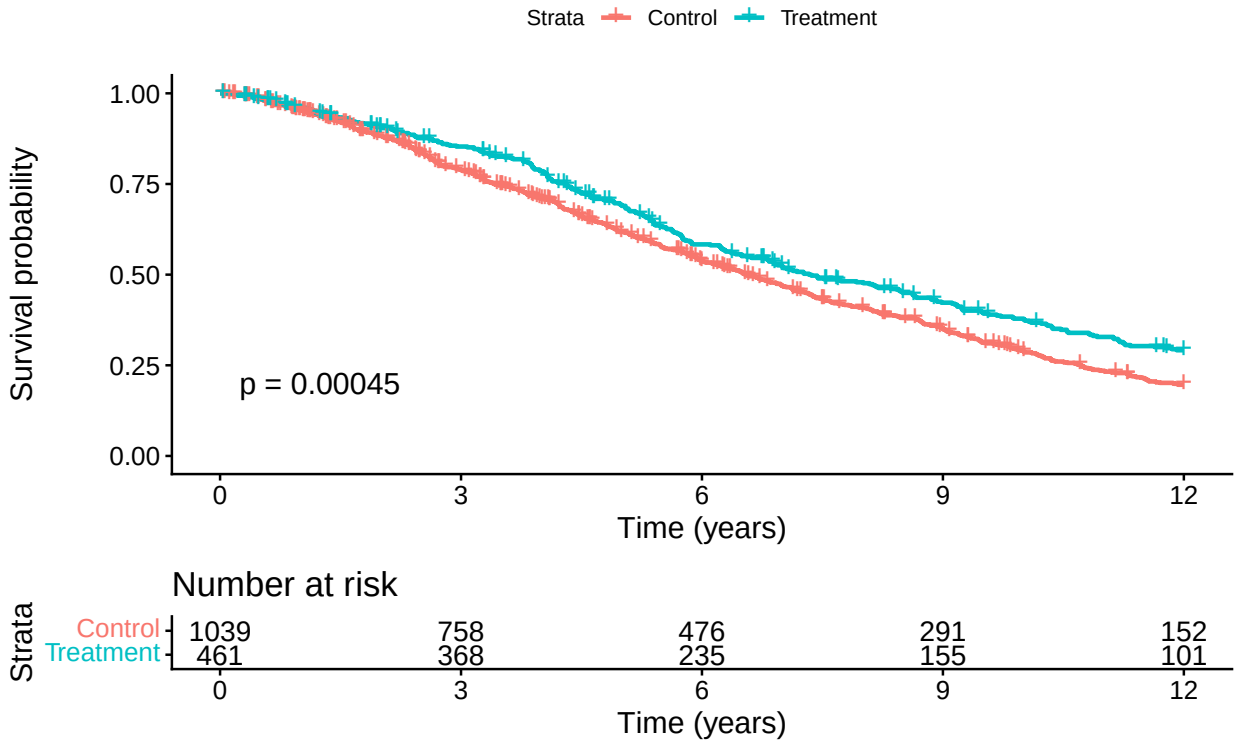


Figure 17.1.: Kaplan-Meier survival curves by treatment arm. Treatment shifts the survival curve upward, but the gap reflects both the causal effect AND any baseline imbalance in age/sex.

The KM curve is descriptive. It is not enough for causal inference here because treatment is related to baseline covariates.

17.4. Cox proportional hazards with adjustment

The familiar regression approach is a Cox proportional hazards model with covariate adjustment:

```
cox_fit <- coxph(Surv(time, event) ~ trt + age + sex, data = df)
tidy(cox_fit, exp = TRUE, conf.int = TRUE) |>
  knitr::kable(digits = 3,
               caption = "Cox PH regression coefficients (exponentiated = hazard ratios)")
```

Table 17.1.: Cox PH regression coefficients (exponentiated = hazard ratios)

term	estimate	std.error	statistic	p.value	conf.low	conf.high
trt	0.669	0.072	-5.608	0	0.582	0.770
age	1.035	0.003	10.754	0	1.029	1.042
sex	1.347	0.064	4.663	0	1.189	1.527

The estimated hazard ratio for treatment is 0.669 with a 95% interval of $[0.582, 0.770]$, against the true $e^{-0.4} = 0.670$. The Cox model is correctly specified here — the DGP is proportional hazards with linear covariate effects — so this is a check that the machinery works, not evidence about robustness. Age carries a hazard ratio of 1.035 per year and sex 1.347, against the true $e^{0.03} = 1.030$ and $e^{0.3} = 1.350$.

17.4.1. The interpretation problem with hazard ratios

The Cox hazard ratio is not a simple mean treatment effect. It compares hazards among those still at risk at time t, and that risk set is itself affected by treatment. I would report it, but not rely on it as the only causal summary.

17.5. Restricted Mean Survival Time (RMST)

The **restricted mean survival time** is the expected survival time up to a cutoff τ:

$$\text{RMST}(\tau) = E[\min(T, \tau)] = \int_0^\tau S(t)\, dt. \tag{17.3}$$

The RMST difference is easier to interpret:

$$\Delta_{\text{RMST}}(\tau) = \text{RMST}_1(\tau) - \text{RMST}_0(\tau) = \int_0^\tau [S_1(t) - S_0(t)]\, dt. \tag{17.4}$$

It also does not require proportional hazards.

```
# Use survRM2 via riskRegression / manually computing from KM
tau <- 5  # restrict to 5 years

# Manual RMST: integrate the KM curve up to tau.
# (For production use, prefer a tested implementation such as
#  survRM2::rmst2; the helper below is for transparency.)
# Read the step function directly from fit$time / fit$surv, anchor
# S(0) = 1 explicitly, and integrate as a left-endpoint rectangle sum
# over the grid (0, event times <= tau, tau). This avoids relying on
# how summary.survfit emits a row at time 0 and never duplicates tau in
# a way that drops the final [last event, tau] interval.
rmst_arm <- function(time, event, tau) {
  fit  <- survfit(Surv(time, event) ~ 1)
  keep <- fit$time <= tau
  t_grid <- c(0, fit$time[keep], tau)
  # Survival height on each interval [t_j, t_{j+1}): S(0) = 1, then the
  # KM value just after each event; the trailing value is a placeholder
  # dropped by head(., -1) below.
  last_s <- if (any(keep)) fit$surv[keep][sum(keep)] else 1
```

```
  s_grid <- c(1, fit$surv[keep], last_s)
  sum(diff(t_grid) * head(s_grid, -1))
}

rmst_trt  <- rmst_arm(df$time[df$trt == 1], df$event[df$trt == 1], tau)
rmst_ctrl <- rmst_arm(df$time[df$trt == 0], df$event[df$trt == 0], tau)

cat(sprintf("RMST( =5) under treatment: %.3f years\n", rmst_trt))
```

```
RMST( =5) under treatment: 4.351 years
```

```
cat(sprintf("RMST( =5) under control:   %.3f years\n", rmst_ctrl))
```

```
RMST( =5) under control:   4.147 years
```

```
cat(sprintf("Difference:                %.3f years\n", rmst_trt - rmst_ctrl))
```

```
Difference:                0.203 years
```

The unadjusted RMST is 4.351 years under treatment and 4.147 under control, a difference of 0.203 years, against a true 0.271. The RMST difference says how many survival years are gained up to the cutoff τ, and this one understates the gain by a quarter, because the treated are older and therefore at higher baseline hazard.

17.5.1. Adjusted RMST via IPW

To adjust for confounding, weight the KM curve by the inverse propensity score:

```
# Propensity score
ps_fit <- glm(trt ~ age + sex, data = df, family = binomial)
df$ps  <- predict(ps_fit, type = "response")
df$ipw <- ifelse(df$trt == 1, 1 / df$ps, 1 / (1 - df$ps))
df$ipw <- pmin(df$ipw, quantile(df$ipw, 0.99))  # trim

km_ipw <- survfit(Surv(time, event) ~ trt, data = df, weights = ipw)
print(km_ipw, rmean = tau)
```

```
Call: survfit(formula = Surv(time, event) ~ trt, data = df, weights = ipw)

      records    n events rmean* se(rmean) median 0.95LCL 0.95UCL
trt=0    1039 1499   1061   4.12    0.0365   6.46    5.95    6.91
trt=1     461 1494    897   4.40    0.0317   7.73    6.94    8.85
    * restricted mean with upper limit =  5
```

The weighted restricted means are 4.40 years under treatment and 4.12 under control, a difference of 0.28 against the true 0.271. Weighting moved the estimate from 0.203 to 0.28 and closed almost the whole gap. Note that the `n` column now reports the weighted totals, 1,494 and 1,499, rather than the 461 and 1,039 actual records — that is the pseudo-population the weights construct.

17.6. Inverse Probability of Censoring Weighting (IPCW)

If censoring is informative, for example if sicker patients drop out earlier, treating censoring as random is biased. IPCW weights observations by the inverse probability of remaining uncensored, conditional on covariates.

```
# Model the censoring hazard
# Use 1 - event as the "censoring" indicator
cens_fit <- coxph(Surv(time, 1 - event) ~ trt + age + sex, data = df)

# Predict survival of censoring at each event time for each unit
df_sub <- df |>
  arrange(time)
times_at_risk <- df_sub$time
# G(t | X): probability of being uncensored at time t given X
# Use survfit on cens_fit to get S_C(t | X)
sc_fit <- survfit(cens_fit, newdata = df_sub)
# We only need each subject's OWN survival at their OWN event time, not the
# full cross product of all N subjects' curves at all N event times.
# summary(sc_fit, times=df_sub$time, extend=TRUE)$surv followed by diag()
# builds that full N x N matrix just to throw away everything off the
# diagonal -- for N in the tens of thousands this is a multi-GB allocation
# and can OOM. Look each subject's own time up in sc_fit's existing
# per-subject step function instead (sc_fit$surv is already an
# n_times x N matrix that survfit(newdata=) computes regardless; this just
# avoids building a second, larger one on top of it). For very large N,
# packages like `riskRegression`/`pec` avoid materializing even that
# per-subject curve matrix and would scale further.
sc_idx  <- findInterval(df_sub$time, sc_fit$time)
sc_at_t <- sc_fit$surv[cbind(pmax(sc_idx, 1), seq_len(nrow(df_sub)))]
sc_at_t[sc_idx == 0] <- 1  # before the first censoring event, S_C = 1

# IPCW weight: 1 / P(C > T_i | X_i) for event times, 0 for censoring times
df_sub$ipcw <- ifelse(df_sub$event == 1, 1 / pmax(sc_at_t, 0.01), 0)
summary(df_sub$ipcw[df_sub$event == 1])
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  1.001   1.075   1.137   1.149   1.219   1.353
```

IPCW upweights observations that remain uncensored despite having a high predicted chance of censoring. The weights here run from 1.001 to 1.353 with a mean of 1.149, which is mild. That

is expected in this simulation: censoring is exponential dropout with a 40-year mean plus an administrative cutoff, and neither depends on the covariates in a way that would make censoring strongly informative. The correction is available and small; in a study where sicker patients drop out, it would not be.

17.7. Doubly-robust survival estimation

The `riskRegression` package combines the outcome model, propensity score, and censoring model:

```
# Average treatment effect on survival probability at t = 5
# ATE(t) = P(T(1) > t) - P(T(0) > t)
df$trt_f <- factor(df$trt)
cox_for_ate <- coxph(Surv(time, event) ~ trt_f + age + sex, data = df,
                     x = TRUE, y = TRUE)
ps_for_ate  <- glm(trt_f ~ age + sex, data = df, family = binomial, x = TRUE)
# With right-censoring, the IPTW/AIPTW survival estimator needs a model for
# the censoring mechanism (a Cox model for time-to-censoring).
cens_for_ate <- coxph(Surv(time, event == 0) ~ age + sex, data = df,
                      x = TRUE, y = TRUE)

ate_surv <- ate(
  event       = cox_for_ate,
  treatment   = ps_for_ate,
  censor      = cens_for_ate,
  data        = df,
  times       = 5,
  estimator   = c("Gformula", "IPTW", "AIPTW"),
  cause       = 1,
  se          = TRUE,
  B           = 0,
  verbose     = FALSE
)

summary(ate_surv)
```

```
     Average treatment effect

 - Treatment            : trt_f (2 levels: "0" "1")
 - Event                : event (cause: 1, censoring: 0)
 - Time  [min;max]      : time [0.0372;12]
 - at risk/time         :   5
   number in treatment  0 561
   number in treatment  1 283

 Estimation procedure
 - Estimators : G-formula, Inverse probability of treatment weighting, Augmented and double rob
```

```
 - Uncertainty: Gaussian approximation
                where the variance is estimated via the influence function

 Testing procedure
 - Null hypothesis     : given two treatments (A,B) and a specific timepoint, equal risks
 - Confidence level    : 0.95

 Results:
 - Difference in standardized risk (B-A) between time zero and 'time'
                reported on the scale [-1;1] (difference between two probabilities)
 (difference in average risks when treating all subjects with the experimental treatment (B),
                               vs. treating all subjects with the reference treatment (A))

 time trt_f=A risk(trt_f=A) trt_f=B risk(trt_f=B) difference            ci
    5       0         0.393       1         0.287     -0.106 [-0.14;-0.07]
  p.value
 5.36e-09

 difference      : estimated difference in standardized risks
 ci              : pointwise confidence intervals
 p.value         : (unadjusted) p-value
```

```
# The summary displays one estimator; put all three side by side
ate_tab <- as.data.frame(data.table::as.data.table(ate_surv, type = "diffRisk"))
knitr::kable(ate_tab[, c("estimator", "estimate", "se", "lower", "upper")],
             digits = 3,
             caption = "Risk difference at t = 5 under each estimator.")
```

Table 17.2.: Risk difference at t = 5 under each estimator.

estimator	estimate	se	lower	upper
GFORMULA	-0.106	0.018	-0.142	-0.070
IPTW	-0.095	0.027	-0.148	-0.041
AIPTW	-0.099	0.027	-0.152	-0.046

The object contains three estimators, shown side-by-side in the table:

- **Gformula** — parametric g-computation using the Cox model only.
- **IPTW** — inverse probability of treatment weighting.
- **AIPTW** — augmented IPTW (doubly robust); the recommended default.

All three agree, and all three are close to the truth: -0.106, -0.095 and -0.099 against a true risk difference of -0.108. Treatment cuts the five-year risk of the event by about ten percentage points. The g-formula has the smallest standard error, 0.018 against 0.027 for the other two, because it leans entirely on the Cox model, which is correctly specified here. That is an efficiency gain bought with an assumption, and AIPTW's wider interval is the price of not needing it.

Agreement across these estimates is reassuring. Disagreement is a warning about model specification.

17.8. Hazard ratio vs RMST: which to report?

For causal reporting, RMST is often easier to defend than the hazard ratio:

Quantity	Pro	Con
Hazard ratio	Compact summary; familiar	Requires PH; no causal interpretation; conditional on survivors
RMST difference	Directly interpretable as "years gained"; no PH assumption	Depends on cutoff τ; less familiar
Survival probability difference at t	Direct probability statement	Depends on choice of t

In practice, report at least:

1. The KM curves with risk tables (description).
2. The Cox HR with adjustment (familiar comparison to literature).
3. The RMST difference at a clinically meaningful τ (interpretable causal contrast).
4. The DR ATE on survival probability at one or two follow-up times.

17.9. Connections

- The G-Methods chapter covers IPTW with time-varying treatments — directly applicable when treatment evolves during follow-up. The IPCW machinery here is the censoring analogue.
- The Heterogeneous Effects chapter ML methods extend to survival via causal survival forests (`grf::causal_survival_forest`).
- The Bayesian Causal Inference chapter extends to survival via Bayesian additive regression trees for time-to-event outcomes (`BART::surv.bart` for discrete-time survival, or `BART::abart` for an accelerated-failure-time model with right-censoring).

17.10. Summary

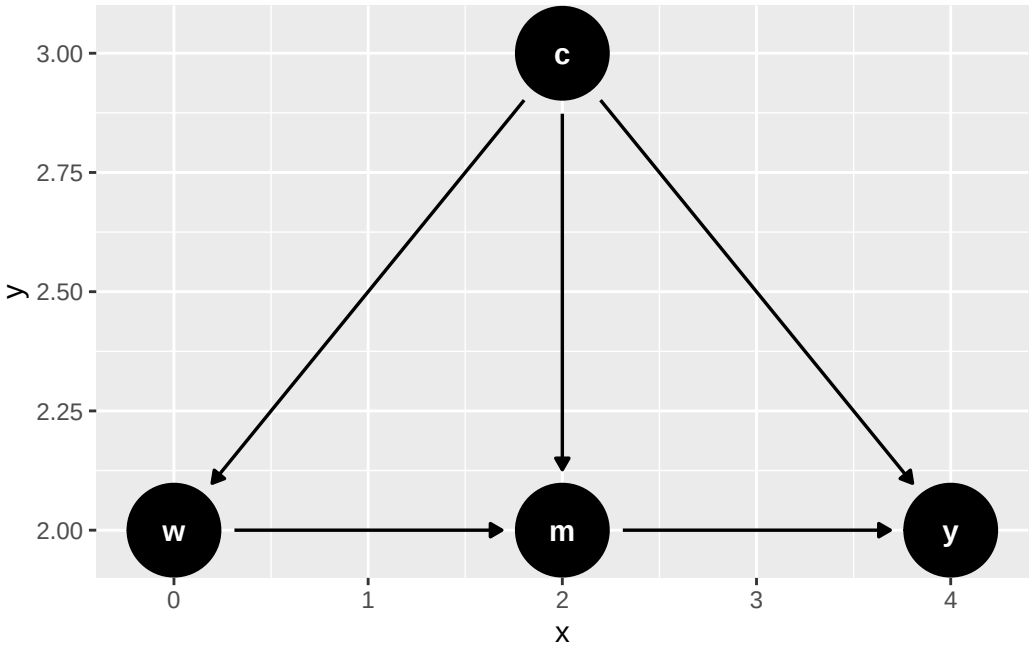
- Survival outcomes require methods that handle censoring.
- Cox models are familiar, but hazard ratios are hard to interpret causally.
- RMST differences are often the clearest causal summary.
- IPCW handles informative censoring.
- `riskRegression::ate` reports g-formula, IPTW, and AIPTW estimates for survival probabilities.

Part VI.

Mediation

18. Causal Mediation Analysis

Mediation asks how a treatment works, not only whether it works. The graph below is the setting: w is the treatment, m a mediator that w affects and that in turn affects the outcome y, and c a set of covariates confounding both the $w \to y$ and the $m \to y$ relationships. The effect of w on y splits into a path through m and a path around it.



18.1. Classical Mediation

Traditionally mediation model can be represented in the following equations:

$$Y = aW + bM + \epsilon_1 \tag{18.1}$$

$$M = cW + \epsilon_2 \tag{18.2}$$

That is, we'd like to study the effect of W on Y, and we see the effect can be a direct effect, and an indirect effect, through M.

Baron and Kenny's (http://davidakenny.net/cm/mediate.htm) method is done in four steps. Modern approach tends to use SEM (structural equation modeling) to model these two equations directly.

We simulate $n = 10{,}000$ observations from exactly this structure: $X \sim N(0,1)$, $M = 0.5X + \varepsilon_2$, and $Y = 0.7M + 0.3X + \varepsilon_1$, with both errors standard normal. So the true direct effect is $a = 0.3$, the mediator coefficient is $b = 0.7$, the treatment-to-mediator coefficient is $c = 0.5$, the indirect effect is $bc = 0.35$, and the total effect is $0.3 + 0.35 = 0.65$. There is no confounding of any kind, which is what makes this the easy case.

```
library(lavaan)
set.seed(1234)
n <- 10000
X <- rnorm(n)
M <- 0.5*X + rnorm(n)
Y <- 0.7*M + 0.3*X + rnorm(n)
Data <- data.frame(X = X, Y = Y, M = M)
model <- '   Y ~ a*X + b*M
             M ~ c*X
           # direct effect (a)
           # indirect effect (b*c)
             bc := b*c
           # total effect
             total := a + (b*c)
         '
```

```
fit <- sem(model, data = Data)
summary(fit)
```

```
lavaan 0.7-2 ended normally after 1 iteration

  Estimator                                         ML
  Optimization method                           NLMINB
  Number of model parameters                         5

  Number of observations                         10000

Model Test User Model:

  Test statistic                                 0.000
  Degrees of freedom                                 0

Parameter Estimates:

  Standard errors                             Standard
  Information                                 Expected
  Information saturated (h1) model          Structured

Regressions:
                   Estimate  Std.Err  z-value  P(>|z|)
  Y ~
```

```
    X           (a)    0.286    0.011   25.233    0.000
    M           (b)    0.699    0.010   70.384    0.000
  M ~
    X           (c)    0.511    0.010   50.161    0.000

Variances:
                   Estimate  Std.Err  z-value  P(>|z|)
   .Y                 0.998    0.014   70.711    0.000
   .M                 1.012    0.014   70.711    0.000

Defined Parameters:
                   Estimate  Std.Err  z-value  P(>|z|)
    bc                 0.357    0.009   40.849    0.000
    total              0.643    0.012   51.956    0.000
```

The estimates are $a = 0.286$, $b = 0.699$ and $c = 0.511$, against true values of 0.3, 0.7 and 0.5. The derived quantities are $bc = 0.357$ against a true 0.35 and a total of 0.643 against 0.65. Everything is recovered, and the model test statistic is 0 on 0 degrees of freedom because the model is saturated — it has as many parameters as moments, so it cannot fail to fit and the fit statistic carries no information.

18.1.1. Problems with Classical Mediation

- Lack of causal claim. We have to assume that there is no unmeasured confounder between M and Y. This is a strong assumption.
- Assumption of homogeneous effect.

18.2. Causal Mediation

18.2.1. CDE

Suppose we can set W and M at will to any (w, m), then we have the potential outcome $Y(w, m)$.

Controlled direct effect (CDE) is defined as

$$CDE(m) = E[Y(1, m) - Y(0, m)] \tag{18.3}$$

That is, setting M to m, what is the effect of W on Y?

18.2.2. Assumptions

- Conditional treatment randomization. Suppose we observe confounders X, which can be a joint set of confounders for the W to Y pathway, or the M to Y pathway.

$$Y(w,m) \perp W|X \tag{18.4}$$

This is the usual conditional independence (unconfoundedness, ignorability, etc.) assumption. This is saying the assignment of treatment, given covariates X, has nothing to do with potential outcomes.

- Conditional mediator randomization.

$$Y(w,m) \perp M|X, W = w \tag{18.5}$$

This is to say, within each strata of X, given treatment status, the assignment of mediator gives no information about potential outcome. This randomization is usually not implemented during many experiments (random trials). This is the assumption that makes a lot of mediation hard to make a causal claim.

- Positivity (overlap). There are two positivity assumptions:

$$P(W = w|X = x) > 0 \tag{18.6}$$

for all x. This is the usual positivity assumption.

$$P(M = m|X = x, W = w) > 0 \tag{18.7}$$

for all x, w, and m. This is the mediator positivity.

18.2.3. Estimation

CDE can be estimated using G-computation method, or IPW, or the doubly-robust methods, one of them is AIPW (augmented IPW).

18.2.4. G-computation

G-computation is to model the outcome equation:

$$\begin{aligned} CDE_G(m) = \sum_x [&E[Y \mid W = 1, M = m, X = x] \\ &- E[Y \mid W = 0, M = m, X = x]]\, P(X = x) \end{aligned} \tag{18.8}$$

18.2.5. IPW

IPW is to model the treatment assignment equation and the mediator assignment equation. The joint denominator factorizes *sequentially* – treatment given covariates, then mediator given treatment and covariates – which is the structure the estimator below relies on:

$$E[Y(w,m)] = E\left[\frac{I(W=w)\,I(M=m)}{g_W(w|X)\,g_M(m|w,X)}\,Y\right] \tag{18.9}$$

Therefore,

$$CDE_{ipw}(m) = E\left[\frac{I(W=1,M=m)}{g_M(m|1,X)\,g_W(1|X)}\,Y - \frac{I(W=0,M=m)}{g_M(m|0,X)\,g_W(0|X)}\,Y\right] \tag{18.10}$$

where g_M is the probability of the mediator and g_W the probability of treatment. **This indicator-based form is for a discrete (here binary) mediator.** For a continuous mediator the indicator $I(M=m)$ and the probability g_M must be replaced by a conditional density (or a kernel/stochastic-intervention formulation); the estimator above does not apply as written.

18.2.6. AIPW

$$CDE_{AIPW} = CDE_G(m) + B(\bar{Q}, g_M, g_W) \tag{18.11}$$

where $\bar{Q}$ is the mean outcome function.

$$\begin{aligned} B(\bar{Q}, g_M, g_W) = &\frac{1}{n}\sum_{i=1}^{n}\frac{I(M_i=m, W_i=1)}{g_M(m|1,X_i)g_W(1|X_i)}[Y_i - \bar{Q}(m,1,X_i)] \\ &- \frac{1}{n}\sum_{i=1}^{n}\frac{I(M_i=m, W_i=0)}{g_M(m|0,X_i)g_W(0|X_i)}[Y_i - \bar{Q}(m,0,X_i)] \end{aligned} \tag{18.12}$$

18.2.7. Example: G-computation

The three estimators below run on one simulated data set of $n = 5000$. There are two independent standard normal covariates, W_1 confounding the treatment-outcome relationship and W_2 confounding the mediator-outcome relationship. Treatment is $A \sim \text{Bernoulli}(\Lambda(-1+W_1/2))$, the mediator is $M \sim \text{Bernoulli}(\Lambda(-2+A/2+W_2/3))$, and the outcome is $Y \sim \text{Bernoulli}(\Lambda(-1+A-M/2+W_1/3+W_2/3))$. All three are binary.

The target is $CDE(0)$, the effect of treatment with the mediator held at 0 for everyone. Since the DGP is fully specified we can integrate it out: $CDE(0) = E_W[\Lambda(W_1/3+W_2/3)-\Lambda(-1+W_1/3+W_2/3)] = 0.222$. For comparison, $CDE(1) = 0.191$ — the effect is smaller when the mediator is held at 1, because the logistic link is flatter away from its centre.

```r
set.seed(1234)
n <- 5000
# confounder of A/Y
W1 <- rnorm(n)
# confounder of M/Y
W2 <- rnorm(n)
# treatment
A <- rbinom(n, 1, plogis(-1 + W1 / 2))
# binary mediator
M <- rbinom(n, 1, plogis(-2 + A / 2 + W2 / 3))
# binary outcome
Y <- rbinom(n, 1, plogis(-1 + A - M / 2 + W1 / 3 + W2 / 3))
full_data <- data.frame(W1 = W1, W2 = W2, A = A, M = M, Y = Y)

# fit outcome regression
or_fit <- glm(Y ~ A + M + W1 + W2, family = binomial(), data = full_data)
# new data setting A and M
data_A1_M0 <- data_A0_M0 <- full_data
data_A1_M0$A <- 1; data_A1_M0$M <- 0
data_A0_M0$A <- 0; data_A0_M0$M <- 0
# predict on new data
Qbar_A1_M0 <- predict(or_fit, newdata = data_A1_M0, type = "response")
Qbar_A0_M0 <- predict(or_fit, newdata = data_A0_M0, type = "response")
# gcomp estimate of CDE(0)
mean(Qbar_A1_M0 - Qbar_A0_M0)
```

```
[1] 0.2515527
```

G-computation gives 0.2516.

18.2.8. Example: IPW

The same target through the treatment and mediator models instead of the outcome model.

```r
# model for P(A = 1 | W)
ps_fit1 <- glm(A ~ W1 + W2, family = binomial(), data = full_data)
P_A1_W <- predict(ps_fit1, type = "response")
P_A0_W <- 1 - P_A1_W
# model for P(M = 0 | A, W)
ps_fit2 <- glm(M ~ A + W1 + W2, family = binomial(), data = full_data)
# P(M = 0 | A = 1, W)
data_A1 <- full_data; data_A1$A <- 1
P_M0_A1_W <- 1 - predict(ps_fit2, newdata = data_A1, type = "response")
# P(M = 0 | A = 0, W)
data_A0 <- full_data; data_A0$A <- 0
```

```
P_M0_A0_W <- 1 - predict(ps_fit2, newdata = data_A0, type = "response")
# ipw estimate of CDE(0)
mean( (A == 1) / P_A1_W * (M == 0) / P_M0_A1_W * Y ) -
mean( (A == 0) / P_A0_W * (M == 0) / P_M0_A0_W * Y )
```

```
[1] 0.2553091
```

IPW gives 0.2553.

18.2.9. Example: AIPW

Combining both, so that either the outcome model or the two treatment models may be wrong.

```
# aipw estimate of E[Y(1,0)]
aiptw_EY_A1_M0 <- mean(Qbar_A1_M0) +
mean( (A == 1) / P_A1_W * (M == 0) / P_M0_A1_W * (Y - Qbar_A1_M0) )
# aipw estimate of E[Y(0,0)]
aiptw_EY_A0_M0 <- mean(Qbar_A0_M0) +
mean( (A == 0) / P_A0_W * (M == 0) / P_M0_A0_W * (Y - Qbar_A0_M0) )
# aipw estimate of CDE(0)
aiptw_EY_A1_M0 - aiptw_EY_A0_M0
```

```
[1] 0.2554265
```

AIPW gives 0.2554.

The three estimates are 0.2516, 0.2553 and 0.2554 against a true $CDE(0)$ of 0.222. They agree closely with each other and all sit about 0.03 above the truth. That is one draw being unlucky, not bias: every model here is correctly specified, and over 400 replications of this design the g-computation estimator averages 0.2206 with a sampling standard deviation of 0.0159, so this seed's 0.2516 is 1.9 standard deviations high. The three estimators agree with each other much more closely than any of them agrees with the truth, because they share the same data and the same fitted nuisance models — agreement across estimators is a check on implementation, not on sampling error.

18.3. NIE and NDE: Natural Direct and Indirect Effects

CDE is to study the effect of treatment, given the level of mediator. Instead, Natural Effect is to set mediator to its natural value with the value of treatment, that is, $M = M(w)$.

$$\begin{aligned} ATE &= NIE + NDE \\ &= (E[Y(1, M(1))] - E[Y(1, M(0))]) \\ &\quad + (E[Y(1, M(0))] - E[Y(0, M(0))]) \end{aligned} \tag{18.13}$$

The advantage of NDE and NIE comparing to CDE is that it's more "natural"; that is, you don't set the level of mediator deterministically. And it can decompose the ATE into direct and indirect effects.

However, there is an additional assumption needed to identify NDE and NIE.

18.3.1. Additional Assumption

$$Y(w, m) \perp M(w^*)|X \tag{18.14}$$

This is the "cross-world" condition: the outcome under (w, m) is independent of M under w^*. These two situations cannot happen in the same world; you cannot set W to both w and w^*. No experiment can implement it.

This cross-world independence is *additional to*, not a replacement for, the usual identifying assumptions. The standard mediation formula for natural effects also requires:

- treatment ignorability (no unmeasured treatment-outcome confounding) given X;
- mediator ignorability (no unmeasured mediator-outcome confounding) given W and X;
- positivity for both the treatment and the mediator; and
- **no treatment-induced confounding** of the mediator-outcome relationship – there must be no variable affected by W that confounds M and Y. (When such a confounder exists, natural effects are not identified by this formula and interventional direct/indirect effects are used instead.)

The cross-world condition by itself is not sufficient.

18.3.2. Estimation

We use the same simulated data. The outcome model now adds $A \times M$ and $M \times W_1$ interactions and the mediator model adds $A \times W_1$ and $W_1 \times W_2$ — both are richer than the DGP requires, which costs a little precision and no consistency. We then average $E[Y \mid A = a, M, W]$ over the distribution of M under $A = a'$, for each of the four (a, a') combinations.

The truths again follow from the DGP by integration: $E[Y(1, M(1))] = 0.478$, $E[Y(1, M(0))] = 0.486$ and $E[Y(0, M(0))] = 0.267$, so the true $NDE = 0.218$, $NIE = -0.008$ and $ATE = 0.211$. The indirect effect is slightly negative because treatment raises the chance of the mediator (from 0.124 to 0.188) while the mediator lowers the outcome — the $-M/2$ term. So this treatment helps directly and hurts a little through its mediator.

```
# fit outcome regression (include interaction because we can)
or_fit <- glm(Y ~ A + M + W1 + W2 + A*M + M*W1,
family = binomial(), data = full_data)
# need E(Y | A = 0/1, M = 0/1, W1 = W1i, W2 = W2i)
get_EY_a_m_Wi <- function(full_data, or_fit, a, m){
data_Aa_Mm_Wi <- full_data
data_Aa_Mm_Wi$A <- a; data_Aa_Mm_Wi$M <- m
predict(or_fit, newdata = data_Aa_Mm_Wi, type = "response")
```

```r
}
EY_A0_M0_Wi <- get_EY_a_m_Wi(full_data, or_fit, a = 0, m = 0)
EY_A0_M1_Wi <- get_EY_a_m_Wi(full_data, or_fit, a = 0, m = 1)
EY_A1_M0_Wi <- get_EY_a_m_Wi(full_data, or_fit, a = 1, m = 0)
EY_A1_M1_Wi <- get_EY_a_m_Wi(full_data, or_fit, a = 1, m = 1)

# include interactions -- why not?
med_fit <- glm(M ~ A*W1 + W1*W2, family = binomial(), data = full_data)
# estimates of P(M = m | A = a, W = W_i)
get_Pm_a_Wi <- function(full_data, med_fit, a, m){
data_Aa_Wi <- full_data; data_Aa_Wi$A <- a
p <- predict(med_fit, newdata = data_Aa_Wi, type = "response")
if(m == 1){
p
}else{
1 - p
}
}
PM0_A0_Wi <- get_Pm_a_Wi(full_data, med_fit, a = 0, m = 0)
PM1_A0_Wi <- get_Pm_a_Wi(full_data, med_fit, a = 0, m = 1)
PM0_A1_Wi <- get_Pm_a_Wi(full_data, med_fit, a = 1, m = 0)
PM1_A1_Wi <- get_Pm_a_Wi(full_data, med_fit, a = 1, m = 1)

# E(E(Y | A = 1, M, W) | A = 1, W)
EY1M1_Wi <- EY_A1_M1_Wi * PM1_A1_Wi + EY_A1_M0_Wi * PM0_A1_Wi
# E(E(Y | A = 0, M, W) | A = 1, W)
EY0M1_Wi <- EY_A0_M1_Wi * PM1_A1_Wi + EY_A0_M0_Wi * PM0_A1_Wi
# E(E(Y | A = 1, M, W) | A = 0, W)
EY1M0_Wi <- EY_A1_M1_Wi * PM1_A0_Wi + EY_A1_M0_Wi * PM0_A0_Wi
# E(E(Y | A = 0, M, W) | A = 0, W)
EY0M0_Wi <- EY_A0_M1_Wi * PM1_A0_Wi + EY_A0_M0_Wi * PM0_A0_Wi

# estimate of E[Y(1, M(1))]
E_Y1M1 <- mean(EY1M1_Wi)
# estimate of E[Y(0, M(1))]
E_Y0M1 <- mean(EY0M1_Wi)
# estimate of E[Y(1, M(0))]
E_Y1M0 <- mean(EY1M0_Wi)
# estimate of E[Y(0, M(0))]
E_Y0M0 <- mean(EY0M0_Wi)

# Decomposition: ATE = NIE + NDE
NIE <- E_Y1M1 - E_Y1M0   # natural indirect effect
NDE <- E_Y1M0 - E_Y0M0   # natural direct effect
ATE <- E_Y1M1 - E_Y0M0   # total effect
cat(sprintf("NDE = %.3f\nNIE = %.3f\nATE = NDE + NIE = %.3f\n", NDE, NIE, ATE))
```

```
NDE = 0.247
NIE = -0.010
ATE = NDE + NIE = 0.238
```

The estimates are $NDE = 0.247$, $NIE = -0.010$ and $ATE = 0.238$, against truths of 0.218, -0.008 and 0.211. The decomposition holds exactly by construction, $0.247 + (-0.010) = 0.238$, and the sign of the indirect effect is recovered. The same upward shift as in the CDE estimates appears here, for the same reason: it is one draw of the data, and every quantity in this section is computed from it.

Note how much smaller the indirect effect is than the direct one, -0.010 against 0.247. Reporting only the total effect of 0.238 would hide that the mediator works against the treatment.

18.4. IIE and IDE: Interventional Direct and Indirect Effects

People are not happy with the cross-world assumption in general. Interventional direct and indirect effects avoid it. The device is a random draw: let M_w^* denote a draw from the distribution of $M(w)$ given $X = x$, rather than the unit's own $M(w)$. The standard definitions (Vansteelandt and Daniel 2017; Díaz et al. 2021) use such a draw in every term:

$$\begin{aligned} IDE &= E[Y(1, M_0^*)] - E[Y(0, M_0^*)] \\ IIE &= E[Y(1, M_1^*)] - E[Y(1, M_0^*)] \end{aligned} \tag{18.15}$$

The direct effect holds the mediator draw fixed at its untreated distribution; the indirect effect moves only that distribution. The two add up to

$$IDE + IIE = E[Y(1, M_1^*)] - E[Y(0, M_0^*)], \tag{18.16}$$

the *overall interventional effect*. It need not equal the ATE: replacing the unit's own $M(w)$ by a random draw breaks the within-unit dependence between $M(w)$ and $Y(w, \cdot)$, and that dependence matters exactly when a treatment-induced confounder Z sits between the treatment and the mediator — the situation in the example below. The payoff is identification without the cross-world assumption, even in the presence of such a Z. These interventional effects are the estimands the `medoutcon` package implements.

18.4.1. Example

This example uses the `medoutcon` vignette's own DGP, which is built to contain exactly the treatment-induced confounder that defeats natural effects. We draw $n = 1000$ observations. Three binary baseline covariates: $W_1 \sim \text{Bernoulli}(0.6)$, $W_2 \sim \text{Bernoulli}(0.3)$, and $W_3 \sim \text{Bernoulli}(\min(0.2 + (W_1 + W_2)/3, 1))$. Treatment is $A \sim \text{Bernoulli}(\Lambda(\sum_j W_j - 2))$.

Then the key variable: $Z \sim \text{Bernoulli}(\Lambda(\overline{-\log 2 + W - A} + 0.2))$ is a mediator-outcome confounder that depends on A. The mediator is $M \sim \text{Bernoulli}(\Lambda(\log 3(W_1 + W_2) + A - Z))$ and the outcome is $Y \sim \text{Bernoulli}(\Lambda(1/(\sum_j W_j - Z + A + M)))$.

Because Z is affected by A and confounds M and Y, natural direct and indirect effects are not identified here by the formula of the previous section. Interventional effects are, which is the reason for this example. We do not have a closed-form truth for this DGP — the reciprocal inside the outcome link makes it awkward — so read the estimates against each other and against zero rather than against a target.

```
library(data.table)
library(tidyverse)
library(medoutcon)
set.seed(1584)

# produces a simple data set based on ca causal model with mediation
make_example_data <- function(n_obs = 1000) {
  ## baseline covariates
  w_1 <- rbinom(n_obs, 1, prob = 0.6)
  w_2 <- rbinom(n_obs, 1, prob = 0.3)
  w_3 <- rbinom(n_obs, 1, prob = pmin(0.2 + (w_1 + w_2) / 3, 1))
  w <- cbind(w_1, w_2, w_3)
  w_names <- paste("W", seq_len(ncol(w)), sep = "_")

  ## exposure
  a <- as.numeric(rbinom(n_obs, 1, plogis(rowSums(w) - 2)))

  ## mediator-outcome confounder affected by treatment
  z <- rbinom(n_obs, 1, plogis(rowMeans(-log(2) + w - a) + 0.2))

  ## mediator -- could be multivariate
  m <- rbinom(n_obs, 1, plogis(rowSums(log(3) * w[, -3] + a - z)))
  m_names <- "M"

  ## outcome
  ## (the reciprocal inside plogis is an idiosyncrasy of the medoutcon
  ##  vignette DGP, not a standard logistic model; a zero denominator
  ##  gives plogis(Inf) = 1, which R handles without error)
  y <- rbinom(n_obs, 1, plogis(1 / (rowSums(w) - z + a + m)))

  ## construct output
  dat <- as.data.table(cbind(w = w, a = a, z = z, m = m, y = y))
  setnames(dat, c(w_names, "A", "Z", m_names, "Y"))
  return(dat)
}
```

```
# set seed and simulate example data
example_data <- make_example_data()
w_names <- str_subset(colnames(example_data), "W")
m_names <- str_subset(colnames(example_data), "M")

# quick look at the data
```

```
head(example_data)
```

```
     W_1   W_2   W_3     A     Z     M     Y
   <num> <num> <num> <num> <num> <num> <num>
1:     1     0     1     0     0     0     1
2:     0     1     0     0     0     1     0
3:     1     1     1     1     0     1     1
4:     0     1     1     0     0     1     0
5:     0     0     0     0     0     1     1
6:     1     0     1     1     0     1     0
```

```
# compute one-step estimate of the interventional direct effect
os_de <- medoutcon(W = example_data[, ..w_names],
                   A = example_data$A,
                   Z = example_data$Z,
                   M = example_data[, ..m_names],
                   Y = example_data$Y,
                   effect = "direct",
                   estimator = "onestep")
summary(os_de)
```

```
# A tibble: 1 x 7
  lwr_ci param_est upr_ci var_est eif_mean estimator param
   <dbl>     <dbl>  <dbl>   <dbl>    <dbl> <chr>     <chr>
1 -0.170   -0.0708 0.0281 0.00254 1.12e-16 onestep   direct_interventional
```

The one-step estimator gives an interventional direct effect of -0.071 with a 95% interval of $[-0.170, 0.028]$. The interval includes zero.

The same estimand can be targeted by TMLE instead:

```
# compute targeted minimum loss estimate of the interventional direct effect
tmle_de <- medoutcon(W = example_data[, ..w_names],
                     A = example_data$A,
                     Z = example_data$Z,
                     M = example_data[, ..m_names],
                     Y = example_data$Y,
                     effect = "direct",
                     estimator = "tmle")

summary(tmle_de)
```

```
# A tibble: 1 x 7
  lwr_ci param_est upr_ci var_est eif_mean estimator param
   <dbl>     <dbl>  <dbl>   <dbl>    <dbl> <chr>     <chr>
1 -0.169   -0.0679 0.0332 0.00266   0.0210 tmle      direct_interventional
```

TMLE gives -0.068 with a 95% interval of $[-0.169, 0.033]$, essentially the same answer as the one-step estimator. Both are indistinguishable from zero at $n = 1000$: the interval is about 0.2 wide for an effect of 0.07, so this sample cannot resolve it.

One diagnostic in the output is worth reading. The `eif_mean` column reports the mean of the estimated efficient influence function, which should be zero at the solution. The one-step estimator gives 1.1×10^{-16}, machine zero, as it must — solving that equation is what a one-step estimator does. TMLE gives 0.021, which is not zero. TMLE reaches its solution by tilting the initial outcome fit rather than by adding a correction term, and here the tilt has not driven the score all the way to zero. The two estimates agree anyway, but a large `eif_mean` is the signal that a TMLE fit has not converged.

Part VII.

Causal Discovery

19. Causal Discovery: Observed Variables

Earlier chapters assumed the graph was known, or at least that we could draw a defensible DAG from subject knowledge. Causal discovery asks what can be learned about the graph from observational data.

19.1. The Problem

Given n i.i.d. observations of p variables $(X_1, \dots, X_p)$, we want to recover the directed acyclic graph (DAG) G that generated the data.

There is an important limit: observational data alone cannot distinguish between DAGs with the same conditional independence structure. The set of observationally equivalent DAGs is a **Markov equivalence class**. The data can identify this class, represented by a CPDAG, not necessarily the one true DAG.

A CPDAG has the same skeleton (undirected edges) as all DAGs in its MEC. Some edges can be oriented (they have the same direction in every member of the MEC); others remain undirected because both directions are consistent with the CI structure.

19.1.1. Why some edges cannot be oriented

Three DAGs are observationally equivalent — they produce the same set of conditional independences — and therefore indistinguishable from data:

$$X \to Y \to Z \qquad X \leftarrow Y \leftarrow Z \qquad X \leftarrow Y \to Z \tag{19.1}$$

All three imply $X \perp\!\!\!\perp Z \mid Y$ and no other CI. Their CPDAG shows $X - Y - Z$ with no arrowheads.

The structure $X \to Y \leftarrow Z$ (a **collider** at Y) is *not* equivalent: here $X \perp\!\!\!\perp Z$ but $X \not\perp\!\!\!\perp Z \mid Y$. Because it has a unique CI signature, the $\to Y \leftarrow$ orientation is identifiable from data. Algorithms orient these colliders; non-collider edges often remain undirected.

19.1.2. What a CPDAG tells you

A directed edge $X \to Y$ in the CPDAG means: in *every* DAG consistent with the data, X causes Y. An undirected edge $X - Y$ means some consistent DAGs have $X \to Y$ and others have $X \leftarrow Y$ — data alone cannot decide.

For causal estimation: once you have a CPDAG, you can use background knowledge (temporal ordering, experimental context) to orient the remaining undirected edges, and then apply identification strategies from earlier chapters.

19.2. Two Discovery Paradigms in R

In R, `pcalg` implements two useful algorithm families:

Paradigm	Algorithm	Decision rule
Constraint-based	PC (Spirtes & Glymour, 1991)	Remove an edge when a CI test reports independence
Score-based	GES (Chickering, 2002)	Add/remove/reverse the edge that most improves a model score (BIC)

Both target the same CPDAG, but they make different mistakes. PC depends on finite-sample conditional independence tests. GES depends on the score. I would usually compare both.

19.3. Simulation

We generate a random Gaussian linear DAG using `pcalg::randomDAG` with 8 nodes and edge probability 0.35. Each edge $j \to i$ generates a structural equation

$$X_i = \sum_{j \in \mathrm{pa}(i)} \beta_{ji} X_j + \varepsilon_i, \quad \varepsilon_i \sim N(0,1), \tag{19.2}$$

with $|\beta_{ji}|$ drawn from $[0.25, 1]$.

```
set.seed(2025)

n_vars    <- 8
n_samples <- 2000
edge_prob <- 0.35
labels    <- paste0("X", 1:n_vars)

true_dag  <- randomDAG(n_vars, prob = edge_prob, lB = 0.25, uB = 1)
graph::nodes(true_dag) <- labels          # rename from "1".."8" to "X1".."X8"
data_mat  <- rmvDAG(n_samples, true_dag, errDist = "normal")
colnames(data_mat) <- labels
```

```
# Adjacency matrix from the true DAG, in the pcalg "ends-at-i" convention:
# entry [i,j] = 1 means an edge ending at i, i.e. j -> i.
adj_true  <- t(as(true_dag, "matrix") != 0) * 1

# igraph representation + shared layout for all comparison panels
ig_true <- graphnel_to_ig(true_dag)
dag_layout <- igraph::layout_with_sugiyama(ig_true)$layout
sprintf("Variables: %d   Samples: %d   True edges: %d",
        n_vars, n_samples, sum(adj_true))
```

```
[1] "Variables: 8   Samples: 2000   True edges: 11"
```

The draw gives 11 edges among the 28 possible pairs of 8 nodes, a density of 0.39, close to the 0.35 we asked for. Both algorithms below see only `data_mat`; `adj_true` is kept aside to score them.

```
plot_ig(ig_true, layout = dag_layout)
```

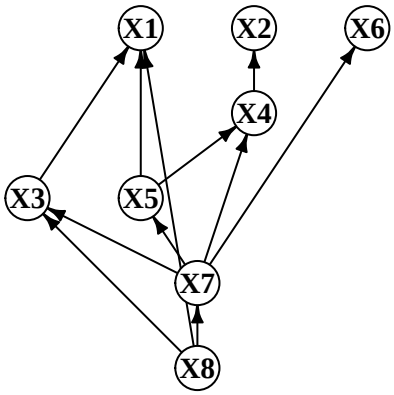


Figure 19.1.: True DAG used for simulation.

19.4. CI-Test and Score Infrastructure

PC uses a **Fisher-Z conditional independence test** appropriate for Gaussian data. Given the partial correlation $\hat{\rho}_{XY|Z}$, the test statistic is

$$T = \sqrt{n - |Z| - 3} \cdot \operatorname{atanh}(\hat{\rho}_{XY|Z}) \;\sim\; N(0,1) \quad \text{under } H_0 : X \perp\!\!\!\perp Y \mid Z. \tag{19.3}$$

Choosing the significance level α involves a bias–variance tradeoff on the skeleton:

An edge is *removed* whenever a CI test fails to reject independence ($p \geq \alpha$), so α controls how easily edges are deleted:

- **Low α (e.g., 0.001)**: independence is rarely rejected, so more tests "find" independence and more edges are removed. Sparser skeleton; risks discarding real but weak edges.
- **High α (e.g., 0.05)**: independence is rejected more often, so fewer edges are removed. Denser skeleton; risks retaining spurious edges.

With $n = 2000$ and $p = 8$ we use $\alpha = 0.01$, a common default.

GES does not use a significance level. It selects the graph that *minimizes* the **Gaussian BIC**

$$\mathrm{BIC}(G) = -2\,\ell(\hat{\theta}; G) + |G| \log n \tag{19.4}$$

(equivalently, maximizes the penalized log-likelihood $\ell - \frac{|G|}{2}\log n$), which penalizes graph complexity directly. Implemented in `pcalg` as `GaussL0penObsScore`.

We wrap `gaussCItest` in a counter to track exactly how many CI tests PC calls:

```
make_counted_ci <- function(suff_stat) {
  count <- 0L
  ci <- function(x, y, S, suffStat) {
    count <<- count + 1L
    gaussCItest(x, y, S, suffStat)
  }
  list(ci = ci, count = function() count)
}

suff_stat <- list(C = cor(data_mat), n = n_samples)
```

19.5. PC Algorithm

The **PC algorithm** (Spirtes & Glymour, 1991) starts from a complete undirected graph and removes edges whenever a conditional independence is found. It searches for separating sets Z of increasing size, conditioning on subsets of the current neighbors. After skeleton discovery, Meek's rules orient as many edges as possible.

How it works, step by step:

1. Start with a complete undirected graph (every pair connected).

2. For each pair (X, Y), test $X \perp\!\!\!\perp Y \mid Z$ for all $Z \subseteq \text{neighbors}(X) \setminus \{Y\}$, starting with $|Z| = 0$. Remove the edge if any such test passes.
3. Repeat with conditioning sets of increasing size until no new edges are removed.
4. Identify colliders: if $X - Z - Y$ and X, Y non-adjacent, orient as $X \to Z \leftarrow Y$ only if Z was not in the separating set of X and Y.
5. Apply Meek's orientation rules to propagate orientations without creating new colliders or cycles.

```
sig_level <- 0.01

ci_pc  <- make_counted_ci(suff_stat)
pc_fit <- pc(suffStat = suff_stat,
             indepTest = ci_pc$ci,
             labels = labels,
             alpha = sig_level,
             verbose = FALSE)

cpdag_pc <- pc_fit@graph
ci_tests_pc <- ci_pc$count()
```

> **Note**
>
> **Reading the output: F1 scores**
> The **F1 score** is the harmonic mean of precision and recall, ranging from 0 (worst) to 1 (perfect).
>
> - **Skeleton F1** measures whether the algorithm found the right edges regardless of direction. An edge is a true positive if it exists in both the estimated skeleton and the true one.
> - **CPDAG F1** is stricter: a directed edge $X \to Y$ is only a true positive if the true DAG also has $X \to Y$; an undirected edge $X - Y$ counts as correct if the true DAG has *either* direction.
>
> A high skeleton F1 with lower CPDAG F1 means the algorithm found the right adjacencies but oriented some of them incorrectly. **Caveat:** this is an *orientation-error-only* score – leaving a truly directed edge undirected is never penalized, only actively wrong directions are. It is therefore not a standard CPDAG accuracy metric (which would also penalize unresolved/uncompelled orientations), and it can flatter an algorithm that simply declines to orient edges. Read it as "of the orientations made, how many are right," not "how much of the true CPDAG was recovered."

```
# Helper: extract directed adjacency matrix from a graphNEL CPDAG.
# pcalg convention: amat[i,j] = 1 means an edge that ENDS at i,
# i.e. a directed edge j -> i (or one endpoint of an undirected edge).
amat_from_graph <- function(g) {
  # as(g, "matrix")[i,j] = 1 means i -> j (from->to). Transpose so that
  # [i,j] = 1 means an edge ending at i (j -> i), matching adj_true's
  # convention; otherwise CPDAG-F1 compares opposite orientations.
```

```
  m <- t(as(g, "matrix") != 0)
  storage.mode(m) <- "integer"
  m
}

# Skeleton: symmetrize (edge exists either way).
amat_to_skel <- function(amat) (amat | t(amat)) * 1L

# F1 scores: skeleton F1 (ignore direction) and CPDAG F1
# (directed edge in estimate must match a directed edge in the truth;
#  undirected in estimate matches either direction in truth).
# CAUTION if reusing this function elsewhere: the CPDAG-F1 component is
# orientation-error-only -- it never penalizes leaving a truly directed edge
# undirected, only actively wrong orientations. It is not a standard CPDAG
# accuracy metric and can flatter an algorithm that declines to orient
# edges. See the discussion above this function for what it does and does
# not measure.
f1_scores <- function(amat_true, amat_est) {
  skel_t <- amat_to_skel(amat_true)
  skel_e <- amat_to_skel(amat_est)
  diag(skel_t) <- diag(skel_e) <- 0
  tp <- sum(skel_t & skel_e) / 2
  fp <- sum(!skel_t & skel_e) / 2
  fn <- sum(skel_t & !skel_e) / 2
  f1_skel <- if (tp == 0) 0 else 2 * tp / (2 * tp + fp + fn)

  # CPDAG F1: walk ordered pairs (i,j) with i < j
  tp <- fp <- fn <- 0
  p <- nrow(amat_true)
  for (i in seq_len(p - 1)) for (j in seq.int(i + 1, p)) {
    t_ij <- amat_true[j, i] == 1   # truth has i -> j?
    t_ji <- amat_true[i, j] == 1   # truth has j -> i?
    e_ij <- amat_est[j, i] == 1
    e_ji <- amat_est[i, j] == 1
    if (!t_ij && !t_ji && !e_ij && !e_ji) next
    truth_undirected <- t_ij && t_ji
    est_undirected   <- e_ij && e_ji
    has_t <- t_ij || t_ji
    has_e <- e_ij || e_ji
    if (has_t && has_e) {
      if (est_undirected || truth_undirected ||
          (t_ij == e_ij && t_ji == e_ji)) {
        tp <- tp + 1
      } else {
        fp <- fp + 1; fn <- fn + 1
      }
    } else if (has_e) {
```

```
      fp <- fp + 1
    } else {
      fn <- fn + 1
    }
  }
  f1_cpdag <- if (tp == 0) 0 else 2 * tp / (2 * tp + fp + fn)

  c(skeleton = f1_skel, cpdag = f1_cpdag)
}

amat_pc  <- amat_from_graph(cpdag_pc)
f1_pc    <- f1_scores(adj_true, amat_pc)
ig_cpdag_pc <- graphnel_to_ig(cpdag_pc)

sprintf("PC:    skeleton F1 = %.3f   CPDAG F1 = %.3f   CI tests = %d",
        f1_pc["skeleton"], f1_pc["cpdag"], ci_tests_pc)
```

```
[1] "PC:    skeleton F1 = 0.952   CPDAG F1 = 0.857   CI tests = 293"
```

PC recovers the skeleton almost perfectly, with an F1 of 0.952, and orients most of it correctly, with a CPDAG F1 of 0.857. It needed 293 conditional independence tests to get there.

Computational complexity: In the worst case PC tests all subsets of neighbors, giving exponential growth with the maximum degree. In sparse graphs this is manageable; in dense graphs it becomes prohibitive.

19.6. GES Algorithm

GES (Greedy Equivalence Search; Chickering, 2002) uses a different strategy. Instead of testing independences and removing edges, it searches the space of CPDAGs by adding and then removing edges that improve a model score. Under faithfulness and a consistent score such as BIC, GES recovers the true CPDAG as $n \to \infty$.

How it works, step by step:

1. **Forward phase.** Start from the empty graph. At each step, add the single edge whose addition most increases the BIC score, restricted to additions that stay within a CPDAG. Stop when no addition improves the score.
2. **Backward phase.** From the forward-phase output, remove the single edge whose removal most increases the score. Stop when no removal improves the score.

The whole search operates on CPDAGs (equivalence classes), not individual DAGs, so it explores a much smaller space than naïve DAG search.

```
score_ges <- new("GaussL0penObsScore", data_mat)
ges_fit   <- ges(score_ges, verbose = FALSE)
cpdag_ges <- as(ges_fit$essgraph, "graphNEL")

amat_ges  <- amat_from_graph(cpdag_ges)
f1_ges    <- f1_scores(adj_true, amat_ges)
ig_cpdag_ges <- graphnel_to_ig(cpdag_ges)

sprintf("GES:   skeleton F1 = %.3f   CPDAG F1 = %.3f   (score-based, no CI tests)",
        f1_ges["skeleton"], f1_ges["cpdag"])
```

```
[1] "GES:   skeleton F1 = 0.800   CPDAG F1 = 0.720   (score-based, no CI tests)"
```

GES gets a skeleton F1 of 0.800 and a CPDAG F1 of 0.720 on this dataset, both below PC's 0.952 and 0.857. Do not read that as a general ranking; the Monte Carlo below shows what happens when we stop looking at one draw.

PC and GES fail differently. In PC, one wrong CI decision can affect later orientation decisions. In GES, a wrong score move is usually more local.

19.7. Comparison

> **Note**
>
> **Reading the CPDAG figure**
> In the side-by-side graph display:
>
> - A **directed edge** $X_i \to X_j$ means this orientation is invariant across all equivalent DAGs.
> - An **undirected edge** $X_i - X_j$ in the CPDAG display represents an *unoriented* edge: both $X_i \to X_j$ and $X_i \leftarrow X_j$ are statistically equivalent. This is **not** a hidden common cause — it means the data cannot determine direction.
> - A **missing edge** means the algorithm found a conditional independence separating those two variables.
>
> Compare the estimated CPDAGs to the true DAG: extra edges are false positives; missing edges are false negatives.

```
kable(data.frame(
  Algorithm   = c("PC", "GES"),
  Paradigm    = c("Constraint-based", "Score-based"),
  Skeleton_F1 = round(c(f1_pc["skeleton"], f1_ges["skeleton"]), 3),
  CPDAG_F1    = round(c(f1_pc["cpdag"],    f1_ges["cpdag"]),    3),
  CI_Tests    = c(ci_tests_pc, NA)
), row.names = FALSE)
```

Table 19.2.: Algorithm comparison on the simulated 8-node DAG (n = 2000, alpha = 0.01)

Algorithm	Paradigm	Skeleton_F1	CPDAG_F1	CI_Tests
PC	Constraint-based	0.952	0.857	293
GES	Score-based	0.800	0.720	NA

```
op <- par(mfrow = c(1, 3), mar = c(1, 1, 3, 1))
plot_ig(ig_true,     layout = dag_layout, main = "True DAG")
plot_ig(ig_cpdag_pc, layout = dag_layout, main = "PC - estimated CPDAG")
plot_ig(ig_cpdag_ges,layout = dag_layout, main = "GES - estimated CPDAG")
par(op)
```

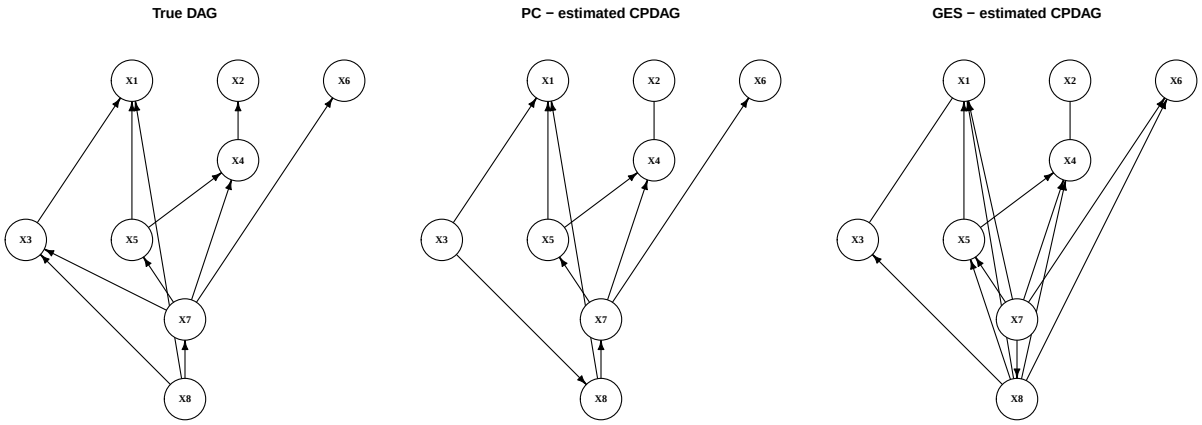


Figure 19.2.: True DAG vs. estimated CPDAGs. Arrows denote oriented edges; lines without arrowheads are unoriented (both directions are CI-equivalent).

> **Note**
>
> **Interpreting the comparison table**
>
> - **Skeleton F1** is the primary metric for comparing algorithms: it measures edge recovery independent of orientation, since orientation accuracy is bounded by the MEC.
> - **CPDAG F1** additionally rewards correctly oriented edges and penalizes wrongly oriented ones (leaving an edge undirected is not penalized under our scoring).
> - **CI tests** is reported only for PC — GES is score-based and does not perform CI tests.
>
> On this dataset PC comes out ahead on both metrics: GES's backward phase prunes or merges a few true adjacencies, which costs it skeleton F1 and, through the lost edges, CPDAG F1 as well. There is no general ranking — both algorithms target the *same* CPDAG, and the BIC score carries no orientation information beyond the conditional-independence structure (all DAGs in one Markov equivalence class receive the same score). Which one gets closer in finite samples depends on the DGP, the sample size, and the tuning (α vs. the BIC penalty).

19.8. Monte Carlo Evaluation

A single dataset can be lucky. We average over 100 replications to get stable estimates.

What to look for in the MC results:

- The **mean skeleton F1** is the headline: how often does each algorithm recover the true adjacencies?
- The distribution of **CI test counts** (shown in Figure 19.3) tracks PC's computational variability; GES has no analogous quantity, so we plot PC alone.
- High variance in F1 across replications suggests the algorithms are sensitive to the particular random graph.

```
evaluate_once <- function(seed, n_vars = 8, n_samples = 2000,
                          edge_prob = 0.35, sig = 0.01) {
  set.seed(seed)
  dag      <- randomDAG(n_vars, prob = edge_prob, lB = 0.25, uB = 1)
  labs     <- paste0("X", 1:n_vars)
  graph::nodes(dag) <- labs
  d        <- rmvDAG(n_samples, dag, errDist = "normal")
  colnames(d) <- labs
  adj      <- t(as(dag, "matrix") != 0) * 1L
  ss       <- list(C = cor(d), n = n_samples)

  ci       <- make_counted_ci(ss)
  pc_f     <- pc(ss, ci$ci, labels = labs, alpha = sig, verbose = FALSE)
  amat_p   <- amat_from_graph(pc_f@graph)

  ges_f    <- ges(new("GaussL0penObsScore", d), verbose = FALSE)
  amat_g   <- amat_from_graph(as(ges_f$essgraph, "graphNEL"))

  f_p <- f1_scores(adj, amat_p); f_g <- f1_scores(adj, amat_g)
  c(f1_skel_pc  = f_p["skeleton"], f1_skel_ges = f_g["skeleton"],
    f1_cpdag_pc = f_p["cpdag"],    f1_cpdag_ges = f_g["cpdag"],
    ci_pc       = ci$count())
}

mc <- as.data.frame(t(sapply(1:100, evaluate_once)))
names(mc) <- c("f1_skel_pc","f1_skel_ges","f1_cpdag_pc","f1_cpdag_ges","ci_pc")

mc_summary <- data.frame(
  Method      = c("PC", "GES"),
  Skeleton_F1 = round(c(mean(mc$f1_skel_pc),  mean(mc$f1_skel_ges)),  3),
  CPDAG_F1    = round(c(mean(mc$f1_cpdag_pc), mean(mc$f1_cpdag_ges)), 3),
  CI_Tests    = c(round(mean(mc$ci_pc), 1), NA)
)
kable(mc_summary,
```

```
      caption = "Monte Carlo averages (100 replications, n = 2000, p = 8)",
      row.names = FALSE)
```

Table 19.3.: Monte Carlo averages (100 replications, n = 2000, p = 8)

Method	Skeleton_F1	CPDAG_F1	CI_Tests
PC	0.947	0.903	230.6
GES	0.933	0.825	NA

The averages change the picture. On skeleton recovery PC gets 0.947 and GES 0.933 — a gap of 0.014, against the 0.152 gap on the single dataset shown above. That one draw was not representative, and it is the reason this section exists. The honest statement is that the two algorithms recover adjacencies about equally well on this class of graphs.

The CPDAG gap is larger and does not close: 0.903 for PC against 0.825 for GES. Both algorithms target the same CPDAG and the BIC score is score-equivalent, so this is not GES orienting *worse* in principle. It follows from the skeleton: edges GES fails to recover cannot be oriented at all, and our CPDAG score counts those as errors.

PC's average cost is 230.6 CI tests, below the 293 it used on the single dataset above — another sign that draw was on the dense side.

```
ggplot(mc, aes(x = ci_pc)) +
  geom_histogram(bins = 20, fill = "steelblue", alpha = 0.75, color = "white") +
  labs(x = "CI tests (PC)", y = "Frequency",
       title = "PC CI-test counts across 100 replications") +
  theme_minimal()
```

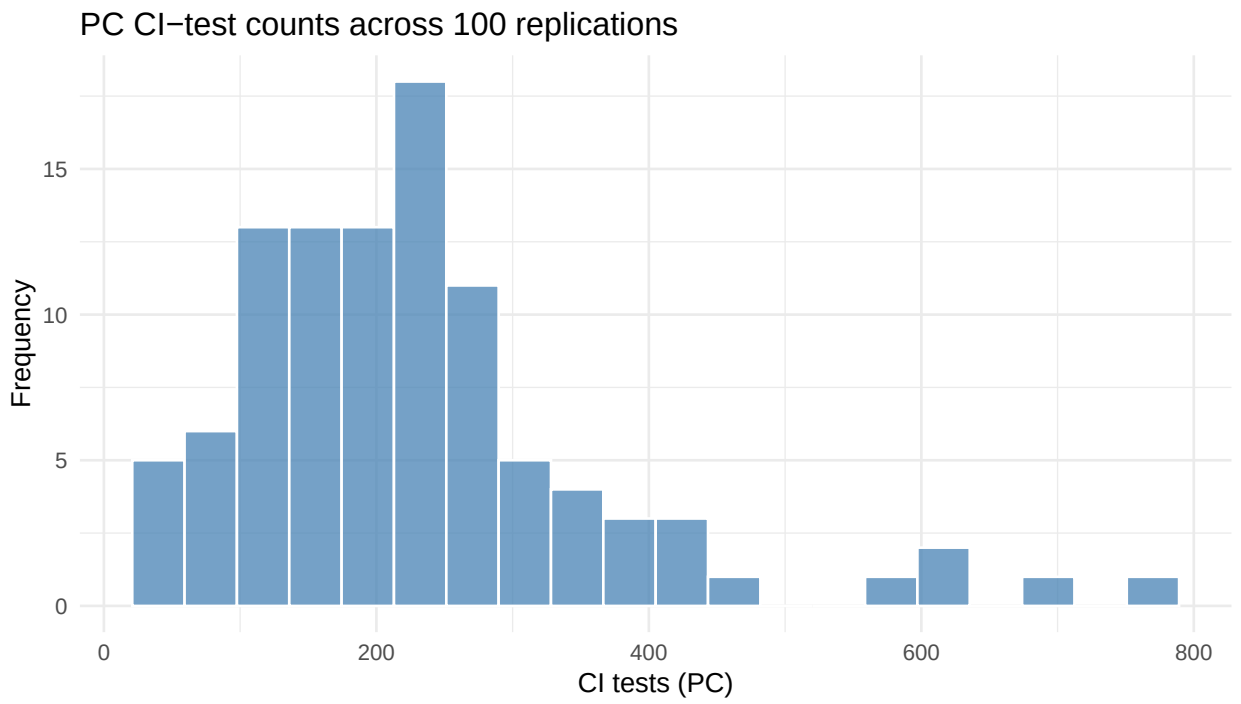


Figure 19.3.: Distribution of PC's CI-test counts across 100 replications. The spread reflects how PC's cost depends on the realized graph density and the order of CI decisions.

The distribution is right-skewed: the mean is 230.6 against a median of 212, and the counts run from 40 to 770 across the 100 replications. Nothing changes between replications except the random graph and the sample drawn from it, so a nineteen-fold spread in cost comes entirely from graph density — a denser draw means larger neighbour sets and therefore more conditioning subsets to test. This is the practical face of the exponential worst case noted earlier. At 8 nodes it is a nuisance; at 50 it is the binding constraint.

19.9. Real Data: Student Performance in PISA

The simulation had a luxury real research never does: a known true DAG, so we could score recovery with F1. On real data there is **no ground truth**, so the questions change. Instead of "how close is the estimate to the truth?" we ask: do PC and GES *agree*; what can *background knowledge* orient; and is the structure *stable* under resampling?

We illustrate on the OECD's **PISA 2022** assessment, restricted to **United States** students and six variables with a clear substantive ordering.

Node	Meaning	Role
`HISEI`	Highest parental occupational status (ISEI)	Family background
`HOMEPOS`	Home possessions index	Family background

Node	Meaning	Role
IMMIG	Immigration status (1 native … 3 first-gen)	Demographic
GRADE	Grade relative to modal grade for age	Schooling
GENDER	1 = female, 0 = male	Demographic
MATH	Mean of the 10 mathematics plausible values	Outcome

```
pisa <- read.csv("data/pisa_usa2022.csv")
Xp   <- as.matrix(pisa[, c("HISEI","HOMEPOS","IMMIG","GRADE","GENDER","MATH")])
wp   <- pisa$W
plabs <- colnames(Xp)
sprintf("Students (complete cases): %d", nrow(Xp))
```

```
[1] "Students (complete cases): 3890"
```

> **Note**
>
> **Three honesty caveats**, kept explicit:
>
> - **Plausible values.** Each student's ability is reported as 10 plausible values (posterior draws), not one score. We use their mean, understating the measurement variance; a full treatment runs discovery on each PV and pools.
> - **Categorical-as-Gaussian.** IMMIG, GRADE, GENDER are discrete; Fisher-Z assumes joint Gaussianity — a standard but imperfect approximation.
> - **One country, one wave.** Pooling would inject country/time structure that appears as spurious edges unless modelled as extra nodes.

19.9.1. Respecting the survey design

PISA is not a simple random sample — students carry final weights W_FSTUWT. The CI tests and the score are functions of the **covariance matrix** and a **sample size**, so we supply design-consistent inputs: a **weighted covariance/correlation** matrix, and **Kish's effective sample size** $n_{\text{eff}} = (\sum_i w_i)^2 / \sum_i w_i^2$.

```
cwp   <- cov.wt(Xp, wt = wp, cor = TRUE)   # design-weighted moments
Cwp   <- cwp$cor
Sigp  <- cwp$cov
neff  <- sum(wp)^2 / sum(wp^2)
sprintf("Raw n = %d   Kish effective n = %.0f", nrow(Xp), neff)
```

```
[1] "Raw n = 3890   Kish effective n = 3481"
```

```
kable(round(Cwp, 2))
```

Table 19.5.: Design-weighted correlation matrix (PISA 2022, USA).

	HISEI	HOMEPOS	IMMIG	GRADE	GENDER	MATH
HISEI	1.00	0.42	-0.17	0.05	-0.02	0.33
HOMEPOS	0.42	1.00	-0.18	0.05	0.03	0.40
IMMIG	-0.17	-0.18	1.00	0.07	0.01	-0.05
GRADE	0.05	0.05	0.07	1.00	0.08	0.17
GENDER	-0.02	0.03	0.01	0.08	1.00	-0.09
MATH	0.33	0.40	-0.05	0.17	-0.09	1.00

The weights cost us little information here: the raw sample of 3,890 students carries an effective sample size of 3,481, about 89% of it.

The correlation matrix already shows what the graph will have to explain. `MATH` correlates 0.40 with `HOMEPOS` and 0.33 with `HISEI`, the two family background measures, which themselves correlate 0.42 with each other. `GRADE` correlates 0.17 with `MATH` and `GENDER` −0.09. `IMMIG` correlates −0.17 and −0.18 with the two background measures but only −0.05 with `MATH` — its association with achievement runs largely through family background, which is the kind of statement a discovery algorithm is meant to formalise.

19.9.2. Two algorithms on the weighted data

PC consumes the weighted correlation and effective n directly. GES is score-based and needs a data matrix, so we build a synthetic sample of n_{eff} rows whose empirical covariance equals the design-weighted Σ_w exactly (a Cholesky recolour) — a device to feed the *same* second moments into a score that expects raw data, adding no information beyond Σ_w.

This is a **moment-based approximation to the PISA design, not design-exact causal discovery.** The weighted covariance plus Kish's n_{eff} captures the final weights but not the full stratification and clustering of the design, and it ignores plausible-value (imputation) uncertainty in the achievement scores. Treat the recovered graph as a weighted-moment exploration, and do not read the effective-n CI tests as exact design-based inference.

```
# Synthetic matrix with covariance == Sigp exactly, neff rows
m_eff <- round(neff)
set.seed(1)
Zp <- scale(matrix(rnorm(m_eff * ncol(Xp)), m_eff), center = TRUE, scale = FALSE)
Zp <- Zp %*% solve(chol(cov(Zp)))     # whiten
Xw <- Zp %*% chol(Sigp)               # recolour to weighted covariance
Xw <- sweep(Xw, 2, cwp$center, "+")
colnames(Xw) <- plabs

pc_pisa   <- pc(list(C = Cwp, n = neff), indepTest = gaussCItest,
                labels = plabs, alpha = 0.01)
```

```
amat_pcp  <- amat_from_graph(pc_pisa@graph)

ges_pisa  <- ges(new("GaussL0penObsScore", Xw))
cpdag_gesp<- as(ges_pisa$essgraph, "graphNEL")
amat_gesp <- amat_from_graph(cpdag_gesp)

skp <- amat_to_skel(amat_pcp); skg <- amat_to_skel(amat_gesp)
diag(skp) <- diag(skg) <- 0
sprintf("Skeleton edges - PC: %d   GES: %d   in both: %d",
        sum(skp)/2, sum(skg)/2, sum(skp & skg)/2)
```

```
[1] "Skeleton edges - PC: 9   GES: 9   in both: 9"
```

The two algorithms find 9 skeleton edges each, and all 9 are the same edges. That is the strongest agreement we can ask for without a ground truth: two methods with different failure modes, one testing conditional independences and one maximising a penalised likelihood, select an identical adjacency structure. Nine edges out of 15 possible pairs is also a genuinely sparse answer, not a near-complete graph.

Agreement on the skeleton does not extend to orientation, as the next figure shows.

```
ig_pcp  <- graphnel_to_ig(pc_pisa@graph)
ig_gesp <- graphnel_to_ig(cpdag_gesp)
lay_p   <- igraph::layout_with_sugiyama(ig_pcp)$layout
op <- par(mfrow = c(1, 2), mar = c(1, 1, 3, 1))
plot_ig(ig_pcp,  layout = lay_p, main = "PC - estimated CPDAG")
plot_ig(ig_gesp, layout = lay_p, main = "GES - estimated CPDAG")
par(op)
```

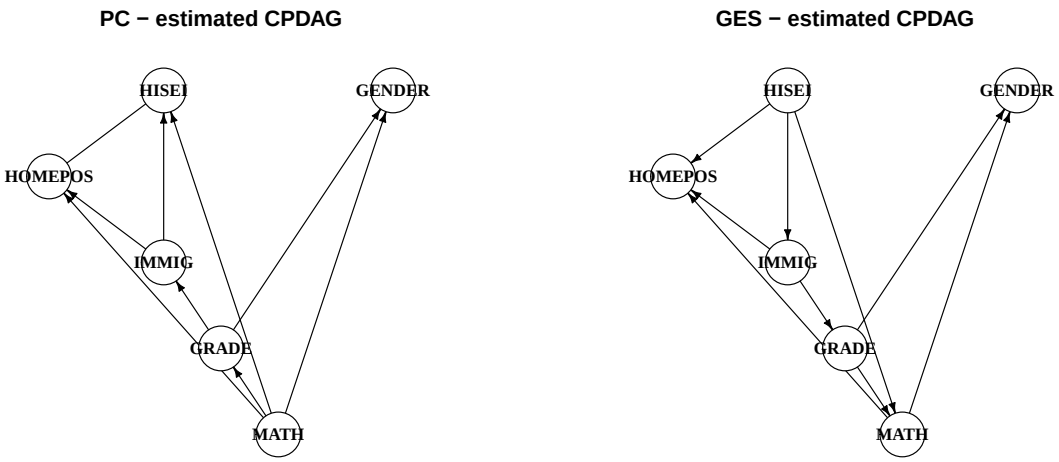


Figure 19.4.: Estimated CPDAGs from PC and GES on PISA 2022 (USA). Arrows are oriented edges; plain lines are unoriented. There is no true-DAG panel — on real data we have no ground truth.

> **Important**
>
> **Read the disagreement, not just the agreement.** A purely data-driven score may orient an edge as `MATH → HISEI` — a child's test score causing their parent's occupational status (GES does exactly this in the right panel). That is causally backwards, and the data cannot protect us: which direction this edge takes is not identifiable from observational data alone, so the orientation the algorithm prints reflects its (possibly mistaken) v-structure and score decisions under misspecification, not evidence about the true direction. This is exactly why background knowledge is not optional.

19.9.3. Orienting with background knowledge

Sex and immigration background are fixed at birth; family socioeconomic status is established long before the test; grade precedes the assessment; the score is the outcome. That gives a **tier ordering**

$$\{\text{GENDER}, \text{IMMIG}\} \prec \{\text{HISEI}, \text{HOMEPOS}\} \prec \text{GRADE} \prec \text{MATH}. \tag{19.5}$$

A cross-tier edge must point earlier→later; a within-tier edge the ordering cannot decide. We impose this on the discovered **skeleton**, because the ordering is knowledge we hold independently of how the data oriented things.

```
tier_p <- c(HISEI = 1, HOMEPOS = 1, IMMIG = 0, GRADE = 2, GENDER = 0, MATH = 3)[plabs]
skel_p <- amat_to_skel(amat_pcp)

edges_df <- data.frame(); undir_flag <- logical(0)
pp <- length(plabs)
for (i in seq_len(pp-1)) for (j in seq.int(i+1, pp)) {
  if (skel_p[i, j] == 0) next
  a <- plabs[i]; b <- plabs[j]
  if (tier_p[a] == tier_p[b]) {
    edges_df <- rbind(edges_df, data.frame(from = a, to = b)); undir_flag <- c(undir_flag, TRU
  } else {
    lo <- if (tier_p[a] < tier_p[b]) a else b
    hi <- if (tier_p[a] < tier_p[b]) b else a
    edges_df <- rbind(edges_df, data.frame(from = lo, to = hi)); undir_flag <- c(undir_flag, F
  }
}
ig_orient <- igraph::graph_from_data_frame(edges_df, vertices = plabs)
igraph::E(ig_orient)$undirected <- undir_flag
```

> **Warning**
>
> **Where data-driven orientation disagreed with us.** Left to itself, PC oriented `HISEI → IMMIG` and `HOMEPOS → IMMIG` — parental status and home possessions causing immigration

background, which our tier ordering puts the other way round. GES did something worse with a different edge: it oriented `GRADE → IMMIG`, schooling causing immigration status (PC got that one right). These mistakes come from v-structure and score decisions made under misspecification; the data is not informative about these directions. Imposing the tier order overrides them. The lesson: **never ship a discovered orientation you have prior reason to disbelieve**.

```
lay_or <- igraph::layout_with_sugiyama(ig_orient)$layout
plot_ig(ig_orient, layout = lay_or, main = "PC skeleton + tier ordering")
```

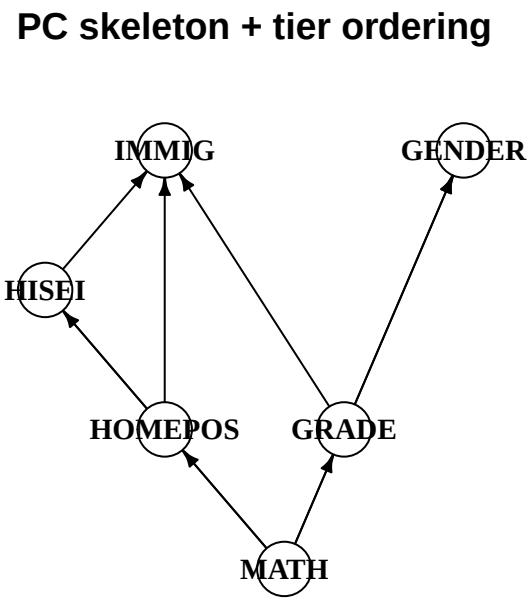


Figure 19.5.: PC skeleton oriented by the temporal/logical tiers. Cross-tier edges point earlier-to-later; HISEI–HOMEPOS stays undirected (same tier).

Every edge into `MATH` is now a candidate causal parent of achievement — the bridge from discovery to estimation (back-door adjustment, e.g. on the parents of the chosen treatment variable). The only edge the ordering leaves undirected, `HISEI - HOMEPOS`, is two same-tier measures of family background that temporal logic cannot separate.

19.9.4. Stability: does the skeleton survive resampling?

A single fit can be lucky. We resample students with replacement, re-estimate the **weighted** PC skeleton, and record how often each edge appears.

```
B_pisa <- 200
boot_once <- function(b) {
  set.seed(1000 + b)
  idx <- sample.int(nrow(Xp), replace = TRUE)
  Xb  <- Xp[idx, ]; wb <- wp[idx]
  cwb <- cov.wt(Xb, wt = wb, cor = TRUE)
  neff_b <- sum(wb)^2 / sum(wb^2)
  fit <- tryCatch(pc(list(C = cwb$cor, n = neff_b), indepTest = gaussCItest,
                     labels = plabs, alpha = 0.01), error = function(e) NULL)
  if (is.null(fit)) return(NULL)
  amat_to_skel(amat_from_graph(fit@graph))
}
ncores <- max(1, parallel::detectCores() - 1)
skels  <- parallel::mclapply(seq_len(B_pisa), boot_once, mc.cores = ncores)
skels  <- skels[!vapply(skels, is.null, logical(1))]
edge_freq <- Reduce(`+`, skels) / length(skels)
```

```
pairs_df <- do.call(rbind, lapply(seq_len(pp-1), function(i)
  do.call(rbind, lapply(seq.int(i+1, pp), function(j)
    data.frame(Edge = paste(plabs[i], "-", plabs[j]),
               Frequency = round(edge_freq[i, j], 2))))))
pairs_df <- pairs_df[pairs_df$Frequency > 0, ]
ggplot(pairs_df, aes(x = reorder(Edge, Frequency), y = Frequency)) +
  geom_col(fill = "steelblue", alpha = 0.8) +
  geom_hline(yintercept = 0.5, linetype = "dashed", colour = "grey50") +
  coord_flip() + ylim(0, 1) +
  labs(x = NULL, y = "Selection frequency across bootstrap resamples") +
  theme_minimal()
```

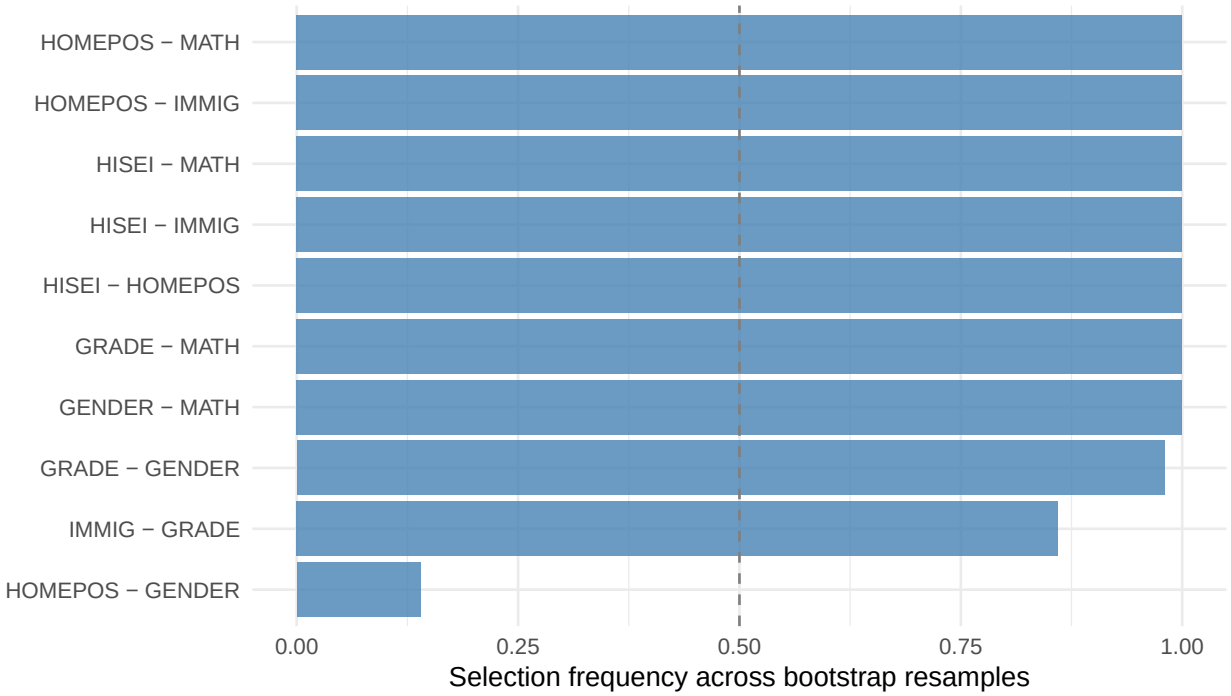


Figure 19.6.: Bootstrap edge-selection frequency (weighted PC, 200 resamples). Edges near 1.0 are robust; near 0.5 are sample-dependent.

Six of the nine edges appear in every one of the 200 resamples: `HISEI-HOMEPOS`, `HISEI-IMMIG`, `HISEI-MATH`, `HOMEPOS-IMMIG`, `HOMEPOS-MATH` and `GRADE-MATH`. `GENDER-MATH` appears in 99.5% and `GRADE-GENDER` in 98.5%. The weakest of the nine is `IMMIG-GRADE` at 86.5% — still stable, but the one to hold most loosely. Two edges that the full sample rejected do appear occasionally, `HOMEPOS-GENDER` in 14% of resamples and `IMMIG-GENDER` in 0.5%; neither is close to the 50% line.

So the family-background-to-math edges are perfectly stable: the SES gradient in achievement is not an artefact of this sample. On real data, causal discovery is best read as **hypothesis generation** — it proposes a skeleton and the orientations the data support, which you then reconcile with subject-matter knowledge before estimating effects.

19.10. Summary

PC and GES target the same Markov equivalence class. PC uses conditional independence tests; GES uses model fit. If they agree on the skeleton, that is reassuring. If they disagree, check sample size, model assumptions, and possible faithfulness problems.

From CPDAG to causal estimation: Once a CPDAG is in hand, the workflow continues as follows:

1. **Use background knowledge to orient undirected edges.** Temporal ordering is the most common source: if X is measured before Y, the edge must be $X \to Y$. Domain expertise, experimental context, or exclusion restrictions can orient others.

2. **Check identifiability.** With a fully oriented DAG, apply the backdoor, front-door, or ID algorithm to determine whether the effect of interest is identifiable.
3. **Estimate.** Use the identified functional with the estimators from earlier chapters (RA, IPW, AIPW, TMLE).

If some edges remain undirected after exhausting background knowledge, you can either report the CPDAG and bound the estimand over the equivalence class, or collect additional interventional data to break the remaining equivalences.

> **Note**
>
> **What cannot be recovered from observational data alone**
> Both algorithms identify the CPDAG, not necessarily the true DAG. Some edge directions require extra assumptions, temporal ordering, or interventional data.
> The next chapter addresses a harder problem: recovering the causal skeleton when some variables are *unobserved*.

20. Causal Discovery: Latent Variables

The previous chapter assumed all causally relevant variables were observed. That is a strong assumption. In economics, ability, demand shocks, and peer effects are often unobserved.

When latent confounders exist, PC and GES can give the wrong graph. Here I use `pcalg` algorithms designed for latent variables.

20.0.1. Why PC fails with latent variables

Suppose the true graph is $X \leftarrow U \rightarrow Y$ where U is unobserved. In the observed data, X and Y are marginally correlated (through U) and no observed conditioning set separates them. PC will incorrectly draw an edge $X - Y$ and may orient it as $X \rightarrow Y$ or $X \leftarrow Y$, neither of which exists in the truth.

With even one hidden common cause, the conditional independences among observed variables may not correspond to any DAG on the observed variables alone. We need a MAG/PAG representation.

20.1. Maximal Ancestral Graphs and PAGs

With latent variables, the appropriate representation is a **Maximal Ancestral Graph (MAG)**. Over a set of *observed* variables $\mathbf{O}$, a MAG encodes:

- $X \rightarrow Y$: X is an ancestor of Y in the full DAG
- $X \leftrightarrow Y$: X and Y have a common hidden ancestor (hidden confounder)
- No edge: $X \perp\!\!\!\perp Y \mid Z$ for some $Z \subseteq \mathbf{O} \setminus \{X, Y\}$

The MAG over observed variables summarizes the causal and confounding relationships visible in the observed data without naming the latent variables.

Just as DAGs have Markov equivalence classes represented by CPDAGs, MAGs have equivalence classes represented by **Partial Ancestral Graphs (PAGs)**. In a PAG, the mark $\circ$ on an edge endpoint means "could be arrowhead or tail in some member of the equivalence class."

PAG mark	Meaning	Economic interpretation
$X \rightarrow Y$	X causes Y in every equivalent MAG	Robust causal direction
$X \circ\!\!\rightarrow Y$	Some MAGs have $X \rightarrow Y$, others $X \leftrightarrow Y$	Direction uncertain
$X \leftrightarrow Y$	Hidden common cause in every equivalent MAG	Definite unmeasured confounder

PAG mark	Meaning	Economic interpretation
$X \circ\!\!-\!\!\circ Y$	Could be $\rightarrow$, $\leftarrow$, or $\leftrightarrow$	Maximally uncertain

Reading a PAG for policy purposes:

- A definite $X \rightarrow Y$ edge is the most useful finding: any intervention on X propagates to Y regardless of which specific MAG the data came from.
- A definite $X \leftrightarrow Y$ edge is a warning: before using regression of Y on X to estimate a causal effect, you must address the hidden confounder — through an instrument, a proxy, or a natural experiment.
- $\circ$ marks indicate what you *don't* know. Additional data, temporal ordering, or experimental variation can resolve them.

20.2. FCI and RFCI: The R Toolkit

R's `pcalg` package provides two algorithms for the latent-variable case:

Algorithm	Output	Cost
FCI (Spirtes, Meek & Richardson, 1995)	Full PAG (skeleton + all orientation marks)	Expensive: the Possible-D-SEP stage performs many additional CI tests
RFCI (Colombo, Maathuis, Kalisch & Richardson, 2012)	PAG with possibly fewer/weaker orientations	Cheaper: skips the Possible-D-SEP tests

RFCI can give fewer orientations than FCI, and its skeleton converges to a (uniquely defined) *supergraph* of the true PAG skeleton — in special configurations it retains an edge FCI would remove, though on many graphs the two coincide. It is a faster screening tool.

20.3. Simulation with Hidden Variables

We reuse the same 8-node Gaussian linear DAG from the previous chapter — same seed, same `randomDAG` call with edge probability 0.35 and coefficients drawn from $[0.25, 1]$, same $n = 2000$ draws with standard normal errors — and then hide two of the nodes, X_3 and X_6, treating them as unobserved confounders. The algorithms below see a 2000×6 matrix; the full 8-column matrix is kept only to construct the truth.

The aim is to see what latent confounding costs. The previous chapter's PC reached a skeleton F1 of 0.952 with all eight variables observed. Nothing about the DGP changes here except which columns we are allowed to look at.

```
set.seed(2025)

n_vars     <- 8
n_samples  <- 2000
edge_prob  <- 0.35
latent_idx <- c(3, 6)
obs_idx    <- setdiff(1:n_vars, latent_idx)
all_labels <- paste0("X", 1:n_vars)
obs_labels <- all_labels[obs_idx]

true_dag   <- randomDAG(n_vars, prob = edge_prob, lB = 0.25, uB = 1)
nodes(true_dag) <- all_labels
data_full  <- rmvDAG(n_samples, true_dag, errDist = "normal")
colnames(data_full) <- all_labels
data_obs   <- data_full[, obs_idx]
n_obs      <- length(obs_idx)

sprintf("Total variables: %d   Hidden: %s   Observed: %d",
        n_vars, paste(all_labels[latent_idx], collapse = ", "), n_obs)
```

```
[1] "Total variables: 8   Hidden: X3, X6   Observed: 6"
```

20.3.1. True structure over observed variables

Two observed variables are adjacent in the MAG iff no subset of the *observed* variables d-separates them in the full DAG. We compute this with `pcalg::dsep`.

```
all_subsets <- function(xs) {
  out <- list(character(0))
  for (k in seq_along(xs)) out <- c(out, combn(xs, k, simplify = FALSE))
  out
}

# Ancestors of a node in a graphNEL DAG (excluding the node itself).
ancestors_of <- function(dag, target) {
  ie <- inEdges(dag)
  visited <- character(0)
  queue   <- target
  while (length(queue)) {
    cur <- queue[1]; queue <- queue[-1]
    if (cur %in% visited) next
    visited <- c(visited, cur)
    queue   <- c(queue, ie[[cur]])
  }
  setdiff(visited, target)
}
```

```r
# Simplified "true MAG" over observed nodes as a pcalg-encoded PAG amat:
# directed edges where one observed node is an ancestor of the other in the
# full DAG, bidirected edges where neither is an ancestor of the other.
#
# NOTE: this is a SIMPLIFIED oracle for the skeleton/F1 comparison below,
# not a full authoritative MAG. A true MAG projection can carry more
# subtle endpoint marks when directed and confounding paths coexist
# (e.g. an edge that is both a causal ancestor and confounded). Use this
# as an observed-variable adjacency oracle, not as a definitive
# MAG/PAG endpoint ground truth.
true_mag_amat <- function(dag, observed_names) {
  p <- length(observed_names)
  amat <- matrix(0L, p, p, dimnames = list(observed_names, observed_names))
  for (i in seq_len(p - 1)) for (j in seq.int(i + 1, p)) {
    u <- observed_names[i]; v <- observed_names[j]
    others <- setdiff(observed_names, c(u, v))
    sep <- any(vapply(all_subsets(others),
                      function(S) dsep(u, v, S, dag),
                      logical(1)))
    if (sep) next
    anc_u <- ancestors_of(dag, u)
    anc_v <- ancestors_of(dag, v)
    # pcalg amat.pag convention: amat[i, j] is the mark AT j
    # (u -> v means arrowhead (2) at v, tail (3) at u)
    if      (u %in% anc_v) { amat[i, j] <- 2L; amat[j, i] <- 3L }   # u -> v
    else if (v %in% anc_u) { amat[i, j] <- 3L; amat[j, i] <- 2L }   # v -> u
    else                   { amat[i, j] <- 2L; amat[j, i] <- 2L }   # u <-> v
  }
  amat
}

# Skeleton version (1 wherever an edge exists, ignoring direction) - used in
# the Monte Carlo F1 comparison below.
true_mag_skel <- function(dag, observed_names) {
  a <- true_mag_amat(dag, observed_names)
  (a != 0L) * 1L
}

amat_true_pag  <- true_mag_amat(true_dag, obs_labels)
amat_true_mag  <- (amat_true_pag != 0L) * 1L  # skeleton for F1 comparisons

# igraph representations used for plotting
ig_true_full <- graphnel_to_ig(true_dag)   # full 8-node DAG
ig_true_pag  <- pag_to_ig(amat_true_pag, obs_labels)   # true MAG over obs nodes

# Shared layout: compute from full DAG, subset coords for observed-node plots
full_layout  <- igraph::layout_with_sugiyama(ig_true_full)$layout
```

```
obs_layout   <- full_layout[obs_idx, ]     # positions of the observed nodes
rownames(obs_layout) <- obs_labels

sprintf("True MAG skeleton edges (over %d observed nodes): %d",
        n_obs, sum(amat_true_mag) / 2)
```

```
[1] "True MAG skeleton edges (over 6 observed nodes): 10"
```

This is the first thing to notice, before any algorithm runs. The full DAG has 11 edges among 8 nodes. The true MAG over the 6 observed nodes has 10 edges among 15 possible pairs, a density of two thirds. Hiding two variables did not simplify the problem; it made the graph much denser, because every pair of observed nodes sharing a hidden parent acquires an edge that was not there before. That densification is the cost of latent confounding, and it is why the F1 scores below sit well under the 0.952 that PC managed with everything observed.

```
op <- par(mfrow = c(1, 2), mar = c(1, 1, 3, 1))
plot_ig(ig_true_full, layout = full_layout, main = "Full DAG (all 8 nodes)")
plot_ig(ig_true_pag,  layout = obs_layout,  main = "True MAG (observed nodes)")
par(op)
```

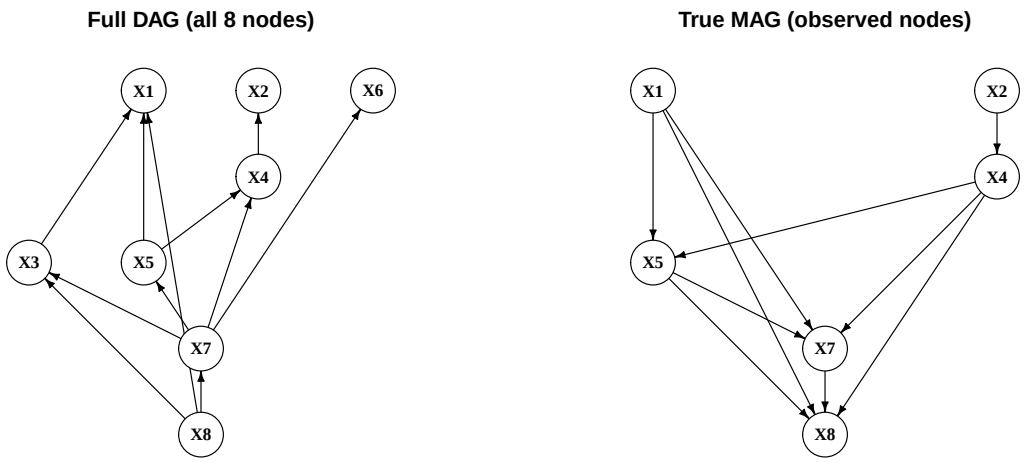


Figure 20.1.: Full DAG (all 8 nodes) and the true MAG skeleton over observed variables. The MAG skeleton includes both direct paths and paths through hidden nodes.

20.4. FCI Algorithm

FCI is the extension of PC for latent variables. It starts from a complete graph, removes edges with CI tests, and then uses orientation rules that can produce bidirected edges for hidden common causes.

How FCI extends PC:

1. **Phase 1 (skeleton):** Same as PC — remove edges via CI tests with growing conditioning sets.
2. **Phase 2 (initial orientation):** Mark all edge endpoints as $\circ$ (uncertain); orient unshielded colliders as in PC.
3. **Phase 3 (Possible-D-SEP pruning):** For each remaining edge $X - Y$, compute the **Possible-D-SEP** sets and run *additional* CI tests conditioning on their subsets, removing further edges. With latent variables, two non-adjacent nodes need not be separable by a subset of their neighbors — this stage is what makes FCI's skeleton differ from PC's, and it is the expensive part (the number of subsets can grow exponentially). Colliders are then re-oriented on the pruned skeleton.
4. **Phase 4 (rule propagation):** Apply the 10 orientation rules of Zhang (2008) (as implemented in `pcalg`) that propagate known marks without creating contradictions, including rules that can produce definite $\rightarrow$ and $\leftrightarrow$ edges. These rules perform no CI tests and are computationally cheap.

The extra marks let FCI say what is known and what remains uncertain, but the output is harder to read than a DAG.

```
make_counted_ci <- function(suff_stat) {
  count <- 0L
  ci <- function(x, y, S, suffStat) {
    count <<- count + 1L
    gaussCItest(x, y, S, suffStat)
  }
  list(ci = ci, count = function() count)
}

sig_level <- 0.01
suff_obs  <- list(C = cor(data_obs), n = n_samples)

ci_fci   <- make_counted_ci(suff_obs)
fci_fit  <- fci(suff_obs, ci_fci$ci, labels = obs_labels,
                alpha = sig_level, verbose = FALSE)
ci_tests_fci <- ci_fci$count()
```

```
# pcalg PAG amat encoding: 0 = no edge, 1 = circle, 2 = arrowhead, 3 = tail.
# An edge between i and j is described by amat[i,j] (the mark AT j)
# and amat[j,i] (the mark AT i).
pag_skeleton_amat <- function(amat) {
  out <- (amat != 0 | t(amat) != 0) * 1L
  diag(out) <- 0L
  out
}

f1_skel <- function(amat_true, amat_est) {
  diag(amat_true) <- diag(amat_est) <- 0
  tp <- sum(amat_true & amat_est) / 2
  fp <- sum(!amat_true & amat_est) / 2
```

```
  fn <- sum(amat_true & !amat_est) / 2
  if (tp == 0) 0 else 2 * tp / (2 * tp + fp + fn)
}

skel_fci  <- pag_skeleton_amat(fci_fit@amat)
f1_fci_v  <- f1_skel(amat_true_mag, skel_fci)
ig_fci    <- pag_to_ig(fci_fit@amat, obs_labels)
sprintf("FCI:    skeleton F1 = %.3f   CI tests = %d", f1_fci_v, ci_tests_fci)
```

```
[1] "FCI:    skeleton F1 = 0.824   CI tests = 165"
```

FCI recovers the skeleton with an F1 of 0.824, using 165 CI tests. That is a long way below PC's 0.952 on the same underlying DAG with nothing hidden, and the dense MAG is why: with two thirds of all pairs adjacent, there are few conditional independences left to find, and each missed edge costs more.

```
plot_ig(ig_fci, layout = obs_layout)
```

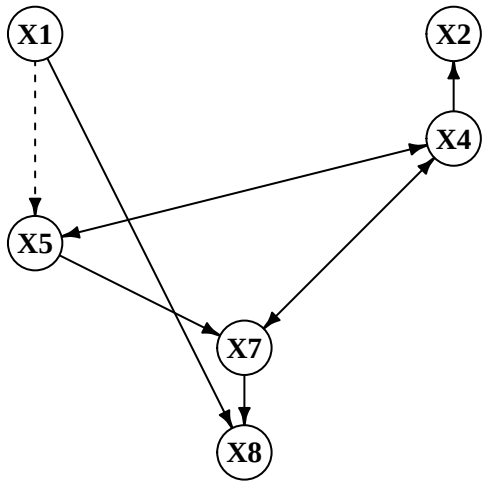


Figure 20.2.: PAG estimated by FCI. Dashed edges indicate endpoints with circle ( ) marks — orientations the algorithm could not determine. Solid double-headed arrows ( ) flag definite hidden common causes.

> **Note**
>
> **Reading the PAG plot**
> In `pcalg`'s plot output:
>
> - A **plain arrow** $X \to Y$ is a definite cause — X is an ancestor of Y in every equivalent MAG. It need not be a *direct* effect: the path may run through latent mediators (the DAG $X \to L \to Y$ with L hidden yields the MAG $X \to Y$).
> - A **double-headed arrow** $X \leftrightarrow Y$ is a definite hidden common cause.
> - A **circle endpoint** is the $\circ$ mark — uncertain at that endpoint.
>
> In practice for economic research: focus first on $\leftrightarrow$ edges — these flag definite confounding and tell you where IV or proxy strategies are needed. Then examine $\to$ edges — these are causal claims that survive across all statistically equivalent structures.

When FCI gives wrong answers: FCI assumes the CI tests are perfectly accurate (no finite-sample error). In practice, with small n or many variables, some false CI decisions propagate through the 10 orientation rules. The skeleton quality (F1) is generally more reliable than the orientation quality.

20.5. RFCI Algorithm

RFCI (Really Fast Causal Inference; Colombo et al., 2012) skips FCI's expensive **Possible-D-SEP pruning stage** entirely — the stage responsible for most of FCI's CI tests. To stay sound without it, RFCI adds a few extra CI tests of its own inside the collider and discriminating-path checks; the orientation-rule propagation is kept.

Key differences from FCI:

1. **Skeleton phase:** the PC-style skeleton search is the same, but RFCI does **not** run the Possible-D-SEP removals — this is where the savings come from, and why its skeleton can retain extra edges (it converges to a supergraph of the PAG skeleton).
2. **Collider check:** before orienting an unshielded triple as a collider, RFCI runs *additional* CI tests to verify the orientation — a modest extra cost that restores soundness without the Possible-D-SEP stage.
3. **Orientation output:** RFCI produces a *partial* PAG. Some edges that FCI orients are left as $\circ\!-\!\circ$ in RFCI; conversely, every mark RFCI does output is asymptotically correct under the same conditions.

In practice, the skeleton is often the main output: it tells us which pairs need substantive attention. Full PAG orientations are useful, but they are harder to defend.

```
ci_rfci    <- make_counted_ci(suff_obs)
rfci_fit   <- rfci(suff_obs, ci_rfci$ci, labels = obs_labels,
                   alpha = sig_level, verbose = FALSE)
ci_tests_rfci <- ci_rfci$count()
```

```
skel_rfci  <- pag_skeleton_amat(rfci_fit@amat)
f1_rfci_v  <- f1_skel(amat_true_mag, skel_rfci)
ig_rfci    <- pag_to_ig(rfci_fit@amat, obs_labels)
sprintf("RFCI:   skeleton F1 = %.3f   CI tests = %d", f1_rfci_v, ci_tests_rfci)
```

```
[1] "RFCI:   skeleton F1 = 0.824   CI tests = 138"
```

RFCI reaches the same skeleton F1 of 0.824 using 138 CI tests instead of 165, a saving of 16%. On this dataset the Possible-D-SEP stage bought FCI nothing: it ran 27 extra tests and removed no edge that mattered for the skeleton score.

> **Note**
>
> **FCI vs. RFCI output: same plot type, fewer marks**
> Both `fci()` and `rfci()` return `fciAlgo` objects and plot with the same `plot()` method. The visual difference is that RFCI's PAG typically has **more circle marks** — it has been more conservative about orientation. Edges that *do* receive an arrowhead or tail mark in RFCI are reliable.
> If you only need the skeleton (the most common use case in large-p screening problems), RFCI is the right default. If you need orientations to plan an IV strategy, run FCI on top.

20.6. Comparison

```
kable(data.frame(
  Algorithm   = c("FCI", "RFCI"),
  Output      = c("Full PAG", "Skeleton + partial PAG"),
  Skeleton_F1 = round(c(f1_fci_v, f1_rfci_v), 3),
  CI_Tests    = c(ci_tests_fci, ci_tests_rfci)
), row.names = FALSE)
```

Table 20.3.: Algorithm comparison on the latent-variable scenario (2 hidden nodes, n = 2000)

Algorithm	Output	Skeleton_F1	CI_Tests
FCI	Full PAG	0.824	165
RFCI	Skeleton + partial PAG	0.824	138

The two algorithms tie on skeleton recovery and RFCI is cheaper. One dataset is not enough to conclude that, which is what the Monte Carlo below is for.

```
op <- par(mfrow = c(1, 3), mar = c(1, 1, 3, 1))
plot_ig(ig_true_pag, layout = obs_layout, main = "True MAG")
plot_ig(ig_fci,      layout = obs_layout, main = "FCI - estimated PAG")
plot_ig(ig_rfci,     layout = obs_layout, main = "RFCI - estimated PAG")
par(op)
```

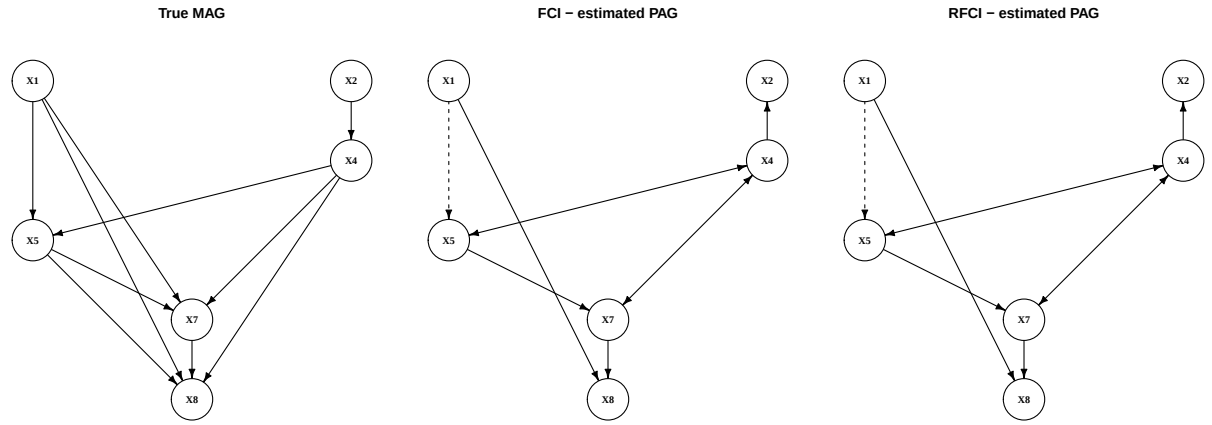


Figure 20.3.: True MAG vs. PAGs recovered by FCI and RFCI. Directed arrows are definite cause-to-effect; bidirected ( ) edges flag hidden common causes; circle ( ) marks indicate orientations the algorithm could not resolve. RFCI typically leaves more circles than FCI because its orientation phase is deliberately more conservative.

20.7. Monte Carlo Evaluation

We repeat the whole exercise 100 times, drawing a fresh random DAG each time and choosing a fresh pair of nodes to hide, so the results do not depend on one graph or one unlucky choice of which variables are unobserved.

```
evaluate_latent <- function(seed, n_vars = 8, n_samples = 2000,
                            edge_prob = 0.35, n_latent = 2, sig = 0.01) {
  set.seed(seed)
  labs <- paste0("X", 1:n_vars)
  dag  <- randomDAG(n_vars, prob = edge_prob, lB = 0.25, uB = 1)
  nodes(dag) <- labs
  d    <- rmvDAG(n_samples, dag, errDist = "normal")
  colnames(d) <- labs

  lat  <- sort(sample.int(n_vars, n_latent))
  obs  <- setdiff(seq_len(n_vars), lat)
  obs_labs <- labs[obs]
  d_obs <- d[, obs]
  ss    <- list(C = cor(d_obs), n = n_samples)
```

```
  true_skel <- true_mag_skel(dag, obs_labs)

  ci_f <- make_counted_ci(ss)
  fci_r <- fci(ss, ci_f$ci, labels = obs_labs, alpha = sig, verbose = FALSE)

  ci_r <- make_counted_ci(ss)
  rfci_r <- rfci(ss, ci_r$ci, labels = obs_labs, alpha = sig, verbose = FALSE)

  c(f1_fci  = f1_skel(true_skel, pag_skeleton_amat(fci_r@amat)),
    f1_rfci = f1_skel(true_skel, pag_skeleton_amat(rfci_r@amat)),
    ci_fci  = ci_f$count(),
    ci_rfci = ci_r$count())
}

mc <- as.data.frame(t(sapply(1:100, evaluate_latent)))

mc_summary <- data.frame(
  Method      = c("FCI", "RFCI"),
  Skeleton_F1 = round(c(mean(mc$f1_fci),  mean(mc$f1_rfci)),  3),
  CI_Tests    = round(c(mean(mc$ci_fci),  mean(mc$ci_rfci)),  1)
)
kable(mc_summary,
      caption = "Monte Carlo averages (100 replications, n = 2000, p = 8, 2 latent)",
      row.names = FALSE)
```

Table 20.4.: Monte Carlo averages (100 replications, n = 2000, p = 8, 2 latent)

Method	Skeleton_F1	CI_Tests
FCI	0.938	150.5
RFCI	0.943	102.7

Averaged over 100 replications, FCI gets a skeleton F1 of 0.938 and RFCI 0.943, while FCI uses 150.5 CI tests and RFCI 102.7. RFCI is 32% cheaper and loses nothing on the skeleton.

Two cautions on reading that. The 0.005 F1 difference is not evidence that RFCI recovers skeletons better; RFCI converges to a *supergraph* of the PAG skeleton, so if anything theory points the other way, and a gap this small on 100 replications is noise. And the single dataset above scored 0.824 for both, well below these averages — that draw was one of the harder ones, as its denser-than-average MAG would suggest.

What does survive is the cost result. Skipping the Possible-D-SEP stage is where RFCI's savings come from, and on these graphs that stage is roughly a third of FCI's work while contributing nothing measurable to skeleton accuracy.

```
mc_long <- data.frame(
  method = rep(c("FCI", "RFCI"), each = nrow(mc)),
```

```
  ci     = c(mc$ci_fci, mc$ci_rfci)
)
ggplot(mc_long, aes(x = ci, fill = method)) +
  geom_histogram(bins = 20, position = "identity", alpha = 0.55, color = "white") +
  scale_fill_manual(values = c(FCI = "steelblue", RFCI = "tomato")) +
  labs(x = "CI tests", y = "Frequency",
       title = "CI Test Counts: FCI vs RFCI (100 replications)") +
  theme_minimal()
```

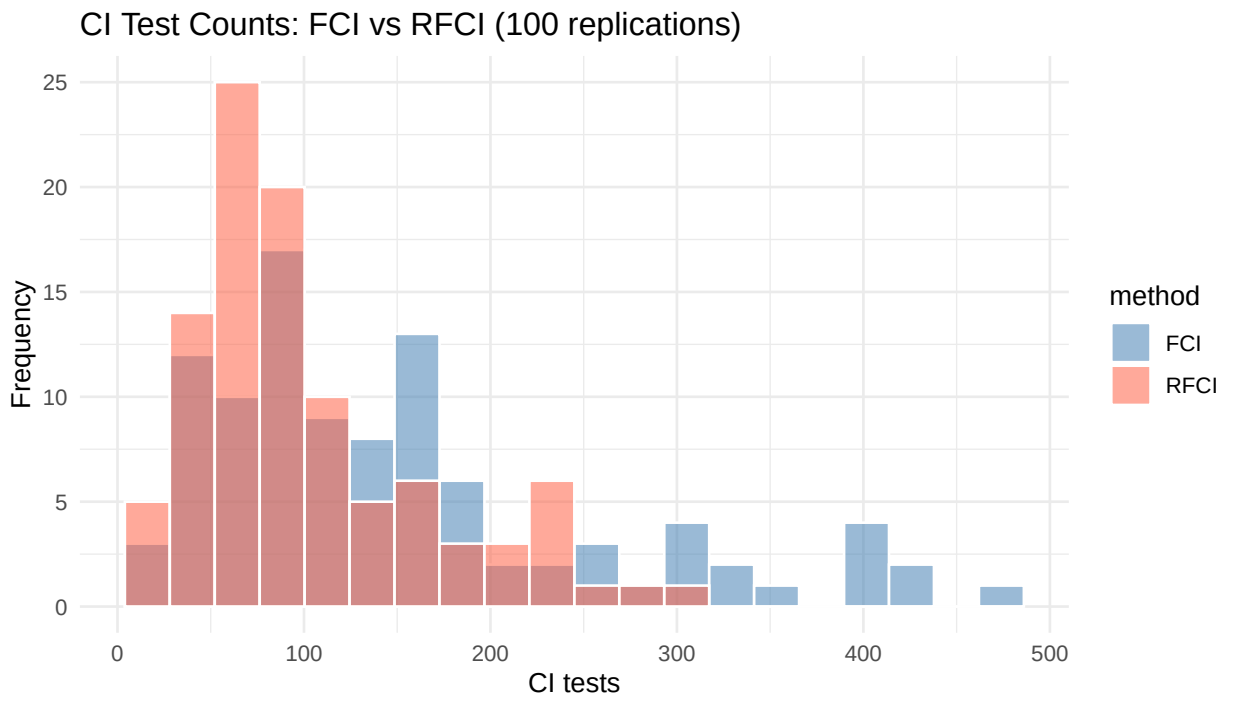


Figure 20.4.: CI-test count distributions across 100 replications. By skipping the Possible-D-SEP tests, RFCI typically — though not always — runs fewer tests than FCI.

The two histograms overlap, which is the point of the figure's caption: RFCI is usually but not always cheaper, because its extra collider-verification tests can outweigh the Possible-D-SEP saving on sparse draws.

20.8. Summary

Property	PC / GES	FCI	RFCI
Latent confounders	assumes none	handles	handles
Output	CPDAG	Full PAG	Skeleton + partial PAG

Property	PC / GES	FCI	RFCI
Skeleton consistency			converges to a slight supergraph
Orientation coverage	full	full (under faithfulness)	partial (conservative)
CI cost	PC: PC's cost; GES: none	skeleton + Possible-D-SEP tests	skeleton only (no Possible-D-SEP)

FCI and RFCI serve different purposes. Use FCI when orientations matter. Use RFCI when the goal is a faster skeleton screen.

20.8.1. Practical decision guide

Use PC or GES when:

- You are confident in causal sufficiency (all relevant variables are measured)
- You want a CPDAG that you can orient further with background knowledge

Use FCI when:

- You suspect hidden confounders but don't know where
- You need edge orientations and $\leftrightarrow$ marks to guide IV or proxy strategies
- Sample size is large enough that CI-test errors don't cascade badly through orientation rules

Use RFCI when:

- You suspect latent confounders and have many variables ($p > 20$)
- The goal is to prune the adjacency graph before applying domain knowledge or estimation
- You want a fast first pass — promote candidates to FCI later

20.8.2. From discovery to estimation

Causal discovery is a first step, not a final answer. A reasonable workflow is:

1. **Discover** — run FCI or RFCI to get candidate adjacencies and, if using FCI, some edge marks
2. **Refine** — apply temporal ordering, institutional knowledge, and exclusion restrictions to orient remaining edges
3. **Identify** — check whether the effect of interest is identified given the refined graph (backdoor, front-door, ID algorithm)
4. **Estimate** — use TMLE, AIPW, or the estimators in earlier chapters

Causal discovery narrows the space of possible structures. Domain knowledge still does the hard work.

> **Note**
>
> **Going further: orientation in the latent case**
> FCI's PAG output distinguishes definite causes ($\rightarrow$), definite hidden common causes ($\leftrightarrow$), and uncertain cases ($\circ$). The PAG can be combined with background knowledge such as temporal ordering to further orient edges before estimation.

20.9. How Much Does Causal Discovery Help in Economics?

Recovering the MAG skeleton is still far from recovering the full DAG. Much is still missing:

What is recovered	What is still missing
RFCI skeleton	Edge directions; direct cause vs. hidden common cause; latent nodes
FCI PAG	Unique MAG; latent nodes; most edges still carry $\circ$ marks
True MAG	Latent nodes and their connections
Full DAG	Nothing — this is the goal

The core econometric question is almost always "what is the causal effect of X on Y?" Causal discovery with latent variables does not answer this directly. A definite $X \leftrightarrow Y$ in a PAG tells you confounding exists — but not how to remove it. Without oriented edges you cannot even check whether the effect is identified, let alone estimate it.

Where these methods do add value in economics:

1. **Falsifying structural models.** If your model implies $X \perp\!\!\!\perp Z \mid W$, test it. A rejection means the model's independence assumptions are inconsistent with the data — discovery tells you *which* assumptions fail before you commit to a full structural estimation.
2. **Flagging where instruments are needed.** A definite $X \leftrightarrow Y$ in the PAG is a data-driven diagnostic: OLS of Y on X is biased, and you need an instrument, proxy, or natural experiment. Discovery does not supply the instrument but tells you where to look for one.
3. **High-dimensional variable selection.** With many candidate controls, knowing which pairs are conditionally independent prunes the problem before applying identification strategies. This is most useful in settings with $p > 20$ variables where theory does not specify the full graph.
4. **Hypothesis generation when theory is silent.** For new economic phenomena — platform markets, fintech, peer effects in novel settings — where theory does not give a strong causal ordering, discovery algorithms generate hypotheses worth investigating with better-powered research designs.

Causal discovery is a complement to IV, RD, DiD, and synthetic control, not a substitute. It is most useful early in a project, as a diagnostic and hypothesis-generating tool.

21. From Graph to Estimate

```
library(dagitty)
library(igraph)
library(causaleffect)
library(ggplot2)
```

The previous chapters covered discovery: algorithms that learn a graph from data. But a graph is not an estimate. After we have a graph, we still need to ask two questions:

1. **Identification**: is the target effect a function of the observed-data distribution?
2. **Estimation**: once the identifying formula is known, estimate it from the data.

In R, `dagitty` handles identification via the backdoor and front-door criteria; `causaleffect` implements the full Pearl–Shpitser ID algorithm for acyclic directed mixed graphs (ADMGs) with hidden confounders. Estimation is done directly from the identifying formula — a step that is explicit in R, making the connection between the formula and the estimator transparent.

21.1. Backdoor: standard adjustment

Start with the simple case. X is a common cause of treatment A and outcome Y, and X is observed. Backdoor adjustment applies.

```
# X is a common cause of A and Y
g_bd <- dagitty("dag { X -> A; X -> Y; A -> Y }")
exposures(g_bd) <- "A"
outcomes(g_bd)  <- "Y"

sets <- adjustmentSets(g_bd, exposure = "A", outcome = "Y")
cat("Adjustment sets:\n")
```

```
Adjustment sets:
```

```
print(sets)
```

```
{ X }
```

`dagitty` finds `{X}` as the minimal adjustment set. Adjusting on X blocks the only open backdoor path $A \leftarrow X \rightarrow Y$.

We simulate data matching that graph. We draw $n = 2000$ observations with $X \sim N(0,1)$, treatment $A \sim \text{Bernoulli}(\Lambda(X))$, and outcome $Y = 2A + X + 0.5\varepsilon$ with $\varepsilon \sim N(0,1)$. The effect is constant, so the true ACE is 2. We then estimate it with AIPW using the adjustment set the graph chose, and take a 95% interval from 500 bootstrap resamples.

```r
set.seed(1)
n     <- 2000
X     <- rnorm(n)
A     <- as.numeric(runif(n) < 1 / (1 + exp(-X)))  # logistic: A depends on X
Y     <- 2 * A + X + 0.5 * rnorm(n)                 # true ACE = 2
data_bd <- data.frame(X = X, A = A, Y = Y)

# AIPW (doubly-robust) estimator for the backdoor-adjusted ACE
# Outcome model: E[Y | A, X]
mu1 <- predict(lm(Y ~ A + X, data = data_bd), newdata = transform(data_bd, A = 1))
mu0 <- predict(lm(Y ~ A + X, data = data_bd), newdata = transform(data_bd, A = 0))

# Propensity score: P(A = 1 | X)
ps  <- predict(glm(A ~ X, data = data_bd, family = binomial), type = "response")

# AIPW influence function
aipw <- mean(
  (data_bd$A / ps) * (Y - mu1) - ((1 - data_bd$A) / (1 - ps)) * (Y - mu0) +
  mu1 - mu0
)

cat(sprintf("Backdoor AIPW ACE (true = 2.0): %.3f\n", aipw))
```

```
Backdoor AIPW ACE (true = 2.0): 1.990
```

```r
# Bootstrap 95% CI
set.seed(99)
boot_bd <- replicate(500, {
  idx  <- sample.int(n, replace = TRUE)
  bd   <- data_bd[idx, ]
  m1   <- predict(lm(Y ~ A + X, data = bd), newdata = transform(bd, A = 1))
  m0   <- predict(lm(Y ~ A + X, data = bd), newdata = transform(bd, A = 0))
  p    <- predict(glm(A ~ X, data = bd, family = binomial), type = "response")
  mean((bd$A / p) * (bd$Y - m1) - ((1 - bd$A) / (1 - p)) * (bd$Y - m0) + m1 - m0)
})
ci_bd <- quantile(boot_bd, c(0.025, 0.975))
cat(sprintf("95%% CI: [%.3f, %.3f]\n", ci_bd[1], ci_bd[2]))
```

```
95% CI: [1.942, 2.043]
```

AIPW returns 1.990 with a 95% bootstrap interval of $[1.942, 2.043]$, covering the true 2. The AIPW estimator is consistent if either the outcome model or the propensity score is correctly specified; here both are, since the DGP is linear in the outcome and logistic in the treatment.

21.2. Front-door: p-fixable effects

Now suppose A and Y have an unmeasured common cause U, but there is a measured mediator M on the path $A \to M \to Y$. Backdoor adjustment fails because U is unobserved. The front-door formula can still identify the effect.

In `dagitty`, the bidirected edge $A \leftrightarrow Y$ (representing the latent U) appears as `A <-> Y`. For `causaleffect`, the same edge is encoded as two directed edges `A -> Y` and `Y -> A` both marked with `description = "U"` in the igraph object.

```
# dagitty for display and adjustment-set check
g_fd <- dagitty("dag { A -> M; M -> Y; A <-> Y }")
exposures(g_fd) <- "A"
outcomes(g_fd)  <- "Y"

# An empty dagitty adjustment-set list prints nothing at all, so report the
# count rather than calling print() on it and appearing to produce no output.
sets_fd <- adjustmentSets(g_fd, exposure = "A", outcome = "Y")
cat("Valid backdoor adjustment sets:", length(sets_fd), "\n")
```

```
Valid backdoor adjustment sets: 0
```

```
# causaleffect for ID algorithm: bidirected A <-> Y = two directed edges + description "U"
g_fd_ig <- graph_from_literal(A -+ M, M -+ Y, A -+ Y, Y -+ A)
# Mark the A<->Y bidirected edge by endpoint name rather than a hardcoded
# edge index: igraph happens to sort edges by source vertex today (giving
# order A->M(1), A->Y(2), M->Y(3), Y->A(4)), but that ordering isn't part of
# igraph's documented contract, so indices like c(2,4) can silently break if
# igraph's internals or this graph's construction ever changes.
edge_ends <- ends(g_fd_ig, E(g_fd_ig))
confounded <- (edge_ends[, 1] == "A" & edge_ends[, 2] == "Y") |
              (edge_ends[, 1] == "Y" & edge_ends[, 2] == "A")
g_fd_ig <- set_edge_attr(g_fd_ig, "description", index = which(confounded), value = "U")

expr_fd <- causal.effect(y = "Y", x = "A", G = g_fd_ig, simp = TRUE)
cat("\nIdentifying expression:\n")
```

```

Identifying expression:
```

```
cat(expr_fd, "\n")
```

```
\sum_{M}P(M|A)\left(\sum_{A}P(Y|A,M)P(A)\right)
```

There are no valid backdoor adjustment sets, as expected: U is unobserved, so no set of measured variables blocks the path it opens. And the expression `causal.effect` returns is the front-door formula, $\sum_M P(M \mid A) \sum_{A'} P(Y \mid A', M)\, P(A')$.

We now simulate data for it. We draw $n = 3000$ observations with an unmeasured confounder $U \sim N(0,1)$, treatment $A \sim \text{Bernoulli}(\Lambda(U))$, mediator $M \sim \text{Bernoulli}(\Lambda(0.5 + 2A))$, and outcome $Y = 1.5M + 2U + 0.5\varepsilon$ with $\varepsilon \sim N(0,1)$. The mediator depends on A alone and the outcome depends on M and U but not directly on A, which is what the front-door criterion requires.

The truth follows from the DGP. Intervening on A changes Y only through M, so the ACE is $1.5 \times [P(M = 1 \mid A = 1) - P(M = 1 \mid A = 0)] = 1.5 \times [\Lambda(2.5) - \Lambda(0.5)] = 1.5 \times 0.302 = 0.453$.

```
set.seed(2)
n     <- 3000
U     <- rnorm(n)                                      # unmeasured confounder
A_fd  <- as.numeric(runif(n) < 1 / (1 + exp(-U)))      # A depends on U
M_fd  <- as.numeric(runif(n) < 1 / (1 + exp(-(0.5 + 2 * A_fd))))  # M depends on A only
Y_fd  <- 1.5 * M_fd + 2 * U + 0.5 * rnorm(n)           # Y depends on M and U

# True ACE: E[Y | do(A=1)] - E[Y | do(A=0)]
# = 1.5 * (E[M | A=1] - E[M | A=0])
true_EMA1  <- 1 / (1 + exp(-(0.5 + 2)))   # scalar P(M=1 | A=1)
true_EMA0  <- 1 / (1 + exp(-0.5))         # scalar P(M=1 | A=0)
true_ace_fd <- 1.5 * (true_EMA1 - true_EMA0)
cat(sprintf("True front-door ACE = %.3f\n", true_ace_fd))
```

```
True front-door ACE = 0.453
```

```
data_fd <- data.frame(A = A_fd, M = M_fd, Y = Y_fd)
```

Estimate the front-door functional using the plug-in formula:

```
front_door_ace <- function(data) {
  pA <- mean(data$A)
  pM_given_A <- prop.table(table(data$A, data$M), margin = 1)
  pY_given_AM <- with(data, tapply(Y, list(A, M), mean))

  ey_do <- function(a) {
    sum(sapply(c(0, 1), function(m) {
      p_m_given_a <- pM_given_A[as.character(a), as.character(m)]
      inner <- sum(sapply(c(0, 1), function(ap)
        pY_given_AM[as.character(ap), as.character(m)] *
```

```
          ifelse(ap == 1, pA, 1 - pA)))
      p_m_given_a * inner
    }))
  }
  ey_do(1) - ey_do(0)
}

est_fd <- front_door_ace(data_fd)

set.seed(77)
boot_fd <- replicate(500, {
  idx <- sample.int(nrow(data_fd), replace = TRUE)
  front_door_ace(data_fd[idx, ])
})
ci_fd  <- quantile(boot_fd, c(0.025, 0.975))

cat(sprintf("Front-door ACE: %.3f  [95%% CI: %.3f, %.3f]\n",
            est_fd, ci_fd[1], ci_fd[2]))
```

```
Front-door ACE: 0.499  [95% CI: 0.432, 0.577]
```

```
# Naïve mean difference for comparison
naive_ace <- mean(Y_fd[A_fd == 1]) - mean(Y_fd[A_fd == 0])
cat(sprintf("Naïve mean difference (biased!): %.3f\n", naive_ace))
```

```
Naïve mean difference (biased!): 2.083
```

```
cat(sprintf("True ACE:                          %.3f\n", true_ace_fd))
```

```
True ACE:                       0.453
```

The front-door estimate is 0.499 with a 95% bootstrap interval of $[0.432, 0.577]$, which covers the true 0.453. The naive mean difference is 2.083 — more than four times the truth, and outside the front-door interval by a factor of four. The formula recovers the effect even though U is never measured; the naive comparison is picking up the path through U, which enters the outcome with coefficient 2 and the treatment through the logit.

The bias here is far larger than in the front-door example of the graphs chapter, where the naive contrast was 0.14 against a truth of 0.055. The difference is the strength of the confounding: U enters Y with coefficient 2 here against 0.7 there. Nothing about the front-door formula changes; what changes is how much work it is doing.

21.3. When identification fails

Not every graph identifies the effect. If A and Y have a hidden common cause and there is no front-door variable, the effect is not identified. The graph below is the **bow**: a directed edge $A \to Y$ plus a bidirected $A \leftrightarrow Y$, and nothing else. Remove the mediator M from the front-door graph and this is what is left.

```
# A → Y with unmeasured confounding (A <-> Y) and no mediator: the "bow"
# graph needs THREE igraph edges - the directed A → Y plus the
# bidirected pair (encoded as two opposite edges marked "U").
# graph_from_literal() silently collapses a duplicated A -+ Y, so we
# build the multigraph with make_graph().
g_unident_ig <- make_graph(c("A","Y",  "A","Y",  "Y","A"), directed = TRUE)
# make_graph() preserves the literal edge order given above (unlike
# graph_from_literal(), which re-sorts), so index 2:3 is reliable as long as
# the c(...) vector above isn't reordered -- mark by endpoint pair anyway so
# a future edit to that vector can't silently mismark the real A->Y edge.
edge_ends <- ends(g_unident_ig, E(g_unident_ig))
# The first A->Y edge is the real causal edge; the second (a duplicate,
# per `duplicated()`) together with Y->A forms the bidirected U pair.
u_edges <- which(((edge_ends[,1]=="A" & edge_ends[,2]=="Y") & duplicated(edge_ends)) |
                   (edge_ends[,1]=="Y" & edge_ends[,2]=="A"))
g_unident_ig <- set_edge_attr(g_unident_ig, "description",
                              index = u_edges, value = "U")

result <- tryCatch(
  causal.effect(y = "Y", x = "A", G = g_unident_ig, simp = TRUE),
  error = function(e) conditionMessage(e)
)
cat("Result from causal.effect:\n", result, "\n")
```

```
Result from causal.effect:
 Not identifiable.
```

`causaleffect` raises an error, printed above as "Not identifiable.", which the `tryCatch` captures. This is a useful failure. The algorithm did not return a formula that would quietly estimate the wrong thing; it reported that no function of the observed distribution equals the causal effect under this graph. No amount of data helps, because the obstacle is the graph and not the sample size.

The honest response in applied work is to report the non-identifiability, then either:

1. Find additional variables that close the unmeasured-confounding paths
2. Apply sensitivity analysis to bound the effect under maintained assumptions
3. Find an instrumental variable for a LATE-style identification (IV chapter)

Saying that the hidden confounding is probably small is sensitivity analysis, not identification.

21.4. End-to-end: discover, identify, estimate

Discovery algorithms return graphs that can be passed to the identification step:

```
library(pcalg)

# Step 1: discover the graph (chapters on causal discovery)
suffStat <- list(C = cor(data_obs), n = nrow(data_obs))
pc_fit   <- pc(suffStat, indepTest = gaussCItest,
               alpha = 0.05, p = ncol(data_obs))

# Step 2: check for backdoor adjustment set using dagitty
# (convert pcalg CPDAG → dagitty for convenience).
# NOTE the t(): as(graphNEL, "matrix") is from→to, but pcalg2dagitty
# expects the amat.cpdag coding, which is its transpose - without it
# every directed edge is silently reversed.
g_dag <- pcalg2dagitty(t(as(pc_fit@graph, "matrix")),
                       colnames(data_obs), type = "cpdag")
sets  <- adjustmentSets(g_dag, exposure = "A", outcome = "Y")

# Step 3: if backdoor set found, estimate with AIPW
# Step 4: if not, call causaleffect::causal.effect() for the ID expression
# Step 5: estimate the returned functional by plug-in or regression
```

In applied work, I would not trust the discovered graph blindly. A more reasonable workflow is:

1. **Hypothesise** a graph based on subject-matter knowledge
2. **Discover** a data-driven graph using PC/GES/FCI
3. **Compare** the two — disagreement points to substantive questions
4. **Identify** under both graphs separately
5. **Estimate** under both, and report the range as part of the result

If the estimate changes a lot across plausible graphs, the data alone is not settling the question.

21.5. Why this matters

The graph-identification-estimation sequence matters because:

- **Backdoor adjustment is not always available.** Many real DAGs have hidden confounders. Front-door, nested, and general ID algorithms are the recourse, but they are too complex to derive by hand for non-trivial graphs.
- **The right estimator depends on the identification strategy.** AIPW-for-backdoor uses a different formula than the front-door plug-in. Matching the estimator to the identification result avoids the common mistake of plugging a graph into the wrong estimator.
- **Reporting becomes more disciplined.** Once the workflow forces you to specify a graph, it becomes harder to wave hands about "unconfoundedness" — you have to say exactly which variables are doing the unconfounding work.

21.6. Summary

- A graph is not an estimate. Identification is the bridge.
- `dagitty` handles backdoor and front-door checks; `causaleffect` handles general ADMG identification.
- Backdoor effects can be estimated with AIPW.
- Front-door effects require estimating the front-door functional.
- If the effect is not identified, report that instead of forcing a biased regression estimate.

Part VIII.

Appendix

22. R Packages Used in This Book

This appendix lists the R packages used in the book. The point is practical: for each method, which package does the computation, and which chapters use it?

Most packages live on CRAN. A few dependencies for `pcalg` (`graph`, `RBGL`, `Rgraphviz`) come from Bioconductor and require `BiocManager`. Two packages are GitHub-only: `npcausal` (`remotes::install_github("ehkennedy/npcausal")`) and `medoutcon` (`remotes::install_github("nhejazi/m`

22.1. Working With The Environment

A minimal reproducible workflow is:

```
cd ~/projects/books/causal_econometrics_guide
# Install Bioconductor dependencies once (needed by pcalg)
Rscript -e 'install.packages("BiocManager"); BiocManager::install(c("graph","RBGL","Rgraphviz")
# Install the CRAN packages used across the book
Rscript -e 'install.packages(c("tidyverse","dagitty","ggdag","causaleffect","igraph",
  "haven","causaldata","hdm","MatchIt","WeightIt","gmm","grf","fixest","did",
  "synthdid","rdrobust","DoubleML","mlr3","mlr3learners","SuperLearner","tmle",
  "mediation","lavaan","lavaan.survey","blavaan","bnlearn","pcalg","MASS"))'
# GitHub-only packages
Rscript -e 'remotes::install_github(c("ehkennedy/npcausal","nhejazi/medoutcon"))'
quarto render
```

The book uses Quarto's `execute: freeze: auto` setting (configured in `_quarto.yml`), so each chapter's computed output is cached under `_freeze/`. Re-rendering only re-executes chunks whose source has changed. When upgrading a package whose API may have moved, delete the relevant `_freeze/<chapter>/` directory to force a re-run.

22.2. Package Map

Package	Used For	Main Book Chapters
`dagitty`	DAG and ADMG construction, adjustment-set search, d-separation	Identification, DAG workflow, Smoking cessation

Package	Used For	Main Book Chapters
`ggdag`	Visual rendering of DAGs/ADMGs built with `dagitty`	Identification, DAG workflow, IV/RDD
`causaleffect`	Pearl–Shpitser ID algorithm for general ADMGs	DAG workflow
`tidyverse` (`dplyr`, `tidyr`, `purrr`, `ggplot2`)	Data wrangling and plotting throughout	Every chapter
`haven`	Reading Stata `.dta` files used as examples	Estimation, DiD, Shift-share IV
`causaldata`	Built-in NHEFS and other causal-inference example datasets	Sensitivity analysis, Smoking cessation, IV/RDD
`hdm`	Pension (`pension`) dataset and high-dimensional inference	Estimation
`SuperLearner`	Stacked machine-learning nuisance estimation	Nonparametric
`npcausal`	Influence-function-based ATE/ATT estimators (Edward Kennedy)	Nonparametric
`tmle`	Targeted maximum likelihood estimation	Nonparametric
`DoubleML`, `mlr3`, `mlr3learners`	Double/debiased ML for partially linear and interactive models	Nonparametric
`did`	Callaway & Sant'Anna estimators; `mpdta` example dataset	DiD
`etwfe`	Wooldridge's extended TWFE for staggered DiD	DiD
`fixest`	High-dimensional fixed-effects OLS, Poisson, IV; clustered SEs	DiD, IV/RDD, Poisson-IV
`synthdid`	Standard DiD, synthetic control, synthetic DiD; Prop 99 example	DiD
`sem`	Classical two-stage least squares via `tsls()`	IV/RDD
`MASS`	Multivariate normal sampling for IV simulations	IV/RDD
`rdrobust`	Local-polynomial RDD, sharp and fuzzy designs	IV/RDD
`gmm`	Generalized method of moments for nonlinear IV	Poisson-IV
`lavaan`	SEM and CFA syntax for classical mediation	Mediation

Package	Used For	Main Book Chapters
`medoutcon`	Causal mediation: controlled, natural, interventional effects	Mediation
`pcalg`	PC, GES, FCI, RFCI causal-discovery algorithms	Causal Discovery (both chapters)
`igraph`	Graph plotting for CPDAGs/PAGs; graph objects for `causaleffect`	Causal Discovery, DAG workflow
`Rgraphviz`, `graph`	Install-time dependencies of `pcalg` (graph classes; pcalg's own plot methods)	Causal Discovery (indirect)

22.3. `dagitty` and `ggdag`

The `dagitty` package provides a small DSL for graphs and the standard graph-theoretic identification queries:

```
library(dagitty)

g <- dagitty("dag {
  X -> A
  X -> Y
  A -> Y
}")

adjustmentSets(g, exposure = "A", outcome = "Y")
```

`adjustmentSets` returns *all* minimal sufficient sets for the backdoor criterion. Bidirected edges encoded as `A <-> Y` represent unobserved common causes (an ADMG), and `dagitty` correctly returns no adjustment set in that case.

`ggdag` consumes `dagitty` objects and renders them through `ggplot2`:

```
library(ggdag)
ggdag(g) + theme_dag_blank()
```

22.4. `causaleffect`

When `dagitty::adjustmentSets` returns an empty list, the effect may still be identified through a non-backdoor route (front-door, more general ID-algorithm patterns). `causaleffect` implements Tikka & Karvanen's R port of the Pearl–Shpitser ID algorithm:

```
library(igraph)
library(causaleffect)

g <- graph_from_literal(A -+ M, M -+ Y, A -+ Y, Y -+ A)
g <- set_edge_attr(g, "description", index = c(2, 4), value = "U")
causal.effect(y = "Y", x = "A", G = g, simp = TRUE)
# → \sum_{M} P(M|A)\left(\sum_{A} P(Y|A,M) P(A)\right)
```

The bidirected edge convention is two reciprocal directed edges with `description = "U"`. The function either returns a symbolic identification expression or raises an error indicating the effect is not identifiable.

22.5. `etwfe` and `fixest`

`fixest` handles the regression models with high-dimensional fixed effects. Its `feols`, `fepois`, and `feglm` use a formula syntax that keeps DiD and IV specifications readable:

```
library(fixest)

feols(y ~ x | id + year, data = df, vcov = ~id)        # TWFE, clustered SE
fepois(y ~ x + offset(log(pop)) | id + year, data = df)  # Poisson with FE
feols(y ~ x | id + year | x_endo ~ z, data = df)         # IV/2SLS
```

`etwfe` wraps `fixest` for Wooldridge's extended two-way fixed-effects DiD:

```
library(etwfe)

mod <- etwfe(fml = lemp ~ lpop, tvar = year, gvar = first.treat,
             data = mpdta, vcov = ~countyreal)
emfx(mod, type = "event")
```

`emfx()` aggregates the cohort × time interaction coefficients into an overall ATT, event-time effects, or calendar-time effects.

22.6. `pcalg`

`pcalg` provides PC, GES, FCI, RFCI, GIES, and LINGAM under one consistent S4 interface. It uses the `graph` package for graph objects and (for its own plot methods) `Rgraphviz`; the discovery chapters in this book plot with `igraph` instead. The Bioconductor dependencies must be installed via `BiocManager` before `pcalg` can be installed from CRAN.

```r
library(pcalg)

# Constraint-based: PC algorithm
pc_fit <- pc(suffStat = list(C = cor(data), n = nrow(data)),
             indepTest = gaussCItest, labels = colnames(data),
             alpha = 0.01)

# Score-based: Greedy Equivalence Search
ges_fit <- ges(new("GaussL0penObsScore", data))

# Latent-variable case: FCI / RFCI
fci_fit  <- fci(suffStat = list(C = cor(data), n = nrow(data)),
                indepTest = gaussCItest, labels = colnames(data), alpha = 0.01)
rfci_fit <- rfci(suffStat = list(C = cor(data), n = nrow(data)),
                 indepTest = gaussCItest, labels = colnames(data), alpha = 0.01)
```

Both observed (PC, GES) and latent (FCI, RFCI) chapters use these as the primary algorithms.

22.7. `lavaan` and `medoutcon`

`lavaan` ports the SEM model-string syntax familiar from EQS, Mplus, and LISREL into R. It is used in the mediation chapter for classical SEM mediation:

```r
library(lavaan)

model <- "
  m ~ a * x
  y ~ b * m + c * x
  indirect := a * b
  total    := c + indirect
"
fit <- sem(model, data = df)
parameterEstimates(fit)
```

For causal mediation under the potential-outcomes framework, the book uses `medoutcon` (Hejazi & van der Laan), which estimates controlled, natural, and interventional direct/indirect effects with cross-fitted nuisance estimators.

22.8. Practical Advice

The packages above cover the examples in this book. A few useful packages outside the main text are:

- `MatchIt` and `WeightIt` for matching and weighting estimators of the ATE/ATT

- `grf` (Generalized Random Forests) for heterogeneous treatment effects and instrumental-forest IV
- `bnlearn` for an alternative causal-discovery toolkit focused on Bayesian networks
- `lavaan.survey` and `blavaan` for survey-weighted and Bayesian SEM
- `mediation` for the classical Imai/Keele/Tingley mediation framework

When upgrading any of these packages, re-render affected chapters after deleting the relevant `_freeze/<chapter>/` directory so that Quarto does not reuse stale cached results.

References

Each chapter lists the works it cites at the bottom of its own page; the complete bibliography lives in `references.bib` in the book repository. Selected entries appear below. The main background references are Imbens and Rubin (2015) and Hernán and Robins (2020).

Adão, Rodrigo, Michal Kolesár, and Eduardo Morales. 2019. "Shift-Share Designs: Theory and Inference." *Quarterly Journal of Economics* 134 (4): 1949–2010.

Angrist, Joshua D., and Jörn-Steffen Pischke. 2009. *Mostly Harmless Econometrics: An Empiricist's Companion*. Princeton University Press.

Autor, David H., David Dorn, and Gordon H. Hanson. 2013. "The China Syndrome: Local Labor Market Effects of Import Competition in the United States." *American Economic Review* 103 (6): 2121–68.

Bartik, Timothy J. 1991. *Who Benefits from State and Local Economic Development Policies?* W.E. Upjohn Institute.

Blandhol, Christine, John Bonney, Magne Mogstad, and Alexander Torgovitsky. 2025. "When Is TSLS Actually LATE?"

Borusyak, Kirill, Peter Hull, and Xavier Jaravel. 2022. "Quasi-Experimental Shift-Share Research Designs." *Review of Economic Studies* 89 (1): 181–213.

Botosaru, Irene, and Laura Liu. 2025. "Time-Varying Heterogeneous Treatment Effects in Event Studies." *arXiv Preprint arXiv:2509.13698*. https://arxiv.org/abs/2509.13698.

———. 2026. "Event Studies with Feedback." *AEA Papers and Proceedings* 116: 70–74. https://doi.org/10.1257/pandp.20261110.

Chernozhukov, Victor, Mert Demirer, Esther Duflo, and Iván Fernández-Val. 2018. "Generic Machine Learning Inference on Heterogeneous Treatment Effects in Randomized Experiments." *NBER Working Paper 24678*.

Díaz, Iván, Nima S. Hejazi, Kara E. Rudolph, and Mark J. van der Laan. 2021. "Nonparametric Efficient Causal Mediation with Intermediate Confounders." *Biometrika* 108 (3): 627–41.

Díaz, Iván, Nicholas Williams, Katherine L. Hoffman, and Edward J. Schenck. 2023. "Nonparametric Causal Effects Based on Longitudinal Modified Treatment Policies." *Journal of the American Statistical Association* 118 (542): 846–57.

Goldsmith-Pinkham, Paul, Isaac Sorkin, and Henry Swift. 2020. "Bartik Instruments: What, When, Why, and How." *American Economic Review* 110 (8): 2586–2624.

Gourieroux, Christian, Alain Monfort, Eric Renault, and Alain Trognon. 1987. "Generalized Residuals." *Journal of Econometrics* 34 (1–2): 5–32.

Hernán, Miguel A. 2010. "The Hazards of Hazard Ratios." *Epidemiology* 21 (1): 13–15.

Hernán, Miguel A., and James M. Robins. 2020. *Causal Inference: What If*. Chapman & Hall/CRC.

Imbens, Guido W., and Donald B. Rubin. 2015. *Causal Inference for Statistics, Social, and Biomedical Sciences: An Introduction*. Cambridge University Press.

Imbens, Guido W., and Jeffrey M. Wooldridge. 2007. "What's New in Econometrics? Lecture 6: Control Functions and Related Methods." National Bureau of Economic Research Summer Institute. https://users.nber.org/~confer/2007/si2007/WNE/lect_6_controlfuncs.pdf.

Kennedy, Edward H. 2020. "Towards Optimal Doubly Robust Estimation of Heterogeneous Causal Effects." *arXiv Preprint arXiv:2004.14497*.

Kennedy, Edward H., Zongming Ma, Matthew D. McHugh, and Dylan S. Small. 2017. "Non-Parametric Methods for Doubly Robust Estimation of Continuous Treatment Effects." *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 79 (4): 1229–45.

Künzel, Sören R., Jasjeet S. Sekhon, Peter J. Bickel, and Bin Yu. 2019. "Metalearners for Estimating Heterogeneous Treatment Effects Using Machine Learning." *Proceedings of the National Academy of Sciences* 116 (10): 4156–65.

Mullahy, John. 1997. "Instrumental-Variable Estimation of Count Data Models: Applications to Models of Cigarette Smoking Behavior." *Review of Economics and Statistics* 79 (4): 586–93.

Nie, Xinkun, and Stefan Wager. 2021. "Quasi-Oracle Estimation of Heterogeneous Treatment Effects." *Biometrika* 108 (2): 299–319.

Ramsey, J. B. 1969. "Tests for Specification Errors in Classical Linear Least-Squares Regression Analysis." *Journal of the Royal Statistical Society, Series B* 31 (2): 350–71.

Słoczyński, Tymon. 2022. "Interpreting OLS Estimands When Treatment Effects Are Heterogeneous: Smaller Groups Get Larger Weights." *The Review of Economics and Statistics* 104 (3): 501–9.

———. 2024. "When Should We (Not) Interpret Linear IV Estimands as LATE?" *Quantitative Economics*.

Terza, Joseph V. 1998. "Estimating Count Models with Endogenous Switching: Sample Selection and Endogenous Treatment Effects." *Journal of Econometrics* 84: 129–54.

Vansteelandt, Stijn, and Rhian M. Daniel. 2017. "Interventional Effects for Mediation Analysis with Multiple Mediators." *Epidemiology* 28 (2): 258–65.

Wooldridge, Jeffrey M. 2010. *Econometric Analysis of Cross Section and Panel Data*. 2nd ed. MIT Press.

———. 2014. "Quasi-Maximum Likelihood Estimation and Testing for Nonlinear Models with Endogenous Explanatory Variables." *Journal of Econometrics* 182 (1): 226–34.

Xu, Ruonan. 2023. "Difference-in-Differences with Interference." *arXiv Preprint arXiv:2306.12003*. https://arxiv.org/abs/2306.12003.

———. 2026. "Dynamic Difference-in-Differences with Interference." *AEA Papers and Proceedings* 116: 58–63. https://doi.org/10.1257/pandp.20261108.

Zhang, Jiji. 2008. "On the Completeness of Orientation Rules for Causal Discovery in the Presence of Latent Confounders and Selection Bias." *Artificial Intelligence* 172 (16–17): 1873–96.